Atmel 8-bit AVR Microcontroller with 2/4/8K Bytes In-System Programmable Flash

ATtiny25/V / ATtiny45/V / ATtiny85/V

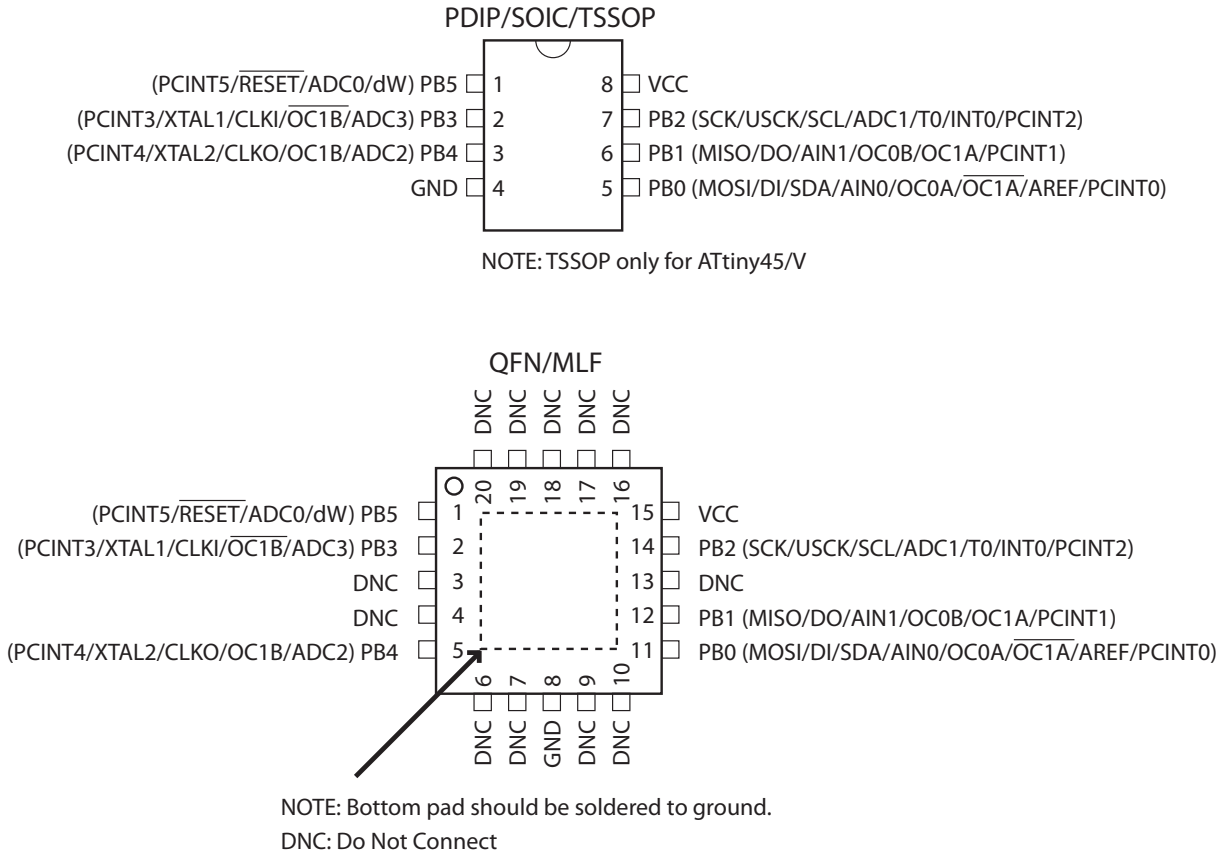
Features

- High Performance, Low Power AVR® 8-Bit Microcontroller
- Advanced RISC Architecture
  - 120 Powerful Instructions – Most Single Clock Cycle Execution
  - 32 x 8 General Purpose Working Registers
  - Fully Static Operation
- Non-volatile Program and Data Memories
  - 2/4/8K Bytes of In-System Programmable Program Memory Flash
    - Endurance: 10,000 Write/Erase Cycles
  - 128/256/512 Bytes In-System Programmable EEPROM
    - Endurance: 100,000 Write/Erase Cycles
  - 128/256/512 Bytes Internal SRAM
  - Programming Lock for Self-Programming Flash Program and EEPROM Data Security
- Peripheral Features
  - 8-bit Timer/Counter with Prescaler and Two PWM Channels
  - 8-bit High Speed Timer/Counter with Separate Prescaler
    - 2 High Frequency PWM Outputs with Separate Output Compare Registers
    - Programmable Dead Time Generator
  - USI – Universal Serial Interface with Start Condition Detector
  - 10-bit ADC
    - 4 Single Ended Channels
    - 2 Differential ADC Channel Pairs with Programmable Gain (1x, 20x)
    - Temperature Measurement
  - Programmable Watchdog Timer with Separate On-chip Oscillator
  - On-chip Analog Comparator
- Special Microcontroller Features
  - debugWIRE On-chip Debug System
  - In-System Programmable via SPI Port
  - External and Internal Interrupt Sources
  - Low Power Idle, ADC Noise Reduction, and Power-down Modes
  - Enhanced Power-on Reset Circuit
  - Programmable Brown-out Detection Circuit
  - Internal Calibrated Oscillator
- I/O and Packages
  - Six Programmable I/O Lines
  - 8-pin PDIP, 8-pin SOIC, 20-pad QFN/MLF, and 8-pin TSSOP (only ATtiny45/V)
- Operating Voltage
  - 1.8 - 5.5V for ATtiny25V/45V/85V
  - 2.7 - 5.5V for ATtiny25/45/85
- Speed Grade
  - ATtiny25V/45V/85V: 0 – 4 MHz @ 1.8 - 5.5V, 0 - 10 MHz @ 2.7 - 5.5V
  - ATtiny25/45/85: 0 – 10 MHz @ 2.7 - 5.5V, 0 - 20 MHz @ 4.5 - 5.5V
- Industrial Temperature Range
- Low Power Consumption
  - Active Mode:
    - 1 MHz, 1.8V: 300 µA
  - Power-down Mode:
    - 0.1 µA at 1.8V

Rev. 2586Q–AVR–08/2013

1. Pin Configurations

Figure 1-1. Pinout ATtiny25/45/85



1.1 Pin Descriptions

1.1.1 VCC

Supply voltage.

1.1.2 GND

Ground.

1.1.3 Port B (PB5:PB0)

Port B is a 6-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port B output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port B pins that are externally pulled low will source current if the pull-up resistors are activated. The Port B pins are tri-stated when a reset condition becomes active, even if the clock is not running.

Port B also serves the functions of various special features of the ATtiny25/45/85 as listed in “Alternate Functions of Port B” on page 60.

On ATtiny25, the programmable I/O ports PB3 and PB4 (pins 2 and 3) are exchanged in ATtiny15 Compatibility Mode for supporting the backward compatibility with ATtiny15.

1.1.4 $\overline{\text{RESET}}$

Reset input. A low level on this pin for longer than the minimum pulse length will generate a reset, even if the clock is not running and provided the reset pin has not been disabled. The minimum pulse length is given in Table 21-4 on page 165. Shorter pulses are not guaranteed to generate a reset.

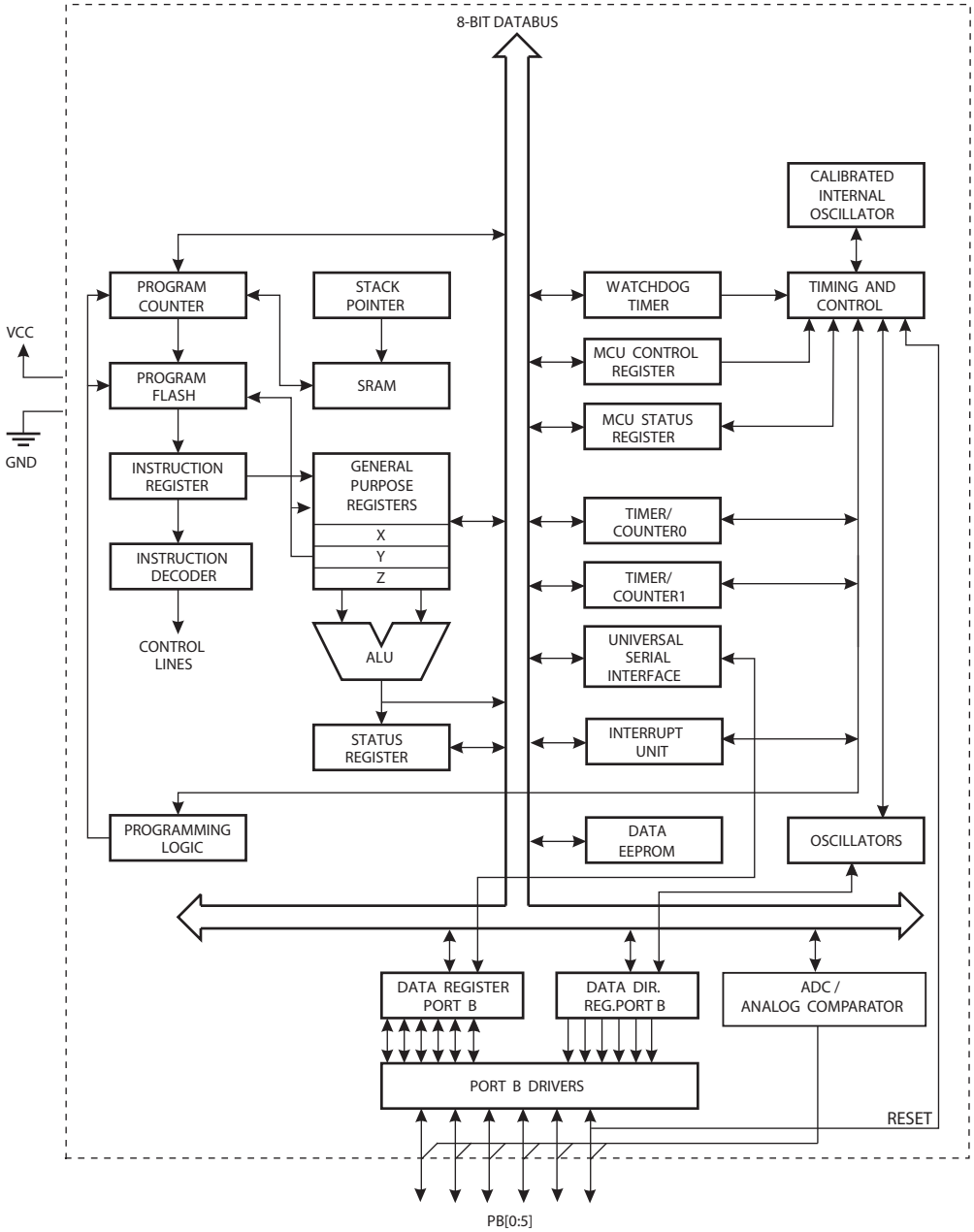
The reset pin can also be used as a (weak) I/O pin.

2. Overview

The ATtiny25/45/85 is a low-power CMOS 8-bit microcontroller based on the AVR enhanced RISC architecture. By executing powerful instructions in a single clock cycle, the ATtiny25/45/85 achieves throughputs approaching 1 MIPS per MHz allowing the system designer to optimize power consumption versus processing speed.

2.1 Block Diagram

Figure 2-1. Block Diagram



The AVR core combines a rich instruction set with 32 general purpose working registers. All 32 registers are directly connected to the Arithmetic Logic Unit (ALU), allowing two independent registers to be accessed in one single instruction executed in one clock cycle. The resulting architecture is more code efficient while achieving throughputs up to ten times faster than conventional CISC microcontrollers.

The ATtiny25/45/85 provides the following features: 2/4/8K bytes of In-System Programmable Flash, 128/256/512 bytes EEPROM, 128/256/256 bytes SRAM, 6 general purpose I/O lines, 32 general purpose working registers, one 8-bit Timer/Counter with compare modes, one 8-bit high speed Timer/Counter, Universal Serial Interface, Internal and External Interrupts, a 4-channel, 10-bit ADC, a programmable Watchdog Timer with internal Oscillator, and three software selectable power saving modes. Idle mode stops the CPU while allowing the SRAM, Timer/Counter, ADC, Analog Comparator, and Interrupt system to continue functioning. Power-down mode saves the register contents, disabling all chip functions until the next Interrupt or Hardware Reset. ADC Noise Reduction mode stops the CPU and all I/O modules except ADC, to minimize switching noise during ADC conversions.

The device is manufactured using Atmel's high density non-volatile memory technology. The On-chip ISP Flash allows the Program memory to be re-programmed In-System through an SPI serial interface, by a conventional non-volatile memory programmer or by an On-chip boot code running on the AVR core.

The ATtiny25/45/85 AVR is supported with a full suite of program and system development tools including: C Compilers, Macro Assemblers, Program Debugger/Simulators and Evaluation kits.

3. About

3.1 Resources

A comprehensive set of development tools, application notes and datasheets are available for download on http://www.atmel.com/avr.

3.2 Code Examples

This documentation contains simple code examples that briefly show how to use various parts of the device. These code examples assume that the part specific header file is included before compilation. Be aware that not all C compiler vendors include bit definitions in the header files and interrupt handling in C is compiler dependent. Please confirm with the C compiler documentation for more details.

For I/O Registers located in the extended I/O map, “IN”, “OUT”, “SBIS”, “SBIC”, “CBI”, and “SBI” instructions must be replaced with instructions that allow access to extended I/O. Typically, this means “LDS” and “STS” combined with “SBRS”, “SBRC”, “SBR”, and “CBR”. Note that not all AVR devices include an extended I/O map.

3.3 Capacitive Touch Sensing

Atmel QTouch Library provides a simple to use solution for touch sensitive interfaces on Atmel AVR microcontrollers. The QTouch Library includes support for QTouch® and QMatrix® acquisition methods.

Touch sensing is easily added to any application by linking the QTouch Library and using the Application Programming Interface (API) of the library to define the touch channels and sensors. The application then calls the API to retrieve channel information and determine the state of the touch sensor.

The QTouch Library is free and can be downloaded from the Atmel website. For more information and details of implementation, refer to the QTouch Library User Guide – also available from the Atmel website.

3.4 Data Retention

Reliability Qualification results show that the projected data retention failure rate is much less than 1 PPM over 20 years at 85°C or 100 years at 25°C.

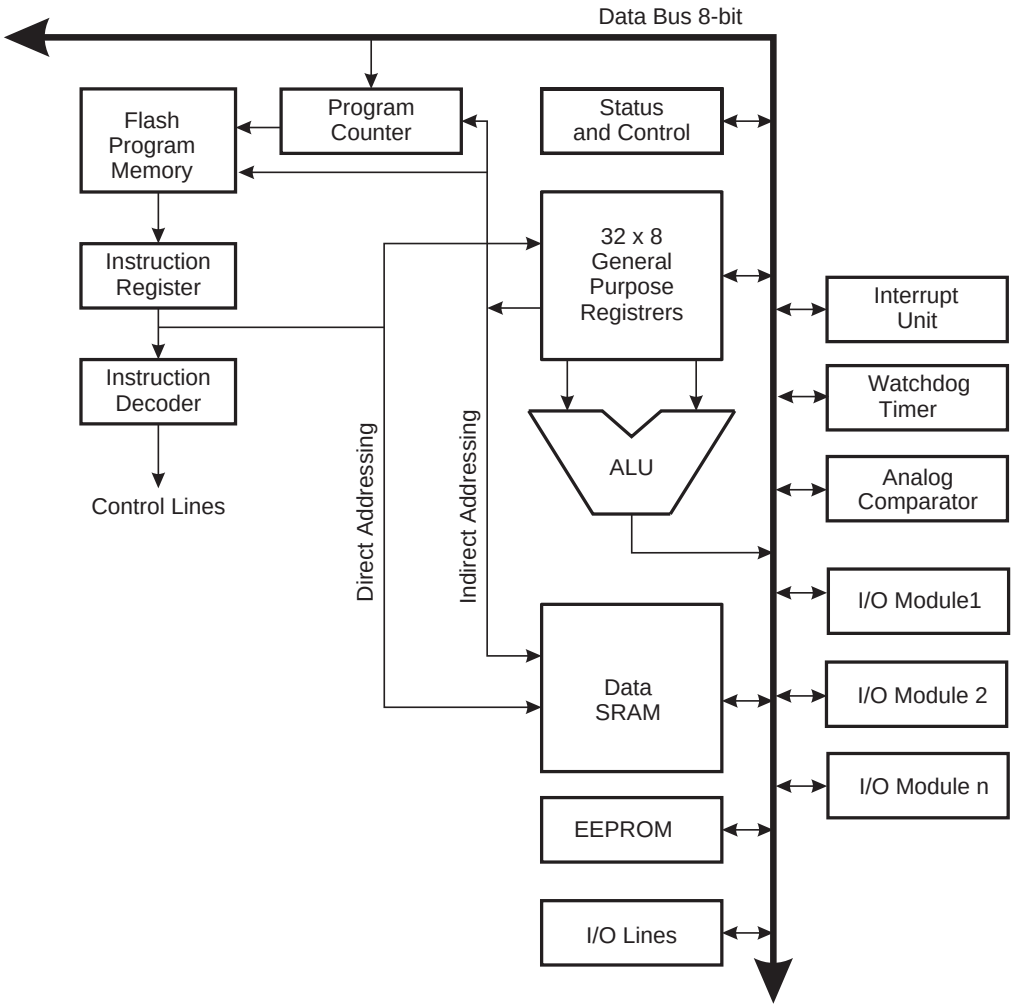
4. AVR CPU Core

4.1 Introduction

This section discusses the AVR core architecture in general. The main function of the CPU core is to ensure correct program execution. The CPU must therefore be able to access memories, perform calculations, control peripherals, and handle interrupts.

4.2 Architectural Overview

Figure 4-1. Block Diagram of the AVR Architecture



In order to maximize performance and parallelism, the AVR uses a Harvard architecture – with separate memories and buses for program and data. Instructions in the Program memory are executed with a single level pipelining. While one instruction is being executed, the next instruction is pre-fetched from the Program memory. This concept enables instructions to be executed in every clock cycle. The Program memory is In-System Reprogrammable Flash memory.

The fast-access Register File contains 32 x 8-bit general purpose working registers with a single clock cycle access time. This allows single-cycle Arithmetic Logic Unit (ALU) operation. In a typical ALU operation, two oper-

ands are output from the Register File, the operation is executed, and the result is stored back in the Register File – in one clock cycle.

Six of the 32 registers can be used as three 16-bit indirect address register pointers for Data Space addressing – enabling efficient address calculations. One of the these address pointers can also be used as an address pointer for look up tables in Flash Program memory. These added function registers are the 16-bit X-, Y-, and Z-register, described later in this section.

The ALU supports arithmetic and logic operations between registers or between a constant and a register. Single register operations can also be executed in the ALU. After an arithmetic operation, the Status Register is updated to reflect information about the result of the operation.

Program flow is provided by conditional and unconditional jump and call instructions, able to directly address the whole address space. Most AVR instructions have a single 16-bit word format, but there are also 32-bit instructions.

During interrupts and subroutine calls, the return address Program Counter (PC) is stored on the Stack. The Stack is effectively allocated in the general data SRAM, and consequently the Stack size is only limited by the total SRAM size and the usage of the SRAM. All user programs must initialize the SP in the Reset routine (before subroutines or interrupts are executed). The Stack Pointer (SP) is read/write accessible in the I/O space. The data SRAM can easily be accessed through the five different addressing modes supported in the AVR architecture.

The memory spaces in the AVR architecture are all linear and regular memory maps.

A flexible interrupt module has its control registers in the I/O space with an additional Global Interrupt Enable bit in the Status Register. All interrupts have a separate Interrupt Vector in the Interrupt Vector table. The interrupts have priority in accordance with their Interrupt Vector position. The lower the Interrupt Vector address, the higher the priority.

The I/O memory space contains 64 addresses for CPU peripheral functions as Control Registers, SPI, and other I/O functions. The I/O memory can be accessed directly, or as the Data Space locations following those of the Register File, 0x20 - 0x5F.

4.3 ALU – Arithmetic Logic Unit

The high-performance AVR ALU operates in direct connection with all the 32 general purpose working registers. Within a single clock cycle, arithmetic operations between general purpose registers or between a register and an immediate are executed. The ALU operations are divided into three main categories – arithmetic, logical, and bit-functions. Some implementations of the architecture also provide a powerful multiplier supporting both signed/unsigned multiplication and fractional format. See the “Instruction Set” section for a detailed description.

4.4 Status Register

The Status Register contains information about the result of the most recently executed arithmetic instruction. This information can be used for altering program flow in order to perform conditional operations. Note that the Status Register is updated after all ALU operations, as specified in the Instruction Set Reference. This will in many cases remove the need for using the dedicated compare instructions, resulting in faster and more compact code.

The Status Register is not automatically stored when entering an interrupt routine and restored when returning from an interrupt. This must be handled by software.

4.4.1 SREG – AVR Status Register

The AVR Status Register – SREG – is defined as:

Bit	7	6	5	4	3	2	1	0	
0x3F	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – I: Global Interrupt Enable**

The Global Interrupt Enable bit must be set for the interrupts to be enabled. The individual interrupt enable control is then performed in separate control registers. If the Global Interrupt Enable Register is cleared, none of the interrupts are enabled independent of the individual interrupt enable settings. The I-bit is cleared by hardware after an interrupt has occurred, and is set by the RETI instruction to enable subsequent interrupts. The I-bit can also be set and cleared by the application with the SEI and CLI instructions, as described in the instruction set reference.

- **Bit 6 – T: Bit Copy Storage**

The Bit Copy instructions BLD (Bit LoaD) and BST (Bit STore) use the T-bit as source or destination for the operated bit. A bit from a register in the Register File can be copied into T by the BST instruction, and a bit in T can be copied into a bit in a register in the Register File by the BLD instruction.

- **Bit 5 – H: Half Carry Flag**

The Half Carry Flag H indicates a Half Carry in some arithmetic operations. Half Carry is useful in BCD arithmetic. See the “Instruction Set Description” for detailed information.

- **Bit 4 – S: Sign Bit, S = N ⊕ V**

The S-bit is always an exclusive or between the Negative Flag N and the Two’s Complement Overflow Flag V. See the “Instruction Set Description” for detailed information.

- **Bit 3 – V: Two’s Complement Overflow Flag**

The Two’s Complement Overflow Flag V supports two’s complement arithmetics. See the “Instruction Set Description” for detailed information.

- **Bit 2 – N: Negative Flag**

The Negative Flag N indicates a negative result in an arithmetic or logic operation. See the “Instruction Set Description” for detailed information.

- **Bit 1 – Z: Zero Flag**

The Zero Flag Z indicates a zero result in an arithmetic or logic operation. See the “Instruction Set Description” for detailed information.

- **Bit 0 – C: Carry Flag**

The Carry Flag C indicates a carry in an arithmetic or logic operation. See the “Instruction Set Description” for detailed information.

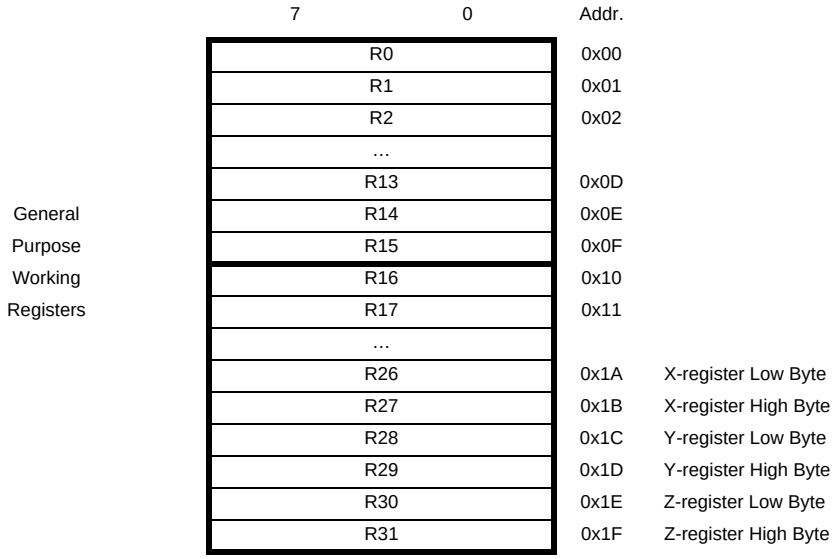
4.5 General Purpose Register File

The Register File is optimized for the AVR Enhanced RISC instruction set. In order to achieve the required performance and flexibility, the following input/output schemes are supported by the Register File:

- One 8-bit output operand and one 8-bit result input
- Two 8-bit output operands and one 8-bit result input
- Two 8-bit output operands and one 16-bit result input
- One 16-bit output operand and one 16-bit result input

Figure 4-2 shows the structure of the 32 general purpose working registers in the CPU.

Figure 4-2. AVR CPU General Purpose Working Registers



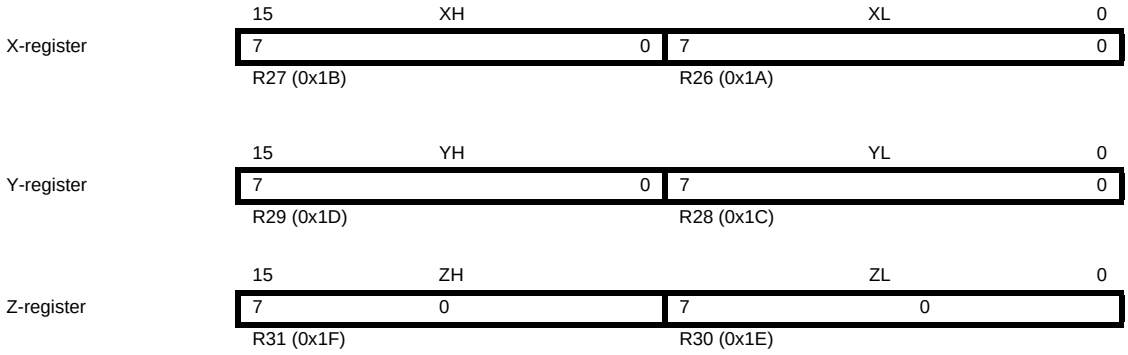
Most of the instructions operating on the Register File have direct access to all registers, and most of them are single cycle instructions.

As shown in Figure 4-2, each register is also assigned a Data memory address, mapping them directly into the first 32 locations of the user Data Space. Although not being physically implemented as SRAM locations, this memory organization provides great flexibility in access of the registers, as the X-, Y- and Z-pointer registers can be set to index any register in the file.

4.5.1 The X-register, Y-register, and Z-register

The registers R26..R31 have some added functions to their general purpose usage. These registers are 16-bit address pointers for indirect addressing of the data space. The three indirect address registers X, Y, and Z are defined as described in Figure 4-3.

Figure 4-3. The X-, Y-, and Z-registers



In the different addressing modes these address registers have functions as fixed displacement, automatic increment, and automatic decrement (see the instruction set reference for details).

4.6 Stack Pointer

The Stack is mainly used for storing temporary data, for storing local variables and for storing return addresses after interrupts and subroutine calls. The Stack Pointer Register always points to the top of the Stack. Note that the Stack is implemented as growing from higher memory locations to lower memory locations. This implies that a Stack PUSH command decreases the Stack Pointer.

The Stack Pointer points to the data SRAM Stack area where the Subroutine and Interrupt Stacks are located. This Stack space in the data SRAM must be defined by the program before any subroutine calls are executed or interrupts are enabled. The Stack Pointer must be set to point above 0x60. The Stack Pointer is decremented by one when data is pushed onto the Stack with the PUSH instruction, and it is decremented by two when the return address is pushed onto the Stack with subroutine call or interrupt. The Stack Pointer is incremented by one when data is popped from the Stack with the POP instruction, and it is incremented by two when data is popped from the Stack with return from subroutine RET or return from interrupt RETI.

The AVR Stack Pointer is implemented as two 8-bit registers in the I/O space. The number of bits actually used is implementation dependent. Note that the data space in some implementations of the AVR architecture is so small that only SPL is needed. In this case, the SPH Register will not be present.

4.6.1 SPH and SPL — Stack Pointer Register

Bit	15	14	13	12	11	10	9	8	
0x3E	**SP15**	**SP14**	**SP13**	**SP12**	**SP11**	**SP10**	**SP9**	**SP8**	**SPH**
0x3D	**SP7**	**SP6**	**SP5**	**SP4**	**SP3**	**SP2**	**SP1**	**SP0**	**SPL**
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	
Initial Value	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	

4.7 Instruction Execution Timing

This section describes the general access timing concepts for instruction execution. The AVR CPU is driven by the CPU clock clk_{CPU}, directly generated from the selected clock source for the chip. No internal clock division is used.

Figure 4-4 shows the parallel instruction fetches and instruction executions enabled by the Harvard architecture and the fast access Register File concept. This is the basic pipelining concept to obtain up to 1 MIPS per MHz with the corresponding unique results for functions per cost, functions per clocks, and functions per power-unit.

Figure 4-4. The Parallel Instruction Fetches and Instruction Executions

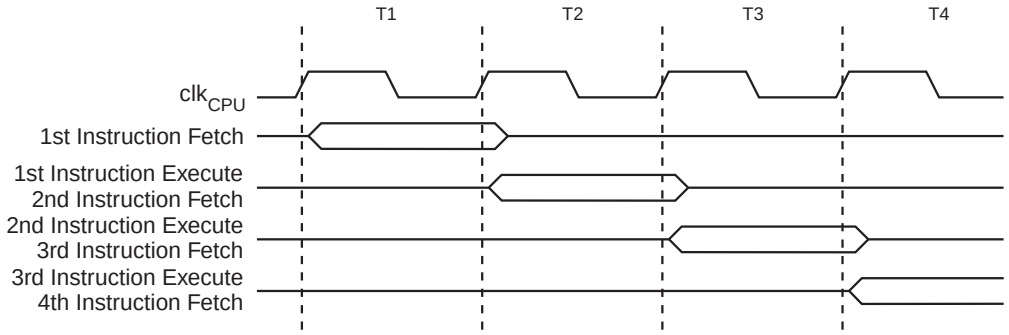
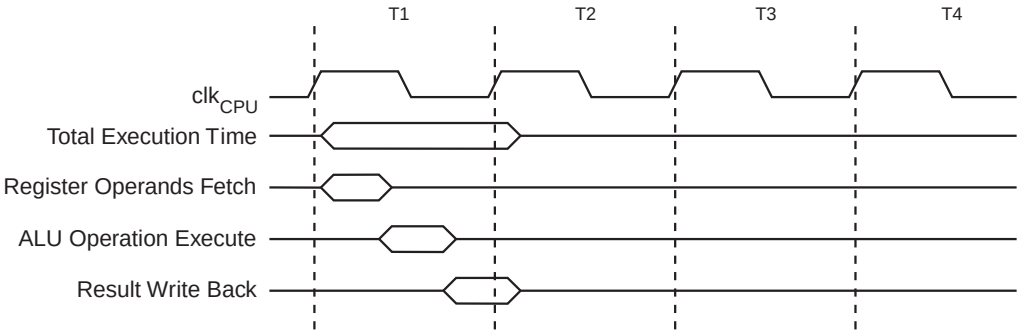


Figure 4-5 shows the internal timing concept for the Register File. In a single clock cycle an ALU operation using two register operands is executed, and the result is stored back to the destination register.

Figure 4-5. Single Cycle ALU Operation



4.8 Reset and Interrupt Handling

The AVR provides several different interrupt sources. These interrupts and the separate Reset Vector each have a separate Program Vector in the Program memory space. All interrupts are assigned individual enable bits which must be written logic one together with the Global Interrupt Enable bit in the Status Register in order to enable the interrupt.

The lowest addresses in the Program memory space are by default defined as the Reset and Interrupt Vectors. The complete list of vectors is shown in “Interrupts” on page 48. The list also determines the priority levels of the different interrupts. The lower the address the higher is the priority level. RESET has the highest priority, and next is INT0 – the External Interrupt Request 0.

When an interrupt occurs, the Global Interrupt Enable I-bit is cleared and all interrupts are disabled. The user software can write logic one to the I-bit to enable nested interrupts. All enabled interrupts can then interrupt the current interrupt routine. The I-bit is automatically set when a Return from Interrupt instruction – RETI – is executed.

There are basically two types of interrupts. The first type is triggered by an event that sets the Interrupt Flag. For these interrupts, the Program Counter is vectored to the actual Interrupt Vector in order to execute the interrupt handling routine, and hardware clears the corresponding Interrupt Flag. Interrupt Flags can also be cleared by writing a logic one to the flag bit position(s) to be cleared. If an interrupt condition occurs while the corresponding interrupt enable bit is cleared, the Interrupt Flag will be set and remembered until the interrupt is enabled, or the flag is cleared by software. Similarly, if one or more interrupt conditions occur while the Global Interrupt Enable bit is cleared, the corresponding Interrupt Flag(s) will be set and remembered until the Global Interrupt Enable bit is set, and will then be executed by order of priority.

The second type of interrupts will trigger as long as the interrupt condition is present. These interrupts do not necessarily have Interrupt Flags. If the interrupt condition disappears before the interrupt is enabled, the interrupt will not be triggered.

When the AVR exits from an interrupt, it will always return to the main program and execute one more instruction before any pending interrupt is served.

Note that the Status Register is not automatically stored when entering an interrupt routine, nor restored when returning from an interrupt routine. This must be handled by software.

When using the CLI instruction to disable interrupts, the interrupts will be immediately disabled. No interrupt will be executed after the CLI instruction, even if it occurs simultaneously with the CLI instruction. The following example shows how this can be used to avoid interrupts during the timed EEPROM write sequence.

Assembly Code Example

```
in  r16, SREG     ; store SREG value
cli    ; disable interrupts during timed sequence
sbi EECR, EEMPE   ; start EEPROM write
sbi EECR, EEPE
out SREG, r16     ; restore SREG value (I-bit)
```

C Code Example

```
char cSREG;
cSREG = SREG; /* store SREG value */
/* disable interrupts during timed sequence */
_CLI();
EECR |= (1<<EEMPE); /* start EEPROM write */
EECR |= (1<<EEPE);
SREG = cSREG; /* restore SREG value (I-bit) */
```

When using the SEI instruction to enable interrupts, the instruction following SEI will be executed before any pending interrupts, as shown in this example.

Assembly Code Example

```
sei  ; set Global Interrupt Enable
sleep; enter sleep, waiting for interrupt
; note: will enter sleep before any pending
; interrupt(s)
```

C Code Example

```
_SEI(); /* set Global Interrupt Enable */
_SLEEP(); /* enter sleep, waiting for interrupt */
/* note: will enter sleep before any pending interrupt(s) */
```

4.8.1 Interrupt Response Time

The interrupt execution response for all the enabled AVR interrupts is four clock cycles minimum. After four clock cycles the Program Vector address for the actual interrupt handling routine is executed. During this four clock cycle period, the Program Counter is pushed onto the Stack. The vector is normally a jump to the interrupt routine, and this jump takes three clock cycles. If an interrupt occurs during execution of a multi-cycle instruction, this instruction is completed before the interrupt is served. If an interrupt occurs when the MCU is in sleep mode, the interrupt execution response time is increased by four clock cycles. This increase comes in addition to the start-up time from the selected sleep mode.

A return from an interrupt handling routine takes four clock cycles. During these four clock cycles, the Program Counter (two bytes) is popped back from the Stack, the Stack Pointer is incremented by two, and the I-bit in SREG is set.

5. AVR Memories

This section describes the different memories in the ATtiny25/45/85. The AVR architecture has two main memory spaces, the Data memory and the Program memory space. In addition, the ATtiny25/45/85 features an EEPROM Memory for data storage. All three memory spaces are linear and regular.

5.1 In-System Re-programmable Flash Program Memory

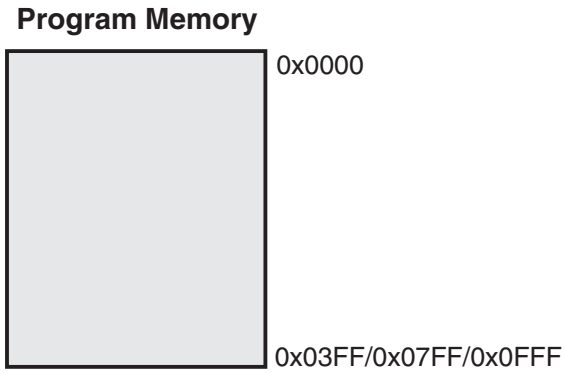
The ATtiny25/45/85 contains 2/4/8K bytes On-chip In-System Reprogrammable Flash memory for program storage. Since all AVR instructions are 16 or 32 bits wide, the Flash is organized as 1024/2048/4096 x 16.

The Flash memory has an endurance of at least 10,000 write/erase cycles. The ATtiny25/45/85 Program Counter (PC) is 10/11/12 bits wide, thus addressing the 1024/2048/4096 Program memory locations. “Memory Programming” on page 147 contains a detailed description on Flash data serial downloading using the SPI pins.

Constant tables can be allocated within the entire Program memory address space (see the LPM – Load Program memory instruction description).

Timing diagrams for instruction fetch and execution are presented in “Instruction Execution Timing” on page 11.

Figure 5-1. Program Memory Map



5.2 SRAM Data Memory

Figure 5-2 shows how the ATtiny25/45/85 SRAM Memory is organized.

The lower 224/352/607 Data memory locations address both the Register File, the I/O memory and the internal data SRAM. The first 32 locations address the Register File, the next 64 locations the standard I/O memory, and the last 128/256/512 locations address the internal data SRAM.

The five different addressing modes for the Data memory cover: Direct, Indirect with Displacement, Indirect, Indirect with Pre-decrement, and Indirect with Post-increment. In the Register File, registers R26 to R31 feature the indirect addressing pointer registers.

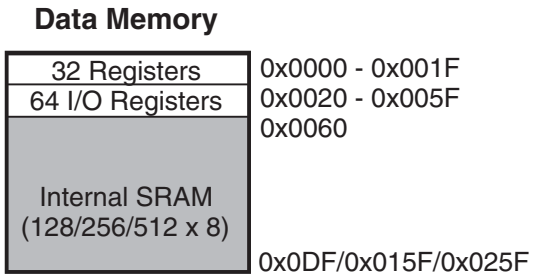
The direct addressing reaches the entire data space.

The Indirect with Displacement mode reaches 63 address locations from the base address given by the Y- or Z-register.

When using register indirect addressing modes with automatic pre-decrement and post-increment, the address registers X, Y, and Z are decremented or incremented.

The 32 general purpose working registers, 64 I/O Registers, and the 128/256/512 bytes of internal data SRAM in the ATtiny25/45/85 are all accessible through all these addressing modes. The Register File is described in “General Purpose Register File” on page 10.

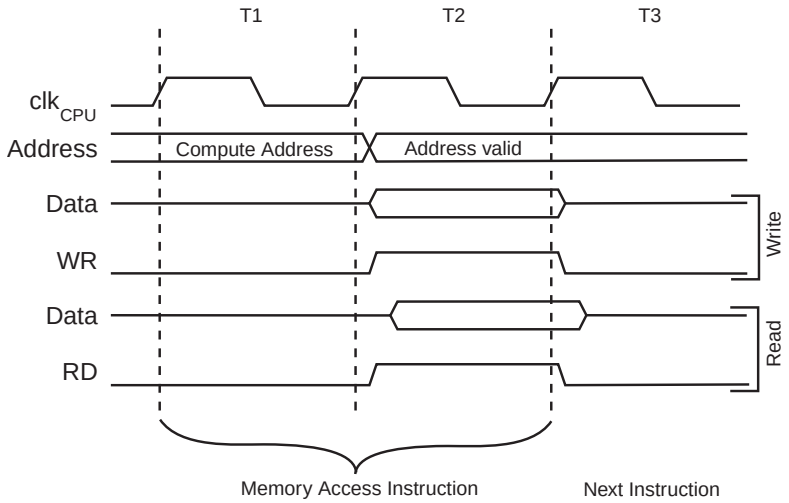
Figure 5-2. Data Memory Map



5.2.1 Data Memory Access Times

This section describes the general access timing concepts for internal memory access. The internal data SRAM access is performed in two clk_{CPU} cycles as described in Figure 5-3.

Figure 5-3. On-chip Data SRAM Access Cycles



5.3 EEPROM Data Memory

The ATtiny25/45/85 contains 128/256/512 bytes of data EEPROM memory. It is organized as a separate data space, in which single bytes can be read and written. The EEPROM has an endurance of at least 100,000 write/erase cycles. The access between the EEPROM and the CPU is described in the following, specifying the EEPROM Address Registers, the EEPROM Data Register, and the EEPROM Control Register. For details see “Serial Downloading” on page 151.

5.3.1 EEPROM Read/Write Access

The EEPROM Access Registers are accessible in the I/O space.

The write access times for the EEPROM are given in Table 5-1 on page 21. A self-timing function, however, lets the user software detect when the next byte can be written. If the user code contains instructions that write the EEPROM, some precautions must be taken. In heavily filtered power supplies, V_{CC} is likely to rise or fall slowly on Power-up/down. This causes the device for some period of time to run at a voltage lower than specified as minimum for the clock frequency used. See “Preventing EEPROM Corruption” on page 19 for details on how to avoid problems in these situations.

In order to prevent unintentional EEPROM writes, a specific write procedure must be followed. Refer to “Atomic Byte Programming” on page 17 and “Split Byte Programming” on page 17 for details on this.

When the EEPROM is read, the CPU is halted for four clock cycles before the next instruction is executed. When the EEPROM is written, the CPU is halted for two clock cycles before the next instruction is executed.

5.3.2 Atomic Byte Programming

Using Atomic Byte Programming is the simplest mode. When writing a byte to the EEPROM, the user must write the address into the EEAR Register and data into EEDR Register. If the EEPMn bits are zero, writing EEPE (within four cycles after EEMPE is written) will trigger the erase/write operation. Both the erase and write cycle are done in one operation and the total programming time is given in Table 5-1 on page 21. The EEPE bit remains set until the erase and write operations are completed. While the device is busy with programming, it is not possible to do any other EEPROM operations.

5.3.3 Split Byte Programming

It is possible to split the erase and write cycle in two different operations. This may be useful if the system requires short access time for some limited period of time (typically if the power supply voltage falls). In order to take advantage of this method, it is required that the locations to be written have been erased before the write operation. But since the erase and write operations are split, it is possible to do the erase operations when the system allows doing time-critical operations (typically after Power-up).

5.3.4 Erase

To erase a byte, the address must be written to EEAR. If the EEPMn bits are 0b01, writing the EEPE (within four cycles after EEMPE is written) will trigger the erase operation only (programming time is given in Table 5-1 on page 21). The EEPE bit remains set until the erase operation completes. While the device is busy programming, it is not possible to do any other EEPROM operations.

5.3.5 Write

To write a location, the user must write the address into EEAR and the data into EEDR. If the EEPMn bits are 0b10, writing the EEPE (within four cycles after EEMPE is written) will trigger the write operation only (programming time is given in Table 5-1 on page 21). The EEPE bit remains set until the write operation completes. If the location to be written has not been erased before write, the data that is stored must be considered as lost. While the device is busy with programming, it is not possible to do any other EEPROM operations.

The calibrated Oscillator is used to time the EEPROM accesses. Make sure the Oscillator frequency is within the requirements described in “OSCCAL – Oscillator Calibration Register” on page 31.

The following code examples show one assembly and one C function for erase, write, or atomic write of the EEPROM. The examples assume that interrupts are controlled (e.g., by disabling interrupts globally) so that no interrupts will occur during execution of these functions.

Assembly Code Example

```
EEPROM_write:
  ; Wait for completion of previous write
  sbic EECR,EEPE
  rjmp EEPROM_write
  ; Set Programming mode
  ldi  r16, (0<<EEPM1)|(0<<EEPM0)
  out  EECR, r16
  ; Set up address (r18:r17) in address register
  out EEARH, r18
  out EEARL, r17
  ; Write data (r19) to data register
  out EEDR, r19
  ; Write logical one to EEMPE
  sbi EECR,EEMPE
  ; Start eeprom write by setting EEPE
  sbi EECR,EEPE
  ret
```

C Code Example

```
void EEPROM_write(unsigned char ucAddress, unsigned char ucData)
{
  /* Wait for completion of previous write */
  while(EECR & (1<<EEPE))
    ;
  /* Set Programming mode */
  EECR = (0<<EEPM1)|(0<<EEPM0);
  /* Set up address and data registers */
  EEAR = ucAddress;
  EEDR = ucData;
  /* Write logical one to EEMPE */
  EECR |= (1<<EEMPE);
  /* Start eeprom write by setting EEPE */
  EECR |= (1<<EEPE);
}
```

The next code examples show assembly and C functions for reading the EEPROM. The examples assume that interrupts are controlled so that no interrupts will occur during execution of these functions.

Assembly Code Example

```
EEPROM_read:
  ; Wait for completion of previous write
  sbic EECR,EEPE
  rjmp EEPROM_read
  ; Set up address (r18:r17) in address register
  out  EEARH, r18
  out  EEARL, r17
  ; Start eeprom read by writing EERE
  sbi  EECR,EERE
  ; Read data from data register
  in   r16,EEDR
  ret
```

C Code Example

```
unsigned char EEPROM_read(unsigned char ucAddress)
{
  /* Wait for completion of previous write */
  while(EECR & (1<<EEPE))
    ;
  /* Set up address register */
  EEAR = ucAddress;
  /* Start eeprom read by writing EERE */
  EECR |= (1<<EERE);
  /* Return data from data register */
  return EEDR;
}
```

5.3.6 Preventing EEPROM Corruption

During periods of low V_{CC}, the EEPROM data can be corrupted because the supply voltage is too low for the CPU and the EEPROM to operate properly. These issues are the same as for board level systems using EEPROM, and the same design solutions should be applied.

An EEPROM data corruption can be caused by two situations when the voltage is too low. First, a regular write sequence to the EEPROM requires a minimum voltage to operate correctly. Secondly, the CPU itself can execute instructions incorrectly, if the supply voltage is too low.

EEPROM data corruption can easily be avoided by following this design recommendation:

Keep the AVR RESET active (low) during periods of insufficient power supply voltage. This can be done by enabling the internal Brown-out Detector (BOD). If the detection level of the internal BOD does not match the needed detection level, an external low V_{CC} reset protection circuit can be used. If a reset occurs while a write operation is in progress, the write operation will be completed provided that the power supply voltage is sufficient.

5.4 I/O Memory

The I/O space definition of the ATtiny25/45/85 is shown in “Register Summary” on page 200.

All ATtiny25/45/85 I/Os and peripherals are placed in the I/O space. All I/O locations may be accessed by the LD/LDS/LDD and ST/STS/STD instructions, transferring data between the 32 general purpose working registers and the I/O space. I/O Registers within the address range 0x00 - 0x1F are directly bit-accessible using the SBI and CBI instructions. In these registers, the value of single bits can be checked by using the SBIS and SBIC instructions. Refer to the instruction set section for more details. When using the I/O specific commands IN and OUT, the I/O addresses 0x00 - 0x3F must be used. When addressing I/O Registers as data space using LD and ST instructions, 0x20 must be added to these addresses.

For compatibility with future devices, reserved bits should be written to zero if accessed. Reserved I/O memory addresses should never be written.

Some of the Status Flags are cleared by writing a logical one to them. Note that the CBI and SBI instructions will only operate on the specified bit, and can therefore be used on registers containing such Status Flags. The CBI and SBI instructions work with registers 0x00 to 0x1F only.

The I/O and Peripherals Control Registers are explained in later sections.

5.5 Register Description

5.5.1 EEARH – EEPROM Address Register

Bit	7	6	5	4	3	2	1	0	
0x1F	–	–	–	–	–	–	–	**EEAR8**	EEARH
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	X/0	

- **Bits 7:1 – Res: Reserved Bits**

These bits are reserved for future use and will always read as zero.

- **Bits 0 – EEAR8: EEPROM Address**

This is the most significant EEPROM address bit of ATtiny85. In devices with less EEPROM, i.e. ATtiny25/ATtiny45, this bit is reserved and will always read zero. The initial value of the EEPROM Address Register (EEAR) is undefined and a proper value must therefore be written before the EEPROM is accessed.

5.5.2 EEARL – EEPROM Address Register

Bit									
0x1E	**EEAR7**	**EEAR6**	**EEAR5**	**EEAR4**	**EEAR3**	**EEAR2**	**EEAR1**	**EEAR0**	EEARL
Rear/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	X	X	X	X	X	X	X	X	

- **Bit 7 – EEAR7: EEPROM Address**

This is the most significant EEPROM address bit of ATtiny45. In devices with less EEPROM, i.e. ATtiny25, this bit is reserved and will always read zero. The initial value of the EEPROM Address Register (EEAR) is undefined and a proper value must therefore be written before the EEPROM is accessed.

- **Bits 6:0 – EEAR[6:0]: EEPROM Address**

These are the (low) bits of the EEPROM Address Register. The EEPROM data bytes are addressed linearly in the range 0...(128/256/512-1). The initial value of EEAR is undefined and a proper value must be therefore be written before the EEPROM may be accessed.

5.5.3 EEDR – EEPROM Data Register

Bit	7	6	5	4	3	2	1	0	
0x1D	EEDR7	EEDR6	EEDR5	EEDR4	EEDR3	EEDR2	EEDR1	EEDR0	EEDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:0 – EEDR[7:0]: EEPROM Data**

For the EEPROM write operation the EEDR Register contains the data to be written to the EEPROM in the address given by the EEAR Register. For the EEPROM read operation, the EEDR contains the data read out from the EEPROM at the address given by EEAR.

5.5.4 EECR – EEPROM Control Register

Bit	7	6	5	4	3	2	1	0	
0x1C	–	–	EEPM1	EEPM0	EERIE	EEMPE	EEPE	EERE	EECR
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	X	X	0	0	X	0	

- **Bit 7 – Res: Reserved Bit**

This bit is reserved for future use and will always read as 0 in ATtiny25/45/85. For compatibility with future AVR devices, always write this bit to zero. After reading, mask out this bit.

- **Bit 6 – Res: Reserved Bit**

This bit is reserved in the ATtiny25/45/85 and will always read as zero.

- **Bits 5:4 – EEPM[1:0]: EEPROM Programming Mode Bits**

The EEPROM Programming mode bits setting defines which programming action that will be triggered when writing EEPE. It is possible to program data in one atomic operation (erase the old value and program the new value) or to split the Erase and Write operations in two different operations. The Programming times for the different modes are shown in Table 5-1. While EEPE is set, any write to EEPMn will be ignored. During reset, the EEPMn bits will be reset to 0b00 unless the EEPROM is busy programming.

Table 5-1. EEPROM Mode Bits

EEPM1	EEPM0	Programming Time	Operation
0	0	3.4 ms	Erase and Write in one operation (Atomic Operation)
0	1	1.8 ms	Erase Only
1	0	1.8 ms	Write Only
1	1	–	Reserved for future use

- **Bit 3 – EERIE: EEPROM Ready Interrupt Enable**

Writing EERIE to one enables the EEPROM Ready Interrupt if the I-bit in SREG is set. Writing EERIE to zero disables the interrupt. The EEPROM Ready Interrupt generates a constant interrupt when Non-volatile memory is ready for programming.

- **Bit 2 – EEMPE: EEPROM Master Program Enable**

The EEMPE bit determines whether writing EEPE to one will have effect or not.

When EEMPE is set, setting EEPE within four clock cycles will program the EEPROM at the selected address. If EEMPE is zero, setting EEPE will have no effect. When EEMPE has been written to one by software, hardware clears the bit to zero after four clock cycles.

- **Bit 1 – EEPE: EEPROM Program Enable**

The EEPROM Program Enable Signal EEPE is the programming enable signal to the EEPROM. When EEPE is written, the EEPROM will be programmed according to the EEPMn bits setting. The EEMPE bit must be written to one before a logical one is written to EEPE, otherwise no EEPROM write takes place. When the write access time has elapsed, the EEPE bit is cleared by hardware. When EEPE has been set, the CPU is halted for two cycles before the next instruction is executed.

- **Bit 0 – EERE: EEPROM Read Enable**

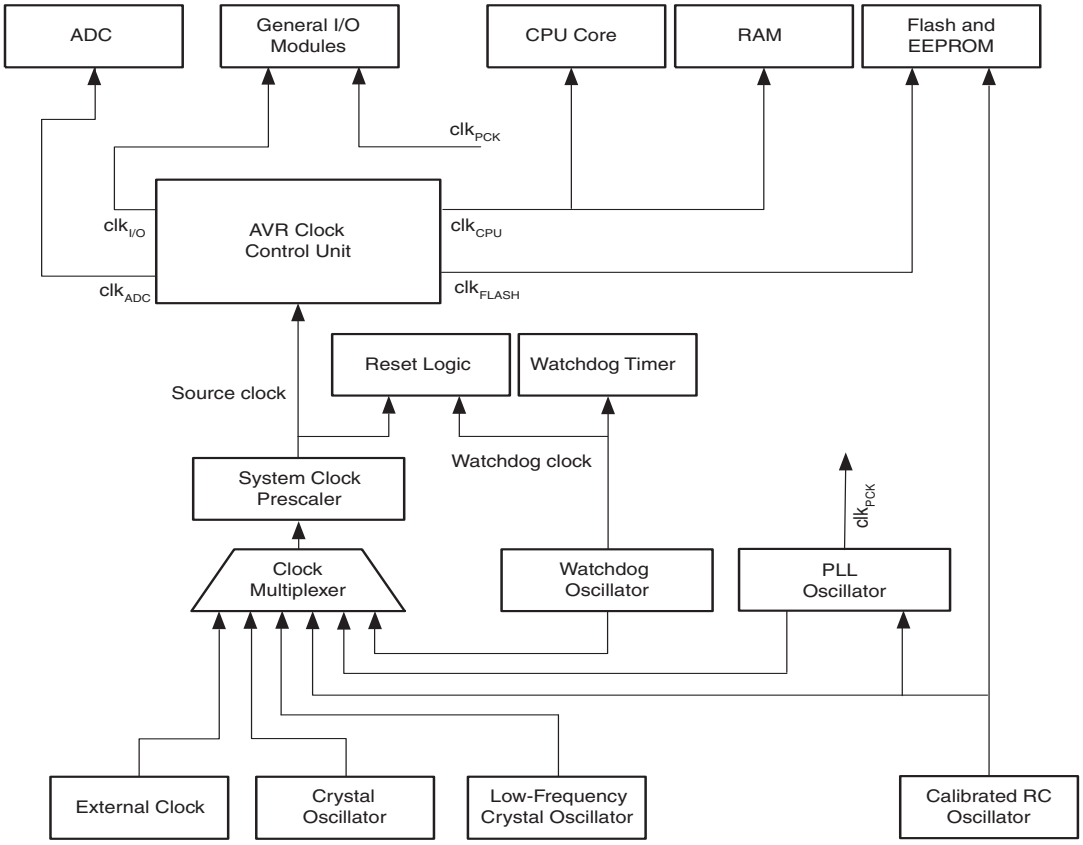
The EEPROM Read Enable Signal – EERE – is the read strobe to the EEPROM. When the correct address is set up in the EEAR Register, the EERE bit must be written to one to trigger the EEPROM read. The EEPROM read access takes one instruction, and the requested data is available immediately. When the EEPROM is read, the CPU is halted for four cycles before the next instruction is executed. The user should poll the EEPE bit before starting the read operation. If a write operation is in progress, it is neither possible to read the EEPROM, nor to change the EEAR Register.

6. System Clock and Clock Options

6.1 Clock Systems and their Distribution

Figure 6-1 presents the principal clock systems in the AVR and their distribution. All of the clocks need not be active at a given time. In order to reduce power consumption, the clocks to modules not being used can be halted by using different sleep modes, as described in “Power Management and Sleep Modes” on page 34. The clock systems are detailed below.

Figure 6-1. Clock Distribution



6.1.1 CPU Clock – clk_{CPU}

The CPU clock is routed to parts of the system concerned with operation of the AVR core. Examples of such modules are the General Purpose Register File, the Status Register and the Data memory holding the Stack Pointer. Halting the CPU clock inhibits the core from performing general operations and calculations.

6.1.2 I/O Clock – $clk_{I/O}$

The I/O clock is used by the majority of the I/O modules, like Timer/Counter. The I/O clock is also used by the External Interrupt module, but note that some external interrupts are detected by asynchronous logic, allowing such interrupts to be detected even if the I/O clock is halted.

6.1.3 Flash Clock – clk_{FLASH}

The Flash clock controls operation of the Flash interface. The Flash clock is usually active simultaneously with the CPU clock.

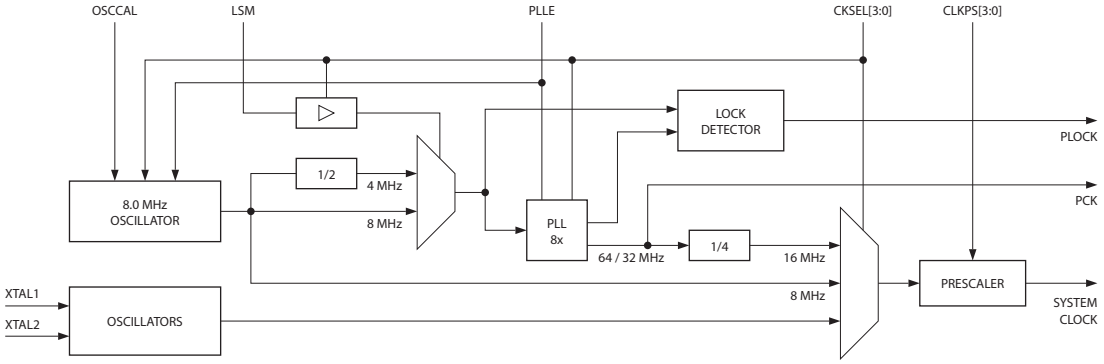
6.1.4 ADC Clock – clk_{ADC}

The ADC is provided with a dedicated clock domain. This allows halting the CPU and I/O clocks in order to reduce noise generated by digital circuitry. This gives more accurate ADC conversion results.

6.1.5 Internal PLL for Fast Peripheral Clock Generation - clk_{PCK}

The internal PLL in ATtiny25/45/85 generates a clock frequency that is 8x multiplied from a source input. By default, the PLL uses the output of the internal, 8.0 MHz RC oscillator as source. Alternatively, if bit LSM of PLLCSR is set the PLL will use the output of the RC oscillator divided by two. Thus the output of the PLL, the fast peripheral clock is 64 MHz. The fast peripheral clock, or a clock prescaled from that, can be selected as the clock source for Timer/Counter1 or as a system clock. See Figure 6-2. The frequency of the fast peripheral clock is divided by two when LSM of PLLCSR is set, resulting in a clock frequency of 32 MHz. Note, that LSM can not be set if PLL_{CLK} is used as system clock.

Figure 6-2. PCK Clocking System.



The PLL is locked on the RC oscillator and adjusting the RC oscillator via OSCCAL register will adjust the fast peripheral clock at the same time. However, even if the RC oscillator is taken to a higher frequency than 8 MHz, the fast peripheral clock frequency saturates at 85 MHz (worst case) and remains oscillating at the maximum frequency. It should be noted that the PLL in this case is not locked any longer with the RC oscillator clock. Therefore, it is recommended not to take the OSCCAL adjustments to a higher frequency than 8 MHz in order to keep the PLL in the correct operating range.

The internal PLL is enabled when:

- The PLLE bit in the register PLLCSR is set.
- The CKSEL fuse is programmed to ‘0001’.
- The CKSEL fuse is programmed to ‘0011’.

The PLLCSR bit PLOCK is set when PLL is locked.

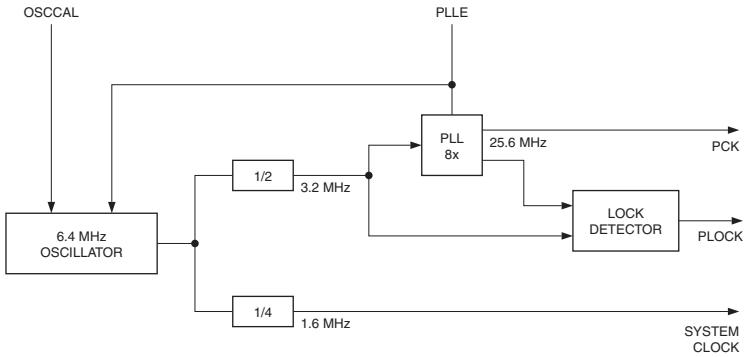
Both internal RC oscillator and PLL are switched off in power down and stand-by sleep modes.

6.1.6 Internal PLL in ATtiny15 Compatibility Mode

Since ATtiny25/45/85 is a migration device for ATtiny15 users there is an ATtiny15 compatibility mode for backward compatibility. The ATtiny15 compatibility mode is selected by programming the CKSEL fuses to ‘0011’.

In the ATtiny15 compatibility mode the frequency of the internal RC oscillator is calibrated down to 6.4 MHz and the multiplication factor of the PLL is set to 4x. See Figure 6-3. With these adjustments the clocking system is ATtiny15-compatible and the resulting fast peripheral clock has a frequency of 25.6 MHz (same as in ATtiny15).

Figure 6-3. PCK Clocking System in ATtiny15 Compatibility Mode.



Note that low speed mode is not implemented in ATtiny15 compatibility mode.

6.2 Clock Sources

The device has the following clock source options, selectable by Flash Fuse bits as shown below. The clock from the selected source is input to the AVR clock generator, and routed to the appropriate modules.

Table 6-1. Device Clocking Options Select

Device Clocking Option	CKSEL[3:0](1)
External Clock (see page 26)	0000
High Frequency PLL Clock (see page 26)	0001
Calibrated Internal Oscillator (see page 27)	0010(2)
Calibrated Internal Oscillator (see page 27)	0011(3)
Internal 128 kHz Oscillator (see page 28)	0100
Low-Frequency Crystal Oscillator (see page 29)	0110
Crystal Oscillator / Ceramic Resonator (see page 29)	1000 – 1111
Reserved	0101, 0111

Note: 1. For all fuses “1” means unprogrammed while “0” means programmed.
2. The device is shipped with this option selected.
3. This will select ATtiny15 Compatibility Mode, where system clock is divided by four, resulting in a 1.6 MHz clock frequency. For more inormation, see “Calibrated Internal Oscillator” on page 27.

The various choices for each clocking option is given in the following sections. When the CPU wakes up from Power-down, the selected clock source is used to time the start-up, ensuring stable Oscillator operation before instruction execution starts. When the CPU starts from reset, there is an additional delay allowing the power to reach a stable level before commencing normal operation. The Watchdog Oscillator is used for timing this real-time part of the start-up time. The number of WDT Oscillator cycles used for each time-out is shown in Table 6-2.

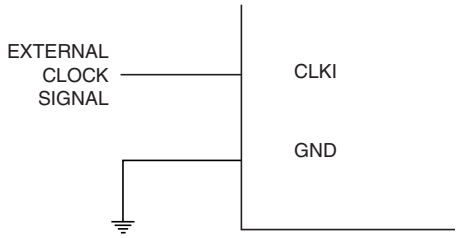
Table 6-2. Number of Watchdog Oscillator Cycles

Typ Time-out	Number of Cycles
4 ms	512
64 ms	8K (8,192)

6.2.1 External Clock

To drive the device from an external clock source, CLKI should be driven as shown in Figure 6-4. To run the device on an external clock, the CKSEL Fuses must be programmed to “00”.

Figure 6-4. External Clock Drive Configuration



When this clock source is selected, start-up times are determined by the SUT Fuses as shown in Table 6-3.

Table 6-3. Start-up Times for the External Clock Selection

SUT[1:0]	Start-up Time from Power-down	Additional Delay from Reset	Recommended Usage
00	6 CK	14CK	BOD enabled
01	6 CK	14CK + 4 ms	Fast rising power
10	6 CK	14CK + 64 ms	Slowly rising power
11	Reserved		

When applying an external clock, it is required to avoid sudden changes in the applied clock frequency to ensure stable operation of the MCU. A variation in frequency of more than 2% from one clock cycle to the next can lead to unpredictable behavior. It is required to ensure that the MCU is kept in Reset during such changes in the clock frequency.

Note that the System Clock Prescaler can be used to implement run-time changes of the internal clock frequency while still ensuring stable operation. Refer to “System Clock Prescaler” on page 31 for details.

6.2.2 High Frequency PLL Clock

There is an internal PLL that provides nominally 64 MHz clock rate locked to the RC Oscillator for the use of the Peripheral Timer/Counter1 and for the system clock source. When selected as a system clock source, by programming the CKSEL fuses to ‘0001’, it is divided by four like shown in Table 6-4.

Table 6-4. High Frequency PLL Clock Operating Modes

CKSEL[3:0]	Nominal Frequency
0001	16 MHz

When this clock source is selected, start-up times are determined by the SUT fuses as shown in Table 6-5.

Table 6-5. Start-up Times for the High Frequency PLL Clock

SUT[1:0]	Start-up Time from Power Down	Additional Delay from Power-On Reset (V_{CC} = 5.0V)	Recommended usage
00	14CK + 1K (1024) CK + 4 ms	4 ms	BOD enabled

Table 6-5. Start-up Times for the High Frequency PLL Clock

SUT[1:0]	Start-up Time from Power Down	Additional Delay from Power-On Reset (V_{CC} = 5.0V)	Recommended usage
01	14CK + 16K (16384) CK + 4 ms	4 ms	Fast rising power
10	14CK + 1K (1024) CK + 64 ms	4 ms	Slowly rising power
11	14CK + 16K (16384) CK + 64 ms	4 ms	Slowly rising power

6.2.3 Calibrated Internal Oscillator

By default, the Internal RC Oscillator provides an approximate 8.0 MHz clock. Though voltage and temperature dependent, this clock can be very accurately calibrated by the user. See “Calibrated Internal RC Oscillator Accuracy” on page 164 and “Internal Oscillator Speed” on page 192 for more details. The device is shipped with the CKDIV8 Fuse programmed. See “System Clock Prescaler” on page 31 for more details.

This clock may be selected as the system clock by programming the CKSEL Fuses as shown in Table 6-6 on page 27. If selected, it will operate with no external components. During reset, hardware loads the pre-programmed calibration value into the OSCCAL Register and thereby automatically calibrates the RC Oscillator. The accuracy of this calibration is shown as Factory calibration in Table 21-2 on page 164.

By changing the OSCCAL register from SW, see “OSCCAL – Oscillator Calibration Register” on page 31, it is possible to get a higher calibration accuracy than by using the factory calibration. The accuracy of this calibration is shown as User calibration in Table 21-2 on page 164.

When this Oscillator is used as the chip clock, the Watchdog Oscillator will still be used for the Watchdog Timer and for the Reset Time-out. For more information on the pre-programmed calibration value, see the section “Calibration Bytes” on page 150.

The internal oscillator can also be set to provide a 6.4 MHz clock by writing CKSEL fuses to “0011”, as shown in Table 6-6 below. This setting is reffered to as ATtiny15 Compatibility Mode and is intended to provide a calibrated clock source at 6.4 MHz, as in ATtiny15. In ATtiny15 Compatibility Mode the PLL uses the internal oscillator running at 6.4 MHz to generate a 25.6 MHz peripheral clock signal for Timer/Counter1 (see “8-bit Timer/Counter1 in ATtiny15 Mode” on page 95). Note that in this mode of operation the 6.4 MHz clock signal is always divided by four, providing a 1.6 MHz system clock.

Table 6-6. Internal Calibrated RC Oscillator Operating Modes

CKSEL[3:0]	Nominal Frequency
0010(1)	8.0 MHz
0011(2)	6.4 MHz

Note: 1. The device is shipped with this option selected.
2. This setting will select ATtiny15 Compatibility Mode, where system clock is divided by four, resulting in a 1.6 MHz clock frequency.

When the calibrated 8 MHz internal oscillator is selected as clock source the start-up times are determined by the SUT Fuses as shown in Table 6-7 below.

Table 6-7. Start-up Times for Internal Calibrated RC Oscillator Clock

SUT[1:0]	Start-up Time from Power-down	Additional Delay from Reset (V_{CC} = 5.0V)	Recommended Usage
00	6 CK	14CK(1)	BOD enabled
01	6 CK	14CK + 4 ms	Fast rising power
10(2)	6 CK	14CK + 64 ms	Slowly rising power
11	Reserved		

Note: 1. If the RSTDISBL fuse is programmed, this start-up time will be increased to 14CK + 4 ms to ensure programming mode can be entered.

2. The device is shipped with this option selected.

In ATtiny15 Compatibility Mode start-up times are determined by SUT fuses as shown in Table 6-8 below.

Table 6-8. Start-up Times for Internal Calibrated RC Oscillator Clock (in ATtiny15 Mode)

SUT[1:0]	Start-up Time from Power-down	Additional Delay from Reset (V_{CC} = 5.0V)	Recommended Usage
00	6 CK	14CK + 64 ms	
01	6 CK	14CK + 64 ms	
10	6 CK	14CK + 4 ms	
11	1 CK	14CK(1)	

Note: 1. If the RSTDISBL fuse is programmed, this start-up time will be increased to 14CK + 4 ms to ensure programming mode can be entered.

In summary, more information on ATtiny15 Compatibility Mode can be found in sections "Port B (PB5:PB0)" on page 2, "Internal PLL in ATtiny15 Compatibility Mode" on page 24, "8-bit Timer/Counter1 in ATtiny15 Mode" on page 95, "Limitations of debugWIRE" on page 140, "Calibration Bytes" on page 150 and in table "Clock Prescaler Select" on page 33.

6.2.4 Internal 128 kHz Oscillator

The 128 kHz internal Oscillator is a low power Oscillator providing a clock of 128 kHz. The frequency is nominal at 3V and 25°C. This clock may be select as the system clock by programming the CKSEL Fuses to "0100".

When this clock source is selected, start-up times are determined by the SUT Fuses as shown in Table 6-9.

Table 6-9. Start-up Times for the 128 kHz Internal Oscillator

SUT[1:0]	Start-up Time from Power-down	Additional Delay from Reset	Recommended Usage
00	6 CK	14CK(1)	BOD enabled
01	6 CK	14CK + 4 ms	Fast rising power
10	6 CK	14CK + 64 ms	Slowly rising power
11	Reserved		

Note: 1. If the RSTDISBL fuse is programmed, this start-up time will be increased to 14CK + 4 ms to ensure programming mode can be entered.

6.2.5 Low-Frequency Crystal Oscillator

To use a 32.768 kHz watch crystal as the clock source for the device, the Low-frequency Crystal Oscillator must be selected by setting CKSEL fuses to '0110'. The crystal should be connected as shown in Figure 6-5. To find suitable load capacitance for a 32.768 kHz crysal, please consult the manufacturer's datasheet.

When this oscillator is selected, start-up times are determined by the SUT fuses as shown in Table 6-10.

Table 6-10. Start-up Times for the Low Frequency Crystal Oscillator Clock Selection

SUT[1:0]	Start-up Time from Power Down	Additional Delay from Reset (V_{CC} = 5.0V)	Recommended usage
00	1K (1024) CK(1)	4 ms	Fast rising power or BOD enabled
01	1K (1024) CK(1)	64 ms	Slowly rising power
10	32K (32768) CK	64 ms	Stable frequency at start-up
11	Reserved		

Note: 1. These options should be used only if frequency stability at start-up is not important.

The Low-frequency Crystal Oscillator provides an internal load capacitance, see Table 6-11 at each TOSC pin.

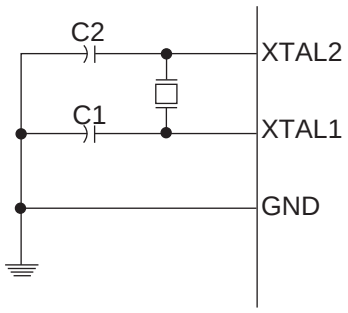
Table 6-11. Capacitance of Low-Frequency Crystal Oscillator

Device	32 kHz Osc. Type	Cap (Xtal1/Tosc1)	Cap (Xtal2/Tosc2)
ATtiny25/45/85	System Osc.	16 pF	6 pF

6.2.6 Crystal Oscillator / Ceramic Resonator

XTAL1 and XTAL2 are input and output, respectively, of an inverting amplifier which can be configured for use as an On-chip Oscillator, as shown in Figure 6-5. Either a quartz crystal or a ceramic resonator may be used.

Figure 6-5. Crystal Oscillator Connections



C1 and C2 should always be equal for both crystals and resonators. The optimal value of the capacitors depends on the crystal or resonator in use, the amount of stray capacitance, and the electromagnetic noise of the environ-

ment. Some initial guidelines for choosing capacitors for use with crystals are given in Table 6-12 below. For ceramic resonators, the capacitor values given by the manufacturer should be used.

Table 6-12. Crystal Oscillator Operating Modes

CKSEL[3:1]	Frequency Range (MHz)	Recommended Range for Capacitors C1 and C2 for Use with Crystals (pF)
100(1)	0.4 - 0.9	–
101	0.9 - 3.0	12 - 22
110	3.0 - 8.0	12 - 22
111	8.0 -	12 - 22

Notes: 1. This option should not be used with crystals, only with ceramic resonators.

The Oscillator can operate in three different modes, each optimized for a specific frequency range. The operating mode is selected by the fuses CKSEL[3:1] as shown in Table 6-12.

The CKSEL0 Fuse together with the SUT[1:0] Fuses select the start-up times as shown in Table 6-13.

Table 6-13. Start-up Times for the Crystal Oscillator Clock Selection

CKSEL0	SUT[1:0]	Start-up Time from Power-down	Additional Delay from Reset	Recommended Usage
0	00	258 CK(1)	14CK + 4 ms	Ceramic resonator, fast rising power
0	01	258 CK(1)	14CK + 64 ms	Ceramic resonator, slowly rising power
0	10	1K (1024) CK(2)	14CK	Ceramic resonator, BOD enabled
0	11	1K (1024)CK(2)	14CK + 4 ms	Ceramic resonator, fast rising power
1	00	1K (1024)CK(2)	14CK + 64 ms	Ceramic resonator, slowly rising power
1	01	16K (16384) CK	14CK	Crystal Oscillator, BOD enabled
1	10	16K (16384) CK	14CK + 4 ms	Crystal Oscillator, fast rising power
1	11	16K (16384) CK	14CK + 64 ms	Crystal Oscillator, slowly rising power

Notes: 1. These options should only be used when not operating close to the maximum frequency of the device, and only if frequency stability at start-up is not important for the application. These options are not suitable for crystals.

2. These options are intended for use with ceramic resonators and will ensure frequency stability at start-up. They can also be used with crystals when not operating close to the maximum frequency of the device, and if frequency stability at start-up is not important for the application.

6.2.7 Default Clock Source

The device is shipped with CKSEL = “0010”, SUT = “10”, and CKDIV8 programmed. The default clock source setting is therefore the Internal RC Oscillator running at 8 MHz with longest start-up time and an initial system clock prescaling of 8, resulting in 1.0 MHz system clock. This default setting ensures that all users can make their desired clock source setting using an In-System or High-voltage Programmer.

6.3 System Clock Prescaler

The ATtiny25/45/85 system clock can be divided by setting the “CLKPR – Clock Prescale Register” on page 32. This feature can be used to decrease power consumption when the requirement for processing power is low. This can be used with all clock source options, and it will affect the clock frequency of the CPU and all synchronous peripherals. $clk_{I/O}$, clk_{ADC}, clk_{CPU}, and clk_{FLASH} are divided by a factor as shown in Table 6-15 on page 33.

6.3.1 Switching Time

When switching between prescaler settings, the System Clock Prescaler ensures that no glitches occur in the clock system and that no intermediate frequency is higher than neither the clock frequency corresponding to the previous setting, nor the clock frequency corresponding to the new setting.

The ripple counter that implements the prescaler runs at the frequency of the undivided clock, which may be faster than the CPU’s clock frequency. Hence, it is not possible to determine the state of the prescaler – even if it were readable, and the exact time it takes to switch from one clock division to another cannot be exactly predicted.

From the time the CLKPS values are written, it takes between T1 + T2 and T1 + 2*T2 before the new clock frequency is active. In this interval, 2 active clock edges are produced. Here, T1 is the previous clock period, and T2 is the period corresponding to the new prescaler setting.

6.4 Clock Output Buffer

The device can output the system clock on the CLKO pin (when not used as XTAL2 pin). To enable the output, the CKOUT Fuse has to be programmed. This mode is suitable when the chip clock is used to drive other circuits on the system. Note that the clock will not be output during reset and that the normal operation of the I/O pin will be overridden when the fuse is programmed. Internal RC Oscillator, WDT Oscillator, PLL, and external clock (CLKI) can be selected when the clock is output on CLKO. Crystal oscillators (XTAL1, XTAL2) can not be used for clock output on CLKO. If the System Clock Prescaler is used, it is the divided system clock that is output.

6.5 Register Description

6.5.1 OSCCAL – Oscillator Calibration Register

Bit	7	6	5	4	3	2	1	0	
0x31	CAL7	CAL6	CAL5	CAL4	CAL3	CAL2	CAL1	CAL0	OSCCAL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	Device Specific Calibration Value								

- **Bits 7:0 – CAL[7:0]: Oscillator Calibration Value**

The Oscillator Calibration Register is used to trim the Calibrated Internal RC Oscillator to remove process variations from the oscillator frequency. A pre-programmed calibration value is automatically written to this register during chip reset, giving the Factory calibrated frequency as specified in Table 21-2 on page 164. The application software can write this register to change the oscillator frequency. The oscillator can be calibrated to frequencies as specified in Table 21-2 on page 164. Calibration outside that range is not guaranteed.

Note that this oscillator is used to time EEPROM and Flash write accesses, and these write times will be affected accordingly. If the EEPROM or Flash are written, do not calibrate to more than 8.8 MHz. Otherwise, the EEPROM or Flash write may fail.

The CAL7 bit determines the range of operation for the oscillator. Setting this bit to 0 gives the lowest frequency range, setting this bit to 1 gives the highest frequency range. The two frequency ranges are overlapping, in other words a setting of OSCCAL = 0x7F gives a higher frequency than OSCCAL = 0x80.

The CAL[6:0] bits are used to tune the frequency within the selected range. A setting of 0x00 gives the lowest frequency in that range, and a setting of 0x7F gives the highest frequency in the range.

To ensure stable operation of the MCU the calibration value should be changed in small. A variation in frequency of more than 2% from one cycle to the next can lead to unpredicatble behavior. Changes in OSCCAL should not exceed 0x20 for each calibration. It is required to ensure that the MCU is kept in Reset during such changes in the clock frequency

Table 6-14. Internal RC Oscillator Frequency Range

OSCCAL Value	Typical Lowest Frequency with Respect to Nominal Frequency	Typical Highest Frequency with Respect to Nominal Frequency
0x00	50%	100%
0x3F	75%	150%
0x7F	100%	200%

6.5.2 CLKPR – Clock Prescale Register

Bit	7	6	5	4	3	2	1	0	
0x26	CLKPCE	–	–	–	CLKPS3	CLKPS2	CLKPS1	CLKPS0	CLKPR
Read/Write	R/W	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	See Bit Description				

- **Bit 7 – CLKPCE: Clock Prescaler Change Enable**

The CLKPCE bit must be written to logic one to enable change of the CLKPS bits. The CLKPCE bit is only updated when the other bits in CLKPR are simultaniosly written to zero. CLKPCE is cleared by hardware four cycles after it is written or when the CLKPS bits are written. Rewriting the CLKPCE bit within this time-out period does neither extend the time-out period, nor clear the CLKPCE bit.

- **Bits 6:4 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bits 3:0 – CLKPS[3:0]: Clock Prescaler Select Bits 3 - 0**

These bits define the division factor between the selected clock source and the internal system clock. These bits can be written run-time to vary the clock frequency to suit the application requirements. As the divider divides the master clock input to the MCU, the speed of all synchronous peripherals is reduced when a division factor is used. The division factors are given in Table 6-15.

To avoid unintentional changes of clock frequency, a special write procedure must be followed to change the CLKPS bits:

1. Write the Clock Prescaler Change Enable (CLKPCE) bit to one and all other bits in CLKPR to zero.
2. Within four cycles, write the desired value to CLKPS while writing a zero to CLKPCE.

Interrupts must be disabled when changing prescaler setting to make sure the write procedure is not interrupted.

The CKDIV8 Fuse determines the initial value of the CLKPS bits. If CKDIV8 is unprogrammed, the CLKPS bits will be reset to “0000”. If CKDIV8 is programmed, CLKPS bits are reset to “0011”, giving a division factor of eight at start up. This feature should be used if the selected clock source has a higher frequency than the maximum frequency of the device at the present operating conditions. Note that any value can be written to the CLKPS bits regardless of the CKDIV8 Fuse setting. The Application software must ensure that a sufficient division factor is

chosen if the selcted clock source has a higher frequency than the maximum frequency of the device at the present operating conditions. The device is shipped with the CKDIV8 Fuse programmed.

Table 6-15. Clock Prescaler Select

CLKPS3	CLKPS2	CLKPS1	CLKPS0	Clock Division Factor
0	0	0	0	1
0	0	0	1	2
0	0	1	0	4
0	0	1	1	8
0	1	0	0	16
0	1	0	1	32
0	1	1	0	64
0	1	1	1	128
1	0	0	0	256
1	0	0	1	Reserved
1	0	1	0	Reserved
1	0	1	1	Reserved
1	1	0	0	Reserved
1	1	0	1	Reserved
1	1	1	0	Reserved
1	1	1	1	Reserved

Note: The prescaler is disabled in ATtiny15 compatibility mode and neither writing to CLKPR, nor programming the CKDIV8 fuse has any effect on the system clock (which will always be 1.6 MHz).

7. Power Management and Sleep Modes

The high performance and industry leading code efficiency makes the AVR microcontrollers an ideal choise for low power applications. In addition, sleep modes enable the application to shut down unused modules in the MCU, thereby saving power. The AVR provides various sleep modes allowing the user to tailor the power consumption to the application's requirements.

7.1 Sleep Modes

Figure 6-1 on page 23 presents the different clock systems and their distribution in ATtiny25/45/85. The figure is helpful in selecting an appropriate sleep mode. Table 7-1 shows the different sleep modes and their wake up sources.

Table 7-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes

	Active Clock Domains					Oscillators	Wake-up Sources					
Sleep Mode	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{PCK}	Main Clock Source Enabled	INT0 and Pin Change	SPM/EEPROM Ready	USI Start Condition	ADC	Other I/O	Watchdog Interrupt
Idle			X	X	X	X	X	X	X	X	X	X
ADC Noise Reduction				X		X	X(1)	X	X	X		X
Power-down							X(1)		X			X

Note: 1. For INT0, only level interrupt.

To enter any of the three sleep modes, the SE bit in MCUCR must be written to logic one and a SLEEP instruction must be executed. The SM[1:0] bits in the MCUCR Register select which sleep mode (Idle, ADC Noise Reduction or Power-down) will be activated by the SLEEP instruction. See Table 7-2 for a summary.

If an enabled interrupt occurs while the MCU is in a sleep mode, the MCU wakes up. The MCU is then halted for four cycles in addition to the start-up time, executes the interrupt routine, and resumes execution from the instruction following SLEEP. The contents of the Register File and SRAM are unaltered when the device wakes up from sleep. If a reset occurs during sleep mode, the MCU wakes up and executes from the Reset Vector.

Note that if a level triggered interrupt is used for wake-up the changed level must be held for some time to wake up the MCU (and for the MCU to enter the interrupt service routine). See "External Interrupts" on page 49 for details.

7.1.1 Idle Mode

When the SM[1:0] bits are written to 00, the SLEEP instruction makes the MCU enter Idle mode, stopping the CPU but allowing Analog Comparator, ADC, USI, Timer/Counter, Watchdog, and the interrupt system to continue operating. This sleep mode basically halts clk_{CPU} and clk_{FLASH}, while allowing the other clocks to run.

Idle mode enables the MCU to wake up from external triggered interrupts as well as internal ones like the Timer Overflow. If wake-up from the Analog Comparator interrupt is not required, the Analog Comparator can be powered down by setting the ACD bit in "ACSR – Analog Comparator Control and Status Register" on page 120. This will reduce power consumption in Idle mode. If the ADC is enabled, a conversion starts automatically when this mode is entered.

7.1.2 ADC Noise Reduction Mode

When the SM[1:0] bits are written to 01, the SLEEP instruction makes the MCU enter ADC Noise Reduction mode, stopping the CPU but allowing the ADC, the external interrupts, and the Watchdog to continue operating (if enabled). This sleep mode halts $clk_{I/O}$, clk_{CPU}, and clk_{FLASH}, while allowing the other clocks to run.

This improves the noise environment for the ADC, enabling higher resolution measurements. If the ADC is enabled, a conversion starts automatically when this mode is entered. Apart form the ADC Conversion Complete interrupt, only an External Reset, a Watchdog Reset, a Brown-out Reset, an SPM/EEPROM ready interrupt, an external level interrupt on INT0 or a pin change interrupt can wake up the MCU from ADC Noise Reduction mode.

7.1.3 Power-down Mode

When the SM[1:0] bits are written to 10, the SLEEP instruction makes the MCU enter Power-down mode. In this mode, the Oscillator is stopped, while the external interrupts, the USI start condition detection and the Watchdog continue operating (if enabled). Only an External Reset, a Watchdog Reset, a Brown-out Reset, USI start condition interupt, an external level interrupt on INT0 or a pin change interrupt can wake up the MCU. This sleep mode halts all generated clocks, allowing operation of asynchronous modules only.

7.2 Software BOD Disable

When the Brown-out Detector (BOD) is enabled by BODLEVEL fuses (see Table 20-4 on page 148), the BOD is actively monitoring the supply voltage during a sleep period. In some devices it is possible to save power by disabling the BOD by software in Power-Down sleep mode. The sleep mode power consumption will then be at the same level as when BOD is globally disabled by fuses.

If BOD is disabled by software, the BOD function is turned off immediately after entering the sleep mode. Upon wake-up from sleep, BOD is automatically enabled again. This ensures safe operation in case the V_{CC} level has dropped during the sleep period.

When the BOD has been disabled, the wake-up time from sleep mode will be the same as that for wakeing up from RESET. The user must manually configure the wake up times such that the bandgap reference has time to start and the BOD is working correctly before the MCU continues executing code. See SUT[1:0] and CKSEL[3:0] fuse bits in table “Fuse Low Byte” on page 149

BOD disable is controlled by the BODS (BOD Sleep) bit of MCU Control Register, see “MCUCR – MCU Control Register” on page 37. Writing this bit to one turns off BOD in Power-Down, while writing a zero keeps the BOD active. The default setting is zero, i.e. BOD active.

Writing to the BODS bit is controlled by a timed sequence and an enable bit, see “MCUCR – MCU Control Register” on page 37.

7.2.1 Limitations

BOD disable functionality has been implemented in the following devices, only:

- ATtiny25, revision E, and newer
- ATtiny45, revision D, and newer
- ATtiny85, revision C, and newer

Revisions are marked on the device package and can be located as follows:

- Bottom side of packages 8P3 and 8S2
- Top side of package 20M1

7.3 Power Reduction Register

The Power Reduction Register (PRR), see “PRR – Power Reduction Register” on page 38, provides a method to reduce power consumption by stopping the clock to individual peripherals. The current state of the peripheral is frozen and the I/O registers can not be read or written. Resources used by the peripheral when stopping the clock will remain occupied, hence the peripheral should in most cases be disabled before stopping the clock. Waking up a module, which is done by clearing the bit in PRR, puts the module in the same state as before shutdown.

Module shutdown can be used in Idle mode and Active mode to significantly reduce the overall power consumption. In all other sleep modes, the clock is already stopped. See “Supply Current of I/O modules” on page 177 for examples.

7.4 Minimizing Power Consumption

There are several issues to consider when trying to minimize the power consumption in an AVR controlled system. In general, sleep modes should be used as much as possible, and the sleep mode should be selected so that as few as possible of the device’s functions are operating. All functions not needed should be disabled. In particular, the following modules may need special consideration when trying to achieve the lowest possible power consumption.

7.4.1 Analog to Digital Converter

If enabled, the ADC will be enabled in all sleep modes. To save power, the ADC should be disabled before entering any sleep mode. When the ADC is turned off and on again, the next conversion will be an extended conversion. Refer to “Analog to Digital Converter” on page 122 for details on ADC operation.

7.4.2 Analog Comparator

When entering Idle mode, the Analog Comparator should be disabled if not used. When entering ADC Noise Reduction mode, the Analog Comparator should be disabled. In the other sleep modes, the Analog Comparator is automatically disabled. However, if the Analog Comparator is set up to use the Internal Voltage Reference as input, the Analog Comparator should be disabled in all sleep modes. Otherwise, the Internal Voltage Reference will be enabled, independent of sleep mode. Refer to “Analog Comparator” on page 119 for details on how to configure the Analog Comparator.

7.4.3 Brown-out Detector

If the Brown-out Detector is not needed in the application, this module should be turned off. If the Brown-out Detector is enabled by the BODLEVEL Fuses, it will be enabled in all sleep modes, and hence, always consume power. In the deeper sleep modes, this will contribute significantly to the total current consumption. See “Brown-out Detection” on page 41 and “Software BOD Disable” on page 35 for details on how to configure the Brown-out Detector.

7.4.4 Internal Voltage Reference

The Internal Voltage Reference will be enabled when needed by the Brown-out Detection, the Analog Comparator or the ADC. If these modules are disabled as described in the sections above, the internal voltage reference will be disabled and it will not be consuming power. When turned on again, the user must allow the reference to start up before the output is used. If the reference is kept on in sleep mode, the output can be used immediately. Refer to "Internal Voltage Reference" on page 42 for details on the start-up time.

7.4.5 Watchdog Timer

If the Watchdog Timer is not needed in the application, this module should be turned off. If the Watchdog Timer is enabled, it will be enabled in all sleep modes, and hence, always consume power. In the deeper sleep modes, this will contribute significantly to the total current consumption. Refer to "Watchdog Timer" on page 42 for details on how to configure the Watchdog Timer.

7.4.6 Port Pins

When entering a sleep mode, all port pins should be configured to use minimum power. The most important thing is then to ensure that no pins drive resistive loads. In sleep modes where both the I/O clock ($clk_{I/O}$) and the ADC clock (clk_{ADC}) are stopped, the input buffers of the device will be disabled. This ensures that no power is consumed by the input logic when not needed. In some cases, the input logic is needed for detecting wake-up conditions, and it will then be enabled. Refer to the section "Digital Input Enable and Sleep Modes" on page 57 for details on which pins are enabled. If the input buffer is enabled and the input signal is left floating or has an analog signal level close to $V_{CC}/2$, the input buffer will use excessive power.

For analog input pins, the digital input buffer should be disabled at all times. An analog signal level close to $V_{CC}/2$ on an input pin can cause significant current even in active mode. Digital input buffers can be disabled by writing to the Digital Input Disable Register (DIDR0). Refer to "DIDR0 – Digital Input Disable Register 0" on page 121 for details.

7.5 Register Description

7.5.1 MCUCR – MCU Control Register

The MCU Control Register contains control bits for power management.

Bit	7	6	5	4	3	2	1	0	
0x35	BODS	PUD	SE	SM1	SM0	BODSE	ISC01	ISC00	MCUCR
Read/Write	R	R/W	R/W	R/W	R/W	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – BODS: BOD Sleep**

BOD disable functionality is available in some devices, only. See "Limitations" on page 36.

In order to disable BOD during sleep (see Table 7-1 on page 34) the BODS bit must be written to logic one. This is controlled by a timed sequence and the enable bit, BODSE in MCUCR. First, both BODS and BODSE must be set to one. Second, within four clock cycles, BODS must be set to one and BODSE must be set to zero. The BODS bit is active three clock cycles after it is set. A sleep instruction must be executed while BODS is active in order to turn off the BOD for the actual sleep mode. The BODS bit is automatically cleared after three clock cycles.

In devices where Sleeping BOD has not been implemented this bit is unused and will always read zero.

- **Bit 5 – SE: Sleep Enable**

The SE bit must be written to logic one to make the MCU enter the sleep mode when the SLEEP instruction is executed. To avoid the MCU entering the sleep mode unless it is the programmer's purpose, it is recommended to

write the Sleep Enable (SE) bit to one just before the execution of the SLEEP instruction and to clear it immediately after waking up.

- **Bits 4:3 – SM[1:0]: Sleep Mode Select Bits 1 and 0**

These bits select between the three available sleep modes as shown in Table 7-2.

Table 7-2. Sleep Mode Select

SM1	SM0	Sleep Mode
0	0	Idle
0	1	ADC Noise Reduction
1	0	Power-down
1	1	Reserved

- **Bit 2 – BODSE: BOD Sleep Enable**

BOD disable functionality is available in some devices, only. See “Limitations” on page 36.

The BODSE bit enables setting of BODS control bit, as explained on BODS bit description. BOD disable is controlled by a timed sequence.

This bit is unused in devices where software BOD disable has not been implemented and will read as zero in those devices.

7.5.2 PRR – Power Reduction Register

The Power Reduction Register provides a method to reduce power consumption by allowing peripheral clock signals to be disabled.

Bit	7	6	5	4	3	2	1	0	
0x20	–	–	–	–	PRTIM1	PRTIM0	PRUSI	PRADC	PRR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:4 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bit 3 – PRTIM1: Power Reduction Timer/Counter1**

Writing a logic one to this bit shuts down the Timer/Counter1 module. When the Timer/Counter1 is enabled, operation will continue like before the shutdown.

- **Bit 2 – PRTIM0: Power Reduction Timer/Counter0**

Writing a logic one to this bit shuts down the Timer/Counter0 module. When the Timer/Counter0 is enabled, operation will continue like before the shutdown.

- **Bit 1 – PRUSI: Power Reduction USI**

Writing a logic one to this bit shuts down the USI by stopping the clock to the module. When waking up the USI again, the USI should be re initialized to ensure proper operation.

- **Bit 0 – PRADC: Power Reduction ADC**

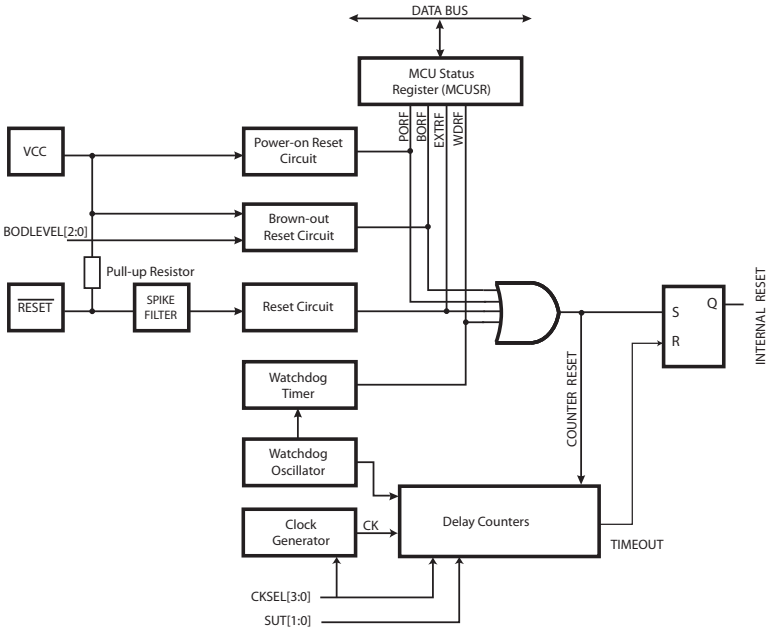
Writing a logic one to this bit shuts down the ADC. The ADC must be disabled before shut down. Note that the ADC clock is also used by some parts of the analog comparator, which means that the analogue comparator can not be used when this bit is high.

8. System Control and Reset

8.1 Resetting the AVR

During reset, all I/O Registers are set to their initial values, and the program starts execution from the Reset Vector. The instruction placed at the Reset Vector must be a RJMP – Relative Jump – instruction to the reset handling routine. If the program never enables an interrupt source, the Interrupt Vectors are not used, and regular program code can be placed at these locations. The circuit diagram in Figure 8-1 shows the reset logic. Electrical parameters of the reset circuitry are given in “System and Reset Characteristics” on page 165.

Figure 8-1. Reset Logic



The I/O ports of the AVR are immediately reset to their initial state when a reset source goes active. This does not require any clock source to be running.

After all reset sources have gone inactive, a delay counter is invoked, stretching the internal reset. This allows the power to reach a stable level before normal operation starts. The time-out period of the delay counter is defined by the user through the SUT and CKSEL Fuses. The different selections for the delay period are presented in “Clock Sources” on page 25.

8.2 Reset Sources

The ATtiny25/45/85 has four sources of reset:

- Power-on Reset. The MCU is reset when the supply voltage is below the Power-on Reset threshold (V_{POT}).
- External Reset. The MCU is reset when a low level is present on the $\overline{\text{RESET}}$ pin for longer than the minimum pulse length.
- Watchdog Reset. The MCU is reset when the Watchdog Timer period expires and the Watchdog is enabled.
- Brown-out Reset. The MCU is reset when the supply voltage V_{CC} is below the Brown-out Reset threshold (V_{BOT}) and the Brown-out Detector is enabled.

8.2.1 Power-on Reset

A Power-on Reset (POR) pulse is generated by an On-chip detection circuit. The detection level is defined in “System and Reset Characteristics” on page 165. The POR is activated whenever V_{CC} is below the detection level. The POR circuit can be used to trigger the Start-up Reset, as well as to detect a failure in supply voltage.

A Power-on Reset (POR) circuit ensures that the device is reset from Power-on. Reaching the Power-on Reset threshold voltage invokes the delay counter, which determines how long the device is kept in RESET after V_{CC} rise. The RESET signal is activated again, without any delay, when V_{CC} decreases below the detection level.

Figure 8-2. MCU Start-up, $\overline{\text{RESET}}$ Tied to V_{CC}

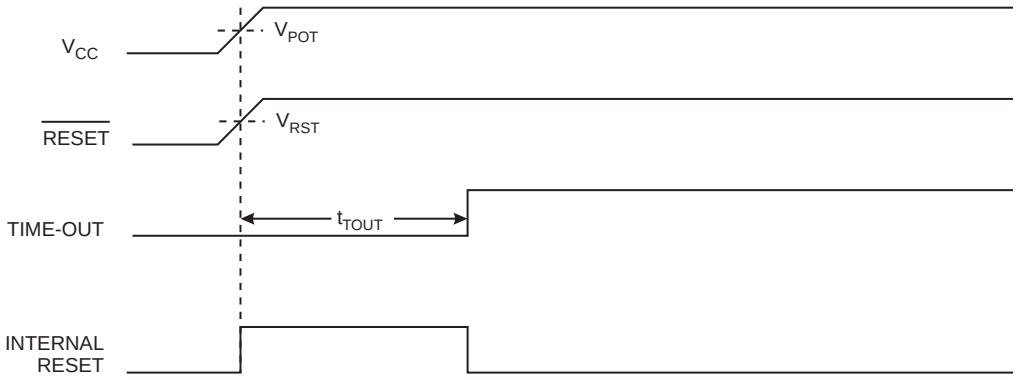
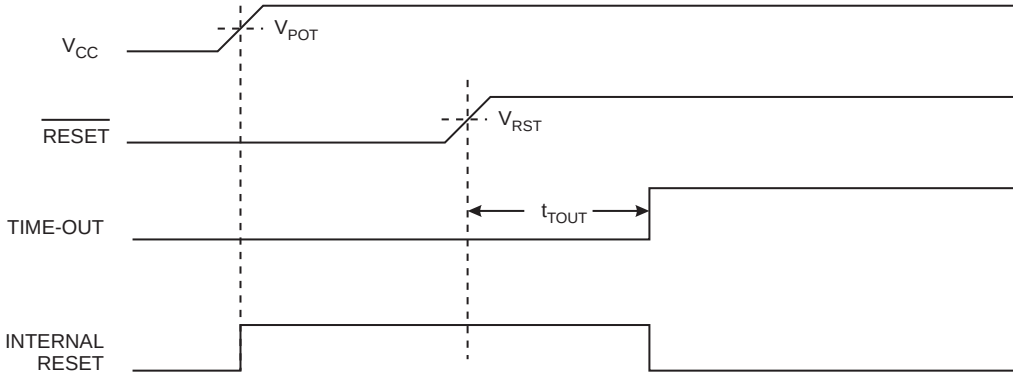


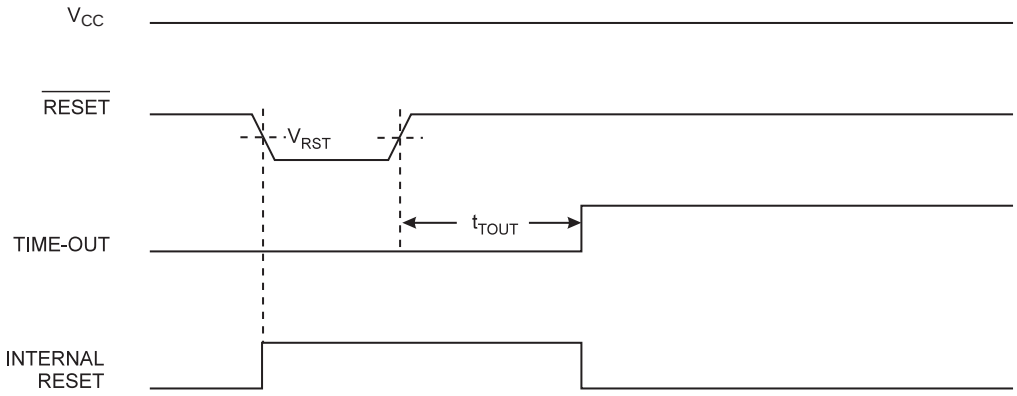
Figure 8-3. MCU Start-up, $\overline{\text{RESET}}$ Extended Externally



8.2.2 External Reset

An External Reset is generated by a low level on the $\overline{\text{RESET}}$ pin if enabled. Reset pulses longer than the minimum pulse width (see “System and Reset Characteristics” on page 165) will generate a reset, even if the clock is not running. Shorter pulses are not guaranteed to generate a reset. When the applied signal reaches the Reset Threshold Voltage – V_{RST} – on its positive edge, the delay counter starts the MCU after the Time-out period – t_{TOUT} – has expired.

Figure 8-4. External Reset During Operation



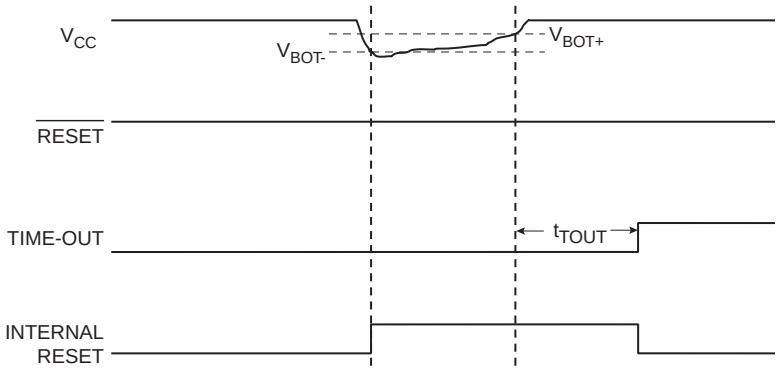
8.2.3 Brown-out Detection

ATtiny25/45/85 has an On-chip Brown-out Detection (BOD) circuit for monitoring the V_{CC} level during operation by comparing it to a fixed trigger level. The trigger level for the BOD can be selected by the BODLEVEL Fuses. The trigger level has a hysteresis to ensure spike free Brown-out Detection. The hysteresis on the detection level should be interpreted as $V_{BOT+} = V_{BOT} + V_{HYST}/2$ and $V_{BOT-} = V_{BOT} - V_{HYST}/2$.

When the BOD is enabled, and V_{CC} decreases to a value below the trigger level (V_{BOT-} in Figure 8-5), the Brown-out Reset is immediately activated. When V_{CC} increases above the trigger level (V_{BOT+} in Figure 8-5), the delay counter starts the MCU after the Time-out period t_{TOUT} has expired.

The BOD circuit will only detect a drop in V_{CC} if the voltage stays below the trigger level for longer than t_{BOD} given in “System and Reset Characteristics” on page 165.

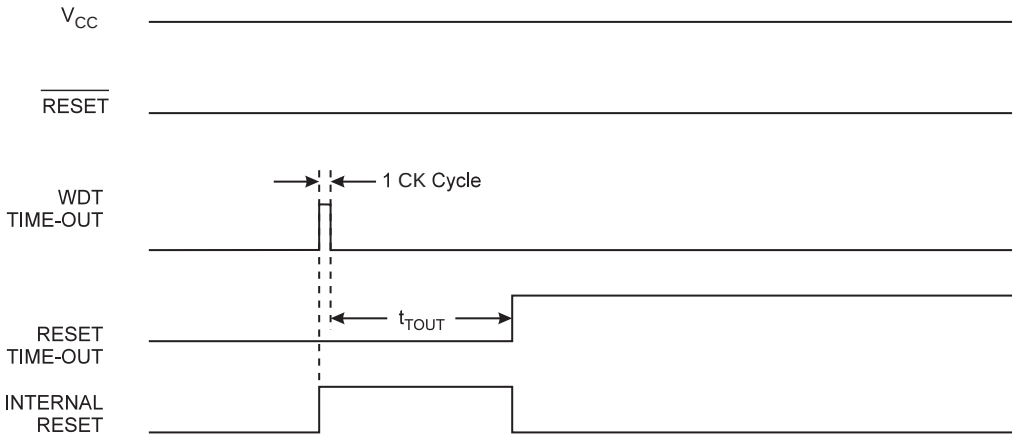
Figure 8-5. Brown-out Reset During Operation



8.2.4 Watchdog Reset

When the Watchdog times out, it will generate a short reset pulse of one CK cycle duration. On the falling edge of this pulse, the delay timer starts counting the Time-out period t_{TOUT}. Refer to “Watchdog Timer” on page 42 for details on operation of the Watchdog Timer.

Figure 8-6. Watchdog Reset During Operation



8.3 Internal Voltage Reference

ATtiny25/45/85 features an internal bandgap reference. This reference is used for Brown-out Detection, and it can be used as an input to the Analog Comparator or the ADC.

8.3.1 Voltage Reference Enable Signals and Start-up Time

The voltage reference has a start-up time that may influence the way it should be used. The start-up time is given in "System and Reset Characteristics" on page 165. To save power, the reference is not always turned on. The reference is on during the following situations:

1. When the BOD is enabled (by programming the BODLEVEL[2:0] Fuse Bits).
2. When the bandgap reference is connected to the Analog Comparator (by setting the ACBG bit in ACSR).
3. When the ADC is enabled.

Thus, when the BOD is not enabled, after setting the ACBG bit or enabling the ADC, the user must always allow the reference to start up before the output from the Analog Comparator or ADC is used. To reduce power consumption in Power-down mode, the user can avoid the three conditions above to ensure that the reference is turned off before entering Power-down mode.

8.4 Watchdog Timer

The Watchdog Timer is clocked from an On-chip Oscillator which runs at 128 kHz. By controlling the Watchdog Timer prescaler, the Watchdog Reset interval can be adjusted as shown in Table 8-3 on page 46. The WDR – Watchdog Reset – instruction resets the Watchdog Timer. The Watchdog Timer is also reset when it is disabled and when a Chip Reset occurs. Ten different clock cycle periods can be selected to determine the reset period. If the reset period expires without another Watchdog Reset, the ATtiny25/45/85 resets and executes from the Reset Vector. For timing details on the Watchdog Reset, refer to Table 8-3 on page 46.

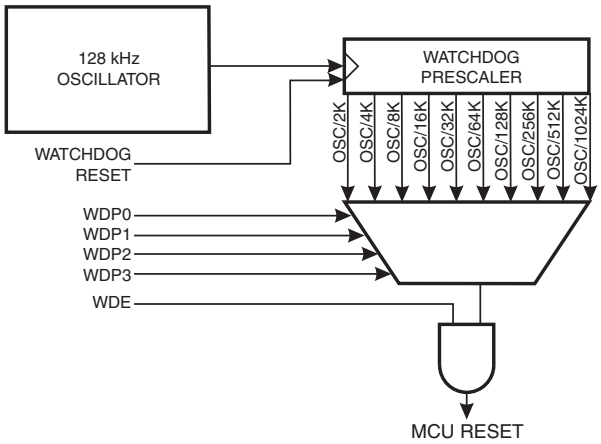
The Wathdog Timer can also be configured to generate an interrupt instead of a reset. This can be very helpful when using the Watchdog to wake-up from Power-down.

To prevent unintentional disabling of the Watchdog or unintentional change of time-out period, two different safety levels are selected by the fuse WDTON as shown in Table 8-1 Refer to “Timed Sequences for Changing the Configuration of the Watchdog Timer” on page 43 for details.

Table 8-1. WDT Configuration as a Function of the Fuse Settings of WDTON

WDTON	Safety Level	WDT Initial State	How to Disable the WDT	How to Change Time-out
Unprogrammed	1	Disabled	Timed sequence	No limitations
Programmed	2	Enabled	Always enabled	Timed sequence

Figure 8-7. Watchdog Timer



8.4.1 Timed Sequences for Changing the Configuration of the Watchdog Timer

The sequence for changing configuration differs slightly between the two safety levels. Separate procedures are described for each level.

8.4.1.1 Safety Level 1

In this mode, the Watchdog Timer is initially disabled, but can be enabled by writing the WDE bit to one without any restriction. A timed sequence is needed when disabling an enabled Watchdog Timer. To disable an enabled Watchdog Timer, the following procedure must be followed:

1. In the same operation, write a logic one to WDCE and WDE. A logic one must be written to WDE regardless of the previous value of the WDE bit.
2. Within the next four clock cycles, in the same operation, write the WDE and WDP bits as desired, but with the WDCE bit cleared.

8.4.1.2 Safety Level 2

In this mode, the Watchdog Timer is always enabled, and the WDE bit will always read as one. A timed sequence is needed when changing the Watchdog Time-out period. To change the Watchdog Time-out, the following procedure must be followed:

1. In the same operation, write a logical one to WDCE and WDE. Even though the WDE always is set, the WDE must be written to one to start the timed sequence.
2. Within the next four clock cycles, in the same operation, write the WDP bits as desired, but with the WDCE bit cleared. The value written to the WDE bit is irrelevant.

8.4.2 Code Example

The following code example shows one assembly and one C function for turning off the WDT. The example assumes that interrupts are controlled (e.g., by disabling interrupts globally) so that no interrupts will occur during execution of these functions.

Assembly Code Example[1]

```
WDT_off:
  wdr
  ; Clear WDRF in MCUSR
  ldi  r16, (0<<WDRF)
  out  MCUSR, r16
  ; Write logical one to WDCE and WDE
  ; Keep old prescaler setting to prevent unintentional Watchdog Reset
  in   r16, WDTCR
  ori  r16, (1<<WDCE)|(1<<WDE)
  out  WDTCR, r16
  ; Turn off WDT
  ldi  r16, (0<<WDE)
  out  WDTCR, r16
  ret
```

C Code Example[1]

```
void WDT_off(void)
{
  _WDR();
  /* Clear WDRF in MCUSR */
  MCUSR = 0x00
  /* Write logical one to WDCE and WDE */
  WDTCR |= (1<<WDCE) | (1<<WDE);
  /* Turn off WDT */
  WDTCR = 0x00;
}
```

Note: 1. See “Code Examples” on page 6.

8.5 Register Description

8.5.1 MCUSR – MCU Status Register

The MCU Status Register provides information on which reset source caused an MCU Reset.

Bit	7	6	5	4	3	2	1	0	
0x34	–	–	–	–	WDRF	BORF	EXTRF	PORF	MCUSR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	See Bit Description				

- **Bits 7:4 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bit 3 – WDRF: Watchdog Reset Flag**

This bit is set if a Watchdog Reset occurs. The bit is reset by a Power-on Reset, or by writing a logic zero to the flag.

- **Bit 2 – BORF: Brown-out Reset Flag**

This bit is set if a Brown-out Reset occurs. The bit is reset by a Power-on Reset, or by writing a logic zero to the flag.

- **Bit 1 – EXTRF: External Reset Flag**

This bit is set if an External Reset occurs. The bit is reset by a Power-on Reset, or by writing a logic zero to the flag.

- **Bit 0 – PORF: Power-on Reset Flag**

This bit is set if a Power-on Reset occurs. The bit is reset only by writing a logic zero to the flag.

To make use of the Reset Flags to identify a reset condition, the user should read and then reset the MCUSR as early as possible in the program. If the register is cleared before another reset occurs, the source of the reset can be found by examining the Reset Flags.

8.5.2 WDTCR – Watchdog Timer Control Register

Bit	7	6	5	4	3	2	1	0	
0x21	**WDIF**	**WDIE**	**WDP3**	**WDCE**	**WDE**	**WDP2**	**WDP1**	**WDP0**	**WDTCR**
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	X	0	0	0	

- **Bit 7 – WDIF: Watchdog Timeout Interrupt Flag**

This bit is set when a time-out occurs in the Watchdog Timer and the Watchdog Timer is configured for interrupt. WDIF is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, WDIF is cleared by writing a logic one to the flag. When the I-bit in SREG and WDIE are set, the Watchdog Time-out Interrupt is executed.

- **Bit 6 – WDIE: Watchdog Timeout Interrupt Enable**

When this bit is written to one, WDE is cleared, and the I-bit in the Status Register is set, the Watchdog Time-out Interrupt is enabled. In this mode the corresponding interrupt is executed instead of a reset if a timeout in the Watchdog Timer occurs.

If WDE is set, WDIE is automatically cleared by hardware when a time-out occurs. This is useful for keeping the Watchdog Reset security while using the interrupt. After the WDIE bit is cleared, the next time-out will generate a reset. To avoid the Watchdog Reset, WDIE must be set after each interrupt.

Table 8-2. Watchdog Timer Configuration

WDE	WDIE	Watchdog Timer State	Action on Time-out
0	0	Stopped	None
0	1	Running	Interrupt
1	0	Running	Reset
1	1	Running	Interrupt

- **Bit 4 – WDCE: Watchdog Change Enable**

This bit must be set when the WDE bit is written to logic zero. Otherwise, the Watchdog will not be disabled. Once written to one, hardware will clear this bit after four clock cycles. Refer to the description of the WDE bit for a Watchdog disable procedure. This bit must also be set when changing the prescaler bits. See “Timed Sequences for Changing the Configuration of the Watchdog Timer” on page 43.

- **Bit 3 – WDE: Watchdog Enable**

When the WDE is written to logic one, the Watchdog Timer is enabled, and if the WDE is written to logic zero, the Watchdog Timer function is disabled. WDE can only be cleared if the WDCE bit has logic level one. To disable an enabled Watchdog Timer, the following procedure must be followed:

1. In the same operation, write a logic one to WDCE and WDE. A logic one must be written to WDE even though it is set to one before the disable operation starts.
2. Within the next four clock cycles, write a logic 0 to WDE. This disables the Watchdog.

In safety level 2, it is not possible to disable the Watchdog Timer, even with the algorithm described above. See “Timed Sequences for Changing the Configuration of the Watchdog Timer” on page 43.

In safety level 1, WDE is overridden by WDRF in MCUSR. See “MCUSR – MCU Status Register” on page 44 for description of WDRF. This means that WDE is always set when WDRF is set. To clear WDE, WDRF must be cleared before disabling the Watchdog with the procedure described above. This feature ensures multiple resets during conditions causing failure, and a safe start-up after the failure.

Note: If the watchdog timer is not going to be used in the application, it is important to go through a watchdog disable procedure in the initialization of the device. If the Watchdog is accidentally enabled, for example by a runaway pointer or brown-out condition, the device will be reset, which in turn will lead to a new watchdog reset. To avoid this situation, the application software should always clear the WDRF flag and the WDE control bit in the initialization routine.

- **Bits 5, 2:0 – WDP[3:0]: Watchdog Timer Prescaler 3, 2, 1, and 0**

The WDP[3:0] bits determine the Watchdog Timer prescaling when the Watchdog Timer is enabled. The different prescaling values and their corresponding Timeout Periods are shown in Table 8-3.

Table 8-3. Watchdog Timer Prescale Select

WDP3	WDP2	WDP1	WDP0	Number of WDT Oscillator Cycles	Typical Time-out at V_{CC} = 5.0V
0	0	0	0	2K (2048) cycles	16 ms
0	0	0	1	4K (4096) cycles	32 ms
0	0	1	0	8K (8192) cycles	64 ms
0	0	1	1	16K (16384) cycles	0.125 s
0	1	0	0	32K (32764) cycles	0.25 s
0	1	0	1	64K (65536) cycles	0.5 s
0	1	1	0	128K (131072) cycles	1.0 s
0	1	1	1	256K (262144) cycles	2.0 s
1	0	0	0	512K (524288) cycles	4.0 s
1	0	0	1	1024K (1048576) cycles	8.0 s

Table 8-3. Watchdog Timer Prescale Select (Continued)

WDP3	WDP2	WDP1	WDP0	Number of WDT Oscillator Cycles	Typical Time-out at V_{CC} = 5.0V
1	0	1	0	Reserved[1]	
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

Note: 1. If selected, one of the valid settings below 0b1010 will be used.

9. Interrupts

This section describes the specifics of the interrupt handling as performed in ATtiny25/45/85. For a general explanation of the AVR interrupt handling, refer to “Reset and Interrupt Handling” on page 12.

9.1 Interrupt Vectors in ATtiny25/45/85

The interrupt vectors of ATtiny25/45/85 are described in Table 9-1below.

Table 9-1. Reset and Interrupt Vectors

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset
2	0x0001	INT0	External Interrupt Request 0
3	0x0002	PCINT0	Pin Change Interrupt Request 0
4	0x0003	TIMER1_COMPA	Timer/Counter1 Compare Match A
5	0x0004	TIMER1_OVF	Timer/Counter1 Overflow
6	0x0005	TIMER0_OVF	Timer/Counter0 Overflow
7	0x0006	EE_RDY	EEPROM Ready
8	0x0007	ANA_COMP	Analog Comparator
9	0x0008	ADC	ADC Conversion Complete
10	0x0009	TIMER1_COMPB	Timer/Counter1 Compare Match B
11	0x000A	TIMER0_COMPA	Timer/Counter0 Compare Match A
12	0x000B	TIMER0_COMPB	Timer/Counter0 Compare Match B
13	0x000C	WDT	Watchdog Time-out
14	0x000D	USI_START	USI START
15	0x000E	USI_OVF	USI Overflow

If the program never enables an interrupt source, the Interrupt Vectors are not used, and regular program code can be placed at these locations.

A typical and general setup for interrupt vector addresses in ATtiny25/45/85 is shown in the program example below.

Assembly Code Example

```
    .org 0x0000                         ;Set address of next statement

      rjmp RESET                        ; Address 0x0000
      rjmp INT0_ISR                     ; Address 0x0001
      rjmp PCINT0_ISR                   ; Address 0x0002
      rjmp TIM1_COMPA_ISR               ; Address 0x0003
      rjmp TIM1_OVF_ISR                 ; Address 0x0004
      rjmp TIM0_OVF_ISR                 ; Address 0x0005
      rjmp EE_RDY_ISR                   ; Address 0x0006
      rjmp ANA_COMP_ISR                 ; Address 0x0007
      rjmp ADC_ISR                      ; Address 0x0008
      rjmp TIM1_COMPB_ISR               ; Address 0x0009
      rjmp TIM0_COMPA_ISR               ; Address 0x000A
      rjmp TIM0_COMPB_ISR               ; Address 0x000B
      rjmp WDT_ISR                      ; Address 0x000C
      rjmp USI_START_ISR                ; Address 0x000D
      rjmp USI_OVF_ISR                  ; Address 0x000E

    RESET:                              ; Main program start
      <instr>                           ; Address 0x000F
      ...
```

Note: See “Code Examples” on page 6.

9.2 External Interrupts

The External Interrupts are triggered by the INT0 pin or any of the PCINT[5:0] pins. Observe that, if enabled, the interrupts will trigger even if the INT0 or PCINT[5:0] pins are configured as outputs. This feature provides a way of generating a software interrupt. Pin change interrupts PCI will trigger if any enabled PCINT[5:0] pin toggles. The PCMSK Register control which pins contribute to the pin change interrupts. Pin change interrupts on PCINT[5:0] are detected asynchronously. This implies that these interrupts can be used for waking the part also from sleep modes other than Idle mode.

The INT0 interrupts can be triggered by a falling or rising edge or a low level. This is set up as indicated in the specification for the MCU Control Register – MCUCR. When the INT0 interrupt is enabled and is configured as level triggered, the interrupt will trigger as long as the pin is held low. Note that recognition of falling or rising edge interrupts on INT0 requires the presence of an I/O clock, described in “Clock Systems and their Distribution” on page 23.

9.2.1 Low Level Interrupt

A low level interrupt on INT0 is detected asynchronously. This implies that this interrupt can be used for waking the part also from sleep modes other than Idle mode. The I/O clock is halted in all sleep modes except Idle mode.

Note that if a level triggered interrupt is used for wake-up from Power-down, the required level must be held long enough for the MCU to complete the wake-up to trigger the level interrupt. If the level disappears before the end of

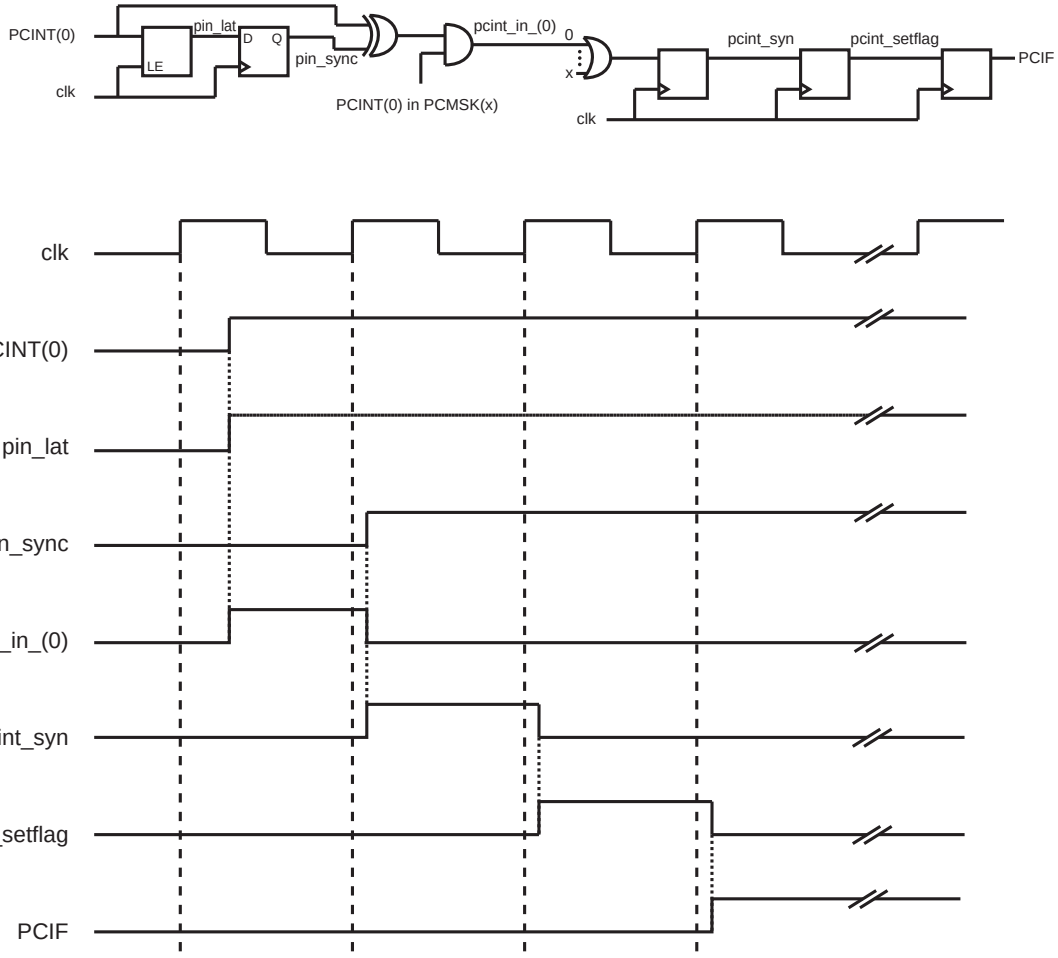
the Start-up Time, the MCU will still wake up, but no interrupt will be generated. The start-up time is defined by the SUT and CKSEL Fuses as described in “System Clock and Clock Options” on page 23.

If the low level on the interrupt pin is removed before the device has woken up then program execution will not be diverted to the interrupt service routine but continue from the instruction following the SLEEP command.

9.2.2 Pin Change Interrupt Timing

An example of timing of a pin change interrupt is shown in Figure 9-1.

Figure 9-1. Timing of pin change interrupts



9.3 Register Description

9.3.1 MCUCR – MCU Control Register

The External Interrupt Control Register A contains control bits for interrupt sense control.

Bit	7	6	5	4	3	2	1	0	
0x35	BODS	PUD	SE	SM1	SM0	BODSE	ISC01	ISC00	MCUCR
Read/Write	R	R/W	R/W	R/W	R/W	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 1:0 – ISC0[1:0]: Interrupt Sense Control 0 Bit 1 and Bit 0**

The External Interrupt 0 is activated by the external pin INT0 if the SREG I-flag and the corresponding interrupt mask are set. The level and edges on the external INT0 pin that activate the interrupt are defined in Table 9-2. The value on the INT0 pin is sampled before detecting edges. If edge or toggle interrupt is selected, pulses that last longer than one clock period will generate an interrupt. Shorter pulses are not guaranteed to generate an interrupt. If low level interrupt is selected, the low level must be held until the completion of the currently executing instruction to generate an interrupt.

Table 9-2. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

9.3.2 GIMSK – General Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x3B	–	INT0	PCIE	–	–	–	–	–	GIMSK
Read/Write	R	R/W	R/W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7, 4:0 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bit 6 – INT0: External Interrupt Request 0 Enable**

When the INT0 bit is set (one) and the I-bit in the Status Register (SREG) is set (one), the external pin interrupt is enabled. The Interrupt Sense Control0 bits 1/0 (ISC01 and ISC00) in the MCU Control Register (MCUCR) define whether the external interrupt is activated on rising and/or falling edge of the INT0 pin or level sensed. Activity on the pin will cause an interrupt request even if INT0 is configured as an output. The corresponding interrupt of External Interrupt Request 0 is executed from the INT0 Interrupt Vector.

- **Bit 5 – PCIE: Pin Change Interrupt Enable**

When the PCIE bit is set (one) and the I-bit in the Status Register (SREG) is set (one), pin change interrupt is enabled. Any change on any enabled PCINT[5:0] pin will cause an interrupt. The corresponding interrupt of Pin Change Interrupt Request is executed from the PCI Interrupt Vector. PCINT[5:0] pins are enabled individually by the PCMSK0 Register.

9.3.3 GIFR – General Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x3A	–	INTF0	PCIF	–	–	–	–	–	GIFR
Read/Write	R	R/W	R/W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7, 4:0 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bit 6 – INTF0: External Interrupt Flag 0**

When an edge or logic change on the INT0 pin triggers an interrupt request, INTF0 becomes set (one). If the I-bit in SREG and the INT0 bit in GIMSK are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it. This flag is always cleared when INT0 is configured as a level interrupt.

- **Bit 5 – PCIF: Pin Change Interrupt Flag**

When a logic change on any PCINT[5:0] pin triggers an interrupt request, PCIF becomes set (one). If the I-bit in SREG and the PCIE bit in GIMSK are set (one), the MCU will jump to the corresponding Interrupt Vector. The flag is cleared when the interrupt routine is executed. Alternatively, the flag can be cleared by writing a logical one to it.

9.3.4 PCMSK – Pin Change Mask Register

Bit	7	6	5	4	3	2	1	0	
0x15	–	–	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	PCMSK
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:6 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bits 5:0 – PCINT[5:0]: Pin Change Enable Mask 5:0**

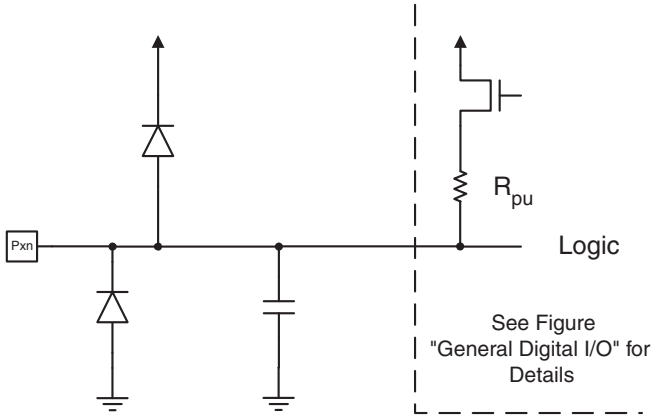
Each PCINT[5:0] bit selects whether pin change interrupt is enabled on the corresponding I/O pin. If PCINT[5:0] is set and the PCIE bit in GIMSK is set, pin change interrupt is enabled on the corresponding I/O pin. If PCINT[5:0] is cleared, pin change interrupt on the corresponding I/O pin is disabled.

10. I/O Ports

10.1 Introduction

All AVR ports have true Read-Modify-Write functionality when used as general digital I/O ports. This means that the direction of one port pin can be changed without unintentionally changing the direction of any other pin with the SBI and CBI instructions. The same applies when changing drive value (if configured as output) or enabling/disabling of pull-up resistors (if configured as input). Each output buffer has symmetrical drive characteristics with both high sink and source capability. The pin driver is strong enough to drive LED displays directly. All port pins have individually selectable pull-up resistors with a supply-voltage invariant resistance. All I/O pins have protection diodes to both V_{CC} and Ground as indicated in Figure 10-1. Refer to “Electrical Characteristics” on page 161 for a complete list of parameters.

Figure 10-1. I/O Pin Equivalent Schematic



All registers and bit references in this section are written in general form. A lower case “x” represents the numbering letter for the port, and a lower case “n” represents the bit number. However, when using the register or bit defines in a program, the precise form must be used. For example, PORTB3 for bit no. 3 in Port B, here documented generally as PORTxn. The physical I/O Registers and bit locations are listed in “Register Description” on page 64.

Three I/O memory address locations are allocated for each port, one each for the Data Register – PORTx, Data Direction Register – DDRx, and the Port Input Pins – PINx. The Port Input Pins I/O location is read only, while the Data Register and the Data Direction Register are read/write. However, writing a logic one to a bit in the PINx Register, will result in a toggle in the corresponding bit in the Data Register. In addition, the Pull-up Disable – PUD bit in MCUCR disables the pull-up function for all pins in all ports when set.

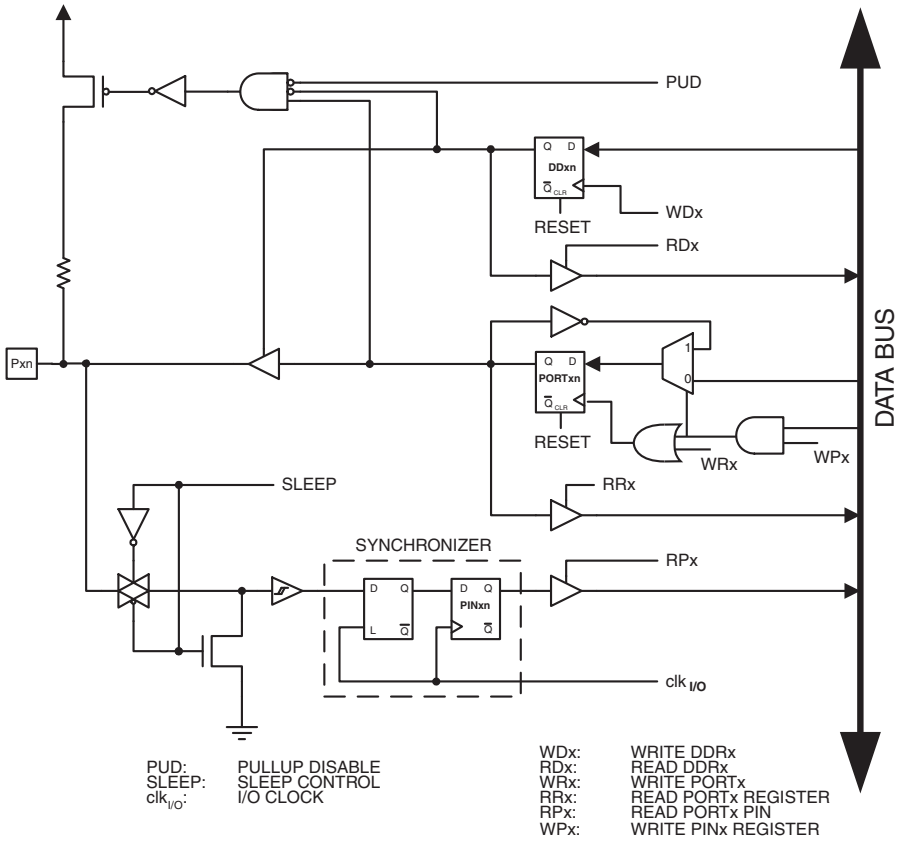
Using the I/O port as General Digital I/O is described in “Ports as General Digital I/O” on page 53. Most port pins are multiplexed with alternate functions for the peripheral features on the device. How each alternate function interferes with the port pin is described in “Alternate Port Functions” on page 57. Refer to the individual module sections for a full description of the alternate functions.

Note that enabling the alternate function of some of the port pins does not affect the use of the other pins in the port as general digital I/O.

10.2 Ports as General Digital I/O

The ports are bi-directional I/O ports with optional internal pull-ups. Figure 10-2 shows a functional description of one I/O-port pin, here generically called Pxn.

Figure 10-2. General Digital I/O[1]



Note: 1. WRx, WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. $clk_{I/O}$, SLEEP, and PUD are common to all ports.

10.2.1 Configuring the Pin

Each port pin consists of three register bits: DDxn, PORTxn, and PINxn. As shown in “Register Description” on page 64, the DDxn bits are accessed at the DDRx I/O address, the PORTxn bits at the PORTx I/O address, and the PINxn bits at the PINx I/O address.

The DDxn bit in the DDRx Register selects the direction of this pin. If DDxn is written logic one, Pxn is configured as an output pin. If DDxn is written logic zero, Pxn is configured as an input pin.

If PORTxn is written logic one when the pin is configured as an input pin, the pull-up resistor is activated. To switch the pull-up resistor off, PORTxn has to be written logic zero or the pin has to be configured as an output pin. The port pins are tri-stated when reset condition becomes active, even if no clocks are running.

If PORTxn is written logic one when the pin is configured as an output pin, the port pin is driven high (one). If PORTxn is written logic zero when the pin is configured as an output pin, the port pin is driven low (zero).

10.2.2 Toggling the Pin

Writing a logic one to PINxn toggles the value of PORTxn, independent on the value of DDRxn. Note that the SBI instruction can be used to toggle one single bit in a port.

10.2.3 Switching Between Input and Output

When switching between tri-state ({DDxn, PORTxn} = 0b00) and output high ({DDxn, PORTxn} = 0b11), an intermediate state with either pull-up enabled {DDxn, PORTxn} = 0b01) or output low ({DDxn, PORTxn} = 0b10) must occur. Normally, the pull-up enabled state is fully acceptable, as a high-impedant environment will not notice the

difference between a strong high driver and a pull-up. If this is not the case, the PUD bit in the MCUCR Register can be set to disable all pull-ups in all ports.

Switching between input with pull-up and output low generates the same problem. The user must use either the tri-state ({DDxn, PORTxn} = 0b00) or the output high state ({DDxn, PORTxn} = 0b10) as an intermediate step.

Table 10-1 summarizes the control signals for the pin value.

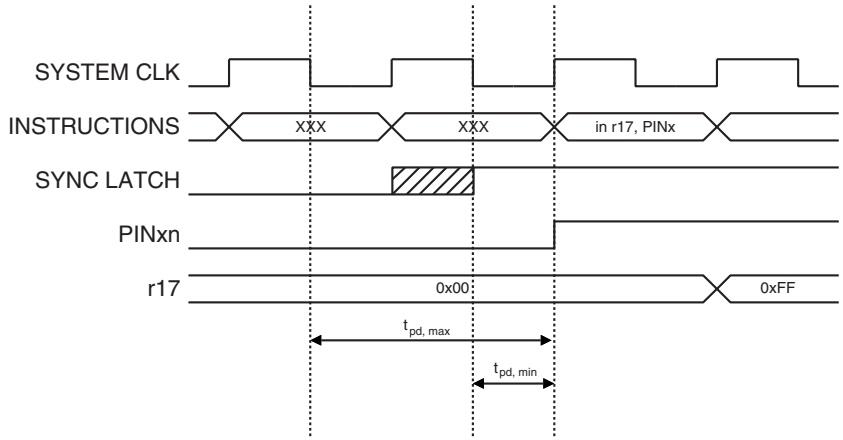
Table 10-1. Port Pin Configurations

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

10.2.4 Reading the Pin Value

Independent of the setting of Data Direction bit DDxn, the port pin can be read through the PINxn Register bit. As shown in Figure 10-2, the PINxn Register bit and the preceding latch constitute a synchronizer. This is needed to avoid metastability if the physical pin changes value near the edge of the internal clock, but it also introduces a delay. Figure 10-3 shows a timing diagram of the synchronization when reading an externally applied pin value. The maximum and minimum propagation delays are denoted $t_{pd,max}$ and $t_{pd,min}$ respectively.

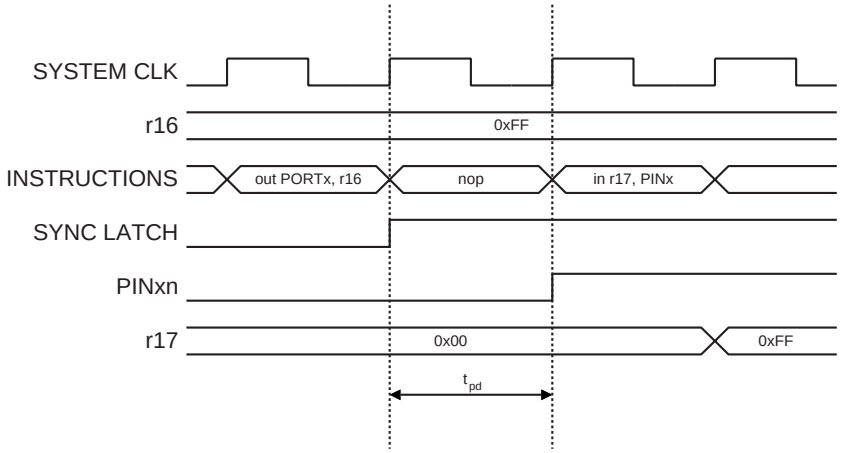
Figure 10-3. Synchronization when Reading an Externally Applied Pin value



Consider the clock period starting shortly after the first falling edge of the system clock. The latch is closed when the clock is low, and goes transparent when the clock is high, as indicated by the shaded region of the “SYNC LATCH” signal. The signal value is latched when the system clock goes low. It is clocked into the PINxn Register at the succeeding positive clock edge. As indicated by the two arrows tpd,max and tpd,min, a single signal transition on the pin will be delayed between ½ and 1½ system clock period depending upon the time of assertion.

When reading back a software assigned pin value, a nop instruction must be inserted as indicated in Figure 10-4. The out instruction sets the “SYNC LATCH” signal at the positive edge of the clock. In this case, the delay tpd through the synchronizer is one system clock period.

Figure 10-4. Synchronization when Reading a Software Assigned Pin Value



The following code example shows how to set port B pins 0 and 1 high, 2 and 3 low, and define the port pins from 4 to 5 as input with a pull-up assigned to port pin 4. The resulting pin values are read back again, but as previously discussed, a *nop* instruction is included to be able to read back the value recently assigned to some of the pins.

Assembly Code Example[1]

```
...
; Define pull-ups and set outputs high
; Define directions for port pins
ldi  r16,(1<<PB4)|(1<<PB1)|(1<<PB0)
ldi  r17,(1<<DDB3)|(1<<DDB2)|(1<<DDB1)|(1<<DDB0)
out  PORTB,r16
out  DDRB,r17
; Insert nop for synchronization
nop
; Read port pins
in   r16,PINB
...
```

Note: 1. For the assembly program, two temporary registers are used to minimize the time from pull-ups are set on pins 0, 1 and 4, until the direction bits are correctly set, defining bit 2 and 3 as low and redefining bits 0 and 1 as strong high drivers.

C Code Example

```
unsigned char i;
  ...
  /* Define pull-ups and set outputs high */
  /* Define directions for port pins */
  PORTB = (1<<PB4)|(1<<PB1)|(1<<PB0);
  DDRB = (1<<DDB3)|(1<<DDB2)|(1<<DDB1)|(1<<DDB0);
  /* Insert nop for synchronization*/
  _NOP();
  /* Read port pins */
  i = PINB;
  ...
```

10.2.5 Digital Input Enable and Sleep Modes

As shown in Figure 10-2, the digital input signal can be clamped to ground at the input of the schmitt-trigger. The signal denoted SLEEP in the figure, is set by the MCU Sleep Controller in Power-down mode to avoid high power consumption if some input signals are left floating, or have an analog signal level close to $V_{CC}/2$.

SLEEP is overridden for port pins enabled as external interrupt pins. If the external interrupt request is not enabled, SLEEP is active also for these pins. SLEEP is also overridden by various other alternate functions as described in “Alternate Port Functions” on page 57.

If a logic high level (“one”) is present on an asynchronous external interrupt pin configured as “Interrupt on Rising Edge, Falling Edge, or Any Logic Change on Pin” while the external interrupt is *not* enabled, the corresponding External Interrupt Flag will be set when resuming from the above mentioned Sleep mode, as the clamping in these sleep mode produces the requested logic change.

10.2.6 Unconnected Pins

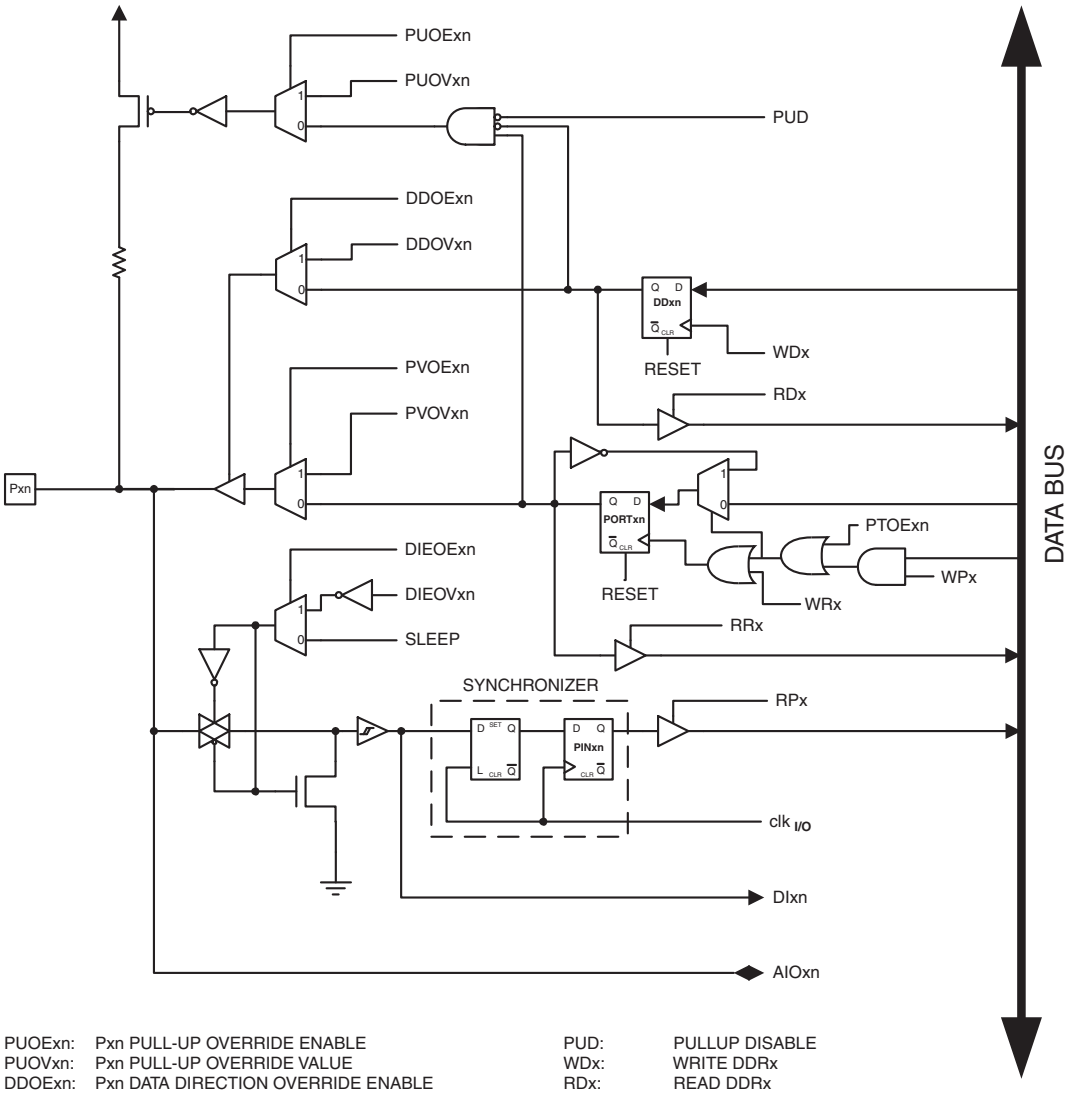
If some pins are unused, it is recommended to ensure that these pins have a defined level. Even though most of the digital inputs are disabled in the deep sleep modes as described above, floating inputs should be avoided to reduce current consumption in all other modes where the digital inputs are enabled (Reset, Active mode and Idle mode).

The simplest method to ensure a defined level of an unused pin, is to enable the internal pull-up. In this case, the pull-up will be disabled during reset. If low power consumption during reset is important, it is recommended to use an external pull-up or pulldown. Connecting unused pins directly to V_{CC} or GND is not recommended, since this may cause excessive currents if the pin is accidentally configured as an output.

10.3 Alternate Port Functions

Most port pins have alternate functions in addition to being general digital I/Os. Figure 10-5 shows how the port pin control signals from the simplified Figure 10-2 can be overridden by alternate functions. The overriding signals may not be present in all port pins, but the figure serves as a generic description applicable to all port pins in the AVR microcontroller family.

Figure 10-5. Alternate Port Functions[1]



PUOExn:	Pxn PULL-UP OVERRIDE ENABLE	PUD:	PULLUP DISABLE
PUOVxn:	Pxn PULL-UP OVERRIDE VALUE	WDx:	WRITE DDRx
DDOExn:	Pxn DATA DIRECTION OVERRIDE ENABLE	RDx:	READ DDRx
DDOVxn:	Pxn DATA DIRECTION OVERRIDE VALUE	RRx:	READ PORTx REGISTER
PVOExn:	Pxn PORT VALUE OVERRIDE ENABLE	WRx:	WRITE PORTx
PVOVxn:	Pxn PORT VALUE OVERRIDE VALUE	RPx:	READ PORTx PIN
DIEOExn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE ENABLE	WPx:	WRITE PINx
DIEOVxn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE VALUE	$clk_{I/O}$:	I/O CLOCK
SLEEP:	SLEEP CONTROL	DIxn:	DIGITAL INPUT PIN n ON PORTx
PTOExn:	Pxn, PORT TOGGLE OVERRIDE ENABLE	AIOxn:	ANALOG INPUT/OUTPUT PIN n ON PORTx

Note: 1. WRx, WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. $clk_{I/O}$, SLEEP, and PUD are common to all ports. All other signals are unique for each pin.

Table 10-2 summarizes the function of the overriding signals. The pin and port indexes from Figure 10-5 are not shown in the succeeding tables. The overriding signals are generated internally in the modules having the alternate function.

Table 10-2. Generic Description of Overriding Signals for Alternate Functions

Signal Name	Full Name	Description
PUOE	Pull-up Override Enable	If this signal is set, the pull-up enable is controlled by the PUOV signal. If this signal is cleared, the pull-up is enabled when {DDxn, PORTxn, PUD} = 0b010.
PUOV	Pull-up Override Value	If PUOE is set, the pull-up is enabled/disabled when PUOV is set/cleared, regardless of the setting of the DDxn, PORTxn, and PUD Register bits.
DDOE	Data Direction Override Enable	If this signal is set, the Output Driver Enable is controlled by the DDOV signal. If this signal is cleared, the Output driver is enabled by the DDxn Register bit.
DDOV	Data Direction Override Value	If DDOE is set, the Output Driver is enabled/disabled when DDOV is set/cleared, regardless of the setting of the DDxn Register bit.
PVOE	Port Value Override Enable	If this signal is set and the Output Driver is enabled, the port value is controlled by the PVOV signal. If PVOE is cleared, and the Output Driver is enabled, the port Value is controlled by the PORTxn Register bit.
PVOV	Port Value Override Value	If PVOE is set, the port value is set to PVOV, regardless of the setting of the PORTxn Register bit.
PTOE	Port Toggle Override Enable	If PTOE is set, the PORTxn Register bit is inverted.
DIEOE	Digital Input Enable Override Enable	If this bit is set, the Digital Input Enable is controlled by the DIEOV signal. If this signal is cleared, the Digital Input Enable is determined by MCU state (Normal mode, sleep mode).
DIEOV	Digital Input Enable Override Value	If DIEOE is set, the Digital Input is enabled/disabled when DIEOV is set/cleared, regardless of the MCU state (Normal mode, sleep mode).
DI	Digital Input	This is the Digital Input to alternate functions. In the figure, the signal is connected to the output of the schmitt-trigger but before the synchronizer. Unless the Digital Input is used as a clock source, the module with the alternate function will use its own synchronizer.
AIO	Analog Input/Output	This is the Analog Input/Output to/from alternate functions. The signal is connected directly to the pad, and can be used bi-directionally.

The following subsections shortly describe the alternate functions for each port, and relate the overriding signals to the alternate function. Refer to the alternate function description for further details.

10.3.1 Alternate Functions of Port B

The Port B pins with alternate function are shown in Table 10-3.

Table 10-3. Port B Pins Alternate Functions

Port Pin	Alternate Function
PB5	RESET: Reset Pin dW: debugWIRE I/O ADC0: ADC Input Channel 0 PCINT5: Pin Change Interrupt, Source 5
PB4	XTAL2: Crystal Oscillator Output CLKO: System Clock Output ADC2: ADC Input Channel 2 OC1B: Timer/Counter1 Compare Match B Output PCINT4: Pin Change Interrupt 0, Source 4
PB3	XTAL1: Crystal Oscillator Input CLKI: External Clock Input ADC3: ADC Input Channel 3 OC1B: Complementary Timer/Counter1 Compare Match B Output PCINT3: Pin Change Interrupt 0, Source 3
PB2	SCK: Serial Clock Input ADC1: ADC Input Channel 1 T0: Timer/Counter0 Clock Source USCK: USI Clock (Three Wire Mode) SCL : USI Clock (Two Wire Mode) INT0: External Interrupt 0 Input PCINT2: Pin Change Interrupt 0, Source 2
PB1	MISO: SPI Master Data Input / Slave Data Output AIN1: Analog Comparator, Negative Input OC0B: Timer/Counter0 Compare Match B Output OC1A: Timer/Counter1 Compare Match A Output DO: USI Data Output (Three Wire Mode) PCINT1:Pin Change Interrupt 0, Source 1
PB0	MOSI:: SPI Master Data Output / Slave Data Input AIN0: Analog Comparator, Positive Input OC0A: Timer/Counter0 Compare Match A output OC1A: Complementary Timer/Counter1 Compare Match A Output DI: USI Data Input (Three Wire Mode) SDA: USI Data Input (Two Wire Mode) AREF: External Analog Reference PCINT0: Pin Change Interrupt 0, Source 0

- **Port B, Bit 5 – RESET/dW/ADC0/PCINT5**
  - RESET: External Reset input is active low and enabled by unprogramming (“1”) the RSTDISBL Fuse. Pullup is activated and output driver and digital input are deactivated when the pin is used as the RESET pin.
  - dW: When the debugWIRE Enable (DWEN) Fuse is programmed and Lock bits are unprogrammed, the debugWIRE system within the target device is activated. The RESET port pin is configured as a wire-AND (open-drain) bi-directional I/O pin with pull-up enabled and becomes the communication gateway between target and emulator.
  - ADC0: Analog to Digital Converter, Channel 0.
  - PCINT5: Pin Change Interrupt source 5.

- **Port B, Bit 4 – XTAL2/CLKO/ADC2/OC1B/PCINT4**
  - XTAL2: Chip Clock Oscillator pin 2. Used as clock pin for all chip clock sources except internal calibrateble RC Oscillator and external clock. When used as a clock pin, the pin can not be used as an I/O pin. When using internal calibratable RC Oscillator or External clock as a Chip clock sources, PB4 serves as an ordinary I/O pin.
  - CLKO: The devided system clock can be output on the pin PB4. The divided system clock will be output if the CKOUT Fuse is programmed, regardless of the PORTB4 and DDB4 settings. It will also be output during reset.
  - ADC2: Analog to Digital Converter, Channel 2.
  - OC1B: Output Compare Match output: The PB4 pin can serve as an external output for the Timer/Counter1 Compare Match B when configured as an output (DDB4 set). The OC1B pin is also the output pin for the PWM mode timer function.
  - PCINT4: Pin Change Interrupt source 4.

- **Port B, Bit 3 – XTAL1/CLKI/ADC3/$\overline{\text{OC1B}}$/PCINT3**
  - XTAL1: Chip Clock Oscillator pin 1. Used for all chip clock sources except internal calibrateble RC oscillator. When used as a clock pin, the pin can not be used as an I/O pin.
  - CLKI: Clock Input from an external clock source, see “External Clock” on page 26.
  - ADC3: Analog to Digital Converter, Channel 3.
  - $\overline{\text{OC1B}}$: Inverted Output Compare Match output: The PB3 pin can serve as an external output for the Timer/Counter1 Compare Match B when configured as an output (DDB3 set). The $\overline{\text{OC1B}}$ pin is also the inverted output pin for the PWM mode timer function.
  - PCINT3: Pin Change Interrupt source 3.

- **Port B, Bit 2 – SCK/ADC1/T0/USCK/SCL/INT0/PCINT2**
  - SCK: Master Clock output, Slave Clock input pin for SPI channel. When the SPI is enabled as a Slave, this pin is configured as an input regardless of the setting of DDB2. When the SPI is enabled as a Master, the data direction of this pin is controlled by DDPB2. When the pin is forced by the SPI to be an input, the pull-up can still be controlled by the PORTB2 bit.
  - ADC1: Analog to Digital Converter, Channel 1.
  - T0: Timer/Counter0 counter source.
  - USCK: Three-wire mode Universal Serial Interface Clock.
  - SCL: Two-wire mode Serial Clock for USI Two-wire mode.
  - INT0: External Interrupt source 0.
  - PCINT2: Pin Change Interrupt source 2.

- **Port B, Bit 1 – MISO/AIN1/OC0B/OC1A/DO/PCINT1**
  - MISO: Master Data input, Slave Data output pin for SPI channel. When the SPI is enabled as a Master, this pin is configured as an input regardless of the setting of DDB1. When the SPI is enabled as a Slave, the data direction of this pin is controlled by DDB1. When the pin is forced by the SPI to be an input, the pull-up can still be controlled by the PORTB1 bit.
  - AIN1: Analog Comparator Negative Input. Configure the port pin as input with the internal pull-up switched off to avoid the digital port function from interfering with the function of the Analog Comparator.
  - OC0B: Output Compare Match output. The PB1 pin can serve as an external output for the Timer/Counter0 Compare Match B. The PB1 pin has to be configured as an output (DDB1 set (one)) to serve this function. The OC0B pin is also the output pin for the PWM mode timer function.

- OC1A: Output Compare Match output: The PB1 pin can serve as an external output for the Timer/Counter1 Compare Match B when configured as an output (DDB1 set). The OC1A pin is also the output pin for the PWM mode timer function.
- DO: Three-wire mode Universal Serial Interface Data output. Three-wire mode Data output overrides PORTB1 value and it is driven to the port when data direction bit DDB1 is set (one). PORTB1 still enables the pull-up, if the direction is input and PORTB1 is set (one).
- PCINT1: Pin Change Interrupt source 1.

- **Port B, Bit 0 – MOSI/AIN0/OC0A/$\overline{\text{OC1A}}$/DI/SDA/AREF/PCINT0**
  - MOSI: SPI Master Data output, Slave Data input for SPI channel. When the SPI is enabled as a Slave, this pin is configured as an input regardless of the setting of DDB0. When the SPI is enabled as a Master, the data direction of this pin is controlled by DDB0. When the pin is forced by the SPI to be an input, the pull-up can still be controlled by the PORTB0 bit.
  - AIN0: Analog Comparator Positive Input. Configure the port pin as input with the internal pull-up switched off to avoid the digital port function from interfering with the function of the Analog Comparator.
  - OC0A: Output Compare Match output. The PB0 pin can serve as an external output for the Timer/Counter0 Compare Match A when configured as an output (DDB0 set (one)). The OC0A pin is also the output pin for the PWM mode timer function.
  - $\overline{\text{OC1A}}$: Inverted Output Compare Match output: The PB0 pin can serve as an external output for the Timer/Counter1 Compare Match B when configured as an output (DDB0 set). The $\overline{\text{OC1A}}$ pin is also the inverted output pin for the PWM mode timer function.
  - SDA: Two-wire mode Serial Interface Data.
  - AREF: External Analog Reference for ADC. Pullup and output driver are disabled on PB0 when the pin is used as an external reference or Internal Voltage Reference with external capacitor at the AREF pin.
  - DI: Data Input in USI Three-wire mode. USI Three-wire mode does not override normal port functions, so pin must be configure as an input for DI function.
  - PCINT0: Pin Change Interrupt source 0.

Table 10-4 and Table 10-5 relate the alternate functions of Port B to the overriding signals shown in Figure 10-5 on page 58.

Table 10-4. Overriding Signals for Alternate Functions in PB[5:3]

Signal Name	PB5/$\overline{\text{RESET}}$/ ADC0/PCINT5	PB4/ADC2/XTAL2/ OC1B/PCINT4	PB3/ADC3/XTAL1/ OC1B/PCINT3
PUOE	$\overline{\text{RSTDISBL}}$[1] • DWEN[1]	0	0
PUOV	1	0	0
DDOE	RSTDISBL[1] • DWEN[1]	0	0
DDOV	debugWire Transmit	0	0
PVOE	0	OC1B Enable	$\overline{\text{OC1B}}$ Enable
PVOV	0	OC1B	OC1B
PTOE	0	0	0
DIEOE	$\overline{\text{RSTDISBL}}$[1] + (PCINT5 • PCIE + ADC0D)	PCINT4 • PCIE + ADC2D	PCINT3 • PCIE + ADC3D
DIEOV	ADC0D	ADC2D	ADC3D
DI	PCINT5 Input	PCINT4 Input	PCINT3 Input
AIO	RESET Input, ADC0 Input	ADC2 Input	ADC3 Input

Note: 1. 1 when the Fuse is “0” (Programmed).

Table 10-5. Overriding Signals for Alternate Functions in PB[2:0]

Signal Name	PB2/SCK/ADC1/T0/ USCK/SCL/INT0/PCINT2	PB1/MISO/DO/AIN1/ OC1A/OC0B/PCINT1	PB0/MOSI/DI/SDA/AIN0/AREF/OC1A/OC0A/ PCINT0
PUOE	USI_TWO_WIRE	0	USI_TWO_WIRE
PUOV	0	0	0
DDOE	USI_TWO_WIRE	0	USI_TWO_WIRE
DDOV	(USI_SCL_HOLD + $\overline{\text{PORTB2}}$) • DDB2	0	($\overline{\text{SDA}}$ + $\overline{\text{PORTB0}}$) • DDB0
PVOE	USI_TWO_WIRE • DDB2	OC0B Enable + OC1A Enable + USI_THREE_WIRE	OC0A Enable + $\overline{\text{OC1A}}$ Enable + (USI_TWO_WIRE • DDB0)
PVOV	0	OC0B + OC1A + DO	OC0A + $\overline{\text{OC1A}}$
PTOE	USITC	0	0
DIEOE	PCINT2 • PCIE + ADC1D + USISIE	PCINT1 • PCIE + AIN1D	PCINT0 • PCIE + AIN0D + USISIE
DIEOV	ADC1D	AIN1D	AIN0D
DI	T0/USCK/SCL/INT0/ PCINT2 Input	PCINT1 Input	DI/SDA/PCINT0 Input
AIO	ADC1 Input	Analog Comparator Negative Input	Analog Comparator Positive Input

10.4 Register Description

10.4.1 MCUCR – MCU Control Register

Bit	7	6	5	4	3	2	1	0	
0x35	BODS	PUD	SE	SM1	SM0	BODSE	ISC01	ISC00	MCUCR
Read/Write	R	R/W	R/W	R/W	R/W	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 6 – PUD: Pull-up Disable**

When this bit is written to one, the pull-ups in the I/O ports are disabled even if the DDxn and PORTxn Registers are configured to enable the pull-ups ({DDxn, PORTxn} = 0b01). See “Configuring the Pin” on page 54 for more details about this feature.

10.4.2 PORTB – Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x18	–	–	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

10.4.3 DDRB – Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x17	–	–	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

10.4.4 PINB – Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x16	–	–	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	N/A	N/A	N/A	N/A	N/A	N/A	

11. 8-bit Timer/Counter0 with PWM

11.1 Features

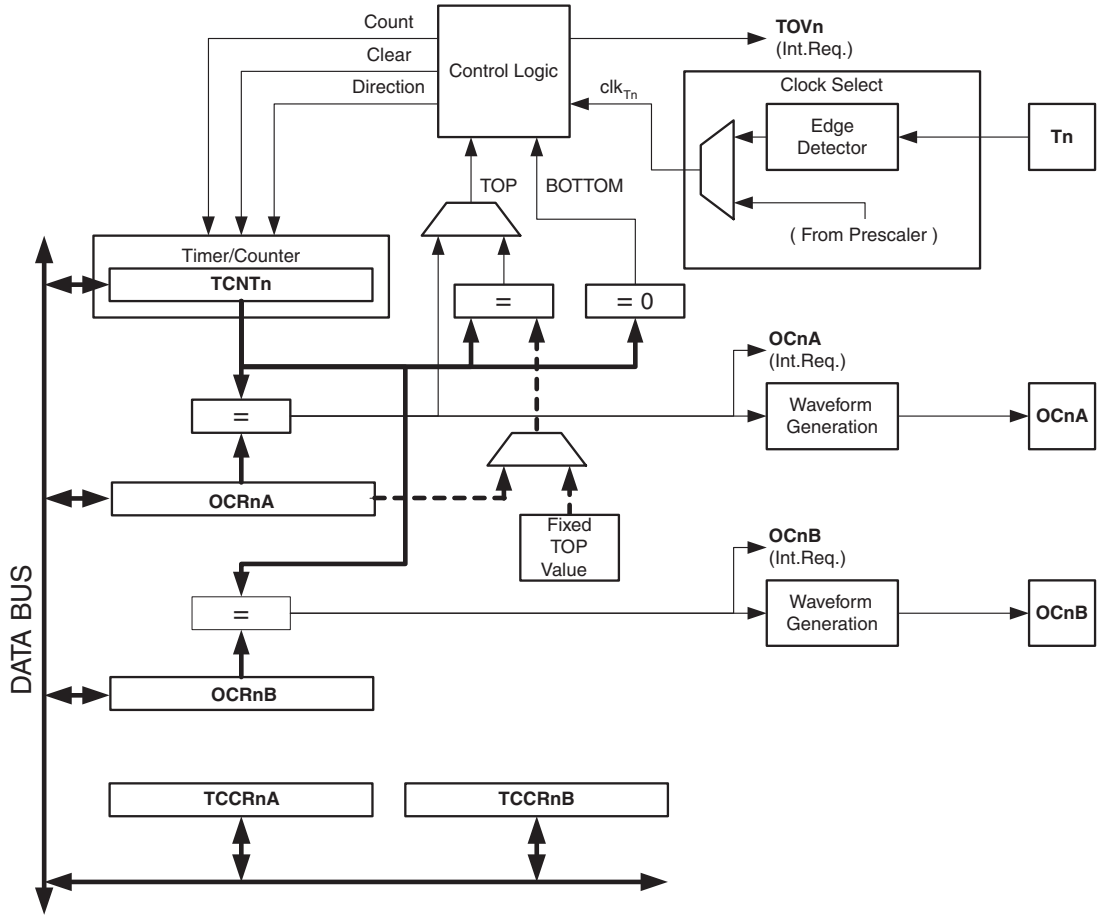
- **Two Independent Output Compare Units**
- **Double Buffered Output Compare Registers**
- **Clear Timer on Compare Match (Auto Reload)**
- **Glitch Free, Phase Correct Pulse Width Modulator (PWM)**
- **Variable PWM Period**
- **Frequency Generator**
- **Three Independent Interrupt Sources (TOV0, OCF0A, and OCF0B)**

11.2 Overview

Timer/Counter0 is a general purpose 8-bit Timer/Counter module, with two independent Output Compare Units, and with PWM support. It allows accurate program execution timing (event management) and wave generation.

A simplified block diagram of the 8-bit Timer/Counter is shown in Figure 11-1. For the actual placement of I/O pins, refer to “Pinout ATtiny25/45/85” on page 2. CPU accessible I/O Registers, including I/O bits and I/O pins, are shown in bold. The device-specific I/O Register and bit locations are listed in the “Register Description” on page 77.

Figure 11-1. 8-bit Timer/Counter Block Diagram



11.2.1 Registers

The Timer/Counter (TCNT0) and Output Compare Registers (OCR0A and OCR0B) are 8-bit registers. Interrupt request (abbreviated to Int.Req. in the figure) signals are all visible in the Timer Interrupt Flag Register (TIFR). All

interrupts are individually masked with the Timer Interrupt Mask Register (TIMSK). TIFR and TIMSK are not shown in the figure.

The Timer/Counter can be clocked internally, via the prescaler, or by an external clock source on the T0 pin. The Clock Select logic block controls which clock source and edge the Timer/Counter uses to increment (or decrement) its value. The Timer/Counter is inactive when no clock source is selected. The output from the Clock Select logic is referred to as the timer clock (clk_{T0}).

The double buffered Output Compare Registers (OCR0A and OCR0B) is compared with the Timer/Counter value at all times. The result of the compare can be used by the Waveform Generator to generate a PWM or variable frequency output on the Output Compare pins (OC0A and OC0B). See “Output Compare Unit” on page 69. for details. The Compare Match event will also set the Compare Flag (OCF0A or OCF0B) which can be used to generate an Output Compare interrupt request.

11.2.2 Definitions

Many register and bit references in this section are written in general form. A lower case “n” replaces the Timer/Counter number, in this case 0. A lower case “x” replaces the Output Compare Unit, in this case Compare Unit A or Compare Unit B. However, when using the register or bit defines in a program, the precise form must be used, i.e., TCNT0 for accessing Timer/Counter0 counter value and so on.

The definitions in Table 11-1 are also used extensively throughout the document.

Table 11-1. Definitions

Constant	Description
BOTTOM	The counter reaches BOTTOM when it becomes 0x00
MAX	The counter reaches its MAXimum when it becomes 0xFF (decimal 255)
TOP	The counter reaches the TOP when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be the fixed value 0xFF (MAX) or the value stored in the OCR0A Register. The assignment depends on the mode of operation

11.3 Timer/Counter0 Prescaler and Clock Sources

The Timer/Counter can be clocked by an internal or an external clock source. The clock source is selected by the Clock Select logic which is controlled by the Clock Select (c) bits located in the Timer/Counter0 Control Register (TCCR0B).

11.3.1 Internal Clock Source with Prescaler

Timer/Counter0 can be clocked directly by the system clock (by setting the CS0[2:0] = 1). This provides the fastest operation, with a maximum timer/counter clock frequency equal to system clock frequency ($f_{CLK_I/O}$). Alternatively, one of four taps from the prescaler can be used as a clock source. The prescaled clock has a frequency of either $f_{CLK_I/O}/8$, $f_{CLK_I/O}/64$, $f_{CLK_I/O}/256$, or $f_{CLK_I/O}/1024$.

11.3.2 Prescaler Reset

The prescaler is free running, i.e. it operates independently of the Clock Select logic of Timer/Counter0. Since the prescaler is not affected by the timer/counter’s clock select, the state of the prescaler will have implications for situations where a prescaled clock is used. One example of a prescaling artifact is when the timer/counter is enabled and clocked by the prescaler (6 > CS0[2:0] > 1). The number of system clock cycles from when the timer is enabled to the first count occurs can be from 1 to N+1 system clock cycles, where N equals the prescaler divisor (8, 64, 256, or 1024).

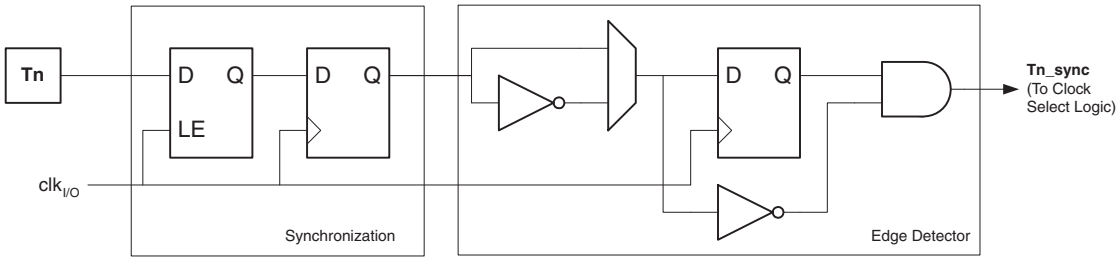
It is possible to use the Prescaler Reset for synchronizing the Timer/Counter to program execution.

11.3.3 External Clock Source

An external clock source applied to the T0 pin can be used as timer/counter clock (clk_{T0}). The T0 pin is sampled once every system clock cycle by the pin synchronization logic. The synchronized (sampled) signal is then passed through the edge detector. Figure 11-2 shows a functional equivalent block diagram of the T0 synchronization and edge detector logic. The registers are clocked at the positive edge of the internal system clock ($clk_{I/O}$). The latch is transparent in the high period of the internal system clock.

The edge detector generates one clk_{T0} pulse for each positive (CS0[2:0] = 7) or negative (CS0[2:0] = 6) edge it detects.

Figure 11-2. T0 Pin Sampling



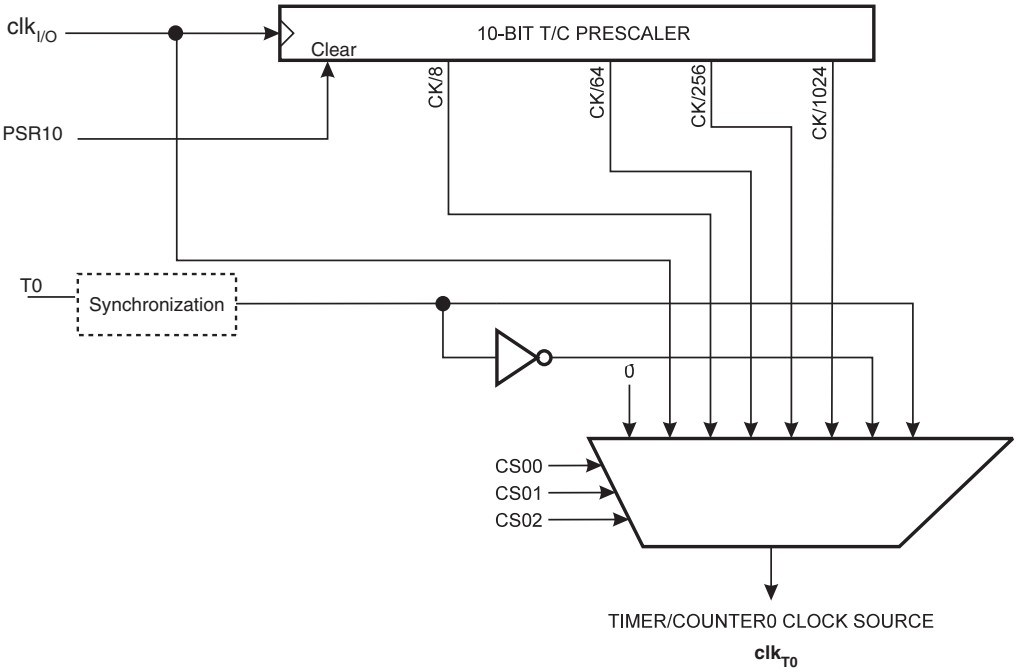
The synchronization and edge detector logic introduces a delay of 2.5 to 3.5 system clock cycles from an edge has been applied to the T0 pin to the counter is updated.

Enabling and disabling of the clock input must be done when T0 has been stable for at least one system clock cycle, otherwise it is a risk that a false timer/counter clock pulse is generated.

Each half period of the external clock applied must be longer than one system clock cycle to ensure correct sampling. The external clock must be guaranteed to have less than half the system clock frequency ($f_{ExtClk} < f_{clk_I/O}/2$) given a 50/50% duty cycle. Since the edge detector uses sampling, the maximum frequency of an external clock it can detect is half the sampling frequency (following the Nyquist sampling theorem). However, due to variation of the system clock frequency and duty cycle caused by oscillator source (crystal, resonator, and capacitors) tolerances, it is recommended that maximum frequency of an external clock source is less than $f_{clk_I/O}/2.5$.

An external clock source can not be prescaled.

Figure 11-3. Timer/Counter0 Prescaler

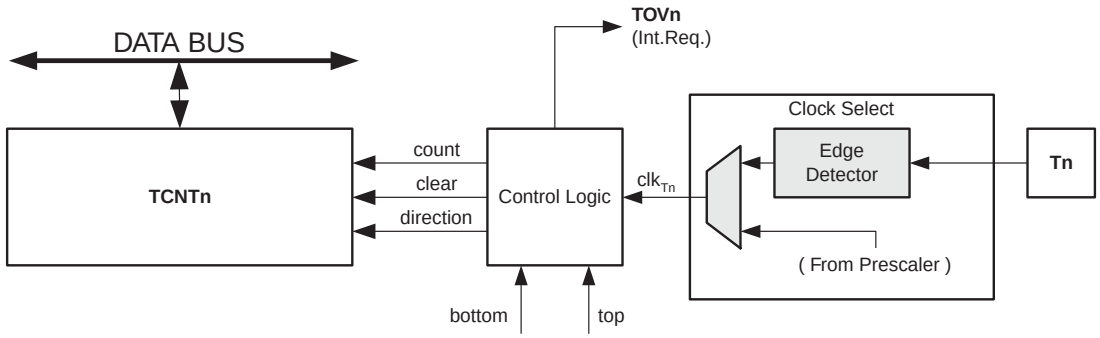


The synchronization logic on the input pins (T0) in Figure 11-3 is shown in Figure 11-2 on page 67.

11.4 Counter Unit

The main part of the 8-bit Timer/Counter is the programmable bi-directional counter unit. Figure 11-4 shows a block diagram of the counter and its surroundings.

Figure 11-4. Counter Unit Block Diagram



Signal description (internal signals):

count	Increment or decrement TCNT0 by 1.
direction	Select between increment and decrement.
clear	Clear TCNT0 (set all bits to zero).
clk_{Tn}	Timer/Counter clock, referred to as clk_{T0} in the following.
top	Signalize that TCNT0 has reached maximum value.
bottom	Signalize that TCNT0 has reached minimum value (zero).

Depending of the mode of operation used, the counter is cleared, incremented, or decremented at each timer clock (clk_{T0}). clk_{T0} can be generated from an external or internal clock source, selected by the Clock Select bits (CS0[2:0]). When no clock source is selected (CS0[2:0] = 0) the timer is stopped. However, the TCNT0 value can

be accessed by the CPU, regardless of whether clk_{T0} is present or not. A CPU write overrides (has priority over) all counter clear or count operations.

The counting sequence is determined by the setting of the WGM01 and WGM00 bits located in the Timer/Counter Control Register (TCCR0A) and the WGM02 bit located in the Timer/Counter Control Register B (TCCR0B). There are close connections between how the counter behaves (counts) and how waveforms are generated on the Output Compare output OC0A. For more details about advanced counting sequences and waveform generation, see “Modes of Operation” on page 71.

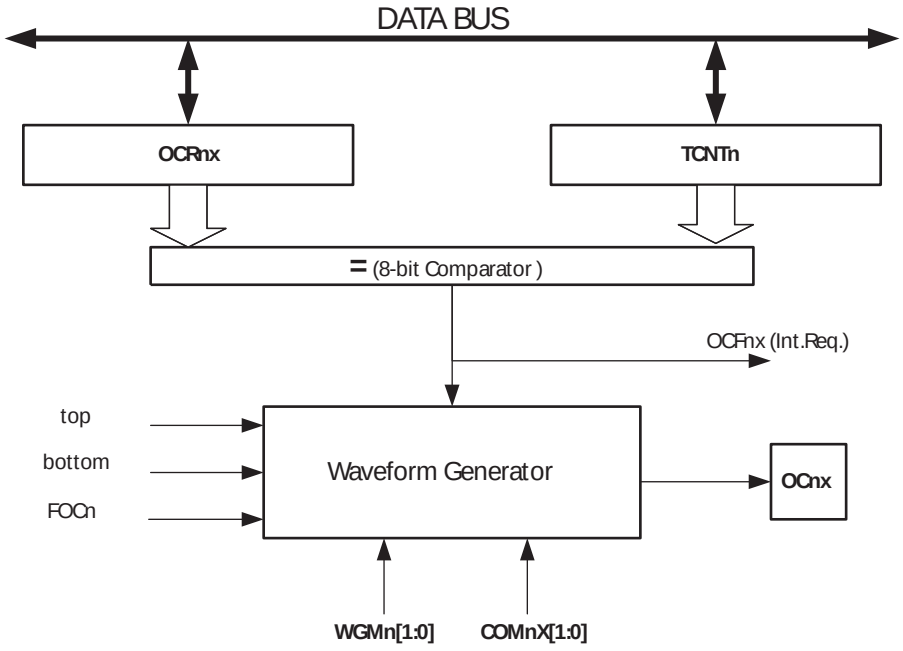
The Timer/Counter Overflow Flag (TOV0) is set according to the mode of operation selected by the WGM0[1:0] bits. TOV0 can be used for generating a CPU interrupt.

11.5 Output Compare Unit

The 8-bit comparator continuously compares TCNT0 with the Output Compare Registers (OCR0A and OCR0B). Whenever TCNT0 equals OCR0A or OCR0B, the comparator signals a match. A match will set the Output Compare Flag (OCF0A or OCF0B) at the next timer clock cycle. If the corresponding interrupt is enabled, the Output Compare Flag generates an Output Compare interrupt. The Output Compare Flag is automatically cleared when the interrupt is executed. Alternatively, the flag can be cleared by software by writing a logical one to its I/O bit location. The Waveform Generator uses the match signal to generate an output according to operating mode set by the WGM0[2:0] bits and Compare Output mode (COM0x[1:0]) bits. The max and bottom signals are used by the Waveform Generator for handling the special cases of the extreme values in some modes of operation (See “Modes of Operation” on page 71.).

Figure 11-5 shows a block diagram of the Output Compare unit.

Figure 11-5. Output Compare Unit, Block Diagram



The OCR0x Registers are double buffered when using any of the Pulse Width Modulation (PWM) modes. For the normal and Clear Timer on Compare (CTC) modes of operation, the double buffering is disabled. The double buffering synchronizes the update of the OCR0x Compare Registers to either top or bottom of the counting sequence. The synchronization prevents the occurrence of odd-length, non-symmetrical PWM pulses, thereby making the output glitch-free.

The OCR0x Register access may seem complex, but this is not case. When the double buffering is enabled, the CPU has access to the OCR0x Buffer Register, and if double buffering is disabled the CPU will access the OCR0x directly.

11.5.1 Force Output Compare

In non-PWM waveform generation modes, the match output of the comparator can be forced by writing a one to the Force Output Compare (FOC0x) bit. Forcing Compare Match will not set the OCF0x Flag or reload/clear the timer, but the OC0x pin will be updated as if a real Compare Match had occurred (the COM0x[1:0] bits settings define whether the OC0x pin is set, cleared or toggled).

11.5.2 Compare Match Blocking by TCNT0 Write

All CPU write operations to the TCNT0 Register will block any Compare Match that occur in the next timer clock cycle, even when the timer is stopped. This feature allows OCR0x to be initialized to the same value as TCNT0 without triggering an interrupt when the Timer/Counter clock is enabled.

11.5.3 Using the Output Compare Unit

Since writing TCNT0 in any mode of operation will block all Compare Matches for one timer clock cycle, there are risks involved when changing TCNT0 when using the Output Compare Unit, independently of whether the Timer/Counter is running or not. If the value written to TCNT0 equals the OCR0x value, the Compare Match will be missed, resulting in incorrect waveform generation. Similarly, do not write the TCNT0 value equal to BOTTOM when the counter is down-counting.

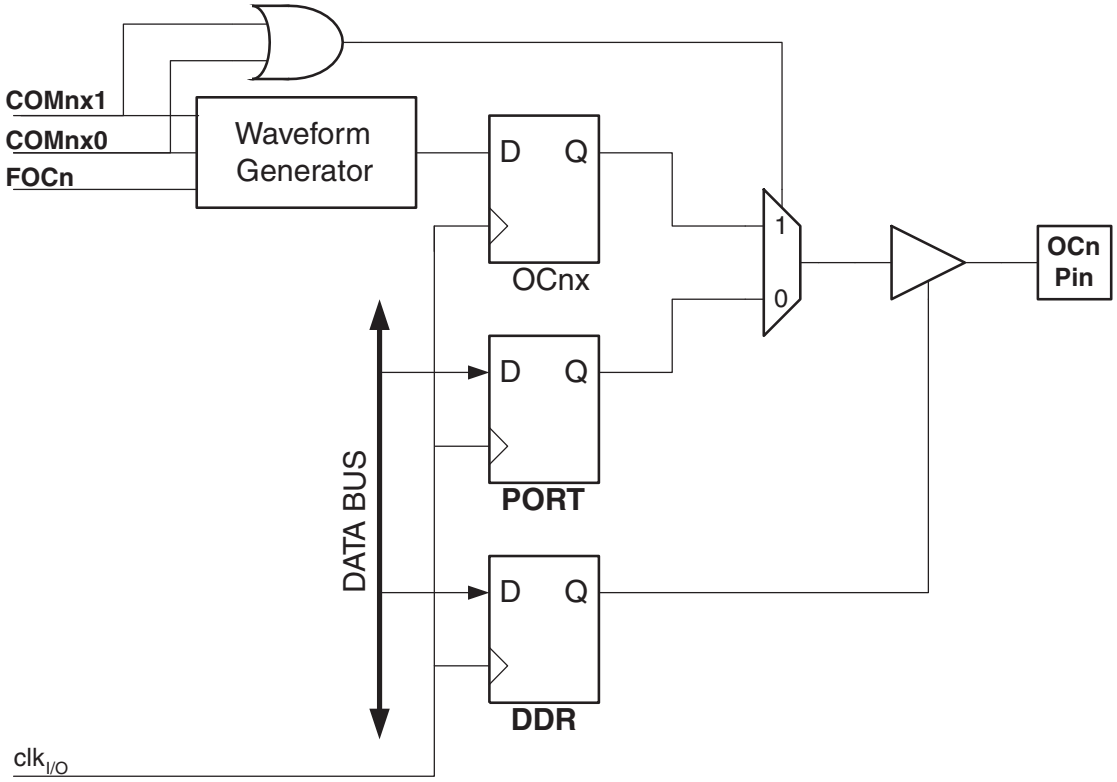
The setup of the OC0x should be performed before setting the Data Direction Register for the port pin to output. The easiest way of setting the OC0x value is to use the Force Output Compare (FOC0x) strobe bits in Normal mode. The OC0x Registers keep their values even when changing between Waveform Generation modes.

Be aware that the COM0x[1:0] bits are not double buffered together with the compare value. Changing the COM0x[1:0] bits will take effect immediately.

11.6 Compare Match Output Unit

The Compare Output mode (COM0x[1:0]) bits have two functions. The Waveform Generator uses the COM0x[1:0] bits for defining the Output Compare (OC0x) state at the next Compare Match. Also, the COM0x[1:0] bits control the OC0x pin output source. Figure 11-6 shows a simplified schematic of the logic affected by the COM0x[1:0] bit setting. The I/O Registers, I/O bits, and I/O pins in the figure are shown in bold. Only the parts of the general I/O Port Control Registers (DDR and PORT) that are affected by the COM0x[1:0] bits are shown. When referring to the OC0x state, the reference is for the internal OC0x Register, not the OC0x pin. If a system reset occur, the OC0x Register is reset to “0”.

Figure 11-6. Compare Match Output Unit, Schematic



The general I/O port function is overridden by the Output Compare (OC0x) from the Waveform Generator if either of the COM0x[1:0] bits are set. However, the OC0x pin direction (input or output) is still controlled by the Data Direction Register (DDR) for the port pin. The Data Direction Register bit for the OC0x pin (DDR_OC0x) must be set as output before the OC0x value is visible on the pin. The port override function is independent of the Waveform Generation mode.

The design of the Output Compare pin logic allows initialization of the OC0x state before the output is enabled. Note that some COM0x[1:0] bit settings are reserved for certain modes of operation. See “Register Description” on page 77.

11.6.1 Compare Output Mode and Waveform Generation

The Waveform Generator uses the COM0x[1:0] bits differently in Normal, CTC, and PWM modes. For all modes, setting the COM0x[1:0] = 0 tells the Waveform Generator that no action on the OC0x Register is to be performed on the next Compare Match. For compare output actions in the non-PWM modes refer to Table 11-2 on page 78. For fast PWM mode, refer to Table 11-3 on page 78, and for phase correct PWM refer to Table 11-4 on page 78.

A change of the COM0x[1:0] bits state will have effect at the first Compare Match after the bits are written. For non-PWM modes, the action can be forced to have immediate effect by using the FOC0x strobe bits.

11.7 Modes of Operation

The mode of operation, i.e., the behavior of the Timer/Counter and the Output Compare pins, is defined by the combination of the Waveform Generation mode (WGM0[2:0]) and Compare Output mode (COM0x[1:0]) bits. The Compare Output mode bits do not affect the counting sequence, while the Waveform Generation mode bits do. The COM0x[1:0] bits control whether the PWM output generated should be inverted or not (inverted or non-inverted PWM). For non-PWM modes the COM0x[1:0] bits control whether the output should be set, cleared, or toggled at a Compare Match (See “Compare Match Output Unit” on page 70.).

For detailed timing information refer to Figure 11-10, Figure 11-11, Figure 11-12 and Figure 11-13 in “Timer/Counter Timing Diagrams” on page 76.

11.7.1 Normal Mode

The simplest mode of operation is the Normal mode (WGM0[2:0] = 0). In this mode the counting direction is always up (incrementing), and no counter clear is performed. The counter simply overruns when it passes its maximum 8-bit value (TOP = 0xFF) and then restarts from the bottom (0x00). In normal operation the Timer/Counter Overflow Flag (TOV0) will be set in the same timer clock cycle as the TCNT0 becomes zero. The TOV0 Flag in this case behaves like a ninth bit, except that it is only set, not cleared. However, combined with the timer overflow interrupt that automatically clears the TOV0 Flag, the timer resolution can be increased by software. There are no special cases to consider in the Normal mode, a new counter value can be written anytime.

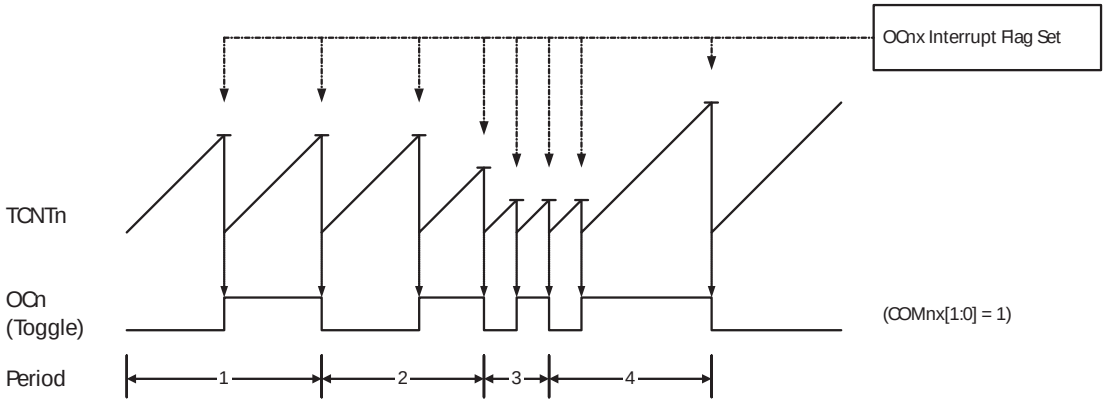
The Output Compare Unit can be used to generate interrupts at some given time. Using the Output Compare to generate waveforms in Normal mode is not recommended, since this will occupy too much of the CPU time.

11.7.2 Clear Timer on Compare Match (CTC) Mode

In Clear Timer on Compare or CTC mode (WGM0[2:0] = 2), the OCR0A Register is used to manipulate the counter resolution. In CTC mode the counter is cleared to zero when the counter value (TCNT0) matches the OCR0A. The OCR0A defines the top value for the counter, hence also its resolution. This mode allows greater control of the Compare Match output frequency. It also simplifies the operation of counting external events.

The timing diagram for the CTC mode is shown in Figure 11-7. The counter value (TCNT0) increases until a Compare Match occurs between TCNT0 and OCR0A, and then counter (TCNT0) is cleared.

Figure 11-7. CTC Mode, Timing Diagram



An interrupt can be generated each time the counter value reaches the TOP value by using the OCF0A Flag. If the interrupt is enabled, the interrupt handler routine can be used for updating the TOP value. However, changing TOP to a value close to BOTTOM when the counter is running with none or a low prescaler value must be done with care since the CTC mode does not have the double buffering feature. If the new value written to OCR0A is lower than the current value of TCNT0, the counter will miss the Compare Match. The counter will then have to count to its maximum value (0xFF) and wrap around starting at 0x00 before the Compare Match can occur.

For generating a waveform output in CTC mode, the OC0A output can be set to toggle its logical level on each Compare Match by setting the Compare Output mode bits to toggle mode (COM0A[1:0] = 1). The OC0A value will not be visible on the port pin unless the data direction for the pin is set to output. The waveform generated will have a maximum frequency of $f_{OC0} = f_{clk_I/O}/2$ when OCR0A is set to zero (0x00). The waveform frequency is defined by the following equation:

$$f_{OCnx} = \frac{f_{clk_I/O}}{2 \cdot N \cdot (1 + OCRnx)}$$

The *N* variable represents the prescale factor (1, 8, 64, 256, or 1024).

As for the Normal mode of operation, the TOV0 Flag is set in the same timer clock cycle that the counter counts from MAX to 0x00.

11.7.3 Fast PWM Mode

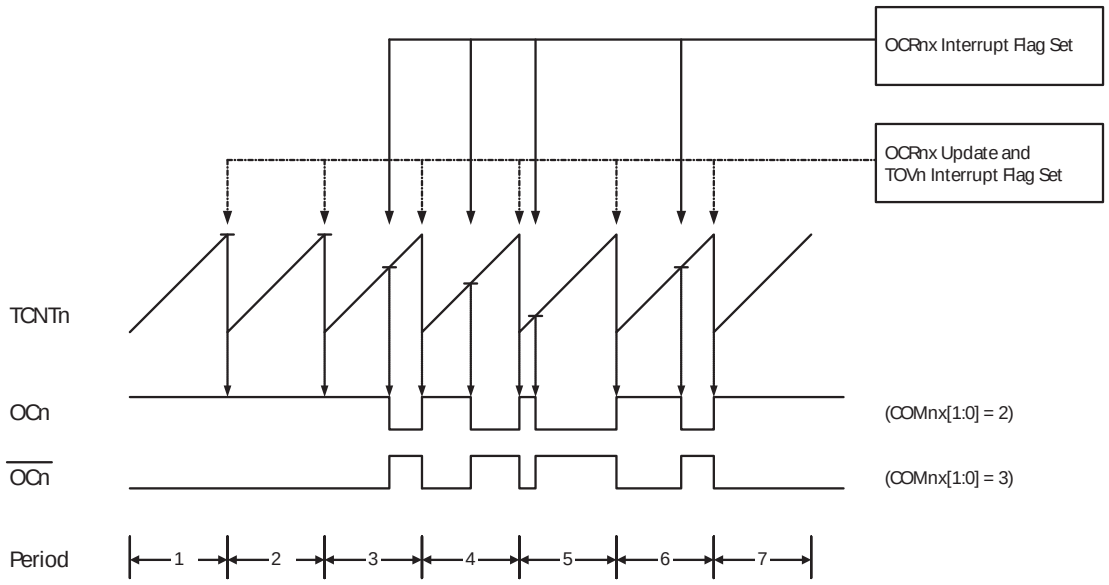
The fast Pulse Width Modulation or fast PWM mode (WGM0[2:0] = 3 or 7) provides a high frequency PWM waveform generation option. The fast PWM differs from the other PWM option by its single-slope operation. The counter counts from BOTTOM to TOP then restarts from BOTTOM. TOP is defined as 0xFF when WGM0[2:0] = 3, and OCR0A when WGM0[2:0] = 7.

In non-inverting Compare Output mode, the Output Compare (OC0x) is cleared on the Compare Match between TCNT0 and OCR0x, and set at BOTTOM. In inverting Compare Output mode, the output is set on Compare Match and cleared at BOTTOM.

Due to the single-slope operation, the operating frequency of the fast PWM mode can be twice as high as the phase correct PWM mode that use dual-slope operation. This high frequency makes the fast PWM mode well suited for power regulation, rectification, and DAC applications. High frequency allows physically small sized external components (coils, capacitors), and therefore reduces total system cost.

In fast PWM mode, the counter is incremented until the counter value matches the TOP value. The counter is then cleared at the following timer clock cycle. The timing diagram for the fast PWM mode is shown in Figure 11-8. The TCNT0 value is in the timing diagram shown as a histogram for illustrating the single-slope operation. The diagram includes non-inverted and inverted PWM outputs. The small horizontal line marks on the TCNT0 slopes represent Compare Matches between OCR0x and TCNT0.

Figure 11-8. Fast PWM Mode, Timing Diagram



The Timer/Counter Overflow Flag (TOV0) is set each time the counter reaches TOP. If the interrupt is enabled, the interrupt handler routine can be used for updating the compare value.

In fast PWM mode, the compare unit allows generation of PWM waveforms on the OC0x pins. Setting the COM0x[1:0] bits to two will produce a non-inverted PWM and an inverted PWM output can be generated by setting the COM0x[1:0] to three: Setting the COM0A[1:0] bits to one allowes the AC0A pin to toggle on Compare Matches if the WGM02 bit is set. This option is not available for the OC0B pin (See Table 11-3 on page 78). The actual OC0x value will only be visible on the port pin if the data direction for the port pin is set as output. The PWM wave-

form is generated by setting (or clearing) the OC0x Register at the Compare Match between OCR0x and TCNT0, and clearing (or setting) the OC0x Register at the timer clock cycle the counter is cleared (changes from TOP to BOTTOM).

The PWM frequency for the output can be calculated by the following equation:

$$f_{OCnxPWM} = \frac{f_{clk_I/O}}{N \cdot 256}$$

The *N* variable represents the prescale factor (1, 8, 64, 256, or 1024).

The extreme values for the OCR0A Register represents special cases when generating a PWM waveform output in the fast PWM mode. If the OCR0A is set equal to BOTTOM, the output will be a narrow spike for each MAX+1 timer clock cycle. Setting the OCR0A equal to MAX will result in a constantly high or low output (depending on the polarity of the output set by the COM0A[1:0] bits.)

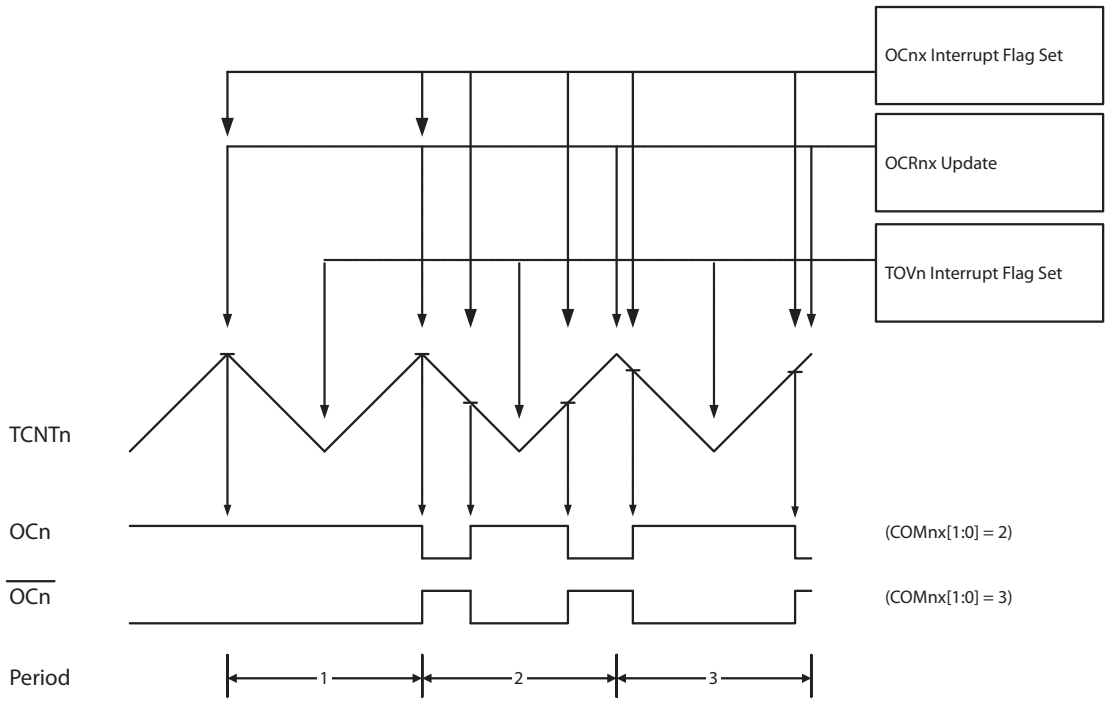
A frequency (with 50% duty cycle) waveform output in fast PWM mode can be achieved by setting OC0x to toggle its logical level on each Compare Match (COM0x[1:0] = 1). The waveform generated will have a maximum frequency of $f_{OC0} = f_{clk_I/O}/2$ when OCR0A is set to zero. This feature is similar to the OC0A toggle in CTC mode, except the double buffer feature of the Output Compare unit is enabled in the fast PWM mode.

11.7.4 Phase Correct PWM Mode

The phase correct PWM mode (WGM0[2:0] = 1 or 5) provides a high resolution phase correct PWM waveform generation option. The phase correct PWM mode is based on a dual-slope operation. The counter counts repeatedly from BOTTOM to TOP and then from TOP to BOTTOM. TOP is defined as 0xFF when WGM0[2:0] = 1, and OCR0A when WGM0[2:0] = 5. In non-inverting Compare Output mode, the Output Compare (OC0x) is cleared on the Compare Match between TCNT0 and OCR0x while upcounting, and set on the Compare Match while downcounting. In inverting Output Compare mode, the operation is inverted. The dual-slope operation has lower maximum operation frequency than single slope operation. However, due to the symmetric feature of the dual-slope PWM modes, these modes are preferred for motor control applications.

In phase correct PWM mode the counter is incremented until the counter value matches TOP. When the counter reaches TOP, it changes the count direction. The TCNT0 value will be equal to TOP for one timer clock cycle. The timing diagram for the phase correct PWM mode is shown on Figure 11-9. The TCNT0 value is in the timing diagram shown as a histogram for illustrating the dual-slope operation. The diagram includes non-inverted and inverted PWM outputs. The small horizontal line marks on the TCNT0 slopes represent Compare Matches between OCR0x and TCNT0.

Figure 11-9. Phase Correct PWM Mode, Timing Diagram



The Timer/Counter Overflow Flag (TOV0) is set each time the counter reaches BOTTOM. The Interrupt Flag can be used to generate an interrupt each time the counter reaches the BOTTOM value.

In phase correct PWM mode, the compare unit allows generation of PWM waveforms on the OC0x pins. Setting the COM0x[1:0] bits to two will produce a non-inverted PWM. An inverted PWM output can be generated by setting the COM0x[1:0] to three: Setting the COM0A0 bits to one allows the OC0A pin to toggle on Compare Matches if the WGM02 bit is set. This option is not available for the OC0B pin (See Table 11-4 on page 78). The actual OC0x value will only be visible on the port pin if the data direction for the port pin is set as output. The PWM waveform is generated by clearing (or setting) the OC0x Register at the Compare Match between OCR0x and TCNT0 when the counter increments, and setting (or clearing) the OC0x Register at Compare Match between OCR0x and TCNT0 when the counter decrements. The PWM frequency for the output when using phase correct PWM can be calculated by the following equation:

$$f_{OCnxPCPWM} = \frac{f_{clk_I/O}}{N \cdot 510}$$

The N variable represents the prescale factor (1, 8, 64, 256, or 1024).

The extreme values for the OCR0A Register represent special cases when generating a PWM waveform output in the phase correct PWM mode. If the OCR0A is set equal to BOTTOM, the output will be continuously low and if set equal to MAX the output will be continuously high for non-inverted PWM mode. For inverted PWM the output will have the opposite logic values.

At the very start of period 2 in Figure 11-9 OCn has a transition from high to low even though there is no Compare Match. The point of this transition is to guaratee symmetry around BOTTOM. There are two cases that give a transition without Compare Match, as follows:

- OCR0A changes its value from MAX, like in Figure 11-9. When the OCR0A value is MAX the OCn pin value is the same as the result of a down-counting Compare Match. To ensure symmetry around BOTTOM the OCn value at MAX must correspond to the result of an up-counting Compare Match.

- The timer starts counting from a value higher than the one in OCR0A, and for that reason misses the Compare Match and hence the OCn change that would have happened on the way up.

11.8 Timer/Counter Timing Diagrams

The Timer/Counter is a synchronous design and the timer clock (clk_{T0}) is therefore shown as a clock enable signal in the following figures. The figures include information on when Interrupt Flags are set. Figure 11-10 contains timing data for basic Timer/Counter operation. The figure shows the count sequence close to the MAX value in all modes other than phase correct PWM mode.

Figure 11-10. Timer/Counter Timing Diagram, no Prescaling

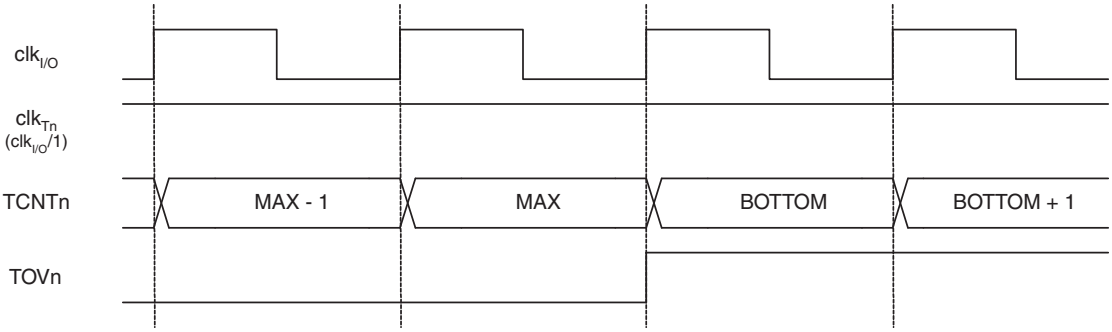


Figure 11-11 shows the same timing data, but with the prescaler enabled.

Figure 11-11. Timer/Counter Timing Diagram, with Prescaler ($f_{clk_I/O}/8$)

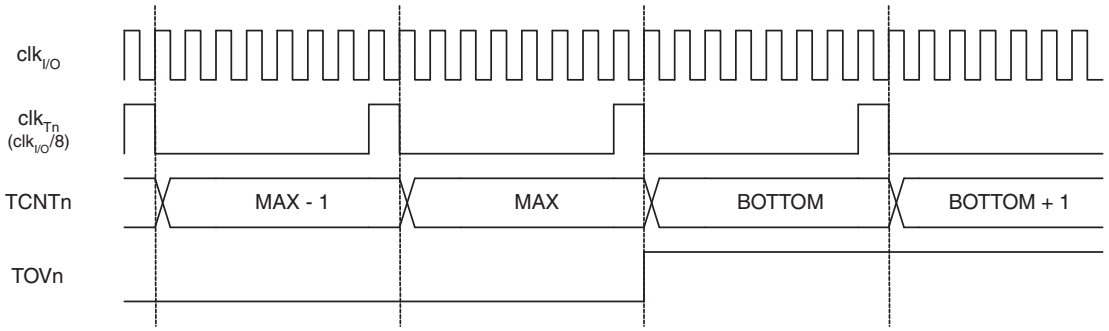


Figure 11-12 shows the setting of OCF0B in all modes and OCF0A in all modes except CTC mode and PWM mode, where OCR0A is TOP.

Figure 11-12. Timer/Counter Timing Diagram, Setting of OCF0x, with Prescaler ($f_{clk_I/O}/8$)

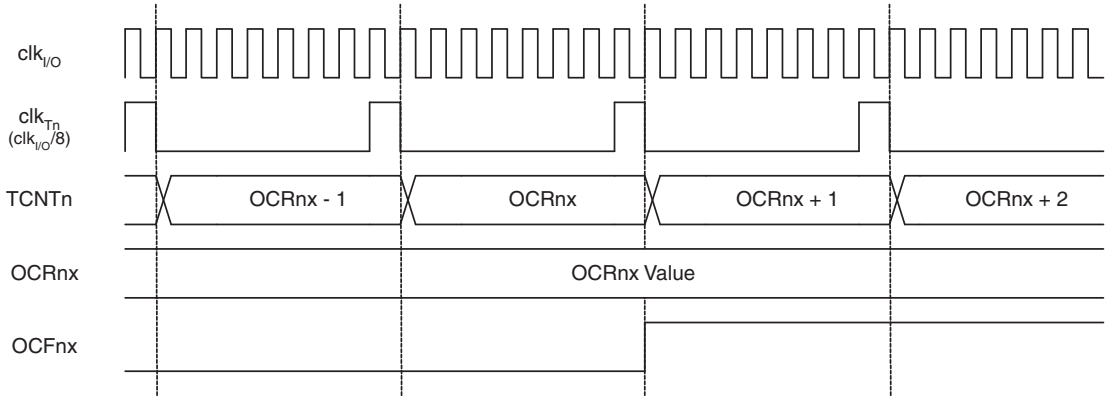
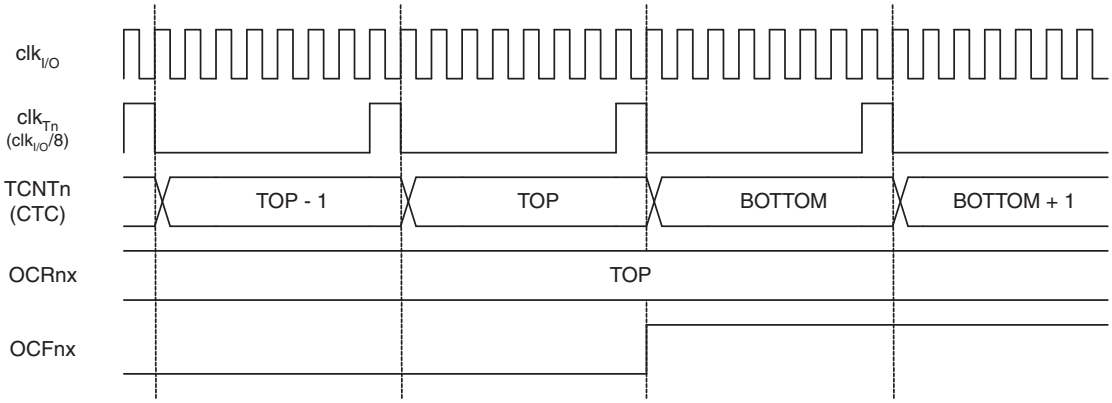


Figure 11-13 shows the setting of OCF0A and the clearing of TCNT0 in CTC mode and fast PWM mode where OCR0A is TOP.

Figure 11-13. Timer/Counter Timing Diagram, Clear Timer on Compare Match mode, with Prescaler ($f_{clk_I/O}/8$)



11.9 Register Description

11.9.1 GTCCR – General Timer/Counter Control Register

Bit	7	6	5	4	3	2	1	0	
0x2C	TSM	PWM1B	COM1B1	COM1B0	FOC1B	FOC1A	PSR1	PSR0	GTCCR
Read/Write	R/W	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – TSM: Timer/Counter Synchronization Mode**

Writing the TSM bit to one activates the Timer/Counter Synchronization Mode. In this mode, the value written to PSR0 is kept, hence keeping the Prescaler Reset signal asserted. This ensures that the timer/counter is halted and can be configured without the risk of advancing during configuration. When the TSM bit is written to zero, the PSR0 bit is cleared by hardware, and the timer/counter start counting.

- **Bit 0 – PSR0: Prescaler Reset Timer/Counter0**

When this bit is one, the Timer/Counter0 prescaler will be Reset. This bit is normally cleared immediately by hardware, except if the TSM bit is set.

11.9.2 TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x2A	COM0A**1**	COM0A**0**	COM0B**1**	COM0B**0**	–	–	**WGM01**	**WGM00**	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:6 – COM0A[1:0]: Compare Match Output A Mode**

- **Bits 5:4 – COM0B[1:0]: Compare Match Output B Mode**

The COM0A[1:0] and COM0B[1:0] bits control the behaviour of Output Compare pins OC0A and OC0B, respectively. If any of the COM0A[1:0] bits are set, the OC0A output overrides the normal port functionality of the I/O pin it is connected to. Similarly, if any of the COM0B[1:0] bits are set, the OC0B output overrides the normal port functionality of the I/O pin it is connected to. However, note that the Data Direction Register (DDR) bit corresponding to the OC0A and OC0B pins must be set in order to enable the output driver.

When OC0A/OC0B is connected to the I/O pin, the function of the COM0A[1:0]/COM0B[1:0] bits depend on the WGM0[2:0] bit setting. Table 11-2 shows the COM0x[1:0] bit functionality when the WGM0[2:0] bits are set to a normal or CTC mode (non-PWM).

Table 11-2. Compare Output Mode, non-PWM Mode

COM0A1 COM0B1	COM0A0 COM0B0	Description
0	0	Normal port operation, OC0A/OC0B disconnected.
0	1	Toggle OC0A/OC0B on Compare Match
1	0	Clear OC0A/OC0B on Compare Match
1	1	Set OC0A/OC0B on Compare Match

Table 11-3 shows the COM0x[1:0] bit functionality when the WGM0[2:0] bits are set to fast PWM mode.

Table 11-3. Compare Output Mode, Fast PWM Mode(1)

COM0A1 COM0B1	COM0A0 COM0B0	Description
0	0	Normal port operation, OC0A/OC0B disconnected.
0	1	Reserved
1	0	Clear OC0A/OC0B on Compare Match, set OC0A/OC0B at BOTTOM (non-inverting mode)
1	1	Set OC0A/OC0B on Compare Match, clear OC0A/OC0B at BOTTOM (inverting mode)

Note: 1. A special case occurs when OCR0A or OCR0B equals TOP and COM0A1/COM0B1 is set. In this case, the compare match is ignored, but the set or clear is done at BOTTOM. See “Fast PWM Mode” on page 73 for more details.

Table 11-4 shows the COM0x[1:0] bit functionality when the WGM0[2:0] bits are set to phase correct PWM mode.

Table 11-4. Compare Output Mode, Phase Correct PWM Mode(1)

COM0A1 COM0B1	COM0A0 COM0B0	Description
0	0	Normal port operation, OC0A/OC0B disconnected.
0	1	Reserved
1	0	Clear OC0A/OC0B on Compare Match when up-counting. Set OC0A/OC0B on Compare Match when down-counting.
1	1	Set OC0A/OC0B on Compare Match when up-counting. Clear OC0A/OC0B on Compare Match when down-counting.

Note: 1. A special case occurs when OCR0A or OCR0B equals TOP and COM0A1/COM0B1 is set. In this case, the Compare Match is ignored, but the set or clear is done at TOP. See “Phase Correct PWM Mode” on page 74 for more details.

- **Bits 3:2 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bits 1:0 – WGM0[1:0]: Waveform Generation Mode**

Combined with the WGM02 bit found in the TCCR0B Register, these bits control the counting sequence of the counter, the source for maximum (TOP) counter value, and what type of waveform generation to be used, see Table 11-5. Modes of operation supported by the Timer/Counter unit are: Normal mode (counter), Clear Timer on Compare Match (CTC) mode, and two types of Pulse Width Modulation (PWM) modes (see “Modes of Operation” on page 71).

Table 11-5. Waveform Generation Mode Bit Description

Mode	WGM 02	WGM 01	WGM 00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on
0	0	0	0	Normal	0xFF	Immediate	MAX(1)
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM(2)
2	0	1	0	CTC	OCRA	Immediate	MAX(1)
3	0	1	1	Fast PWM	0xFF	BOTTOM(2)	MAX(1)
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM(2)
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM(2)	TOP

Notes: 1. MAX = 0xFF
2. BOTTOM = 0x00

11.9.3 TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x33	FOC0A	FOC0B	–	–	**WGM02**	**CS02**	**CS01**	**CS00**	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – FOC0A: Force Output Compare A**

The FOC0A bit is only active when the WGM bits specify a non-PWM mode.

However, for ensuring compatibility with future devices, this bit must be set to zero when TCCR0B is written when operating in PWM mode. When writing a logical one to the FOC0A bit, an immediate Compare Match is forced on the Waveform Generation unit. The OC0A output is changed according to its COM0A[1:0] bits setting. Note that the FOC0A bit is implemented as a strobe. Therefore it is the value present in the COM0A[1:0] bits that determines the effect of the forced compare.

A FOC0A strobe will not generate any interrupt, nor will it clear the timer in CTC mode using OCR0A as TOP.

The FOC0A bit is always read as zero.

- **Bit 6 – FOC0B: Force Output Compare B**

The FOC0B bit is only active when the WGM bits specify a non-PWM mode.

However, for ensuring compatibility with future devices, this bit must be set to zero when TCCR0B is written when operating in PWM mode. When writing a logical one to the FOC0B bit, an immediate Compare Match is forced on the Waveform Generation unit. The OC0B output is changed according to its COM0B[1:0] bits setting. Note that the FOC0B bit is implemented as a strobe. Therefore it is the value present in the COM0B[1:0] bits that determines the effect of the forced compare.

A FOC0B strobe will not generate any interrupt, nor will it clear the timer in CTC mode using OCR0B as TOP.

The FOC0B bit is always read as zero.

- **Bits 5:4 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bit 3 – WGM02: Waveform Generation Mode**

See the description in the “TCCR0A – Timer/Counter Control Register A” on page 77.

- **Bits 2:0 – CS0[2:0]: Clock Select**

The three Clock Select bits select the clock source to be used by the Timer/Counter.

Table 11-6. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

If external pin modes are used for the Timer/Counter0, transitions on the T0 pin will clock the counter even if the pin is configured as an output. This feature allows software control of the counting.

11.9.4 TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x32	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Timer/Counter Register gives direct access, both for read and write operations, to the Timer/Counter unit 8-bit counter. Writing to the TCNT0 Register blocks (removes) the Compare Match on the following timer clock. Modifying the counter (TCNT0) while the counter is running, introduces a risk of missing a Compare Match between TCNT0 and the OCR0x Registers.

11.9.5 OCR0A – Output Compare Register A

Bit	7	6	5	4	3	2	1	0	
0x29	OCR0A[7:0]								OCR0A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register A contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0A pin.

11.9.6 OCR0B – Output Compare Register B

Bit	7	6	5	4	3	2	1	0	
0x28	OCR0B[7:0]								OCR0B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register B contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0B pin.

11.9.7 TIMSK – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x39	–	OCIE1A	OCIE1B	OCIE0A	OCIE0B	TOIE1	TOIE0	–	TIMSK
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 4 – OCIE0A: Timer/Counter0 Output Compare Match A Interrupt Enable**

When the OCIE0A bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Compare Match A interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter0 occurs, i.e., when the OCF0A bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

- **Bit 3 – OCIE0B: Timer/Counter Output Compare Match B Interrupt Enable**

When the OCIE0B bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter Compare Match B interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter occurs, i.e., when the OCF0B bit is set in the Timer/Counter Interrupt Flag Register – TIFR0.

- **Bit 1 – TOIE0: Timer/Counter0 Overflow Interrupt Enable**

When the TOIE0 bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Overflow interrupt is enabled. The corresponding interrupt is executed if an overflow in Timer/Counter0 occurs, i.e., when the TOV0 bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

11.9.8 TIFR – Timer/Counter Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x38	–	OCF1A	OCF1B	OCF0A	OCF0B	TOV1	TOV0	–	TIFR
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 4 – OCF0A: Output Compare Flag 0 A**

The OCF0A bit is set when a Compare Match occurs between the Timer/Counter0 and the data in OCR0A – Output Compare Register0. OCF0A is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0A is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0A (Timer/Counter0 Compare Match Interrupt Enable), and OCF0A are set, the Timer/Counter0 Compare Match Interrupt is executed.

- **Bit 3 – OCF0B: Output Compare Flag 0 B**

The OCF0B bit is set when a Compare Match occurs between the Timer/Counter and the data in OCR0B – Output Compare Register0 B. OCF0B is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0B is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0B (Timer/Counter Compare B Match Interrupt Enable), and OCF0B are set, the Timer/Counter Compare Match Interrupt is executed.

- **Bit 1 – TOV0: Timer/Counter0 Overflow Flag**

The bit TOV0 is set when an overflow occurs in Timer/Counter0. TOV0 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, TOV0 is cleared by writing a logic one to the flag. When the SREG I-bit, TOIE0 (Timer/Counter0 Overflow Interrupt Enable), and TOV0 are set, the Timer/Counter0 Overflow interrupt is executed.

The setting of this flag is dependent of the WGM0[2:0] bit setting. Refer to Table 11-5, “Waveform Generation Mode Bit Description” on page 79.

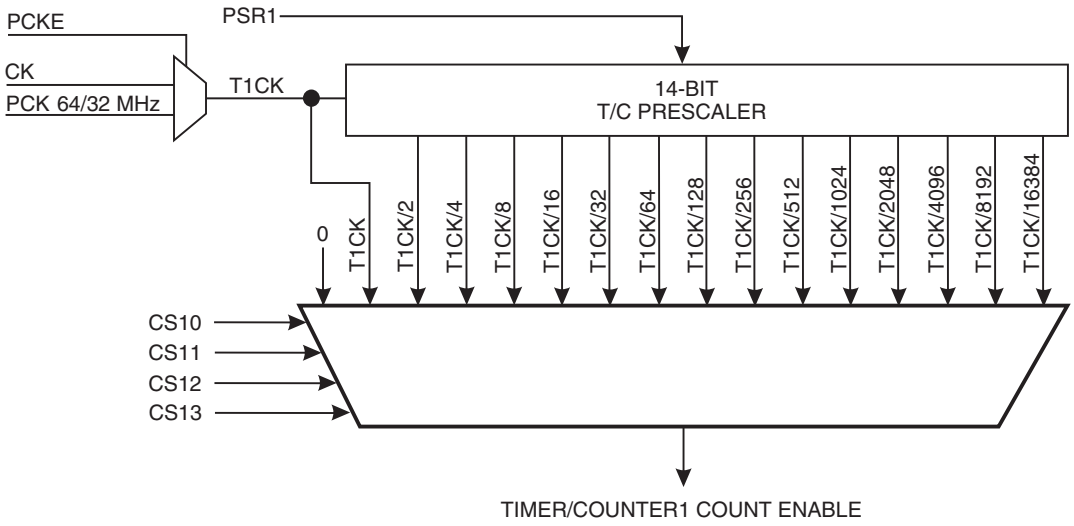
12. 8-bit Timer/Counter1

The Timer/Counter1 is a general purpose 8-bit Timer/Counter module that has a separate prescaling selection from the separate prescaler.

12.1 Timer/Counter1 Prescaler

Figure 12-1 shows the Timer/Counter1 prescaler that supports two clocking modes, a synchronous clocking mode and an asynchronous clocking mode. The synchronous clocking mode uses the system clock (CK) as the clock timebase and asynchronous mode uses the fast peripheral clock (PCK) as the clock time base. The PCKE bit from the PLLCSR register enables the asynchronous mode when it is set (‘1’).

Figure 12-1. Timer/Counter1 Prescaler



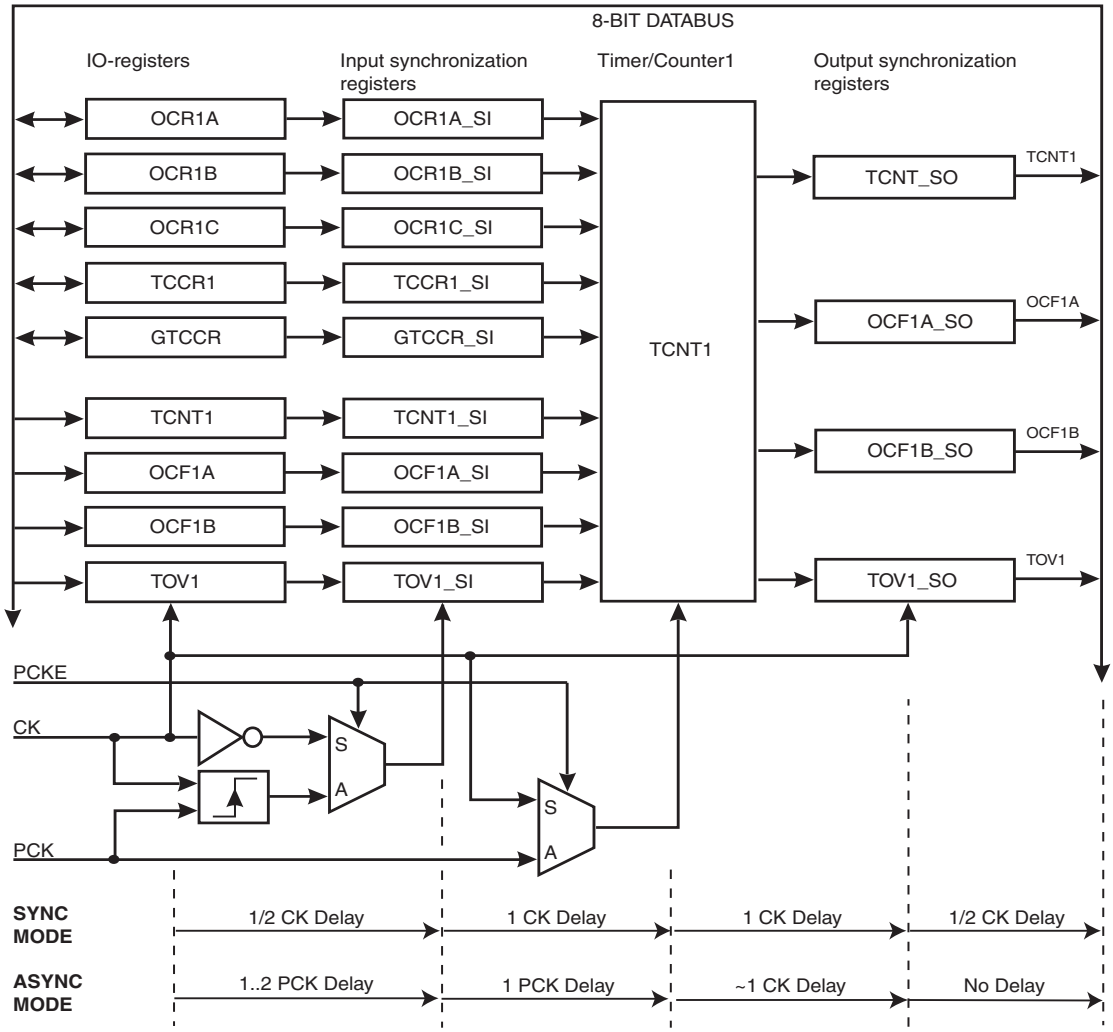
In the asynchronous clocking mode the clock selections are from PCK to PCK/16384 and stop, and in the synchronous clocking mode the clock selections are from CK to CK/16384 and stop. The clock options are described in Table 12-5 on page 89 and the Timer/Counter1 Control Register, TCCR1. Setting the PSR1 bit in GTCCR register resets the prescaler. The PCKE bit in the PLLCSR register enables the asynchronous mode. The frequency of the fast peripheral clock is 64 MHz (or 32 MHz in Low Speed Mode).

12.2 Counter and Compare Units

The Timer/Counter1 general operation is described in the asynchronous mode and the operation in the synchronous mode is mentioned only if there are differences between these two modes. Figure 12-2 shows Timer/Counter 1 synchronization register block diagram and synchronization delays in between registers. Note that all clock gating details are not shown in the figure. The Timer/Counter1 register values go through the internal synchronization registers, which cause the input synchronization delay, before affecting the counter operation. The registers TCCR1, GTCCR, OCR1A, OCR1B, and OCR1C can be read back right after writing the register. The read back values are delayed for the Timer/Counter1 (TCNT1) register and flags (OCF1A, OCF1B, and TOV1), because of the input and output synchronization.

The Timer/Counter1 features a high resolution and a high accuracy usage with the lower prescaling opportunities. It can also support two accurate, high speed, 8-bit Pulse Width Modulators using clock speeds up to 64 MHz (or 32 MHz in Low Speed Mode). In this mode, Timer/Counter1 and the output compare registers serve as dual standalone PWMs with non-overlapping non-inverted and inverted outputs. Refer to page 86 for a detailed description on this function. Similarly, the high prescaling opportunities make this unit useful for lower speed functions or exact timing functions with infrequent actions.

Figure 12-2. Timer/Counter 1 Synchronization Register Block Diagram.

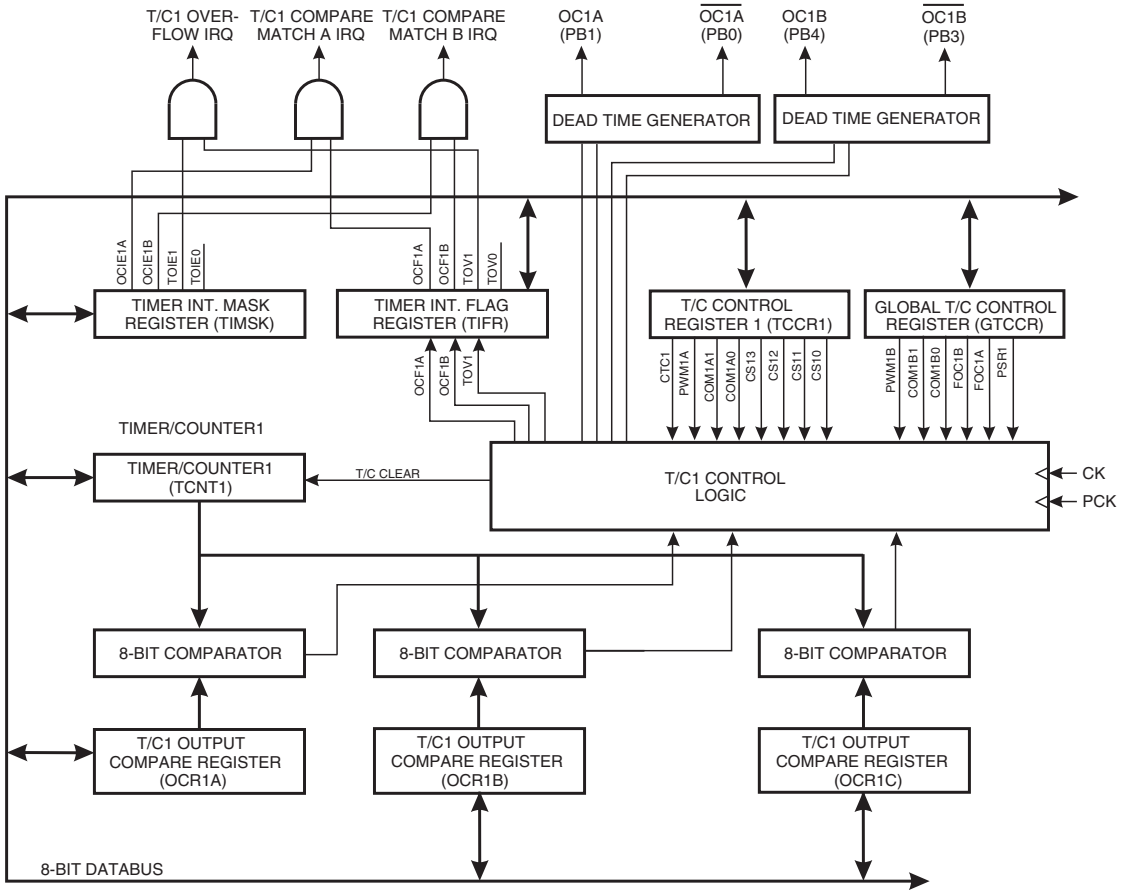


Timer/Counter1 and the prescaler allow running the CPU from any clock source while the prescaler is operating on the fast 64 MHz (or 32 MHz in Low Speed Mode) PCK clock in the asynchronous mode.

Note that the system clock frequency must be lower than one third of the PCK frequency. The synchronization mechanism of the asynchronous Timer/Counter1 needs at least two edges of the PCK when the system clock is high. If the frequency of the system clock is too high, it is a risk that data or control values are lost.

The following Figure 12-3 shows the block diagram for Timer/Counter1.

Figure 12-3. Timer/Counter1 Block Diagram



Three status flags (overflow and compare matches) are found in the Timer/Counter Interrupt Flag Register - TIFR. Control signals are found in the Timer/Counter Control Registers TCCR1 and GTCCR. The interrupt enable/disable settings are found in the Timer/Counter Interrupt Mask Register - TIMSK.

The Timer/Counter1 contains three Output Compare Registers, OCR1A, OCR1B, and OCR1C as the data source to be compared with the Timer/Counter1 contents. In normal mode the Output Compare functions are operational with all three output compare registers. OCR1A determines action on the OC1A pin (PB1), and it can generate Timer1 OC1A interrupt in normal mode and in PWM mode. Likewise, OCR1B determines action on the OC1B pin (PB4) and it can generate Timer1 OC1B interrupt in normal mode and in PWM mode. OCR1C holds the Timer/Counter maximum value, i.e. the clear on compare match value. In the normal mode an overflow interrupt (TOV1) is generated when Timer/Counter1 counts from $FF to $00, while in the PWM mode the overflow interrupt is generated when Timer/Counter1 counts either from $FF to $00 or from OCR1C to $00. The inverted PWM outputs $\overline{OC1A}$ and $\overline{OC1B}$ are not connected in normal mode.

In PWM mode, OCR1A and OCR1B provide the data values against which the Timer Counter value is compared. Upon compare match the PWM outputs (OC1A, $\overline{OC1A}$, OC1B, $\overline{OC1B}$) are generated. In PWM mode, the Timer Counter counts up to the value specified in the output compare register OCR1C and starts again from $00. This feature allows limiting the counter “full” value to a specified value, lower than $FF. Together with the many prescaler options, flexible PWM frequency selection is provided. Table 12-3 on page 88 lists clock selection and OCR1C values to obtain PWM frequencies from 20 kHz to 250 kHz in 10 kHz steps and from 250 kHz to 500 kHz in 50 kHz steps. Higher PWM frequencies can be obtained at the expense of resolution.

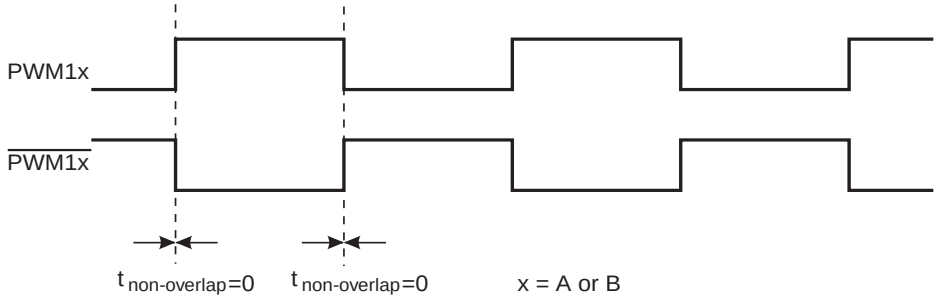
12.2.1 Timer/Counter1 Initialization for Asynchronous Mode

To set Timer/Counter1 in asynchronous mode first enable PLL and then wait 100 µs for PLL to stabilize. Next, poll the PLOCK bit until it is set and then set the PCKE bit.

12.2.2 Timer/Counter1 in PWM Mode

When the PWM mode is selected, Timer/Counter1 and the Output Compare Register C - OCR1C form a dual 8-bit, free-running and glitch-free PWM generator with outputs on the PB1(OC1A) and PB4(OC1B) pins and inverted outputs on pins PB0($\overline{OC1A}$) and PB3($\overline{OC1B}$). As default non-overlapping times for complementary output pairs are zero, but they can be inserted using a Dead Time Generator (see description on page 100).

Figure 12-4. The PWM Output Pair



When the counter value match the contents of OCR1A or OCR1B, the OC1A and OC1B outputs are set or cleared according to the COM1A1/COM1A0 or COM1B1/COM1B0 bits in the Timer/Counter1 Control Register A - TCCR1, as shown in Table 12-1.

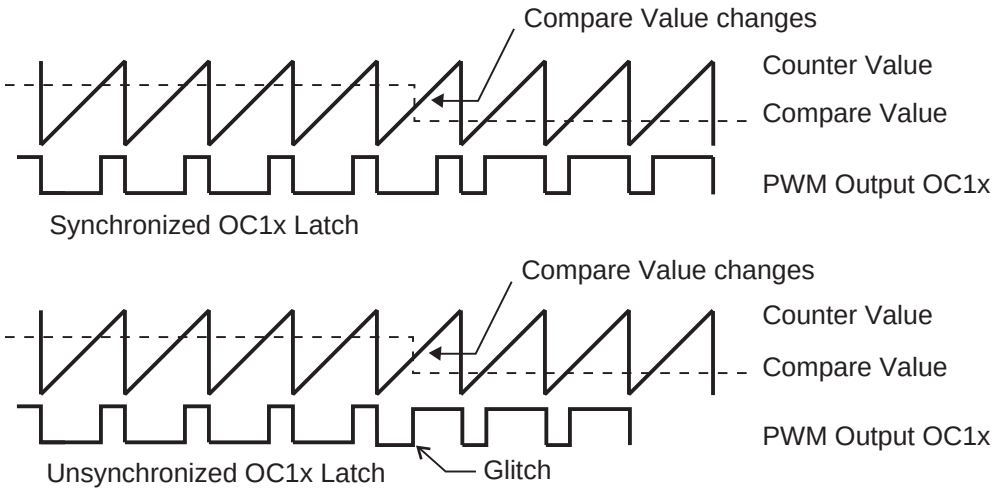
Timer/Counter1 acts as an up-counter, counting from $00 up to the value specified in the output compare register OCR1C, and starting from $00 up again. A compare match with OC1C will set an overflow interrupt flag (TOV1) after a synchronization delay following the compare event.

Table 12-1. Compare Mode Select in PWM Mode

COM1x1	COM1x0	Effect on Output Compare Pins
0	0	OC1x not connected. $\overline{OC1x}$ not connected.
0	1	OC1x cleared on compare match. Set whenTCNT1 = $00. $\overline{OC1x}$ set on compare match. Cleared when TCNT1 = $00.
1	0	OC1x cleared on compare match. Set when TCNT1 = $00. $\overline{OC1x}$ not connected.
1	1	OC1x Set on compare match. Cleared when TCNT1= $00. $\overline{OC1x}$ not connected.

Note that in PWM mode, writing to the Output Compare Registers OCR1A or OCR1B, the data value is first transferred to a temporary location. The value is latched into OCR1A or OCR1B when the Timer/Counter reaches OCR1C. This prevents the occurrence of odd-length PWM pulses (glitches) in the event of an unsynchronized OCR1A or OCR1B. See Figure 12-5 for an example.

Figure 12-5. Effects of Unsynchronized OCR Latching



During the time between the write and the latch operation, a read from OCR1A or OCR1B will read the contents of the temporary location. This means that the most recently written value always will read out of OCR1A or OCR1B.

When OCR1A or OCR1B contain $00 or the top value, as specified in OCR1C register, the output PB1(OC1A) or PB4(OC1B) is held low or high according to the settings of COM1A1/COM1A0. This is shown in Table 12-2.

Table 12-2. PWM Outputs OCR1x = $00 or OCR1C, x = A or B

COM1x1	COM1x0	OCR1x	Output OC1x	Output $\overline{OC1x}$
0	1	$00	L	H
0	1	OCR1C	H	L
1	0	$00	L	Not connected.
1	0	OCR1C	H	Not connected.
1	1	$00	H	Not connected.
1	1	OCR1C	L	Not connected.

In PWM mode, the Timer Overflow Flag - TOV1 is set when the TCNT1 counts to the OCR1C value and the TCNT1 is reset to $00. The Timer Overflow Interrupt1 is executed when TOV1 is set provided that Timer Overflow Interrupt and global interrupts are enabled. This also applies to the Timer Output Compare flags and interrupts.

The frequency of the PWM will be Timer Clock 1 Frequency divided by (OCR1C value + 1). See the following equation:

$$f_{PWM} = \frac{f_{TCK1}}{(OCR1C + 1)}$$

Resolution shows how many bits are required to express the value in the OCR1C register and can be calculated using the following equation:

$$R = \log 2(OCR1C + 1)$$

Table 12-3. Timer/Counter1 Clock Prescale Select in the Asynchronous Mode

PWM Frequency	Clock Selection	CS1[3:0]	OCR1C	RESOLUTION
20 kHz	PCK/16	0101	199	7.6
30 kHz	PCK/16	0101	132	7.1
40 kHz	PCK/8	0100	199	7.6
50 kHz	PCK/8	0100	159	7.3
60 kHz	PCK/8	0100	132	7.1
70 kHz	PCK/4	0011	228	7.8
80 kHz	PCK/4	0011	199	7.6
90 kHz	PCK/4	0011	177	7.5
100 kHz	PCK/4	0011	159	7.3
110 kHz	PCK/4	0011	144	7.2
120 kHz	PCK/4	0011	132	7.1
130 kHz	PCK/2	0010	245	7.9
140 kHz	PCK/2	0010	228	7.8
150 kHz	PCK/2	0010	212	7.7
160 kHz	PCK/2	0010	199	7.6
170 kHz	PCK/2	0010	187	7.6
180 kHz	PCK/2	0010	177	7.5
190 kHz	PCK/2	0010	167	7.4
200 kHz	PCK/2	0010	159	7.3
250 kHz	PCK	0001	255	8.0
300 kHz	PCK	0001	212	7.7
350 kHz	PCK	0001	182	7.5
400 kHz	PCK	0001	159	7.3
450 kHz	PCK	0001	141	7.1
500 kHz	PCK	0001	127	7.0

12.3 Register Description

12.3.1 TCCR1 – Timer/Counter1 Control Register

Bit	7	6	5	4	3	2	1	0	
0x30	CTC1	PWM1A	COM1A1	COM1A0	CS13	CS12	CS11	CS10	TCCR1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – CTC1 : Clear Timer/Counter on Compare Match**

When the CTC1 control bit is set (one), Timer/Counter1 is reset to $00 in the CPU clock cycle after a compare match with OCR1C register value. If the control bit is cleared, Timer/Counter1 continues counting and is unaffected by a compare match.

- **Bit 6 – PWM1A: Pulse Width Modulator A Enable**

When set (one) this bit enables PWM mode based on comparator OCR1A in Timer/Counter1 and the counter value is reset to $00 in the CPU clock cycle after a compare match with OCR1C register value.

- **Bits 5:4 – COM1A[1:0]: Comparator A Output Mode, Bits 1 and 0**

The COM1A1 and COM1A0 control bits determine any output pin action following a compare match with compare register A in Timer/Counter1. Since the output pin action is an alternative function to an I/O port, the corresponding direction control bit must be set (one) in order to control an output pin.

In Normal mode, the COM1A1 and COM1A0 control bits determine the output pin actions that affect pin PB1 (OC1A) as described in Table 12-4. Note that $\overline{\text{OC1A}}$ is not connected in normal mode.

Table 12-4. Comparator A Mode Select in Normal Mode

COM1A1	COM1A0	Description
0	0	Timer/Counter Comparator A disconnected from output pin OC1A.
0	1	Toggle the OC1A output line.
1	0	Clear the OC1A output line.
1	1	Set the OC1A output line

In PWM mode, these bits have different functions. Refer to Table 12-1 on page 86 for a detailed description.

- **Bits 3:0 - CS1[3:0]: Clock Select Bits 3, 2, 1, and 0**

The Clock Select bits 3, 2, 1, and 0 define the prescaling source of Timer/Counter1.

Table 12-5. Timer/Counter1 Prescale Select

CS13	CS12	CS11	CS10	Asynchronous Clocking Mode	Synchronous Clocking Mode
0	0	0	0	T/C1 stopped	T/C1 stopped
0	0	0	1	PCK	CK
0	0	1	0	PCK/2	CK/2
0	0	1	1	PCK/4	CK/4
0	1	0	0	PCK/8	CK/8
0	1	0	1	PCK/16	CK/16
0	1	1	0	PCK/32	CK/32

Table 12-5. Timer/Counter1 Prescale Select (Continued)

CS13	CS12	CS11	CS10	Asynchronous Clocking Mode	Synchronous Clocking Mode
0	1	1	1	PCK/64	CK/64
1	0	0	0	PCK/128	CK/128
1	0	0	1	PCK/256	CK/256
1	0	1	0	PCK/512	CK/512
1	0	1	1	PCK/1024	CK/1024
1	1	0	0	PCK/2048	CK/2048
1	1	0	1	PCK/4096	CK/4096
1	1	1	0	PCK/8192	CK/8192
1	1	1	1	PCK/16384	CK/16384

The Stop condition provides a Timer Enable/Disable function.

12.3.2 GTCCR – General Timer/Counter1 Control Register

Bit	7	6	5	4	3	2	1	0	
0x2C	TSM	PWM1B	COM1B1	COM1B0	FOC1B	FOC1A	PSR1	PSR0	GTCCR
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 6 – PWM1B: Pulse Width Modulator B Enable**

When set (one) this bit enables PWM mode based on comparator OCR1B in Timer/Counter1 and the counter value is reset to $00 in the CPU clock cycle after a compare match with OCR1C register value.

- **Bits 5:4 – COM1B[1:0]: Comparator B Output Mode, Bits 1 and 0**

The COM1B1 and COM1B0 control bits determine any output pin action following a compare match with compare register B in Timer/Counter1. Since the output pin action is an alternative function to an I/O port, the corresponding direction control bit must be set (one) in order to control an output pin.

In Normal mode, the COM1B1 and COM1B0 control bits determine the output pin actions that affect pin PB4 (OC1B) as described in Table 12-6. Note that $\overline{\text{OC1B}}$ is not connected in normal mode.

Table 12-6. Comparator B Mode Select in Normal Mode

COM1B1	COM1B0	Description
0	0	Timer/Counter Comparator B disconnected from output pin OC1B.
0	1	Toggle the OC1B output line.
1	0	Clear the OC1B output line.
1	1	Set the OC1B output line

In PWM mode, these bits have different functions. Refer to Table 12-1 on page 86 for a detailed description.

- **Bit 3 – FOC1B: Force Output Compare Match 1B**

Writing a logical one to this bit forces a change in the compare match output pin PB4 (OC1B) according to the values already set in COM1B1 and COM1B0. If COM1B1 and COM1B0 written in the same cycle as FOC1B, the new settings will be used. The Force Output Compare bit can be used to change the output pin value regardless of the timer value. The automatic action programmed in COM1B1 and COM1B0 takes place as if a compare match had

occurred, but no interrupt is generated. The FOC1B bit always reads as zero. FOC1B is not in use if PWM1B bit is set.

- **Bit 2 – FOC1A: Force Output Compare Match 1A**

Writing a logical one to this bit forces a change in the compare match output pin PB1 (OC1A) according to the values already set in COM1A1 and COM1A0. If COM1A1 and COM1A0 written in the same cycle as FOC1A, the new settings will be used. The Force Output Compare bit can be used to change the output pin value regardless of the timer value. The automatic action programmed in COM1A1 and COM1A0 takes place as if a compare match had occurred, but no interrupt is generated. The FOC1A bit always reads as zero. FOC1A is not in use if PWM1A bit is set.

- **Bit 1 – PSR1 : Prescaler Reset Timer/Counter1**

When this bit is set (one), the Timer/Counter prescaler (TCNT1 is unaffected) will be reset. The bit will be cleared by hardware after the operation is performed. Writing a zero to this bit will have no effect. This bit will always read as zero.

12.3.3 TCNT1 – Timer/Counter1

Bit	7	6	5	4	3	2	1	0	
0x2F	MSB							LSB	TCNT1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

This 8-bit register contains the value of Timer/Counter1.

Timer/Counter1 is realized as an up counter with read and write access. Due to synchronization of the CPU, Timer/Counter1 data written into Timer/Counter1 is delayed by one and half CPU clock cycles in synchronous mode and at most one CPU clock cycles for asynchronous mode.

12.3.4 OCR1A –Timer/Counter1 Output Compare RegisterA

Bit	7	6	5	4	3	2	1	0	
0x2E	MSB							LSB	OCR1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The output compare register A is an 8-bit read/write register.

The Timer/Counter Output Compare Register A contains data to be continuously compared with Timer/Counter1. Actions on compare matches are specified in TCCR1. A compare match does only occur if Timer/Counter1 counts to the OCR1A value. A software write that sets TCNT1 and OCR1A to the same value does not generate a compare match.

A compare match will set the compare interrupt flag OCF1A after a synchronization delay following the compare event.

12.3.5 OCR1B – Timer/Counter1 Output Compare RegisterB

Bit	7	6	5	4	3	2	1	0	
0x2B	MSB							LSB	OCR1B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The output compare register B is an 8-bit read/write register.

The Timer/Counter Output Compare Register B contains data to be continuously compared with Timer/Counter1. Actions on compare matches are specified in TCCR1. A compare match does only occur if Timer/Counter1 counts

to the OCR1B value. A software write that sets TCNT1 and OCR1B to the same value does not generate a compare match.

A compare match will set the compare interrupt flag OCF1B after a synchronization delay following the compare event.

12.3.6 OCR1C – Timer/Counter1 Output Compare RegisterC

Bit	7	6	5	4	3	2	1	0	
0x2D	MSB							LSB	OCR1C
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	1	1	1	1	1	1	1	1	

The output compare register C is an 8-bit read/write register.

The Timer/Counter Output Compare Register C contains data to be continuously compared with Timer/Counter1. A compare match does only occur if Timer/Counter1 counts to the OCR1C value. A software write that sets TCNT1 and OCR1C to the same value does not generate a compare match. If the CTC1 bit in TCCR1 is set, a compare match will clear TCNT1.

This register has the same function in normal mode and PWM mode.

12.3.7 TIMSK – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x39	–	OCIE1A	OCIE1B	OCIE0A	OCIE0B	TOIE1	TOIE0	–	TIMSK
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

- **Bit 6 – OCIE1A: Timer/Counter1 Output Compare Interrupt Enable**

When the OCIE1A bit is set (one) and the I-bit in the Status Register is set (one), the Timer/Counter1 Compare MatchA, interrupt is enabled. The corresponding interrupt at vector $003 is executed if a compare matchA occurs. The Compare Flag in Timer/Counter1 is set (one) in the Timer/Counter Interrupt Flag Register.

- **Bit 5 – OCIE1B: Timer/Counter1 Output Compare Interrupt Enable**

When the OCIE1B bit is set (one) and the I-bit in the Status Register is set (one), the Timer/Counter1 Compare MatchB, interrupt is enabled. The corresponding interrupt at vector $009 is executed if a compare matchB occurs. The Compare Flag in Timer/Counter1 is set (one) in the Timer/Counter Interrupt Flag Register.

- **Bit 2 – TOIE1: Timer/Counter1 Overflow Interrupt Enable**

When the TOIE1 bit is set (one) and the I-bit in the Status Register is set (one), the Timer/Counter1 Overflow interrupt is enabled. The corresponding interrupt (at vector $004) is executed if an overflow in Timer/Counter1 occurs. The Overflow Flag (Timer1) is set (one) in the Timer/Counter Interrupt Flag Register - TIFR.

- **Bit 0 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

12.3.8 TIFR – Timer/Counter Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x38	–	OCF1A	OCF1B	OCF0A	OCF0B	TOV1	TOV0	–	TIFR
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

- **Bit 6 – OCF1A: Output Compare Flag 1A**

The OCF1A bit is set (one) when compare match occurs between Timer/Counter1 and the data value in OCR1A - Output Compare Register 1A. OCF1A is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF1A is cleared, after synchronization clock cycle, by writing a logic one to the flag. When the I-bit in SREG, OCIE1A, and OCF1A are set (one), the Timer/Counter1 A compare match interrupt is executed.

- **Bit 5 – OCF1B: Output Compare Flag 1B**

The OCF1B bit is set (one) when compare match occurs between Timer/Counter1 and the data value in OCR1B - Output Compare Register 1A. OCF1B is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF1B is cleared, after synchronization clock cycle, by writing a logic one to the flag. When the I-bit in SREG, OCIE1B, and OCF1B are set (one), the Timer/Counter1 B compare match interrupt is executed.

- **Bit 2 – TOV1: Timer/Counter1 Overflow Flag**

In normal mode (PWM1A=0 and PWM1B=0) the bit TOV1 is set (one) when an overflow occurs in Timer/Counter1. The bit TOV1 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, TOV1 is cleared, after synchronization clock cycle, by writing a logical one to the flag.

In PWM mode (either PWM1A=1 or PWM1B=1) the bit TOV1 is set (one) when compare match occurs between Timer/Counter1 and data value in OCR1C - Output Compare Register 1C.

When the SREG I-bit, and TOIE1 (Timer/Counter1 Overflow Interrupt Enable), and TOV1 are set (one), the Timer/Counter1 Overflow interrupt is executed.

- **Bit 0 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

12.3.9 PLLCSR – PLL Control and Status Register

Bit	7	6	5	4	3	2	1	0	
0x27	**LSM**	**-**	**-**	**-**	**-**	**PCKE**	**PLLE**	**PLOCK**	**PLLCSR**
Read/Write	R/W	R	R	R	R	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0/1	0	

- **Bit 7 – LSM: Low Speed Mode**

The high speed mode is enabled as default and the fast peripheral clock is 64 MHz, but the low speed mode can be set by writing the LSM bit to one. Then the fast peripheral clock is scaled down to 32 MHz. The low speed mode must be set, if the supply voltage is below 2.7 volts, because the Timer/Counter1 is not running fast enough on low voltage levels. It is highly recommended that Timer/Counter1 is stopped whenever the LSM bit is changed.

Note, that LSM can not be set if PLL_{CLK} is used as system clock.

- **Bit 6:3 – Res : Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and always read as zero.

- **Bit 2 – PCKE: PCK Enable**

The PCKE bit change the Timer/Counter1 clock source. When it is set, the asynchronous clock mode is enabled and fast 64 MHz (or 32 MHz in Low Speed Mode) PCK clock is used as Timer/Counter1 clock source. If this bit is cleared, the synchronous clock mode is enabled, and system clock CK is used as Timer/Counter1 clock source. This bit can be set only if PLLE bit is set. It is safe to set this bit only when the PLL is locked i.e the PLOCK bit is 1. The bit PCKE can only be set, if the PLL has been enabled earlier.

- **Bit 1 – PLLE: PLL Enable**

When the PLLE is set, the PLL is started and if needed internal RC-oscillator is started as a PLL reference clock. If PLL is selected as a system clock source the value for this bit is always 1.

- **Bit 0 – PLOCK: PLL Lock Detector**

When the PLOCK bit is set, the PLL is locked to the reference clock. The PLOCK bit should be ignored during initial PLL lock-in sequence when PLL frequency overshoots and undershoots, before reaching steady state. The steady state is obtained within 100 µs. After PLL lock-in it is recommended to check the PLOCK bit before enabling PCK for Timer/Counter1.

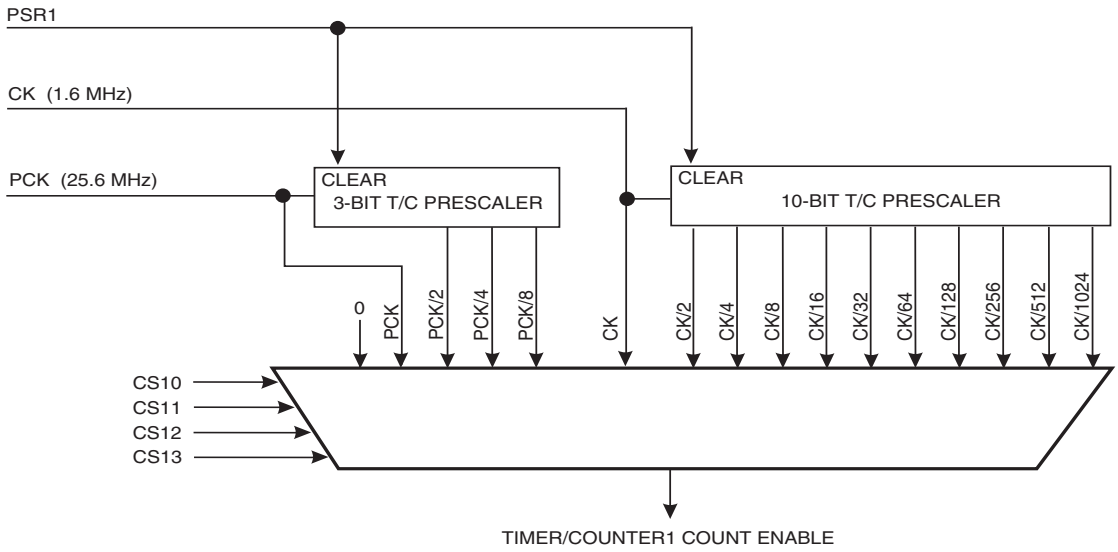
13. 8-bit Timer/Counter1 in ATtiny15 Mode

The ATtiny15 compatibility mode is selected by writing the code “0011” to the CKSEL fuses (if any other code is written, the Timer/Counter1 is working in normal mode). When selected the ATtiny15 compatibility mode provides an ATtiny15 backward compatible prescaler and Timer/Counter. Furthermore, the clocking system has same clock frequencies as in ATtiny15.

13.1 Timer/Counter1 Prescaler

Figure 13-1 shows an ATtiny15 compatible prescaler. It has two prescaler units, a 10-bit prescaler for the system clock (CK) and a 3-bit prescaler for the fast peripheral clock (PCK). The clocking system of the Timer/Counter1 is always synchronous in the ATtiny15 compatibility mode, because the same RC Oscillator is used as a PLL clock source (generates the input clock for the prescaler) and the AVR core.

Figure 13-1. Timer/Counter1 Prescaler



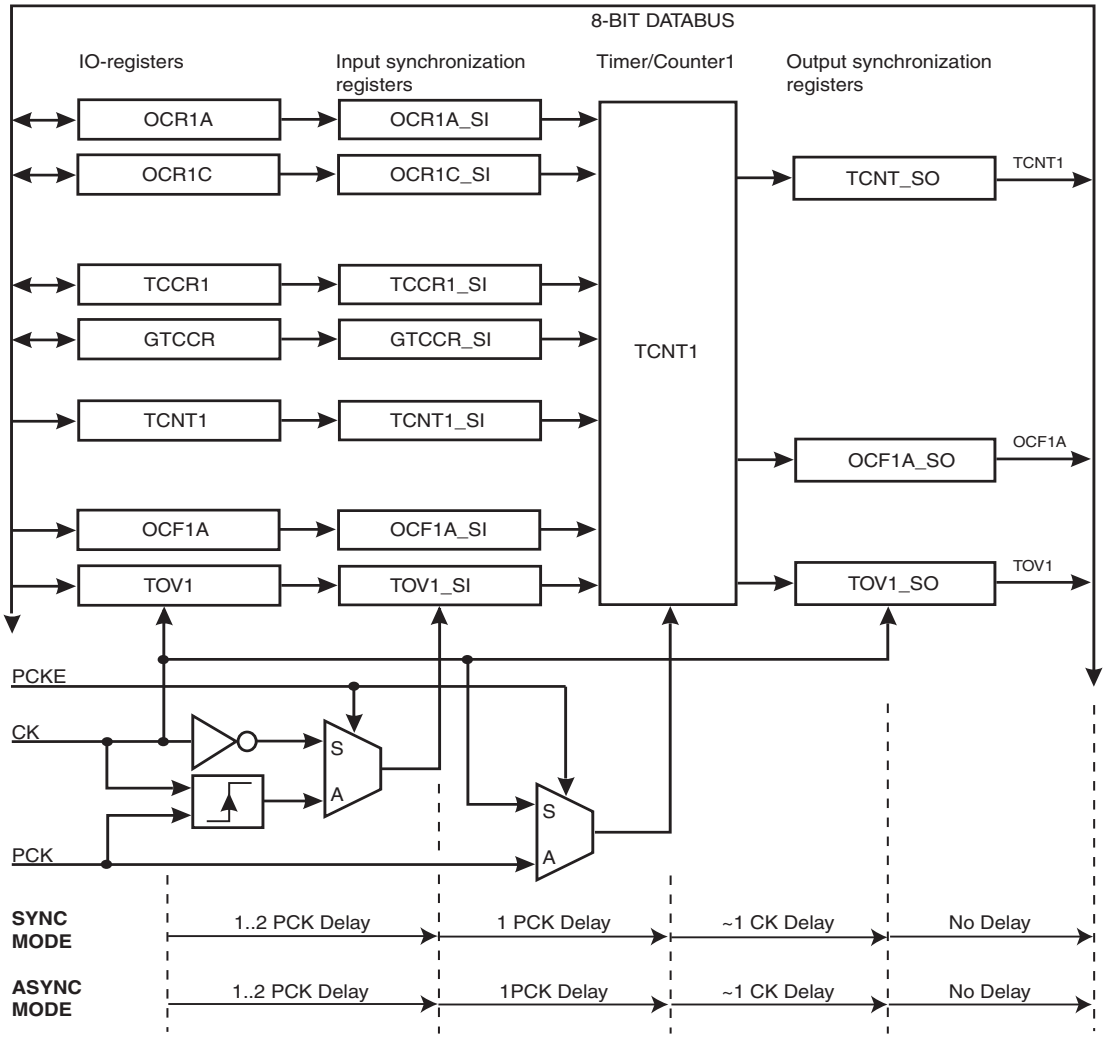
The same clock selections as in ATtiny15 can be chosen for Timer/Counter1 from the output multiplexer, because the frequency of the fast peripheral clock is 25.6 MHz and the prescaler is similar in the ATtiny15 compatibility mode. The clock selections are PCK, PCK/2, PCK/4, PCK/8, CK, CK/2, CK/4, CK/8, CK/16, CK/32, CK/64, CK/128, CK/256, CK/512, CK/1024 and stop.

13.2 Counter and Compare Units

Figure 13-2 shows Timer/Counter 1 synchronization register block diagram and synchronization delays in between registers. Note that all clock gating details are not shown in the figure. The Timer/Counter1 register values go through the internal synchronization registers, which cause the input synchronization delay, before affecting the counter operation. The registers TCCR1, GTCCR, OCR1A and OCR1C can be read back right after writing the register. The read back values are delayed for the Timer/Counter1 (TCNT1) register and flags (OCF1A and TOV1), because of the input and output synchronization.

The Timer/Counter1 features a high resolution and a high accuracy usage with the lower prescaling opportunities. It can also support an accurate, high speed, 8-bit Pulse Width Modulator (PWM) using clock speeds up to 25.6 MHz. In this mode, Timer/Counter1 and the Output Compare Registers serve as a stand-alone PWM. Refer to “Timer/Counter1 in PWM Mode” on page 97 for a detailed description on this function. Similarly, the high prescaling opportunities make this unit useful for lower speed functions or exact timing functions with infrequent actions.

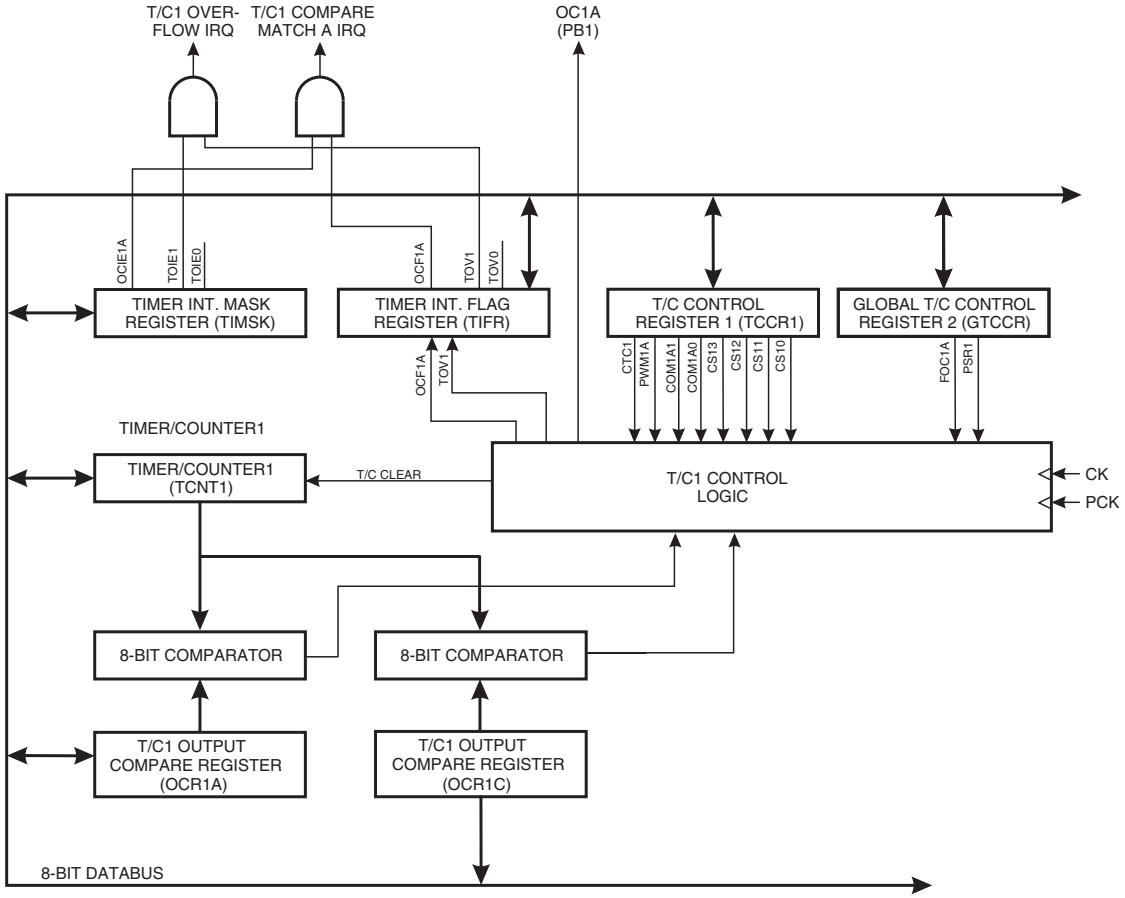
Figure 13-2. Timer/Counter 1 Synchronization Register Block Diagram.



Timer/Counter1 and the prescaler allow running the CPU from any clock source while the prescaler is operating on the fast 25.6 MHz PCK clock in the asynchronous mode.

The following Figure 13-3 shows the block diagram for Timer/Counter1.

Figure 13-3. Timer/Counter1 Block Diagram



Two status flags (overflow and compare match) are found in the Timer/Counter Interrupt Flag Register - TIFR. Control signals are found in the Timer/Counter Control Registers TCCR1 and GTCCR. The interrupt enable/disable settings are found in the Timer/Counter Interrupt Mask Register - TIMSK.

The Timer/Counter1 contains two Output Compare Registers, OCR1A and OCR1C as the data source to be compared with the Timer/Counter1 contents. In normal mode the Output Compare functions are operational with OCR1A only. OCR1A determines action on the OC1A pin (PB1), and it can generate Timer1 OC1A interrupt in normal mode and in PWM mode. OCR1C holds the Timer/Counter maximum value, i.e. the clear on compare match value. In the normal mode an overflow interrupt (TOV1) is generated when Timer/Counter1 counts from $FF to $00, while in the PWM mode the overflow interrupt is generated when the Timer/Counter1 counts either from $FF to $00 or from OCR1C to $00.

In PWM mode, OCR1A provides the data values against which the Timer Counter value is compared. Upon compare match the PWM outputs (OC1A) is generated. In PWM mode, the Timer Counter counts up to the value specified in the output compare register OCR1C and starts again from $00. This feature allows limiting the counter “full” value to a specified value, lower than $FF. Together with the many prescaler options, flexible PWM frequency selection is provided. Table 12-3 on page 88 lists clock selection and OCR1C values to obtain PWM frequencies from 20 kHz to 250 kHz in 10 kHz steps and from 250 kHz to 500 kHz in 50 kHz steps. Higher PWM frequencies can be obtained at the expense of resolution.

13.2.1 Timer/Counter1 in PWM Mode

When the PWM mode is selected, Timer/Counter1 and the Output Compare Register A - OCR1A form an 8-bit, free-running and glitch-free PWM generator with output on the PB1(OC1A).

When the counter value match the content of OCR1A, the OC1A and output is set or cleared according to the COM1A1/COM1A0 bits in the Timer/Counter1 Control Register A - TCCR1, as shown in Table 13-1.

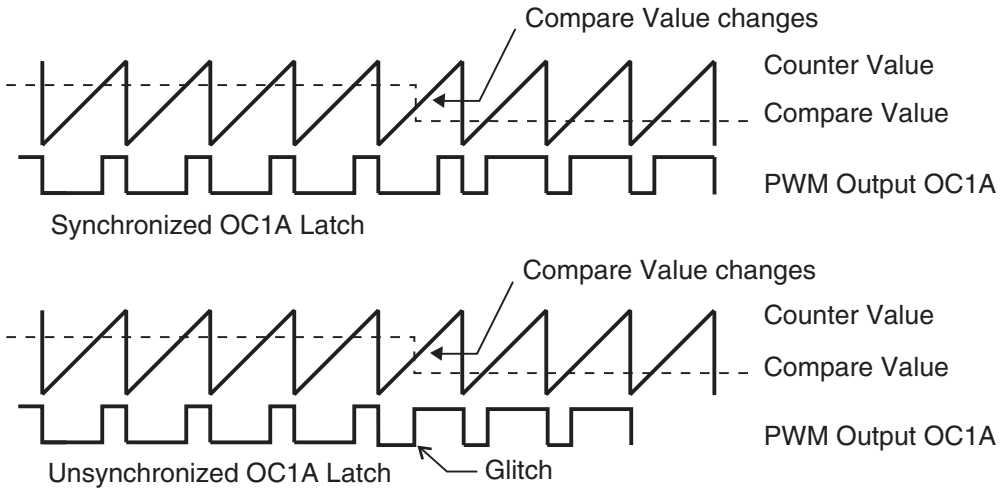
Timer/Counter1 acts as an up-counter, counting from $00 up to the value specified in the output compare register OCR1C, and starting from $00 up again. A compare match with OCR1C will set an overflow interrupt flag (TOV1) after a synchronization delay following the compare event.

Table 13-1. Compare Mode Select in PWM Mode

COM1A1	COM1A0	Effect on Output Compare Pin
0	0	OC1A not connected.
0	1	OC1A not connected.
1	0	OC1A cleared on compare match. Set when TCNT1 = $00.
1	1	OC1A set on compare match. Cleared when TCNT1 = $00.

Note that in PWM mode, writing to the Output Compare Register OCR1A, the data value is first transferred to a temporary location. The value is latched into OCR1A when the Timer/Counter reaches OCR1C. This prevents the occurrence of odd-length PWM pulses (glitches) in the event of an unsynchronized OCR1A. See Figure 13-4 for an e xample.

Figure 13-4. Effects of Unsynchronized OCR Latching



During the time between the write and the latch operation, a read from OCR1A will read the contents of the temporary location. This means that the most recently written value always will read out of OCR1A.

When OCR1A contains $00 or the top value, as specified in OCR1C register, the output PB1(OC1A) is held low or high according to the settings of COM1A1/COM1A0. This is shown in Table 13-2.

Table 13-2. PWM Outputs OCR1A = $00 or OCR1C

COM1A1	COM1A0	OCR1A	Output OC1A
0	1	$00	L
0	1	OCR1C	H
1	0	$00	L

Table 13-2. PWM Outputs OCR1A = $00 or OCR1C

COM1A1	COM1A0	OCR1A	Output OC1A
1	0	OCR1C	H
1	1	$00	H
1	1	OCR1C	L

In PWM mode, the Timer Overflow Flag - TOV1 is set when the TCNT1 counts to the OCR1C value and the TCNT1 is reset to $00. The Timer Overflow Interrupt1 is executed when TOV1 is set provided that Timer Overflow Interrupt and global interrupts are enabled. This also applies to the Timer Output Compare flags and interrupts.

The PWM frequency can be derived from the timer/counter clock frequency using the following equation:

$$f = \frac{f_{TCK1}}{(OCR1C + 1)}$$

The duty cycle of the PWM waveform can be calculated using the following equation:

$$D = \frac{(OCR1A + 1) \times T_{TCK1} - T_{PCK}}{(OCR1C + 1) \times T_{TCK1}}$$

...where T_{PCK} is the period of the fast peripheral clock (1/25.6 MHz = 39.1 ns).

Resolution indicates how many bits are required to express the value in the OCR1C register. It can be calculated using the following equation:

$$R = \log 2(OCR1C + 1)$$

Table 13-3. Timer/Counter1 Clock Prescale Select in the Asynchronous Mode

PWM Frequency	Clock Selection	CS1[3:0]	OCR1C	RESOLUTION
20 kHz	PCK/16	0101	199	7.6
30 kHz	PCK/16	0101	132	7.1
40 kHz	PCK/8	0100	199	7.6
50 kHz	PCK/8	0100	159	7.3
60 kHz	PCK/8	0100	132	7.1
70 kHz	PCK/4	0011	228	7.8
80 kHz	PCK/4	0011	199	7.6
90 kHz	PCK/4	0011	177	7.5
100 kHz	PCK/4	0011	159	7.3
110 kHz	PCK/4	0011	144	7.2
120 kHz	PCK/4	0011	132	7.1
130 kHz	PCK/2	0010	245	7.9
140 kHz	PCK/2	0010	228	7.8
150 kHz	PCK/2	0010	212	7.7

Table 13-3. Timer/Counter1 Clock Prescale Select in the Asynchronous Mode (Continued)

PWM Frequency	Clock Selection	CS1[3:0]	OCR1C	RESOLUTION
160 kHz	PCK/2	0010	199	7.6
170 kHz	PCK/2	0010	187	7.6
180 kHz	PCK/2	0010	177	7.5
190 kHz	PCK/2	0010	167	7.4
200 kHz	PCK/2	0010	159	7.3
250 kHz	PCK	0001	255	8.0
300 kHz	PCK	0001	212	7.7
350 kHz	PCK	0001	182	7.5
400 kHz	PCK	0001	159	7.3
450 kHz	PCK	0001	141	7.1
500 kHz	PCK	0001	127	7.0

13.3 Register Description

13.3.1 TCCR1 – Timer/Counter1 Control Register

Bit	7	6	5	4	3	2	1	0	
0x30	CTC1	PWM1A	COM1A1	COM1A0	CS13	CS12	CS11	CS10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – CTC1 : Clear Timer/Counter on Compare Match**

When the CTC1 control bit is set (one), Timer/Counter1 is reset to $00 in the CPU clock cycle after a compare match with OCR1A register. If the control bit is cleared, Timer/Counter1 continues counting and is unaffected by a compare match.

- **Bit 6 – PWM1A: Pulse Width Modulator A Enable**

When set (one) this bit enables PWM mode based on comparator OCR1A in Timer/Counter1 and the counter value is reset to $00 in the CPU clock cycle after a compare match with OCR1C register value.

- **Bits 5:4 – COM1A[1:0]: Comparator A Output Mode, Bits 1 and 0**

The COM1A1 and COM1A0 control bits determine any output pin action following a compare match with compare register A in Timer/Counter1. Output pin actions affect pin PB1 (OC1A). Since this is an alternative function to an I/O port, the corresponding direction control bit must be set (one) in order to control an output pin.

Table 13-4. Comparator A Mode Select

COM1A1	COM1A0	Description
0	0	Timer/Counter Comparator A disconnected from output pin OC1A.
0	1	Toggle the OC1A output line.
1	0	Clear the OC1A output line.
1	1	Set the OC1A output line

In PWM mode, these bits have different functions. Refer to Table 13-1 on page 98 for a detailed description.

- **Bits 3:0 – CS1[3:0]: Clock Select Bits 3, 2, 1, and 0**

The Clock Select bits 3, 2, 1, and 0 define the prescaling source of Timer/Counter1.

Table 13-5. Timer/Counter1 Prescale Select

CS13	CS12	CS11	CS10	T/C1 Clock
0	0	0	0	T/C1 stopped
0	0	0	1	PCK
0	0	1	0	PCK/2
0	0	1	1	PCK/4
0	1	0	0	PCK/8
0	1	0	1	CK
0	1	1	0	CK/2
0	1	1	1	CK/4
1	0	0	0	CK/8
1	0	0	1	CK/16
1	0	1	0	CK/32
1	0	1	1	CK/64
1	1	0	0	CK/128
1	1	0	1	CK/256
1	1	1	0	CK/512
1	1	1	1	CK/1024

The Stop condition provides a Timer Enable/Disable function.

13.3.2 GTCCR – General Timer/Counter1 Control Register

Bit	7	6	5	4	3	2	1	0	
0x2C	TSM	PWM1B	COM1B1	COM1B0	FOC1B	FOC1A	PSR1	PSR0	GTCCR
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 2 – FOC1A: Force Output Compare Match 1A**

Writing a logical one to this bit forces a change in the compare match output pin PB1 (OC1A) according to the values already set in COM1A1 and COM1A0. If COM1A1 and COM1A0 written in the same cycle as FOC1A, the new settings will be used. The Force Output Compare bit can be used to change the output pin value regardless of the timer value. The automatic action programmed in COM1A1 and COM1A0 takes place as if a compare match had occurred, but no interrupt is generated. The FOC1A bit always reads as zero. FOC1A is not in use if PWM1A bit is set.

- **Bit 1 – PSR1 : Prescaler Reset Timer/Counter1**

When this bit is set (one), the Timer/Counter prescaler (TCNT1 is unaffected) will be reset. The bit will be cleared by hardware after the operation is performed. Writing a zero to this bit will have no effect. This bit will always read as zero.

13.3.3 TCNT1 – Timer/Counter1

Bit	7	6	5	4	3	2	1	0	
0x2F	MSB							LSB	TCNT1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

This 8-bit register contains the value of Timer/Counter1.

Timer/Counter1 is realized as an up counter with read and write access. Due to synchronization of the CPU, Timer/Counter1 data written into Timer/Counter1 is delayed by one CPU clock cycle in synchronous mode and at most two CPU clock cycles for asynchronous mode.

13.3.4 OCR1A – Timer/Counter1 Output Compare RegisterA

Bit	7	6	5	4	3	2	1	0	
0x2E	MSB							LSB	OCR1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The output compare register A is an 8-bit read/write register.

The Timer/Counter Output Compare Register A contains data to be continuously compared with Timer/Counter1. Actions on compare matches are specified in TCCR1. A compare match does only occur if Timer/Counter1 counts to the OCR1A value. A software write that sets TCNT1 and OCR1A to the same value does not generate a compare match.

A compare match will set the compare interrupt flag OCF1A after a synchronization delay following the compare event.

13.3.5 OCR1C – Timer/Counter1 Output Compare Register C

Bit	7	6	5	4	3	2	1	0	
0x2D	MSB							LSB	OCR1C
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	1	1	1	1	1	1	1	1	

The Output Compare Register B - OCR1B from ATtiny15 is replaced with the output compare register C - OCR1C that is an 8-bit read/write register. This register has the same function as the Output Compare Register B in ATtiny15.

The Timer/Counter Output Compare Register C contains data to be continuously compared with Timer/Counter1. A compare match does only occur if Timer/Counter1 counts to the OCR1C value. A software write that sets TCNT1 and OCR1C to the same value does not generate a compare match. If the CTC1 bit in TCCR1 is set, a compare match will clear TCNT1.

13.3.6 TIMSK – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x39	–	OCIE1A	OCIE1B	OCIE0A	OCIE0B	TOIE1	TOIE0	–	TIMSK
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

- **Bit 6 – OCIE1A: Timer/Counter1 Output Compare Interrupt Enable**

When the OCIE1A bit is set (one) and the I-bit in the Status Register is set (one), the Timer/Counter1 Compare MatchA, interrupt is enabled. The corresponding interrupt at vector $003 is executed if a compare matchA occurs. The Compare Flag in Timer/Counter1 is set (one) in the Timer/Counter Interrupt Flag Register.

- **Bit 2 – TOIE1: Timer/Counter1 Overflow Interrupt Enable**

When the TOIE1 bit is set (one) and the I-bit in the Status Register is set (one), the Timer/Counter1 Overflow interrupt is enabled. The corresponding interrupt (at vector $004) is executed if an overflow in Timer/Counter1 occurs. The Overflow Flag (Timer1) is set (one) in the Timer/Counter Interrupt Flag Register - TIFR.

- **Bit 0 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

13.3.7 TIFR – Timer/Counter Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x38	–	OCF1A	OCF1B	OCF0A	OCF0B	TOV1	TOV0	–	TIFR
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0	0	

- **Bit 7 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

- **Bit 6 – OCF1A: Output Compare Flag 1A**

The OCF1A bit is set (one) when compare match occurs between Timer/Counter1 and the data value in OCR1A - Output Compare Register 1A. OCF1A is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF1A is cleared, after synchronization clock cycle, by writing a logic one to the flag. When the I-bit in SREG, OCIE1A, and OCF1A are set (one), the Timer/Counter1 A compare match interrupt is executed.

- **Bit 2 – TOV1: Timer/Counter1 Overflow Flag**

The bit TOV1 is set (one) when an overflow occurs in Timer/Counter1. TOV1 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, TOV1 is cleared, after synchronization clock cycle, by writing a logical one to the flag. When the SREG I-bit, and TOIE1 (Timer/Counter1 Overflow Interrupt Enable), and TOV1 are set (one), the Timer/Counter1 Overflow interrupt is executed.

- **Bit 0 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and always reads as zero.

13.3.8 PLLCSR – PLL Control and Status Register

Bit	7	6	5	4	3	2	1	0	
0x27	LSM	–	–	–	–	PCKE	PLLE	PLOCK	PLLCSR
Read/Write	R/W	R	R	R	R	R/W	R/W	R	
Initial value	0	0	0	0	0	0	0/1	0	

- **Bits 6:3 – Res : Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and always read as zero.

- **Bit 2 – PCKE: PCK Enable**

The bit PCKE is always set in the ATtiny15 compatibility mode.

- **Bit 1 – PLLE: PLL Enable**

The PLL is always enabled in the ATtiny15 compatibility mode.

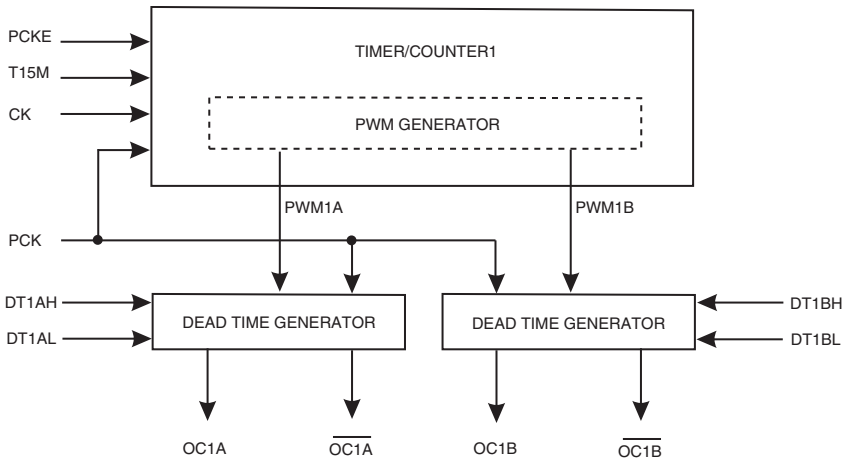
- **Bit 0 – PLOCK: PLL Lock Detector**

When the PLOCK bit is set, the PLL is locked to the reference clock. The PLOCK bit should be ignored during initial PLL lock-in sequence when PLL frequency overshoots and undershoots, before reaching steady state. The steady state is obtained within 100 µs. After PLL lock-in it is recommended to check the PLOCK bit before enabling PCK for Timer/Counter1.

14. Dead Time Generator

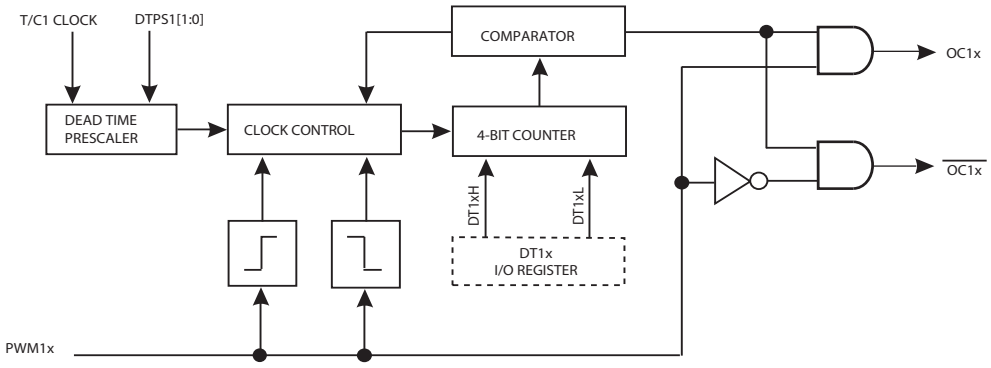
The Dead Time Generator is provided for the Timer/Counter1 PWM output pairs to allow driving external power control switches safely. The Dead Time Generator is a separate block that can be connected to Timer/Counter1 and it is used to insert dead times (non-overlapping times) for the Timer/Counter1 complementary output pairs (OC1A-$\overline{\text{OC1A}}$ and OC1B-$\overline{\text{OC1B}}$). The sharing of tasks is as follows: the timer/counter generates the PWM output and the Dead Time Generator generates the non-overlapping PWM output pair from the timer/counter PWM signal. Two Dead Time Generators are provided, one for each PWM output. The non-overlap time is adjustable and the PWM output and it's complementary output are adjusted separately, and independently for both PWM outputs.

Figure 14-1. Timer/Counter1 & Dead Time Generators



The dead time generation is based on the 4-bit down counters that count the dead time, as shown in Figure 46. There is a dedicated prescaler in front of the Dead Time Generator that can divide the Timer/Counter1 clock (PCK or CK) by 1, 2, 4 or 8. This provides for large range of dead times that can be generated. The prescaler is controlled by two control bits DTPS1[1:0] from the I/O register at address 0x23. The block has also a rising and falling edge detector that is used to start the dead time counting period. Depending on the edge, one of the transitions on the rising edges, OC1x or $\overline{\text{OC1x}}$ is delayed until the counter has counted to zero. The comparator is used to compare the counter with zero and stop the dead time insertion when zero has been reached. The counter is loaded with a 4-bit DT1xH or DT1xL value from DT1x I/O register, depending on the edge of the PWM generator output when the dead time insertion is started.

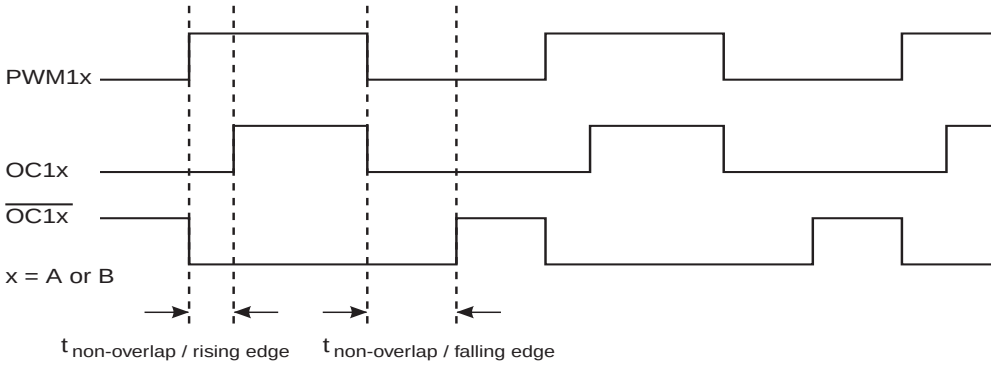
Figure 14-2. Dead Time Generator



The length of the counting period is user adjustable by selecting the dead time prescaler setting in 0x23 register, and selecting then the dead time value in I/O register DT1x. The DT1x register consists of two 4-bit fields, DT1xH and DT1xL that control the dead time periods of the PWM output and its' complementary output separately. Thus

the rising edge of OC1x and $\overline{OC1x}$ can have different dead time periods. The dead time is adjusted as the number of prescaled dead time generator clock cycles.

Figure 14-3. The Complementary Output Pair



14.1 Register Description

14.1.1 DTPS1 – Timer/Counter1 Dead Time Prescaler Register 1

Bit	7	6	5	4	3	2	1	0	
0x23							DTPS11	DTPS10	DTPS1
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The dead time prescaler register, DTPS1 is a 2-bit read/write register.

- **Bits 1:0 – DTPS1[1:0]: Dead Time Prescaler**

The dedicated Dead Time prescaler in front of the Dead Time Generator can divide the Timer/Counter1 clock (PCK or CK) by 1, 2, 4 or 8 providing a large range of dead times that can be generated. The Dead Time prescaler is controlled by two bits DTPS1[1:0] from the Dead Time Prescaler register. These bits define the division factor of the Dead Time prescaler. The division factors are given in table 46.

Table 14-1. Division factors of the Dead Time prescaler

DTPS11	DTPS10	Prescaler divides the T/C1 clock by
0	0	1x (no division)
0	1	2x
1	0	4x
1	1	8x

14.1.2 DT1A – Timer/Counter1 Dead Time A

Bit	7	6	5	4	3	2	1	0	
0x25	DT1AH3	DT1AH2	DT1AH1	DT1AH0	DT1AL3	DT1AL2	DT1AL1	DT1AL0	DT1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The dead time value register A is an 8-bit read/write register.

The dead time delay of is adjusted by the dead time value register, DT1A. The register consists of two fields, DT1AH[3:0] and DT1AL[3:0], one for each complementary output. Therefore a different dead time delay can be adjusted for the rising edge of OC1A and the rising edge of $\overline{OC1A}$.

- **Bits 7:4 – DT1AH[3:0]: Dead Time Value for OC1A Output**

The dead time value for the OC1A output. The dead time delay is set as a number of the prescaled timer/counter clocks. The minimum dead time is zero and the maximum dead time is the prescaled time/counter clock period multiplied by 15.

- **Bits 3:0 – DT1AL[3:0]: Dead Time Value for $\overline{OC1A}$ Output**

The dead time value for the $\overline{OC1A}$ output. The dead time delay is set as a number of the prescaled timer/counter clocks. The minimum dead time is zero and the maximum dead time is the prescaled time/counter clock period multiplied by 15.

14.1.3 DT1B – Timer/Counter1 Dead Time B

Bit	7	6	5	4	3	2	1	0	
0x24	DT1BH3	DT1BH2	DT1BH1	DT1BH0	DT1BL3	DT1BL2	DT1BL1	DT1BL0	DT1B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	

The dead time value register Bis an 8-bit read/write register.

The dead time delay of is adjusted by the dead time value register, DT1B. The register consists of two fields, DT1BH[3:0] and DT1BL[3:0], one for each complementary output. Therefore a different dead time delay can be adjusted for the rising edge of OC1A and the rising edge of $\overline{OC1A}$.

- **Bits 7:4 – DT1BH[3:0]: Dead Time Value for OC1B Output**

The dead time value for the OC1B output. The dead time delay is set as a number of the prescaled timer/counter clocks. The minimum dead time is zero and the maximum dead time is the prescaled time/counter clock period multiplied by 15.

- **Bits 3:0 – DT1BL[3:0]: Dead Time Value for $\overline{OC1B}$ Output**

The dead time value for the $\overline{OC1B}$ output. The dead time delay is set as a number of the prescaled timer/counter clocks. The minimum dead time is zero and the maximum dead time is the prescaled time/counter clock period multiplied by 15.

15. USI – Universal Serial Interface

15.1 Features

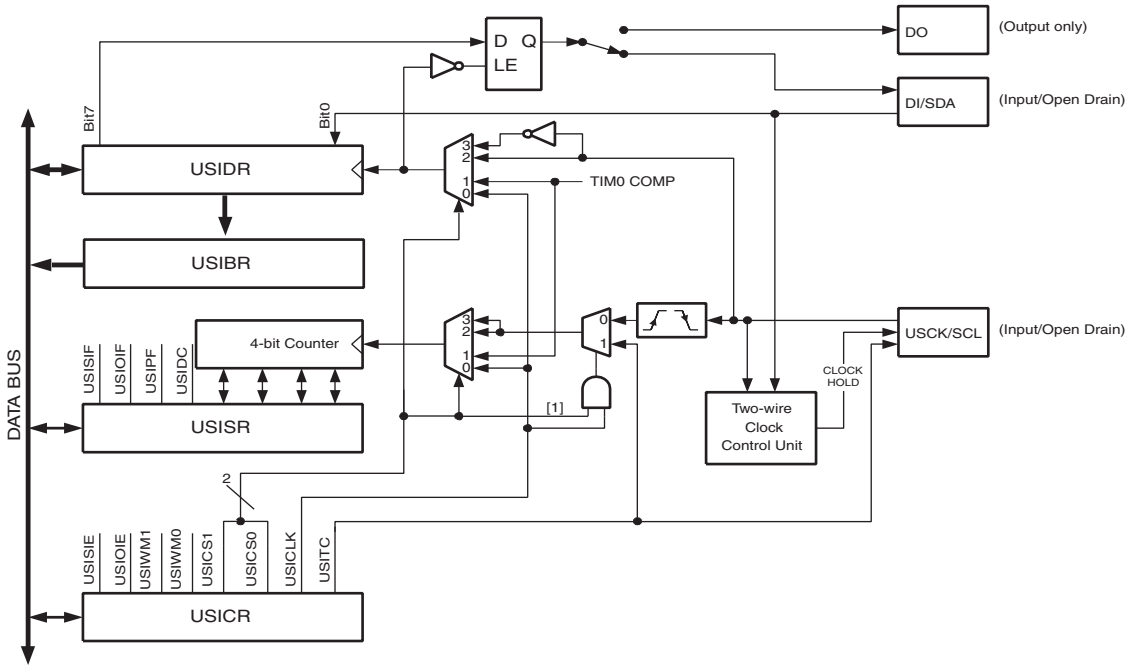
- **Two-wire Synchronous Data Transfer (Master or Slave)**
- **Three-wire Synchronous Data Transfer (Master or Slave)**
- **Data Received Interrupt**
- **Wakeup from Idle Mode**
- **Wake-up from All Sleep Modes In Two-wire Mode**
- **Two-wire Start Condition Detector with Interrupt Capability**

15.2 Overview

The Universal Serial Interface (USI), provides the basic hardware resources needed for serial communication. Combined with a minimum of control software, the USI allows significantly higher transfer rates and uses less code space than solutions based on software only. Interrupts are included to minimize the processor load.

A simplified block diagram of the USI is shown in Figure 15-1 For actual placement of I/O pins refer to “Pinout ATtiny25/45/85” on page 2. Device-specific I/O Register and bit locations are listed in the “Register Descriptions” on page 115.

Figure 15-1. Universal Serial Interface, Block Diagram



The 8-bit USI Data Register (USIDR) contains the incoming and outgoing data. It is directly accessible via the data bus but a copy of the contents is also placed in the USI Buffer Register (USIBR) where it can be retrieved later. If reading the USI Data Register directly, the register must be read as quickly as possible to ensure that no data is lost.

The most significant bit of the USI Data Register is connected to one of two output pins (depending on the mode configuration, see “USICR – USI Control Register” on page 116). There is a transparent latch between the output of the USI Data Register and the output pin, which delays the change of data output to the opposite clock edge of the data input sampling. The serial input is always sampled from the Data Input (DI) pin independent of the configuration.

The 4-bit counter can be both read and written via the data bus, and it can generate an overflow interrupt. Both the USI Data Register and the counter are clocked simultaneously by the same clock source. This allows the counter to count the number of bits received or transmitted and generate an interrupt when the transfer is complete. Note that when an external clock source is selected the counter counts both clock edges. This means the counter registers the number of clock edges and not the number of data bits. The clock can be selected from three different sources: The USCK pin, Timer/Counter0 Compare Match or from software.

The two-wire clock control unit can be configured to generate an interrupt when a start condition has been detected on the two-wire bus. It can also be set to generate wait states by holding the clock pin low after a start condition is detected, or after the counter overflows.

15.3 Functional Descriptions

15.3.1 Three-wire Mode

The USI three-wire mode is compliant to the Serial Peripheral Interface (SPI) mode 0 and 1, but does not have the slave select (SS) pin functionality. However, this feature can be implemented in software, if required. Pin names used in this mode are DI, DO, and USCK.

Figure 15-2. Three-wire Mode Operation, Simplified Diagram

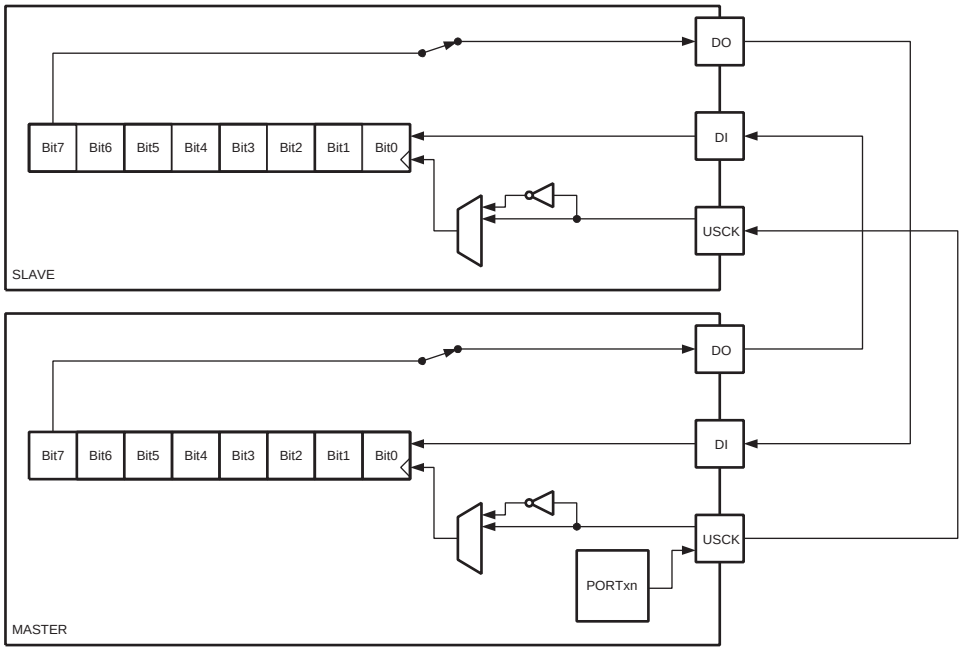
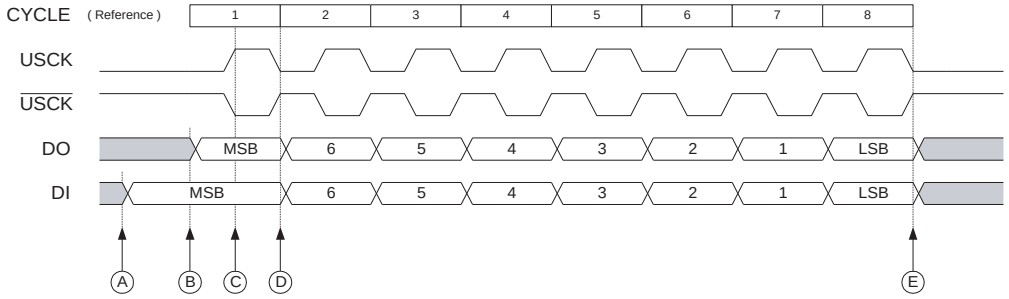


Figure 15-2 shows two USI units operating in three-wire mode, one as Master and one as Slave. The two USI Data Registers are interconnected in such way that after eight USCK clocks, the data in each register has been interchanged. The same clock also increments the USI's 4-bit counter. The Counter Overflow (interrupt) Flag, or USIOIF, can therefore be used to determine when a transfer is completed. The clock is generated by the Master device software by toggling the USCK pin via the PORTB register or by writing a one to bit USITC bit in USICR.

Figure 15-3. Three-Wire Mode, Timing Diagram



The three-wire mode timing is shown in Figure 15-3 At the top of the figure is a USCK cycle reference. One bit is shifted into the USI Data Register (USIDR) for each of these cycles. The USCK timing is shown for both external clock modes. In external clock mode 0 (USICS0 = 0), DI is sampled at positive edges, and DO is changed (USI Data Register is shifted by one) at negative edges. In external clock mode 1 (USICS0 = 1) the opposite edges with respect to mode 0 are used. In other words, data is sampled at negative and changes the output at positive edges. The USI clock modes corresponds to the SPI data mode 0 and 1.

Referring to the timing diagram (Figure 15-3), a bus transfer involves the following steps:

1. The slave and master devices set up their data outputs and, depending on the protocol used, enable their output drivers (mark A and B). The output is set up by writing the data to be transmitted to the USI Data Register. The output is enabled by setting the corresponding bit in the Data Direction Register of Port B. Note that there is not a preferred order of points A and B in the figure, but both must be at least one half USCK cycle before point C, where the data is sampled. This is in order to ensure that the data setup requirement is satisfied. The 4-bit counter is reset to zero.
2. The master software generates a clock pulse by toggling the USCK line twice (C and D). The bit values on the data input (DI) pins are sampled by the USI on the first edge (C), and the data output is changed on the opposite edge (D). The 4-bit counter will count both edges.
3. Step 2. is repeated eight times for a complete register (byte) transfer.
4. After eight clock pulses (i.e., 16 clock edges) the counter will overflow and indicate that the transfer has been completed. If USI Buffer Registers are not used the data bytes that have been transferred must now be processed before a new transfer can be initiated. The overflow interrupt will wake up the processor if it is set to Idle mode. Depending of the protocol used the slave device can now set its output to high impedance.

15.3.2 SPI Master Operation Example

The following code demonstrates how to use the USI as an SPI Master:

```
SPITransfer:
    out    USIDR,r16
    ldi    r16,(1<<USIOIF)
    out    USISR,r16
    ldi    r16,(1<<USIWM0)|(1<<USICS1)|(1<<USICLK)|(1<<USITC)
SPITransfer_loop:
    out    USICR,r16
    in    r16, USISR
    sbrs   r16, USIOIF
    rjmp   SPITransfer_loop
    in     r16,USIDR
    ret
```

The code is size optimized using only eight instructions (plus return). The code example assumes that the DO and USCK pins have been enabled as outputs in DDRB. The value stored in register r16 prior to the function is called is transferred to the slave device, and when the transfer is completed the data received from the slave is stored back into the register r16.

The second and third instructions clear the USI Counter Overflow Flag and the USI counter value. The fourth and fifth instructions set three-wire mode, positive edge clock, count at USITC strobe, and toggle USCK. The loop is repeated 16 times.

The following code demonstrates how to use the USI as an SPI master with maximum speed ($f_{SCK} = f_{CK}/2$):

```
SPITransfer_Fast:
    out    USIDR,r16
    ldi    r16,(1<<USIWM0)|(0<<USICS0)|(1<<USITC)
    ldi    r17,(1<<USIWM0)|(0<<USICS0)|(1<<USITC)|(1<<USICLK)

    out    USICR,r16 ; MSB
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16
    out    USICR,r17
    out    USICR,r16 ; LSB
    out    USICR,r17

    in     r16,USIDR
ret
```

15.3.3 SPI Slave Operation Example

The following code demonstrates how to use the USI as an SPI slave:

```
init:
    ldi    r16,(1<<USIWM0)|(1<<USICS1)
    out    USICR,r16
...
SlaveSPITransfer:
    out    USIDR,r16
    ldi    r16,(1<<USIOIF)
    out  USISR,r16
SlaveSPITransfer_loop:
    in     r16, USISR
```

```
sbrs   r16, USIOIF
rjmp   SlaveSPITransfer_loop
in     r16,USIDR
ret
```

The code is size optimized using only eight instructions (plus return). The code example assumes that the DO and USCK pins have been enabled as outputs in DDRB. The value stored in register r16 prior to the function is called is transferred to the master device, and when the transfer is completed the data received from the master is stored back into the register r16.

Note that the first two instructions is for initialization, only, and need only be executed once. These instructions set three-wire mode and positive edge clock. The loop is repeated until the USI Counter Overflow Flag is set.

15.3.4 Two-wire Mode

The USI two-wire mode is compliant to the Inter IC (TWI) bus protocol, but without slew rate limiting on outputs and without input noise filtering. Pin names used in this mode are SCL and SDA.

Figure 15-4. Two-wire Mode Operation, Simplified Diagram

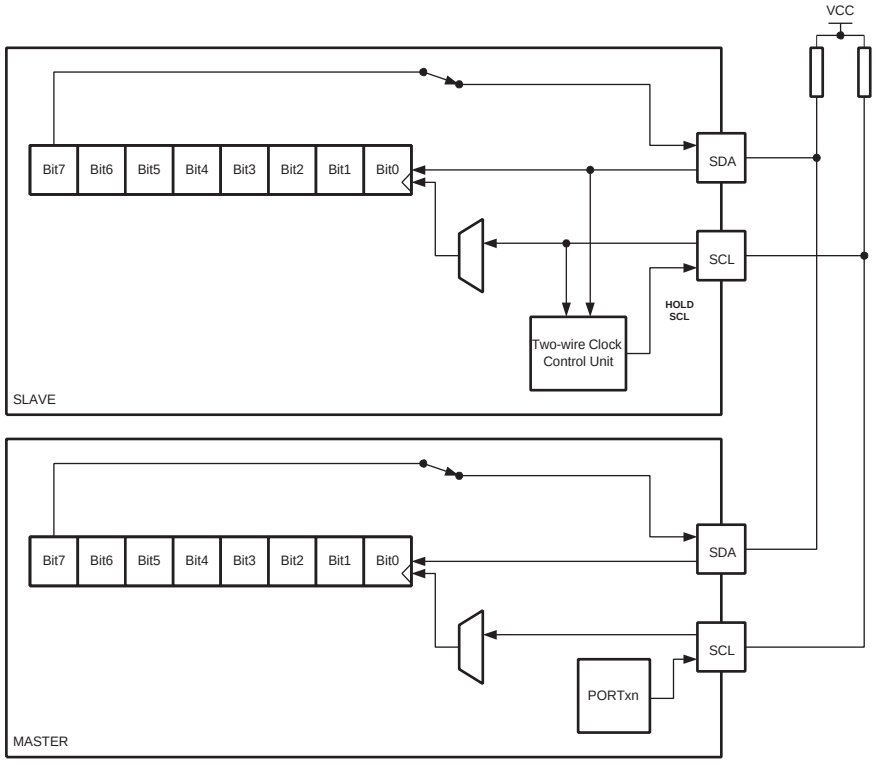


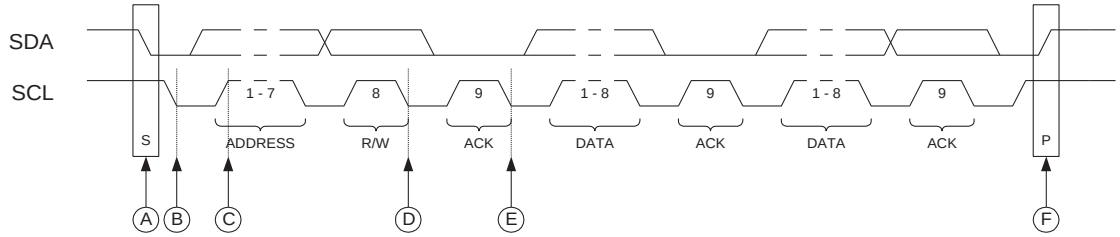
Figure 15-4 shows two USI units operating in two-wire mode, one as master and one as slave. It is only the physical layer that is shown since the system operation is highly dependent of the communication scheme used. The main differences between the master and slave operation at this level is the serial clock generation which is always done by the master. Only the slave uses the clock control unit.

Clock generation must be implemented in software, but the shift operation is done automatically in both devices. Note that clocking only on negative edges for shifting data is of practical use in this mode. The slave can insert wait states at start or end of transfer by forcing the SCL clock low. This means that the master must always check if the SCL line was actually released after it has generated a positive edge.

Since the clock also increments the counter, a counter overflow can be used to indicate that the transfer is completed. The clock is generated by the master by toggling the USCK pin via the PORTB register.

The data direction is not given by the physical layer. A protocol, like the one used by the TWI-bus, must be implemented to control the data flow.

Figure 15-5. Two-wire Mode, Typical Timing Diagram



Referring to the timing diagram (Figure 15-5), a bus transfer involves the following steps:

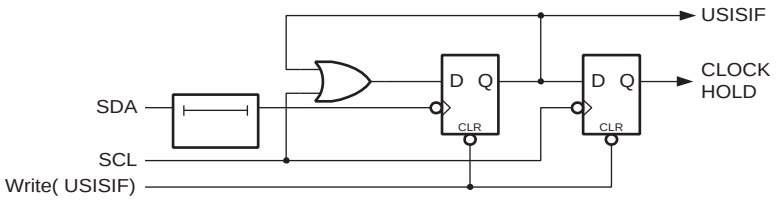
1. The start condition is generated by the master by forcing the SDA low line while keeping the SCL line high (A). SDA can be forced low either by writing a zero to bit 7 of the USI Data Register, or by setting the corresponding bit in the PORTB register to zero. Note that the Data Direction Register bit must be set to one for the output to be enabled. The start detector logic of the slave device (see Figure 15-6 on page 114) detects the start condition and sets the USISIF Flag. The flag can generate an interrupt if necessary.
2. In addition, the start detector will hold the SCL line low after the master has forced a negative edge on this line (B). This allows the slave to wake up from sleep or complete other tasks before setting up the USI Data Register to receive the address. This is done by clearing the start condition flag and resetting the counter.
3. The master set the first bit to be transferred and releases the SCL line (C). The slave samples the data and shifts it into the USI Data Register at the positive edge of the SCL clock.
4. After eight bits containing slave address and data direction (read or write) have been transferred, the slave counter overflows and the SCL line is forced low (D). If the slave is not the one the master has addressed, it releases the SCL line and waits for a new start condition.
5. When the slave is addressed, it holds the SDA line low during the acknowledgment cycle before holding the SCL line low again (i.e., the USI Counter Register must be set to 14 before releasing SCL at (D)). Depending on the R/W bit the master or slave enables its output. If the bit is set, a master read operation is in progress (i.e., the slave drives the SDA line) The slave can hold the SCL line low after the acknowledge (E).
6. Multiple bytes can now be transmitted, all in same direction, until a stop condition is given by the master (F), or a new start condition is given.

If the slave is not able to receive more data it does not acknowledge the data byte it has last received. When the master does a read operation it must terminate the operation by forcing the acknowledge bit low after the last byte transmitted.

15.3.5 Start Condition Detector

The start condition detector is shown in Figure 15-6. The SDA line is delayed (in the range of 50 to 300 ns) to ensure valid sampling of the SCL line. The start condition detector is only enabled in two-wire mode.

Figure 15-6. Start Condition Detector, Logic Diagram



The start condition detector is working asynchronously and can therefore wake up the processor from power-down sleep mode. However, the protocol used might have restrictions on the SCL hold time. Therefore, when using this feature the oscillator start-up time (set by CKSEL fuses, see “Clock Systems and their Distribution” on page 23) must also be taken into consideration. Refer to the description of the USISIF bit on page 115 for further details.

15.3.6 Clock speed considerations

Maximum frequency for SCL and SCK is f_{CK} / 2. This is also the maximum data transmit and receive rate in both two- and three-wire mode. In two-wire slave mode the Two-wire Clock Control Unit will hold the SCL low until the slave is ready to receive more data. This may reduce the actual data rate in two-wire mode.

15.4 Alternative USI Usage

The flexible design of the USI allows it to be used for other tasks when serial communication is not needed. Below are some examples.

15.4.1 Half-Duplex Asynchronous Data Transfer

Using the USI Data Register in three-wire mode it is possible to implement a more compact and higher performance UART than by software, only.

15.4.2 4-Bit Counter

The 4-bit counter can be used as a stand-alone counter with overflow interrupt. Note that if the counter is clocked externally, both clock edges will increment the counter value.

15.4.3 12-Bit Timer/Counter

Combining the 4-bit USI counter with one of the 8-bit timer/counters creates a 12-bit counter.

15.4.4 Edge Triggered External Interrupt

By setting the counter to maximum value (F) it can function as an additional external interrupt. The Overflow Flag and Interrupt Enable bit are then used for the external interrupt. This feature is selected by the USICS1 bit.

15.4.5 Software Interrupt

The counter overflow interrupt can be used as a software interrupt triggered by a clock strobe.

15.5 Register Descriptions

15.5.1 USIDR – USI Data Register

Bit	7	6	5	4	3	2	1	0	
0x0F	MSB							LSB	USIDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The USI Data Register can be accessed directly but a copy of the data can also be found in the USI Buffer Register.

Depending on the USICS[1:0] bits of the USI Control Register a (left) shift operation may be performed. The shift operation can be synchronised to an external clock edge, to a Timer/Counter0 Compare Match, or directly to software via the USICLK bit. If a serial clock occurs at the same cycle the register is written, the register will contain the value written and no shift is performed.

Note that even when no wire mode is selected (USIWM[1:0] = 0) both the external data input (DI/SDA) and the external clock input (USCK/SCL) can still be used by the USI Data Register.

The output pin (DO or SDA, depending on the wire mode) is connected via the output latch to the most significant bit (bit 7) of the USI Data Register. The output latch ensures that data input is sampled and data output is changed on opposite clock edges. The latch is open (transparent) during the first half of a serial clock cycle when an external clock source is selected (USICS1 = 1) and constantly open when an internal clock source is used (USICS1 = 0). The output will be changed immediately when a new MSB is written as long as the latch is open.

Note that the Data Direction Register bit corresponding to the output pin must be set to one in order to enable data output from the USI Data Register.

15.5.2 USIBR – USI Buffer Register

Bit	7	6	5	4	3	2	1	0	
0x10	MSB							LSB	USIBR
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

Instead of reading data from the USI Data Register the USI Buffer Register can be used. This makes controlling the USI less time critical and gives the CPU more time to handle other program tasks. USI flags as set similarly as when reading the USIDR register.

The content of the USI Data Register is loaded to the USI Buffer Register when the transfer has been completed.

15.5.3 USISR – USI Status Register

Bit	7	6	5	4	3	2	1	0	
0x0E	USISIF	USIOIF	USIPF	USIDC	USICNT3	USICNT2	USICNT1	USICNT0	USISR
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Status Register contains interrupt flags, line status flags and the counter value.

- **Bit 7 – USISIF: Start Condition Interrupt Flag**

When two-wire mode is selected, the USISIF Flag is set (to one) when a start condition has been detected. When three-wire mode or output disable mode has been selected any edge on the SCK pin will set the flag.

If USISIE bit in USICR and the Global Interrupt Enable Flag are set, an interrupt will be generated when this flag is set. The flag will only be cleared by writing a logical one to the USISIF bit. Clearing this bit will release the start detection hold of USCL in two-wire mode.

A start condition interrupt will wakeup the processor from all sleep modes.

- **Bit 6 – USIOIF: Counter Overflow Interrupt Flag**

This flag is set (one) when the 4-bit counter overflows (i.e., at the transition from 15 to 0). If the USIOIE bit in USICR and the Global Interrupt Enable Flag are set an interrupt will also be generated when the flag is set. The flag will only be cleared if a one is written to the USIOIF bit. Clearing this bit will release the counter overflow hold of SCL in two-wire mode.

A counter overflow interrupt will wakeup the processor from Idle sleep mode.

- **Bit 5 – USIPF: Stop Condition Flag**

When two-wire mode is selected, the USIPF Flag is set (one) when a stop condition has been detected. The flag is cleared by writing a one to this bit. Note that this is not an interrupt flag. This signal is useful when implementing two-wire bus master arbitration.

- **Bit 4 – USIDC: Data Output Collision**

This bit is logical one when bit 7 in the USI Data Register differs from the physical pin value. The flag is only valid when two-wire mode is used. This signal is useful when implementing Two-wire bus master arbitration.

- **Bits 3:0 – USICNT[3:0]: Counter Value**

These bits reflect the current 4-bit counter value. The 4-bit counter value can directly be read or written by the CPU.

The 4-bit counter increments by one for each clock generated either by the external clock edge detector, by a Timer/Counter0 Compare Match, or by software using USICLK or USITC strobe bits. The clock source depends on the setting of the USICS[1:0] bits.

For external clock operation a special feature is added that allows the clock to be generated by writing to the USITC strobe bit. This feature is enabled by choosing an external clock source (USICS1 = 1) and writing a one to the USICLK bit.

Note that even when no wire mode is selected (USIWM[1:0] = 0) the external clock input (USCK/SCL) can still be used by the counter.

15.5.4 USICR – USI Control Register

Bit	7	6	5	4	3	2	1	0	
0x0D	**USISIE**	**USIOIE**	**USIWM1**	**USIWM0**	**USICS1**	**USICS0**	**USICLK**	**USITC**	**USICR**
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	W	W	
Initial Value	0	0	0	0	0	0	0	0	

The USI Control Register includes bits for interrupt enable, setting the wire mode, selecting the clock and clock strobe.

- **Bit 7 – USISIE: Start Condition Interrupt Enable**

Setting this bit to one enables the start condition detector interrupt. If there is a pending interrupt and USISIE and the Global Interrupt Enable Flag are set to one the interrupt will be executed immediately. Refer to the USISIF bit description on page 115 for further details.

- **Bit 6 – USIOIE: Counter Overflow Interrupt Enable**

Setting this bit to one enables the counter overflow interrupt. If there is a pending interrupt and USIOIE and the Global Interrupt Enable Flag are set to one the interrupt will be executed immediately. Refer to the USIOIF bit description on page 116 for further details.

- **Bits 5:4 – USIWM[1:0]: Wire Mode**

These bits set the type of wire mode to be used, as shown in Table 15-1 below.

Table 15-1. Relationship between USIWM[1:0] and USI Operation

USIWM1	USIWM0	Description
0	0	Outputs, clock hold, and start detector disabled. Port pins operates as normal.
0	1	Three-wire mode. Uses DO, DI, and USCK pins. The *Data Output* (DO) pin overrides the corresponding bit in the PORTB register. However, the corresponding DDRB bit still controls the data direction. When the port pin is set as input the pin pull-up is controlled by the PORTB bit. The *Data Input* (DI) and *Serial Clock* (USCK) pins do not affect the normal port operation. When operating as master, clock pulses are software generated by toggling the PORTB register, while the data direction is set to output. The USITC bit in the USICR Register can be used for this purpose.
1	0	Two-wire mode. Uses SDA (DI) and SCL (USCK) pins [1]. The *Serial Data* (SDA) and the *Serial Clock* (SCL) pins are bi-directional and use open-collector output drives. The output drivers are enabled by setting the corresponding bit for SDA and SCL in the DDRB register. When the output driver is enabled for the SDA pin it will force the line SDA low if the output of the USI Data Register or the corresponding bit in the PORTB register is zero. Otherwise, the SDA line will not be driven (i.e., it is released). When the SCL pin output driver is enabled the SCL line will be forced low if the corresponding bit in the PORTB register is zero, or by the start detector. Otherwise the SCL line will not be driven. The SCL line is held low when a start detector detects a start condition and the output is enabled. Clearing the Start Condition Flag (USISIF) releases the line. The SDA and SCL pin inputs is not affected by enabling this mode. Pull-ups on the SDA and SCL port pin are disabled in Two-wire mode.
1	1	Two-wire mode. Uses SDA and SCL pins. Same operation as in two-wire mode above, except that the SCL line is also held low when a counter overflow occurs, and until the Counter Overflow Flag (USIOIF) is cleared.

Note: 1. The DI and USCK pins are renamed to *Serial Data* (SDA) and *Serial Clock* (SCL) respectively to avoid confusion between the modes of operation.

Basically only the function of the outputs are affected by these bits. Data and clock inputs are not affected by the mode selected and will always have the same function. The counter and USI Data Register can therefore be clocked externally and data input sampled, even when outputs are disabled.

- **Bits 3:2 – USICS[1:0]: Clock Source Select**

These bits set the clock source for the USI Data Register and counter. The data output latch ensures that the output is changed at the opposite edge of the sampling of the data input (DI/SDA) when using external clock source (USCK/SCL). When software strobe or Timer/Counter0 Compare Match clock option is selected, the output latch is transparent and therefore the output is changed immediately.

Clearing the USICS[1:0] bits enables software strobe option. When using this option, writing a one to the USICLK bit clocks both the USI Data Register and the counter. For external clock source (USICS1 = 1), the USICLK bit is no longer used as a strobe, but selects between external clocking and software clocking by the USITC strobe bit.

Table 15-2 shows the relationship between the USICS[1:0] and USICLK setting and clock source used for the USI Data Register and the 4-bit counter.

Table 15-2. Relationship between the USICS[1:0] and USICLK Setting

USICS1	USICS0	USICLK	Clock Source	4-bit Counter Clock Source
0	0	0	No Clock	No Clock
0	0	1	Software clock strobe (USICLK)	Software clock strobe (USICLK)
0	1	X	Timer/Counter0 Compare Match	Timer/Counter0 Compare Match
1	0	0	External, positive edge	External, both edges
1	1	0	External, negative edge	External, both edges
1	0	1	External, positive edge	Software clock strobe (USITC)
1	1	1	External, negative edge	Software clock strobe (USITC)

- **Bit 1 – USICLK: Clock Strobe**

Writing a one to this bit location strobes the USI Data Register to shift one step and the counter to increment by one, provided that the software clock strobe option has been selected by writing USICS[1:0] bits to zero. The output will change immediately when the clock strobe is executed, i.e., during the same instruction cycle. The value shifted into the USI Data Register is sampled the previous instruction cycle.

When an external clock source is selected (USICS1 = 1), the USICLK function is changed from a clock strobe to a Clock Select Register. Setting the USICLK bit in this case will select the USITC strobe bit as clock source for the 4-bit counter (see Table 15-2).

The bit will be read as zero.

- **Bit 0 – USITC: Toggle Clock Port Pin**

Writing a one to this bit location toggles the USCK/SCL value either from 0 to 1, or from 1 to 0. The toggling is independent of the setting in the Data Direction Register, but if the PORT value is to be shown on the pin the corresponding DDR pin must be set as output (to one). This feature allows easy clock generation when implementing master devices.

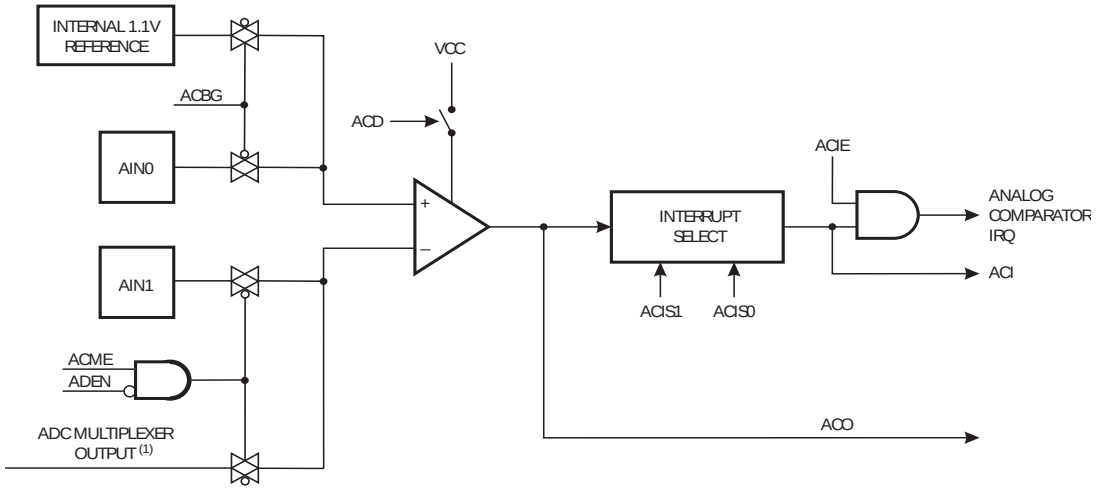
When an external clock source is selected (USICS1 = 1) and the USICLK bit is set to one, writing to the USITC strobe bit will directly clock the 4-bit counter. This allows an early detection of when the transfer is done when operating as a master device.

The bit will read as zero.

16. Analog Comparator

The Analog Comparator compares the input values on the positive pin AIN0 and negative pin AIN1. When the voltage on the positive pin AIN0 is higher than the voltage on the negative pin AIN1, the Analog Comparator output, ACO, is set. The comparator can trigger a separate interrupt, exclusive to the Analog Comparator. The user can select Interrupt triggering on comparator output rise, fall or toggle. A block diagram of the comparator and its surrounding logic is shown in Figure 16-1.

Figure 16-1. Analog Comparator Block Diagram



Notes: 1. See Table 16-1 below.

See Figure 1-1 on page 2 and Table 10-5 on page 63 for Analog Comparator pin placement.

16.1 Analog Comparator Multiplexed Input

When the Analog to Digital Converter (ADC) is configurated as single ended input channel, it is possible to select any of the ADC[3:0] pins to replace the negative input to the Analog Comparator. The ADC multiplexer is used to select this input, and consequently, the ADC must be switched off to utilize this feature. If the Analog Comparator Multiplexer Enable bit (ACME in ADCSRB) is set and the ADC is switched off (ADEN in ADCSRA is zero), MUX[1:0] in ADMUX select the input pin to replace the negative input to the Analog Comparator, as shown in Table 16-1. If ACME is cleared or ADEN is set, AIN1 is applied to the negative input to the Analog Comparator.

Table 16-1. Analog Comparator Multiplexed Input

ACME	ADEN	MUX[1:0]	Analog Comparator Negative Input
0	x	xx	AIN1
1	1	xx	AIN1
1	0	00	ADC0
1	0	01	ADC1
1	0	10	ADC2
1	0	11	ADC3

16.2 Register Description

16.2.1 ADCSRB – ADC Control and Status Register B

Bit	7	6	5	4	3	2	1	0	
0x03	BIN	ACME	IPR	–	–	ADTS2	ADTS1	ADTS0	ADCSRB
Read/Write	R/W	R/W	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 6 – ACME: Analog Comparator Multiplexer Enable**

When this bit is written logic one and the ADC is switched off (ADEN in ADCSRA is zero), the ADC multiplexer selects the negative input to the Analog Comparator. When this bit is written logic zero, AIN1 is applied to the negative input of the Analog Comparator. For a detailed description of this bit, see “Analog Comparator Multiplexed Input” on page 119.

16.2.2 ACSR – Analog Comparator Control and Status Register

Bit	7	6	5	4	3	2	1	0	
0x08	**ACD**	**ACBG**	**ACO**	**ACI**	**ACIE**	**–**	**ACIS1**	**ACIS0**	**ACSR**
Read/Write	R/W	R/W	R	R/W	R/W	R	R/W	R/W	
Initial Value	0	0	N/A	0	0	0	0	0	

- **Bit 7 – ACD: Analog Comparator Disable**

When this bit is written logic one, the power to the Analog Comparator is switched off. This bit can be set at any time to turn off the Analog Comparator. This will reduce power consumption in Active and Idle mode. When changing the ACD bit, the Analog Comparator Interrupt must be disabled by clearing the ACIE bit in ACSR. Otherwise an interrupt can occur when the bit is changed.

- **Bit 6 – ACBG: Analog Comparator Bandgap Select**

When this bit is set, a fixed bandgap reference voltage replaces the positive input to the Analog Comparator. When this bit is cleared, AIN0 is applied to the positive input of the Analog Comparator. When the bandgap reference is used as input to the Analog Comparator, it will take a certain time for the voltage to stabilize. If not stabilized, the first conversion may give a wrong value. See “Internal Voltage Reference” on page 42.

- **Bit 5 – ACO: Analog Comparator Output**

The output of the Analog Comparator is synchronized and then directly connected to ACO. The synchronization introduces a delay of 1 - 2 clock cycles.

- **Bit 4 – ACI: Analog Comparator Interrupt Flag**

This bit is set by hardware when a comparator output event triggers the interrupt mode defined by ACIS1 and ACIS0. The Analog Comparator interrupt routine is executed if the ACIE bit is set and the I-bit in SREG is set. ACI is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, ACI is cleared by writing a logic one to the flag.

- **Bit 3 – ACIE: Analog Comparator Interrupt Enable**

When the ACIE bit is written logic one and the I-bit in the Status Register is set, the Analog Comparator interrupt is activated. When written logic zero, the interrupt is disabled.

- **Bit 2 – Res: Reserved Bit**

This bit is a reserved bit in the ATtiny25/45/85 and will always read as zero.

- **Bits 1:0 – ACIS[1:0]: Analog Comparator Interrupt Mode Select**

These bits determine which comparator events that trigger the Analog Comparator interrupt. The different settings are shown in Table 16-2.

Table 16-2. ACIS1/ACIS0 Settings

ACIS1	ACIS0	Interrupt Mode
0	0	Comparator Interrupt on Output Toggle.
0	1	Reserved
1	0	Comparator Interrupt on Falling Output Edge.
1	1	Comparator Interrupt on Rising Output Edge.

When changing the ACIS1/ACIS0 bits, the Analog Comparator Interrupt must be disabled by clearing its Interrupt Enable bit in the ACSR Register. Otherwise an interrupt can occur when the bits are changed.

16.2.3 DIDR0 – Digital Input Disable Register 0

Bit	7	6	5	4	3	2	1	0	
0x14	–	–	ADC0D	ADC2D	ADC3D	ADC1D	AIN1D	AIN0D	DIDR0
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 1:0 – AIN1D, AIN0D: AIN[1:0] Digital Input Disable**

When this bit is written logic one, the digital input buffer on the AIN1/0 pin is disabled. The corresponding PIN Register bit will always read as zero when this bit is set. When an analog signal is applied to the AIN1/0 pin and the digital input from this pin is not needed, this bit should be written logic one to reduce power consumption in the digital input buffer.

17. Analog to Digital Converter

17.1 Features

- **10-bit Resolution**
- **1 LSB Integral Non-linearity**
- **± 2 LSB Absolute Accuracy**
- **65 - 260 µs Conversion Time**
- **Up to 15 kSPS at Maximum Resolution**
- **Four Multiplexed Single Ended Input Channels**
- **Two differential input channels with selectable gain**
- **Temperature sensor input channel**
- **Optional Left Adjustment for ADC Result Readout**
- **0 - V_{CC} ADC Input Voltage Range**
- **Selectable 1.1V / 2.56V ADC Voltage Reference**
- **Free Running or Single Conversion Mode**
- **ADC Start Conversion by Auto Triggering on Interrupt Sources**
- **Interrupt on ADC Conversion Complete**
- **Sleep Mode Noise Cancele**
- **Unipolar / Bibilar Input Mode**
- **Input Polarity Reversal Mode**

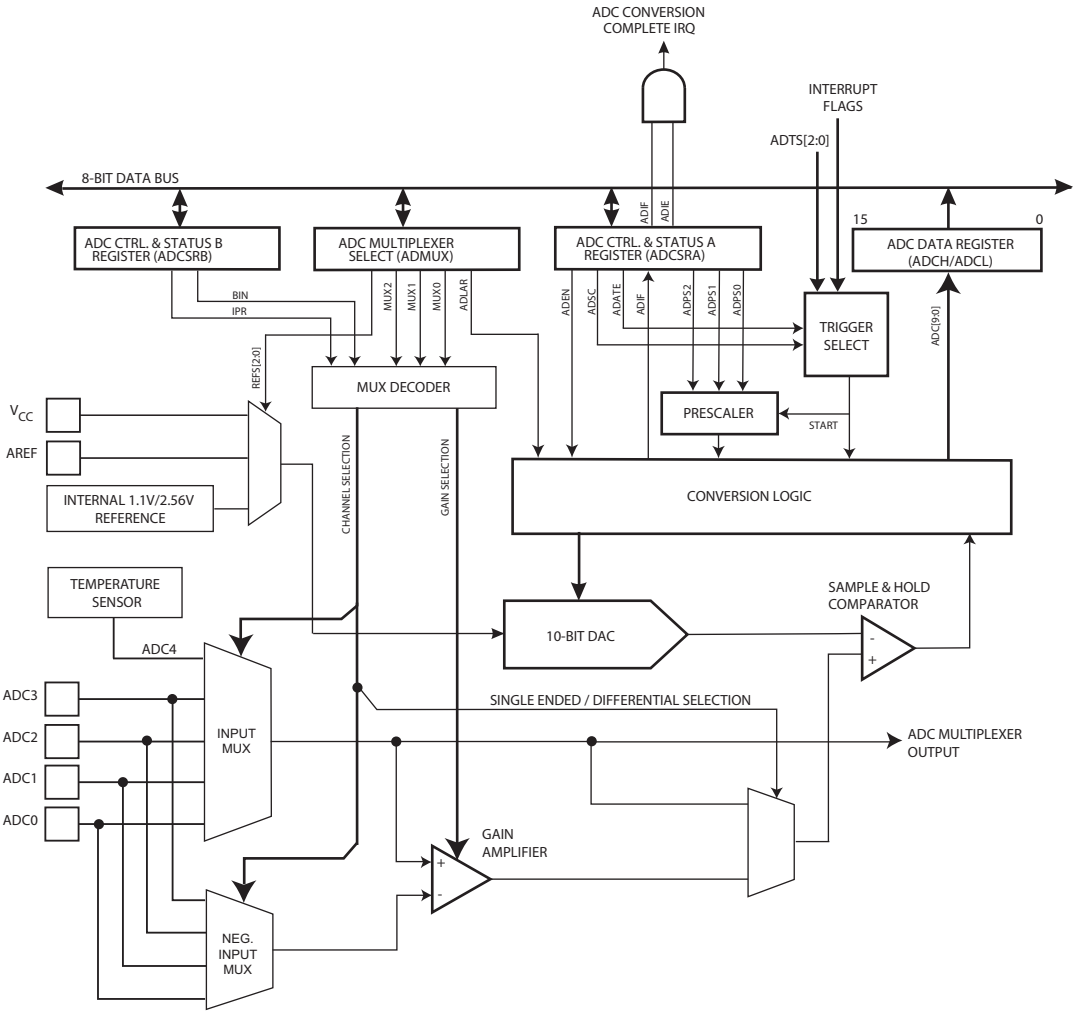
17.2 Overview

The ATtiny25/45/85 features a 10-bit successive approximation Analog to Digital Converter (ADC). The ADC is connected to a 4-channel Analog Multiplexer which allows one differential voltage input and four single-ended voltage inputs constructed from the pins of Port B. The differential input (PB3, PB4 or PB2, PB5) is equipped with a programmable gain stage, providing amplification step of 26 dB (20x) on the differential input voltage before the A/D conversion. The single-ended voltage inputs refer to 0V (GND).

The ADC contains a Sample and Hold circuit which ensures that the input voltage to the ADC is held at a constant level during conversion. A block diagram of the ADC is shown in Figure 17-1 on page 123.

Internal reference voltages of nominally 1.1V / 2.56V are provided on-chip. Alternatively, V_{CC} can be used as reference voltage for single ended channels. There is also an option to use an external voltage reference and turn-off the internal voltage reference.

Figure 17-1. Analog to Digital Converter Block Schematic



17.3 Operation

The ADC converts an analog input voltage to a 10-bit digital value through successive approximation. The minimum value represents GND and the maximum value represents the voltage on V_{CC}, the voltage on the AREF pin or an internal 1.1V / 2.56V voltage reference.

The voltage reference for the ADC may be selected by writing to the REFS[2:0] bits in ADMUX. The VCC supply, the AREF pin or an internal 1.1V / 2.56V voltage reference may be selected as the ADC voltage reference. Optionally the internal 2.56V voltage reference may be decoupled by an external capacitor at the AREF pin to improve noise immunity.

The analog input channel and differential gain are selected by writing to the MUX[3:0] bits in ADMUX. Any of the four ADC input pins ADC[3:0] can be selected as single ended inputs to the ADC. ADC2 or ADC0 can be selected as positive input and ADC0, ADC1, ADC2 or ADC3 can be selected as negative input to the differential gain amplifier.

If differential channels are selected, the differential gain stage amplifies the voltage difference between the selected input pair by the selected gain factor, 1x or 20x, according to the setting of the MUX[3:0] bits in ADMUX. This amplified value then becomes the analog input to the ADC. If single ended channels are used, the gain amplifier is bypassed altogether.

If ADC0 or ADC2 is selected as both the positive and negative input to the differential gain amplifier (ADC0-ADC0 or ADC2-ADC2), the remaining offset in the gain stage and conversion circuitry can be measured directly as the result of the conversion. This figure can be subtracted from subsequent conversions with the same gain setting to reduce offset error to below 1 LSW.

The on-chip temperature sensor is selected by writing the code “1111” to the MUX[3:0] bits in ADMUX register when the ADC4 channel is used as an ADC input.

The ADC is enabled by setting the ADC Enable bit, ADEN in ADCSRA. Voltage reference and input channel selections will not go into effect until ADEN is set. The ADC does not consume power when ADEN is cleared, so it is recommended to switch off the ADC before entering power saving sleep modes.

The ADC generates a 10-bit result which is presented in the ADC Data Registers, ADCH and ADCL. By default, the result is presented right adjusted, but can optionally be presented left adjusted by setting the ADLAR bit in ADMUX.

If the result is left adjusted and no more than 8-bit precision is required, it is sufficient to read ADCH. Otherwise, ADCL must be read first, then ADCH, to ensure that the content of the data registers belongs to the same conversion. Once ADCL is read, ADC access to data registers is blocked. This means that if ADCL has been read, and a conversion completes before ADCH is read, neither register is updated and the result from the conversion is lost. When ADCH is read, ADC access to the ADCH and ADCL Registers is re-enabled.

The ADC has its own interrupt which can be triggered when a conversion completes. When ADC access to the data registers is prohibited between reading of ADCH and ADCL, the interrupt will trigger even if the result is lost.

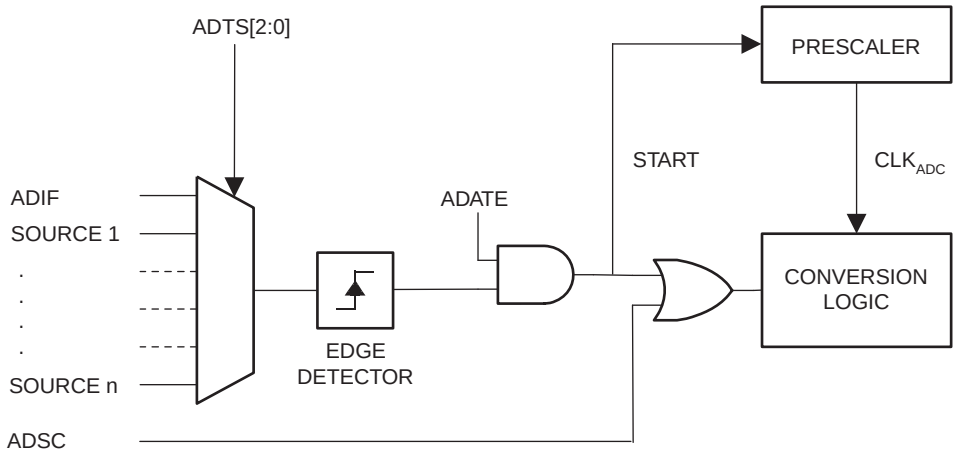
17.4 Starting a Conversion

A single conversion is started by writing a logical one to the ADC Start Conversion bit, ADSC. This bit stays high as long as the conversion is in progress and will be cleared by hardware when the conversion is completed. If a different data channel is selected while a conversion is in progress, the ADC will finish the current conversion before performing the channel change.

Alternatively, a conversion can be triggered automatically by various sources. Auto Triggering is enabled by setting the ADC Auto Trigger Enable bit, ADATE in ADCSRA. The trigger source is selected by setting the ADC Trigger Select bits, ADTS in ADCSRB (see description of the ADTS bits for a list of the trigger sources). When a positive edge occurs on the selected trigger signal, the ADC prescaler is reset and a conversion is started. This provides a method of starting conversions at fixed intervals. If the trigger signal still is set when the conversion completes, a new conversion will not be started. If another positive edge occurs on the trigger signal during conversion, the edge will be ignored. Note that an Interrupt Flag will be set even if the specific interrupt is disabled or the Global Interrupt Enable bit in SREG is cleared. A conversion can thus be triggered without causing an interrupt. However, the Interrupt Flag must be cleared in order to trigger a new conversion at the next interrupt event.

Using the ADC Interrupt Flag as a trigger source makes the ADC start a new conversion as soon as the ongoing conversion has finished. The ADC then operates in Free Running mode, constantly sampling and updating the ADC Data Register. The first conversion must be started by writing a logical one to the ADSC bit in ADCSRA. In this mode the ADC will perform successive conversions independently of whether the ADC Interrupt Flag, ADIF is cleared or not.

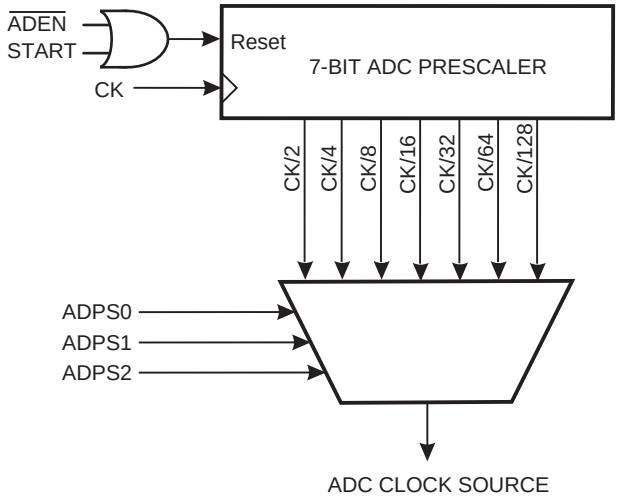
Figure 17-2. ADC Auto Trigger Logic



If Auto Triggering is enabled, single conversions can be started by writing ADSC in ADCSRA to one. ADSC can also be used to determine if a conversion is in progress. The ADSC bit will be read as one during a conversion, independently of how the conversion was started.

17.5 Prescaling and Conversion Timing

Figure 17-3. ADC Prescaler



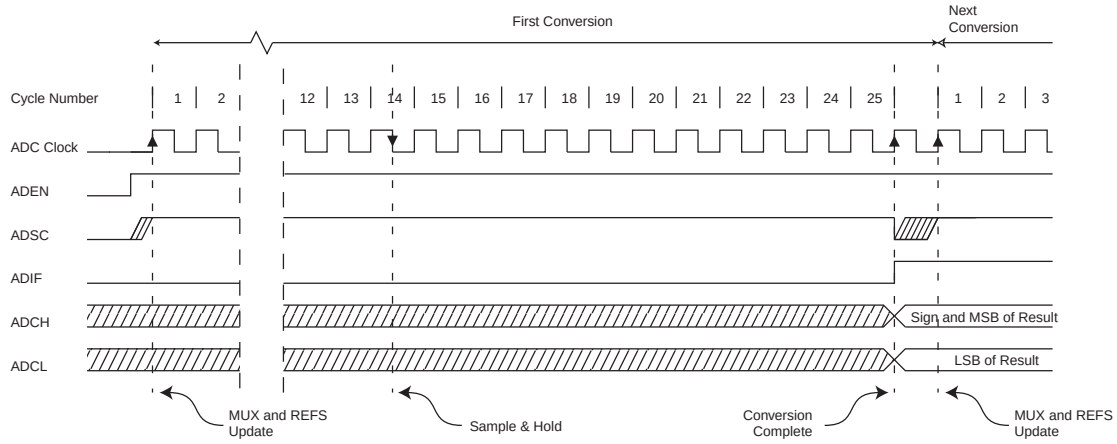
By default, the successive approximation circuitry requires an input clock frequency between 50 kHz and 200 kHz to get maximum resolution. If a lower resolution than 10 bits is needed, the input clock frequency to the ADC can be higher than 200 kHz to get a higher sample rate. It is not recommended to use a higher input clock frequency than 1 MHz.

The ADC module contains a prescaler, which generates an acceptable ADC clock frequency from any CPU frequency above 100 kHz. The prescaling is set by the ADPS bits in ADCSRA. The prescaler starts counting from the moment the ADC is switched on by setting the ADEN bit in ADCSRA. The prescaler keeps running for as long as the ADEN bit is set, and is continuously reset when ADEN is low.

When initiating a single ended conversion by setting the ADSC bit in ADCSRA, the conversion starts at the following rising edge of the ADC clock cycle.

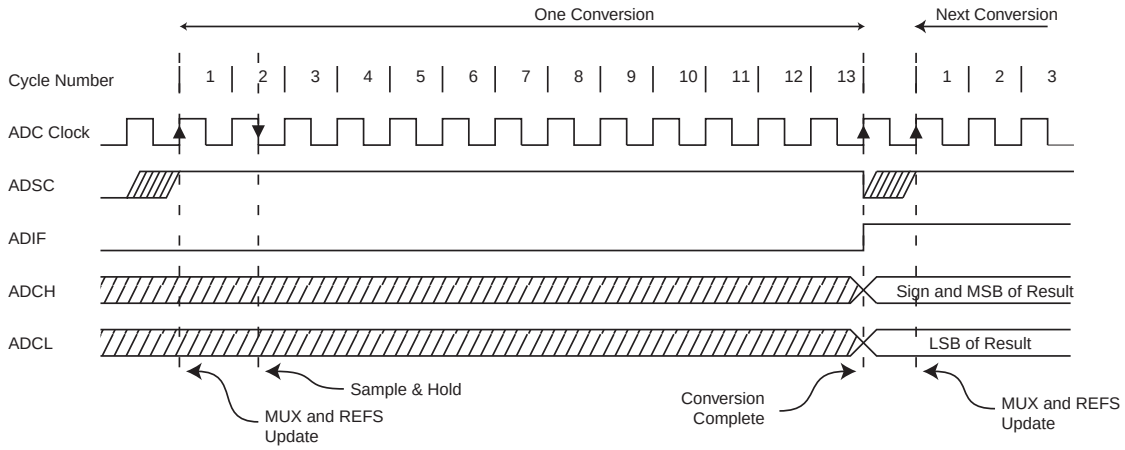
A normal conversion takes 13 ADC clock cycles. The first conversion after the ADC is switched on (ADEN in ADCSRA is set) takes 25 ADC clock cycles in order to initialize the analog circuitry, as shown in Figure 17-4 below.

Figure 17-4. ADC Timing Diagram, First Conversion (Single Conversion Mode)



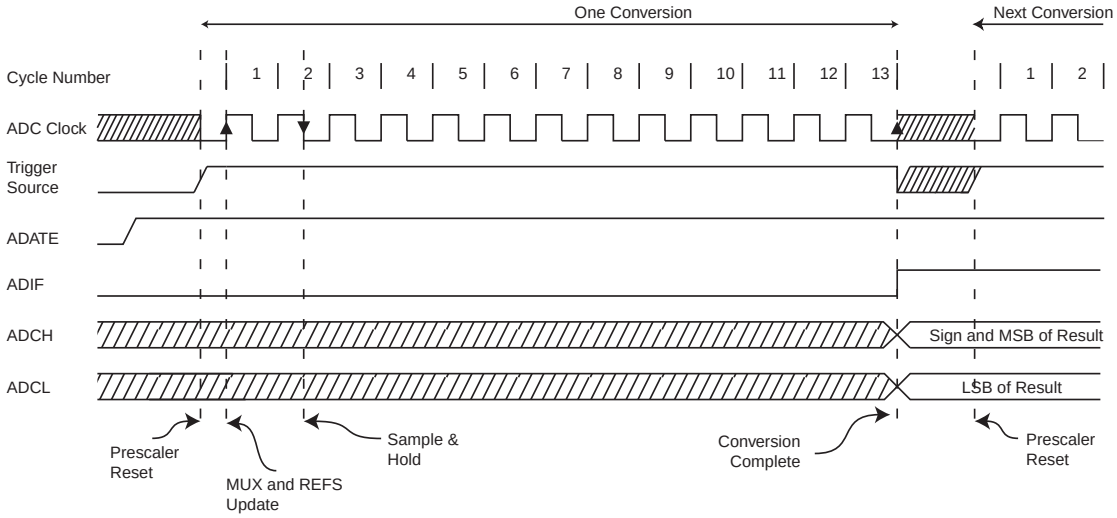
The actual sample-and-hold takes place 1.5 ADC clock cycles after the start of a normal conversion and 13.5 ADC clock cycles after the start of an first conversion. When a conversion is complete, the result is written to the ADC Data Registers, and ADIF is set. In Single Conversion mode, ADSC is cleared simultaneously. The software may then set ADSC again, and a new conversion will be initiated on the first rising ADC clock edge.

Figure 17-5. ADC Timing Diagram, Single Conversion



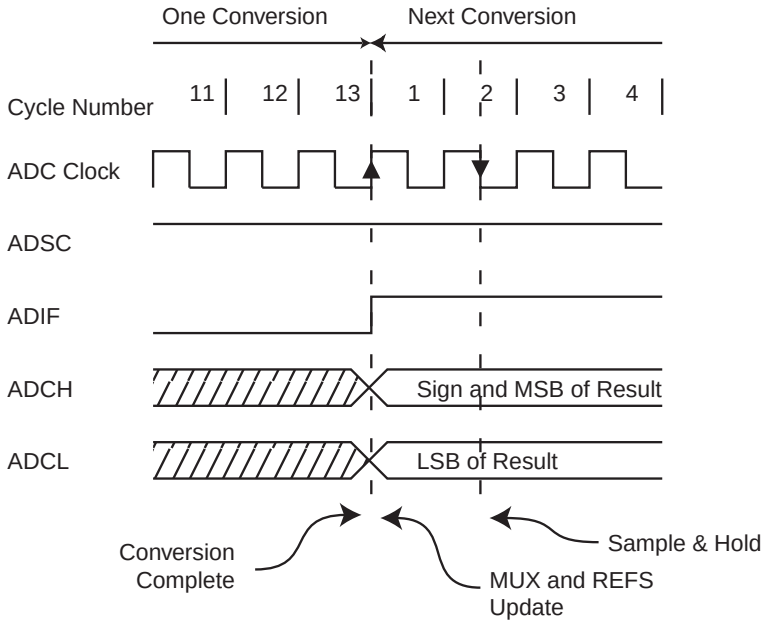
When Auto Triggering is used, the prescaler is reset when the trigger event occurs. This assures a fixed delay from the trigger event to the start of conversion. In this mode, the sample-and-hold takes place two ADC clock cycles after the rising edge on the trigger source signal. Three additional CPU clock cycles are used for synchronization logic.

Figure 17-6. ADC Timing Diagram, Auto Triggered Conversion



In Free Running mode, a new conversion will be started immediately after the conversion completes, while ADSC remains high.

Figure 17-7. ADC Timing Diagram, Free Running Conversion



For a summary of conversion times, see Table 17-1.

Table 17-1. ADC Conversion Time

Condition	Sample & Hold (Cycles from Start of Conversion)	Total Conversion Time (Cycles)
First conversion	13.5	25
Normal conversions	1.5	13
Auto Triggered conversions	2	13.5

17.6 Changing Channel or Reference Selection

The MUX[3:0] and REFS[2:0] bits in the ADMUX Register are single buffered through a temporary register to which the CPU has random access. This ensures that the channels and voltage reference selection only takes place at a safe point during the conversion. The channel and voltage reference selection is continuously updated until a conversion is started. Once the conversion starts, the channel and voltage reference selection is locked to ensure a sufficient sampling time for the ADC. Continuous updating resumes in the last ADC clock cycle before the conversion completes (ADIF in ADCSRA is set). Note that the conversion starts on the following rising ADC clock edge after ADSC is written. The user is thus advised not to write new channel or voltage reference selection values to ADMUX until one ADC clock cycle after ADSC is written.

If Auto Triggering is used, the exact time of the triggering event can be indeterministic. Special care must be taken when updating the ADMUX Register, in order to control which conversion will be affected by the new settings.

If both ADATE and ADEN is written to one, an interrupt event can occur at any time. If the ADMUX Register is changed in this period, the user cannot tell if the next conversion is based on the old or the new settings. ADMUX can be safely updated in the following ways:

a. When ADATE or ADEN is cleared.
b. During conversion, minimum one ADC clock cycle after the trigger event.
c. After a conversion, before the Interrupt Flag used as trigger source is cleared.

When updating ADMUX in one of these conditions, the new settings will affect the next ADC conversion.

17.6.1 ADC Input Channels

When changing channel selections, the user should observe the following guidelines to ensure that the correct channel is selected:

In Single Conversion mode, always select the channel before starting the conversion. The channel selection may be changed one ADC clock cycle after writing one to ADSC. However, the simplest method is to wait for the conversion to complete before changing the channel selection.

In Free Running mode, always select the channel before starting the first conversion. The channel selection may be changed one ADC clock cycle after writing one to ADSC. However, the simplest method is to wait for the first conversion to complete, and then change the channel selection. Since the next conversion has already started automatically, the next result will reflect the previous channel selection. Subsequent conversions will reflect the new channel selection.

17.6.2 ADC Voltage Reference

The voltage reference for the ADC (V_{REF}) indicates the conversion range for the ADC. Single ended channels that exceed V_{REF} will result in codes close to 0x3FF. V_{REF} can be selected as either V_{CC}, or internal 1.1V / 2.56V voltage reference, or external AREF pin. The first ADC conversion result after switching voltage reference source may be inaccurate, and the user is advised to discard this result.

17.7 ADC Noise Canceler

The ADC features a noise canceler that enables conversion during sleep mode to reduce noise induced from the CPU core and other I/O peripherals. The noise canceler can be used with ADC Noise Reduction and Idle mode. To make use of this feature, the following procedure should be used:

- Make sure that the ADC is enabled and is not busy converting. Single Conversion mode must be selected and the ADC conversion complete interrupt must be enabled.
- Enter ADC Noise Reduction mode (or Idle mode). The ADC will start a conversion once the CPU has been halted.
- If no other interrupts occur before the ADC conversion completes, the ADC interrupt will wake up the CPU and execute the ADC Conversion Complete interrupt routine. If another interrupt wakes up the CPU before the ADC

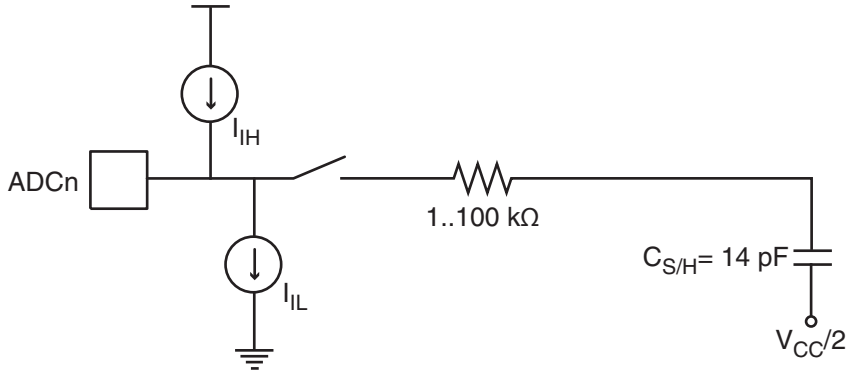
conversion is complete, that interrupt will be executed, and an ADC Conversion Complete interrupt request will be generated when the ADC conversion completes. The CPU will remain in active mode until a new sleep command is executed.

Note that the ADC will not be automatically turned off when entering other sleep modes than Idle mode and ADC Noise Reduction mode. The user is advised to write zero to ADEN before entering such sleep modes to avoid excessive power consumption.

17.8 Analog Input Circuitry

The analog input circuitry for single ended channels is illustrated in Figure 17-8. An analog source applied to ADCn is subjected to the pin capacitance and input leakage of that pin, regardless of whether that channel is selected as input for the ADC. When the channel is selected, the source must drive the S/H capacitor through the series resistance (combined resistance in the input path).

Figure 17-8. Analog Input Circuitry



The ADC is optimized for analog signals with an output impedance of approximately 10 kΩ or less. If such a source is used, the sampling time will be negligible. If a source with higher impedance is used, the sampling time will depend on how long time the source needs to charge the S/H capacitor, with can vary widely. The user is recommended to only use low impedant sources with slowly varying signals, since this minimizes the required charge transfer to the S/H capacitor.

Signal components higher than the Nyquist frequency ($f_{ADC}/2$) should not be present to avoid distortion from unpredictable signal convolution. The user is advised to remove high frequency components with a low-pass filter before applying the signals as inputs to the ADC.

17.9 Noise Canceling Techniques

Digital circuitry inside and outside the device generates EMI which might affect the accuracy of analog measurements. If conversion accuracy is critical, the noise level can be reduced by applying the following techniques:

- Keep analog signal paths as short as possible.
- Make sure analog tracks run over the analog ground plane.
- Keep analog tracks well away from high-speed switching digital tracks.
- If any port pin is used as a digital output, it mustn't switch while a conversion is in progress.
- Place bypass capacitors as close to V_{CC} and GND pins as possible.

Where high ADC accuracy is required it is recommended to use ADC Noise Reduction Mode, as described in Section 17.7 on page 128. This is especially the case when system clock frequency is above 1 MHz, or when the ADC

is used for reading the internal temperature sensor, as described in Section 17.12 on page 133. A good system design with properly placed, external bypass capacitors does reduce the need for using ADC Noise Reduction Mode

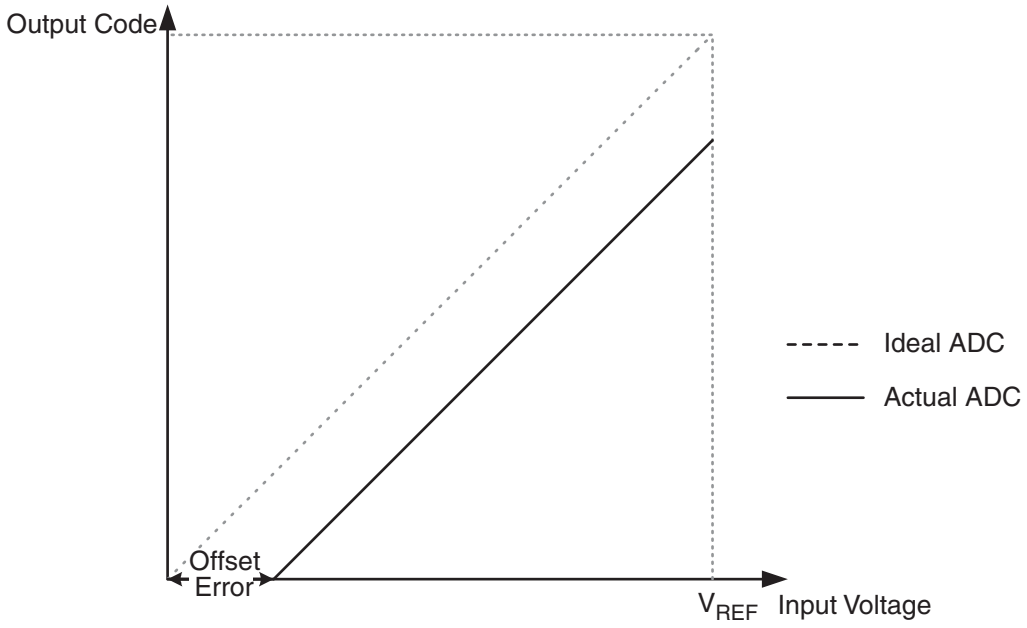
17.10 ADC Accuracy Definitions

An n-bit single-ended ADC converts a voltage linearly between GND and V_{REF} in 2^n steps (LSBs). The lowest code is read as 0, and the highest code is read as 2^n-1.

Several parameters describe the deviation from the ideal behavior, as follows:

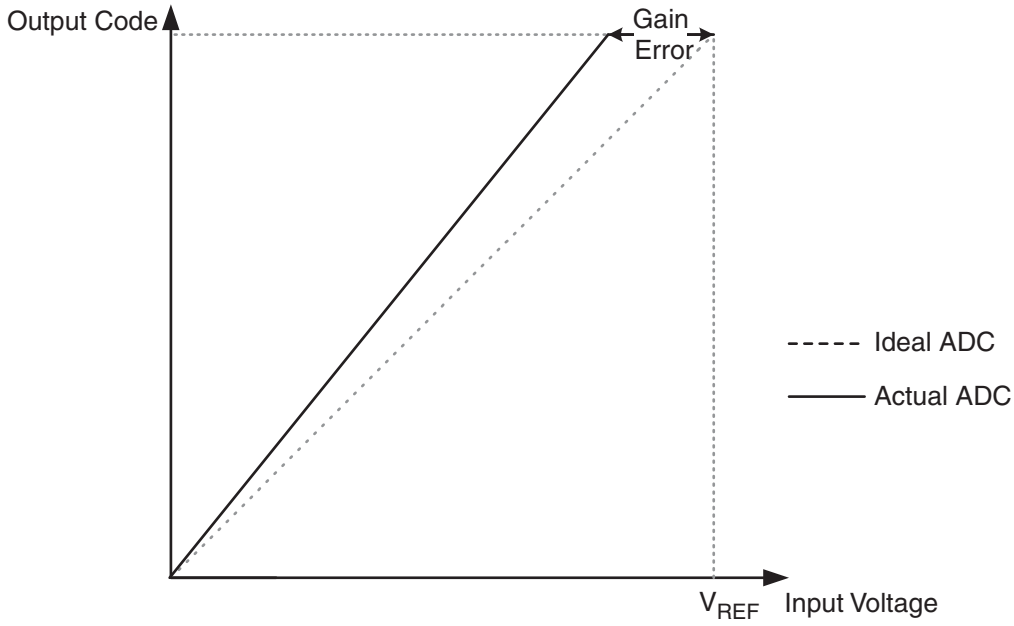
- Offset: The deviation of the first transition (0x000 to 0x001) compared to the ideal transition (at 0.5 LSB). Ideal value: 0 LSB.

Figure 17-9. Offset Error



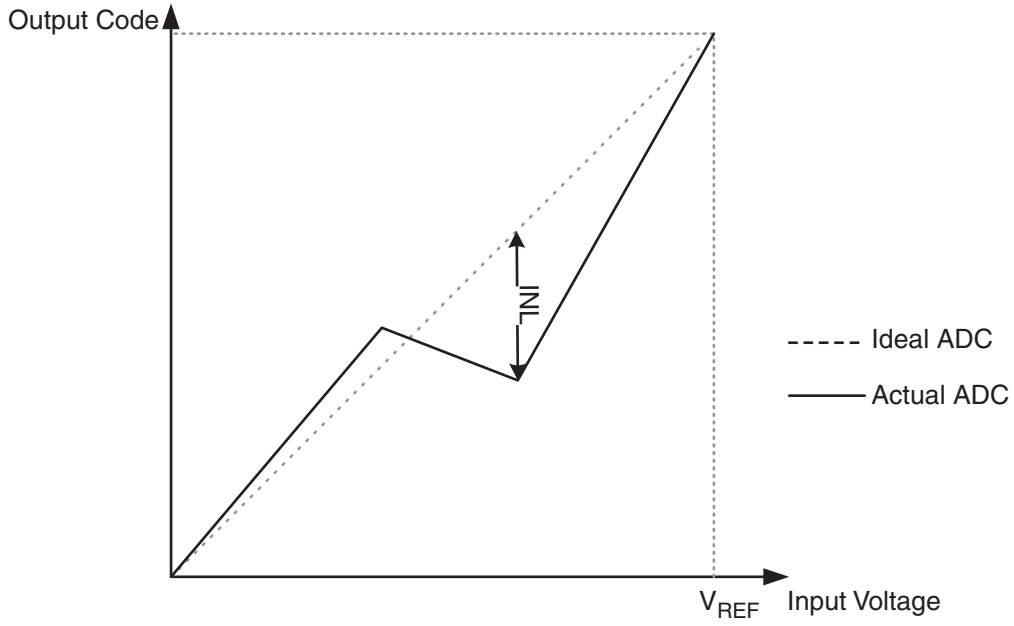
- Gain Error: After adjusting for offset, the Gain Error is found as the deviation of the last transition (0x3FE to 0x3FF) compared to the ideal transition (at 1.5 LSB below maximum). Ideal value: 0 LSB

Figure 17-10. Gain Error



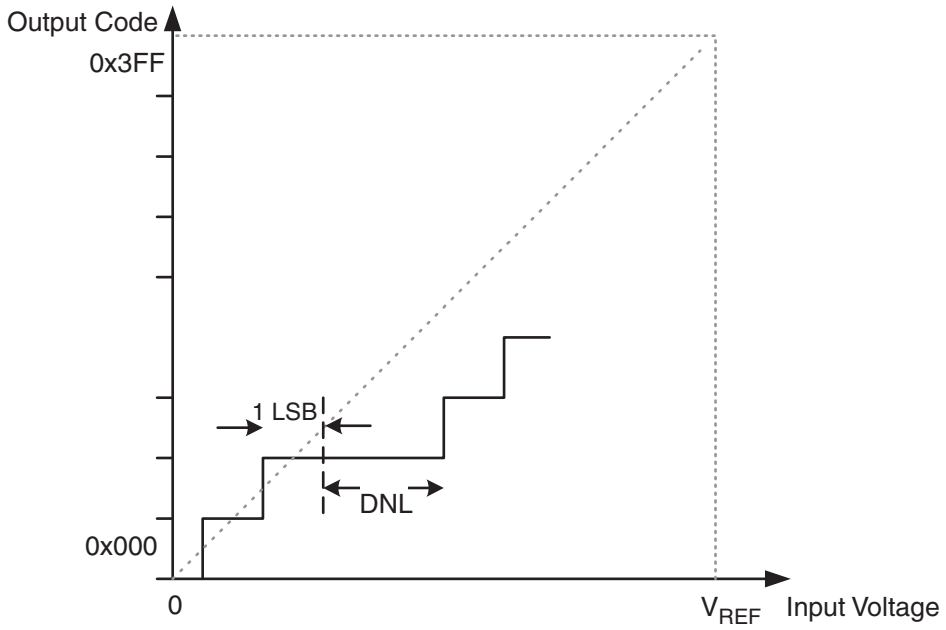
- Integral Non-linearity (INL): After adjusting for offset and gain error, the INL is the maximum deviation of an actual transition compared to an ideal transition for any code. Ideal value: 0 LSB.

Figure 17-11. Integral Non-linearity (INL)



- Differential Non-linearity (DNL): The maximum deviation of the actual code width (the interval between two adjacent transitions) from the ideal code width (1 LSB). Ideal value: 0 LSB.

Figure 17-12. Differential Non-linearity (DNL)



- Quantization Error: Due to the quantization of the input voltage into a finite number of codes, a range of input voltages (1 LSB wide) will code to the same value. Always ± 0.5 LSB.

- Absolute Accuracy: The maximum deviation of an actual (unadjusted) transition compared to an ideal transition for any code. This is the compound effect of offset, gain error, differential error, non-linearity, and quantization error. Ideal value: ± 0.5 LSB.

17.11 ADC Conversion Result

After the conversion is complete (ADIF is high), the conversion result can be found in the ADC Result Registers (ADCL, ADCH). The form of the conversion result depends on the type of the conversio as there are three types of conversions: single ended conversion, unipolar differential conversion and bipolar differential conversion.

17.11.1 Single Ended Conversion

For single ended conversion, the result is

$$ADC = \frac{V_{IN} \cdot 1024}{V_{REF}}$$

where V_{IN} is the voltage on the selected input pin and V_{REF} the selected voltage reference (see Table 17-3 on page 134 and Table 17-4 on page 135). 0x000 represents analog ground, and 0x3FF represents the selected voltage reference minus one LSB. The result is presented in one-sided form, from 0x3FF to 0x000.

17.11.2 Unipolar Differential Conversion

If differential channels and an unipolar input mode are used, the result is

$$ADC = \frac{(V_{POS} - V_{NEG}) \cdot 1024}{V_{REF}} \cdot GAIN$$

where V_{POS} is the voltage on the positive input pin, V_{NEG} the voltage on the negative input pin, and V_{REF} the selected voltage reference (see Table 17-3 on page 134 and Table 17-4 on page 135). The voltage on the positive pin must always be larger than the voltage on the negative pin or otherwise the voltage difference is saturated to zero. The result is presented in one-sided form, from 0x000 (0d) to 0x3FF (+1023d). The GAIN is either 1x or 20x.

17.11.3 Bipolar Differential Conversion

As default the ADC converter operates in the unipolar input mode, but the bipolar input mode can be selected by writting the BIN bit in the ADCSRB to one. In the bipolar input mode two-sided voltage differences are allowed and thus the voltage on the negative input pin can also be larger than the voltage on the positive input pin. If differential channels and a bipolar input mode are used, the result is

$$ADC = \frac{(V_{POS} - V_{NEG}) \cdot 512}{V_{REF}} \cdot GAIN$$

where V_{POS} is the voltage on the positive input pin, V_{NEG} the voltage on the negative input pin, and V_{REF} the selected voltage reference. The result is presented in two's complement form, from 0x200 (-512d) through 0x000 (+0d) to 0x1FF (+511d). The GAIN is either 1x or 20x.

However, if the signal is not bipolar by nature (9 bits + sign as the 10th bit), this scheme loses one bit of the converter dynamic range. Then, if the user wants to perform the conversion with the maximum dynamic range, the user can perform a quick polarity check of the result and use the unipolar differential conversion with selectable differential input pairs (see the Input Polarity Reversal mode ie. the IPR bit in the "ADCSRB – ADC Control and Status Register B" on page 137). When the polarity check is performed, it is sufficient to read the MSB of the result (ADC9 in ADCH). If the bit is one, the result is negative, and if this bit is zero, the result is positive.

17.12 Temperature Measurement

The temperature measurement is based on an on-chip temperature sensor that is coupled to a single ended ADC4 channel. Selecting the ADC4 channel by writing the MUX[3:0] bits in ADMUX register to "1111" enables the temperature sensor. The internal 1.1V reference must also be selected for the ADC reference source in the temperature sensor measurement. When the temperature sensor is enabled, the ADC converter can be used in single conversion mode to measure the voltage over the temperature sensor.

The measured voltage has a linear relationship to the temperature as described in Table 17-2 The sensitivity is approximately 1 LSB / °C and the accuracy depends on the method of user calibration. Typically, the measurement accuracy after a single temperature calibration is ±10°C, assuming calibration at room temperature. Better accuracies are achieved by using two temperature points for calibration.

Table 17-2. Temperature vs. Sensor Output Voltage (Typical Case)

Temperature	-40°C	+25°C	+85°C
ADC	230 LSB	300 LSB	370 LSB

The values described in Table 17-2 are typical values. However, due to process variation the temperature sensor output voltage varies from one chip to another. To be capable of achieving more accurate results the temperature measurement can be calibrated in the application software. The sofware calibration can be done using the formula:

```
T = k * [(ADCH << 8) | ADCL] + TOS
```

where ADCH and ADCL are the ADC data registers, k is the fixed slope coefficient and T_{OS} is the temperature sensor offset. Typically, k is very close to 1.0 and in single-point calibration the coefficient may be omitted. Where higher accuracy is required the slope coefficient should be evaluated based on measurements at two temperatures.

17.13 Register Description

17.13.1 ADMUX – ADC Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0	
0x07	REFS1	REFS0	ADLAR	REFS2	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:6, 4 – REFS[2:0]: Voltage Reference Selection Bits**

These bits select the voltage reference (V_{REF}) for the ADC, as shown in Table 17-3. If these bits are changed during a conversion, the change will not go in effect until this conversion is complete (ADIF in ADCSR is set). Whenever these bits are changed, the next conversion will take 25 ADC clock cycles. When differential channels and gain are used, using V_{CC} or an external AREF higher than (V_{CC} - 1V) as a voltage reference is not recommended as this will affect the ADC accuracy.

Table 17-3. Voltage Reference Selections for ADC

REFS2	REFS1	REFS0	Voltage Reference (V_{REF}) Selection
X	0	0	V_{CC} used as Voltage Reference, disconnected from PB0 (AREF).
X	0	1	External Voltage Reference at PB0 (AREF) pin, Internal Voltage Reference turned off.
0	1	0	Internal 1.1V Voltage Reference.
0	1	1	Reserved
1	1	0	Internal 2.56V Voltage Reference without external bypass capacitor, disconnected from PB0 (AREF)(1).
1	1	1	Internal 2.56V Voltage Reference with external bypass capacitor at PB0 (AREF) pin(1).

Note: 1. The device requries a supply voltage of 3V in order to generate 2.56V reference voltage.

- **Bit 5 – ADLAR: ADC Left Adjust Result**

The ADLAR bit affects the presentation of the ADC conversion result in the ADC Data Register. Write one to ADLAR to left adjust the result. Otherwise, the result is right adjusted. Changing the ADLAR bit will affect the ADC Data Register immediately, regardless of any ongoing conversions. For a complete description of this bit, see “ADCL and ADCH – The ADC Data Register” on page 137.

- **Bits 3:0 – MUX[3:0]: Analog Channel and Gain Selection Bits**

The value of these bits selects which combination of analog inputs are connected to the ADC. In case of differential input (ADC0 - ADC1 or ADC2 - ADC3), gain selection is also made with these bits. Selecting ADC2 or ADC0 as both inputs to the differential gain stage enables offset measurements. Selecting the single-ended channel ADC4

enables the temperature sensor. Refer to Table 17-4 for details. If these bits are changed during a conversion, the change will not go into effect until this conversion is complete (ADIF in ADCSRA is set).

Table 17-4. Input Channel Selections

MUX[3:0]	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
0000	ADC0 (PB5)	N/A		
0001	ADC1 (PB2)			
0010	ADC2 (PB4)			
0011	ADC3 (PB3)			
0100	N/A	ADC2 (PB4)	ADC2 (PB4)	1x
0101 (1)		ADC2 (PB4)	ADC2 (PB4)	20x
0110		ADC2 (PB4)	ADC3 (PB3)	1x
0111		ADC2 (PB4)	ADC3 (PB3)	20x
1000		ADC0 (PB5)	ADC0 (PB5)	1x
1001		ADC0 (PB5)	ADC0 (PB5)	20x
1010		ADC0 (PB5)	ADC1 (PB2)	1x
1011		ADC0 (PB5)	ADC1 (PB2)	20x
1100 (2)	V_{BG}	N/A		
1101	GND			
1110	N/A			
1111 (3)	ADC4			

Note:
1. For offset calibration, only. See “Operation” on page 123.
2. After switching to internal voltage reference the ADC requires a settling time of 1ms before measurements are stable. Conversions starting before this may not be reliable. The ADC must be enabled during the settling time.
3. For temperature sensor.

17.13.2 ADCSRA – ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0	
0x06	**ADEN**	**ADSC**	**ADATE**	**ADIF**	**ADIE**	**ADPS2**	**ADPS1**	**ADPS0**	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – ADEN: ADC Enable**

Writing this bit to one enables the ADC. By writing it to zero, the ADC is turned off. Turning the ADC off while a conversion is in progress, will terminate this conversion.

- **Bit 6 – ADSC: ADC Start Conversion**

In Single Conversion mode, write this bit to one to start each conversion. In Free Running mode, write this bit to one to start the first conversion. The first conversion after ADSC has been written after the ADC has been enabled, or if ADSC is written at the same time as the ADC is enabled, will take 25 ADC clock cycles instead of the normal 13. This first conversion performs initialization of the ADC.

ADSC will read as one as long as a conversion is in progress. When the conversion is complete, it returns to zero. Writing zero to this bit has no effect.

- **Bit 5 – ADATE: ADC Auto Trigger Enable**

When this bit is written to one, Auto Triggering of the ADC is enabled. The ADC will start a conversion on a positive edge of the selected trigger signal. The trigger source is selected by setting the ADC Trigger Select bits, ADTS in ADCSRB.

- **Bit 4 – ADIF: ADC Interrupt Flag**

This bit is set when an ADC conversion completes and the data registers are updated. The ADC Conversion Complete Interrupt is executed if the ADIE bit and the I-bit in SREG are set. ADIF is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, ADIF is cleared by writing a logical one to the flag. Beware that if doing a Read-Modify-Write on ADCSRA, a pending interrupt can be disabled. This also applies if the SBI and CBI instructions are used.

- **Bit 3 – ADIE: ADC Interrupt Enable**

When this bit is written to one and the I-bit in SREG is set, the ADC Conversion Complete Interrupt is activated.

- **Bits 2:0 – ADPS[2:0]: ADC Prescaler Select Bits**

These bits determine the division factor between the system clock frequency and the input clock to the ADC.

Table 17-5. ADC Prescaler Selections

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

17.13.3 ADCL and ADCH – The ADC Data Register

17.13.3.1 ADLAR = 0

Bit	15	14	13	12	11	10	9	8	
0x05	–	–	–	–	–	–	ADC9	ADC8	ADCH
0x04	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R	R	R	R	
	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

17.13.3.2 ADLAR = 1

Bit	15	14	13	12	11	10	9	8	
0x05	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
0x04	ADC1	ADC0	–	–	–	–	–	–	ADCL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R	R	R	R	
	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

When an ADC conversion is complete, the result is found in these two registers.

When ADCL is read, the ADC Data Register is not updated until ADCH is read. Consequently, if the result is left adjusted and no more than 8-bit precision is required, it is sufficient to read ADCH. Otherwise, ADCL must be read first, then ADCH.

The ADLAR bit in ADMUX, and the MUXn bits in ADMUX affect the way the result is read from the registers. If ADLAR is set, the result is left adjusted. If ADLAR is cleared (default), the result is right adjusted.

- **Bits 9:0 - ADC[9:0]: ADC Conversion Result**

These bits represent the result from the conversion, as detailed in “ADC Conversion Result” on page 132.

17.13.4 ADCSRB – ADC Control and Status Register B

Bit	7	6	5	4	3	2	1	0	
0x03	BIN	ACME	IPR	–	–	ADTS2	ADTS1	ADTS0	ADCSRB
Read/Write	R/W	R/W	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – BIN: Bipolar Input Mode**

The gain stage is working in the unipolar mode as default, but the bipolar mode can be selected by writing the BIN bit in the ADCSRB register. In the unipolar mode only one-sided conversions are supported and the voltage on the positive input must always be larger than the voltage on the negative input. Otherwise the result is saturated to the voltage reference. In the bipolar mode two-sided conversions are supported and the result is represented in the two’s complement form. In the unipolar mode the resolution is 10 bits and the bipolar mode the resolution is 9 bits + 1 sign bit.

- **Bit 5 – IPR: Input Polarity Reversal**

The Input Polarity mode allows software selectable differential input pairs and full 10 bit ADC resolution, in the unipolar input mode, assuming a pre-determined input polarity. If the input polarity is not known it is actually possible to determine the polarity first by using the bipolar input mode (with 9 bit resolution + 1 sign bit ADC measurement). And once determined, set or clear the polarity reversal bit, as needed, for a succeeding 10 bit unipolar measurement.

- **Bits 4:3 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and will always read as zero.

- **Bits 2:0 – ADTS[2:0]: ADC Auto Trigger Source**

If ADATE in ADCSRA is written to one, the value of these bits selects which source will trigger an ADC conversion. If ADATE is cleared, the ADTS[2:0] settings will have no effect. A conversion will be triggered by the rising edge of the selected Interrupt Flag. Note that switching from a trigger source that is cleared to a trigger source that is set, will generate a positive edge on the trigger signal. If ADEN in ADCSRA is set, this will start a conversion. Switching to Free Running mode (ADTS[2:0]=0) will not cause a trigger event, even if the ADC Interrupt Flag is set.

Table 17-6. ADC Auto Trigger Source Selections

ADTS2	ADTS1	ADTS0	Trigger Source
0	0	0	Free Running mode
0	0	1	Analog Comparator
0	1	0	External Interrupt Request 0
0	1	1	Timer/Counter0 Compare Match A
1	0	0	Timer/Counter0 Overflow
1	0	1	Timer/Counter0 Compare Match B
1	1	0	Pin Change Interrupt Request

17.13.5 DIDR0 – Digital Input Disable Register 0

Bit	7	6	5	4	3	2	1	0	
0x14	–	–	ADC0D	ADC2D	ADC3D	ADC1D	AIN1D	AIN0D	DIDR0
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 5:2 – ADC3D:ADC0D: ADC[3:0] Digital Input Disable**

When this bit is written logic one, the digital input buffer on the corresponding ADC pin is disabled. The corresponding PIN register bit will always read as zero when this bit is set. When an analog signal is applied to the ADC[3:0] pin and the digital input from this pin is not needed, this bit should be written logic one to reduce power consumption in the digital input buffer.

18. debugWIRE On-chip Debug System

18.1 Features

- **Complete Program Flow Control**
- **Emulates All On-chip Functions, Both Digital and Analog , except RESET Pin**
- **Real-time Operation**
- **Symbolic Debugging Support (Both at C and Assembler Source Level, or for Other HLLs)**
- **Unlimited Number of Program Break Points (Using Software Break Points)**
- **Non-intrusive Operation**
- **Electrical Characteristics Identical to Real Device**
- **Automatic Configuration System**
- **High-Speed Operation**
- **Programming of Non-volatile Memories**

18.2 Overview

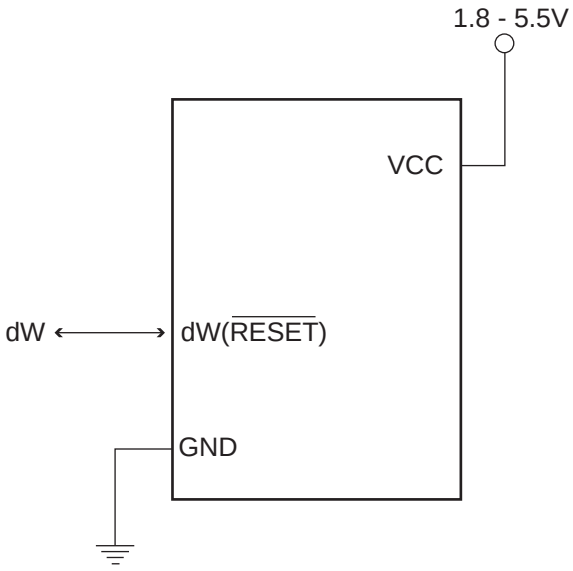
The debugWIRE On-chip debug system uses a One-wire, bi-directional interface to control the program flow, execute AVR instructions in the CPU and to program the different non-volatile memories.

18.3 Physical Interface

When the debugWIRE Enable (DWEN) Fuse is programmed and Lock bits are unprogrammed, the debugWIRE system within the target device is activated. The RESET port pin is configured as a wire-AND (open-drain) bi-directional I/O pin with pull-up enabled and becomes the communication gateway between target and emulator.

Figure 18-1 shows the schematic of a target MCU, with debugWIRE enabled, and the emulator connector. The system clock is not affected by debugWIRE and will always be the clock source selected by the CKSEL Fuses.

Figure 18-1. The debugWIRE Setup



When designing a system where debugWIRE will be used, the following must be observed:

- Pull-Up resistor on the dW/(RESET) line must be in the range of 10k to 20 kΩ. However, the pull-up resistor is optional.
- Connecting the RESET pin directly to V_{CC} will not work.
- Capacitors inserted on the RESET pin must be disconnected when using debugWire.
- All external reset sources must be disconnected.

18.4 Software Break Points

debugWIRE supports Program memory Break Points by the AVR Break instruction. Setting a Break Point in AVR Studio® will insert a BREAK instruction in the Program memory. The instruction replaced by the BREAK instruction will be stored. When program execution is continued, the stored instruction will be executed before continuing from the Program memory. A break can be inserted manually by putting the BREAK instruction in the program.

The Flash must be re-programmed each time a Break Point is changed. This is automatically handled by AVR Studio through the debugWIRE interface. The use of Break Points will therefore reduce the Flash Data retention. Devices used for debugging purposes should not be shipped to end customers.

18.5 Limitations of debugWIRE

The debugWIRE communication pin (dW) is physically located on the same pin as External Reset (RESET). An External Reset source is therefore not supported when the debugWIRE is enabled.

The debugWIRE system accurately emulates all I/O functions when running at full speed, i.e., when the program in the CPU is running. When the CPU is stopped, care must be taken while accessing some of the I/O Registers via the debugger (AVR Studio). See the debugWIRE documentation for detailed description of the limitations.

The debugWIRE interface is asynchronous, which means that the debugger needs to synchronize to the system clock. If the system clock is changed by software (e.g. by writing CLKPS bits) communication via debugWIRE may fail. Also, clock frequencies below 100kHz may cause communication problems.

A programmed DWEN Fuse enables some parts of the clock system to be running in all sleep modes. This will increase the power consumption while in sleep. Thus, the DWEN Fuse should be disabled when debugWire is not used.

18.6 Register Description

The following section describes the registers used with the debugWire.

18.6.1 DWDR – debugWire Data Register

Bit	7	6	5	4	3	2	1	0	
0x22	DWDR7	DWDR6	DWDR5	DWDR4	DWDR3	DWDR2	DWDR1	DWDR0	DWDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The DWDR Register provides a communication channel from the running program in the MCU to the debugger. This register is only accessible by the debugWIRE and can therefore not be used as a general purpose register in the normal operations.

19. Self-Programming the Flash

The device provides a Self-Programming mechanism for downloading and uploading program code by the MCU itself. The Self-Programming can use any available data interface and associated protocol to read code and write (program) that code into the Program memory. The SPM instruction is disabled by default but it can be enabled by programming the SELFPRGEN fuse (to “0”).

The Program memory is updated in a page by page fashion. Before programming a page with the data stored in the temporary page buffer, the page must be erased. The temporary page buffer is filled one word at a time using SPM and the buffer can be filled either before the Page Erase command or between a Page Erase and a Page Write operation:

Alternative 1, fill the buffer before a Page Erase

- Fill temporary page buffer
- Perform a Page Erase
- Perform a Page Write

Alternative 2, fill the buffer after Page Erase

- Perform a Page Erase
- Fill temporary page buffer
- Perform a Page Write

If only a part of the page needs to be changed, the rest of the page must be stored (for example in the temporary page buffer) before the erase, and then be re-written. When using alternative 1, the Boot Loader provides an effective Read-Modify-Write feature which allows the user software to first read the page, do the necessary changes, and then write back the modified data. If alternative 2 is used, it is not possible to read the old data while loading since the page is already erased. The temporary page buffer can be accessed in a random sequence. It is essential that the page address used in both the Page Erase and Page Write operation is addressing the same page.

19.1 Performing Page Erase by SPM

To execute Page Erase, set up the address in the Z-pointer, write “00000011” to SPMCSR and execute SPM within four clock cycles after writing SPMCSR. The data in R1 and R0 is ignored. The page address must be written to PCPAGE in the Z-register. Other bits in the Z-pointer will be ignored during this operation.

Note: The CPU is halted during the Page Erase operation.

19.2 Filling the Temporary Buffer (Page Loading)

To write an instruction word, set up the address in the Z-pointer and data in R1:R0, write “00000001” to SPMCSR and execute SPM within four clock cycles after writing SPMCSR. The content of PCWORD in the Z-register is used to address the data in the temporary buffer. The temporary buffer will auto-erase after a Page Write operation or by writing the CTPB bit in SPMCSR. It is also erased after a system reset. Note that it is not possible to write more than one time to each address without erasing the temporary buffer.

If the EEPROM is written in the middle of an SPM Page Load operation, all data loaded will be lost.

19.3 Performing a Page Write

To execute Page Write, set up the address in the Z-pointer, write “00000101” to SPMCSR and execute SPM within four clock cycles after writing SPMCSR. The data in R1 and R0 is ignored. The page address must be written to PCPAGE. Other bits in the Z-pointer must be written to zero during this operation.

Note: The CPU is halted during the Page Write operation.

19.4 Addressing the Flash During Self-Programming

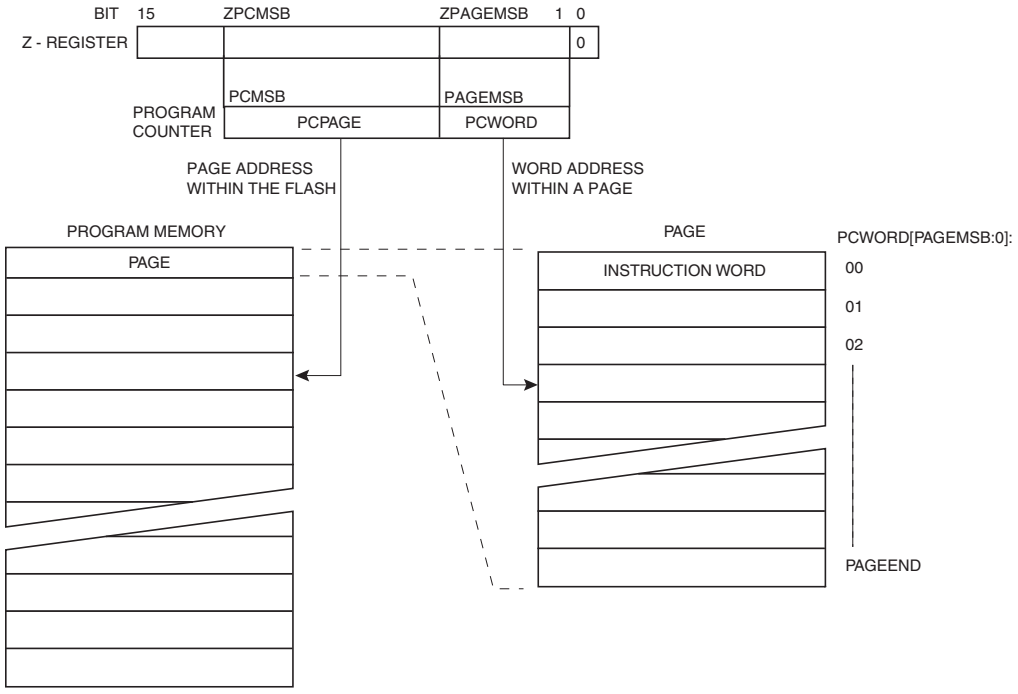
The Z-pointer is used to address the SPM commands.

Bit	15	14	13	12	11	10	9	8
ZH (R31)	Z15	Z14	Z13	Z12	Z11	Z10	Z9	Z8
ZL (R30)	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0
	7	6	5	4	3	2	1	0

Since the Flash is organized in pages (see Table 20-8 on page 150), the Program Counter can be treated as having two different sections. One section, consisting of the least significant bits, is addressing the words within a page, while the most significant bits are addressing the pages. This is shown in Figure 19-1. Note that the Page Erase and Page Write operations are addressed independently. Therefore it is of major importance that the software addresses the same page in both the Page Erase and Page Write operation.

The LPM instruction uses the Z-pointer to store the address. Since this instruction addresses the Flash byte-by-byte, also the LSB (bit Z0) of the Z-pointer is used.

Figure 19-1. Addressing the Flash During SPM(1)



Note: 1. The different variables used in Figure 19-1 are listed in Table 20-8 on page 150.

19.5 EEPROM Write Prevents Writing to SPMCSR

Note that an EEPROM write operation will block all software programming to Flash. Reading the Fuses and Lock bits from software will also be prevented during the EEPROM write operation. It is recommended that the user checks the status bit (EEPE) in the EECR Register and verifies that the bit is cleared before writing to the SPMCSR Register.

19.6 Reading Lock, Fuse and Signature Data from Software

It is possible to read fuse and lock bits from firmware. In addition, firmware can also read data from the device signature imprint table (see page 149).

Note: Fuse and Lock bits that are programmed, will be read as zero. Fuse and Lock bits that are unprogrammed, will be read as one.

19.6.1 Reading Lock Bits from Firmware

Issuing an LPM instruction within three CPU cycles after RFLB and SELFPRGEN bits have been set in SPMCSR will return lock bit values in the destination register. The RFLB and SELFPRGEN bits automatically clear upon completion of reading the lock bits, or if no LPM instruction is executed within three CPU cycles, or if no SPM instruction is executed within four CPU cycles. When RFLB and SELFPRGEN are cleared LPM functions normally.

To read the lock bits, follow the below procedure:

1. Load the Z-pointer with 0x0001.
2. Set RFLB and SELFPRGEN bits in SPMCSR.
3. Issue an LPM instruction within three clock cycles.
4. Read the lock bits from the LPM destination register.

If successful, the contents of the destination register are as follows.

Bit	7	6	5	4	3	2	1	0
Rd	–	–	–	–	–	–	LB2	LB1

See section “Program And Data Memory Lock Bits” on page 147 for more information.

19.6.2 Reading Fuse Bits from Firmware

The algorithm for reading fuse bytes is similar to the one described above for reading lock bits, only the addresses are different. To read the Fuse Low Byte (FLB), follow the below procedure:

1. Load the Z-pointer with 0x0000.
2. Set RFLB and SELFPRGEN bits in SPMCSR.
3. Issue an LPM instruction within three clock cycles.
4. Read the FLB from the LPM destination register.

If successful, the contents of the destination register are as follows.

Bit	7	6	5	4	3	2	1	0
Rd	FLB7	FLB6	FLB5	FLB4	FLB3	FLB2	FLB1	FLB0

Refer to Table 20-5 on page 149 for a detailed description and mapping of the Fuse Low Byte.

To read the Fuse High Byte (FHB), simply replace the address in the Z-pointer with 0x0003 and repeat the procedure above. If successful, the contents of the destination register are as follows.

Bit	7	6	5	4	3	2	1	0
Rd	FHB7	FHB6	FHB5	FHB4	FHB3	FHB2	FHB1	FHB0

Refer to Table 20-4 on page 148 for detailed description and mapping of the Fuse High Byte.

To read the Fuse Extended Byte (FEB), replace the address in the Z-pointer with 0x0002 and repeat the previous procedure. If successful, the contents of the destination register are as follows.

Bit	7	6	5	4	3	2	1	0
Rd	FEB7	FEB6	FEB5	FEB4	FEB3	FEB2	FEB1	FEB0

Refer to Table 20-3 on page 148 for detailed description and mapping of the Fuse Extended Byte.

19.6.3 Reading Device Signature Imprint Table from Firmware

To read the contents of the device signature imprint table, follow the below procedure:

1. Load the Z-pointer with the table index.
2. Set RSIG and SPMEN bits in SPMCSR.
3. Issue an LPM instruction within three clock cycles.
4. Read table data from the LPM destination register.

See program example below.

Assembly Code Example[1]

```
DSIT_read:
  ; Uses Z-pointer as table index
  ldi  ZH, 0
  ldi  ZL, 1
  ; Preload SPMCSR bits into R16, then write to SPMCSR
  ldi  r16, (1<<RSIG)|(1<<SPMEN)
  out  SPMCSR, r16
  ; Issue LPM. Table data will be returned into r17
  lpm  r17, Z
  ret
```

Note: 1. See "Code Examples" on page 6.

If successful, the contents of the destination register are as described in section "Device Signature Imprint Table" on page 149.

19.7 Preventing Flash Corruption

During periods of low V_{CC}, the Flash program can be corrupted because the supply voltage is too low for the CPU and the Flash to operate properly. These issues are the same as for board level systems using the Flash, and the same design solutions should be applied.

A Flash program corruption can be caused by two situations when the voltage is too low. First, a regular write sequence to the Flash requires a minimum voltage to operate correctly. Secondly, the CPU itself can execute instructions incorrectly, if the supply voltage for executing instructions is too low.

Flash corruption can easily be avoided by following these design recommendations (one is sufficient):

1. Keep the AVR RESET active (low) during periods of insufficient power supply voltage. This can be done by enabling the internal Brown-out Detector (BOD) if the operating voltage matches the detection level. If

not, an external low V_{CC} reset protection circuit can be used. If a reset occurs while a write operation is in progress, the write operation will be completed provided that the power supply voltage is sufficient.

2. Keep the AVR core in Power-down sleep mode during periods of low V_{CC}. This will prevent the CPU from attempting to decode and execute instructions, effectively protecting the SPMCSR Register and thus the Flash from unintentional writes.

19.8 Programming Time for Flash when Using SPM

The calibrated RC Oscillator is used to time Flash accesses. Table 19-1 shows the typical programming time for Flash accesses from the CPU.

Table 19-1. SPM Programming Time[1]

Symbol	Min Programming Time	Max Programming Time
Flash write (Page Erase, Page Write, and write Lock bits by SPM)	3.7 ms	4.5 ms

Note: 1. Minimum and maximum programming time is per individual operation.

19.9 Register Description

19.9.1 SPMCSR – Store Program Memory Control and Status Register

The Store Program Memory Control and Status Register contains the control bits needed to control the Program memory operations.

Bit	7	6	5	4	3	2	1	0	
0x37	–	–	RSIG	CTPB	RFLB	PGWRT	PGERS	SPMEN	SPMCSR
Read/Write	R	R	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:6 – Res: Reserved Bits**

These bits are reserved bits in the ATtiny25/45/85 and always read as zero.

- **Bit 5 – RSIG: Read Device Signature Imprint Table**

Issuing an LPM instruction within three cycles after RSIG and SPMEN bits have been set in SPMCSR will return the selected data (depending on Z-pointer value) from the device signature imprint table into the destination register. See “Device Signature Imprint Table” on page 149 for details.

- **Bit 4 – CTPB: Clear Temporary Page Buffer**

If the CTPB bit is written while filling the temporary page buffer, the temporary page buffer will be cleared and the data will be lost.

- **Bit 3 – RFLB: Read Fuse and Lock Bits**

An LPM instruction within three cycles after RFLB and SPMEN are set in the SPMCSR Register, will read either the Lock bits or the Fuse bits (depending on Z0 in the Z-pointer) into the destination register. See “EEPROM Write Prevents Writing to SPMCSR” on page 142 for details.

- **Bit 2 – PGWRT: Page Write**

If this bit is written to one at the same time as SPMEN, the next SPM instruction within four clock cycles executes Page Write, with the data stored in the temporary buffer. The page address is taken from the high part of the Z-pointer. The data in R1 and R0 are ignored. The PGWRT bit will auto-clear upon completion of a Page Write, or if no SPM instruction is executed within four clock cycles. The CPU is halted during the entire Page Write operation.

- **Bit 1 – PGERS: Page Erase**

If this bit is written to one at the same time as SPMEN, the next SPM instruction within four clock cycles executes Page Erase. The page address is taken from the high part of the Z-pointer. The data in R1 and R0 are ignored. The PGERS bit will auto-clear upon completion of a Page Erase, or if no SPM instruction is executed within four clock cycles. The CPU is halted during the entire Page Write operation.

- **Bit 0 – SPMEN: Store Program Memory Enable**

This bit enables the SPM instruction for the next four clock cycles. If set to one together with RSIG, CTPB, RFLB, PGWRT or PGERS, the following LPM/SPM instruction will have a special meaning, as described elsewhere.

If only SPMEN is written, the following SPM instruction will store the value in R1:R0 in the temporary page buffer addressed by the Z-pointer. The LSB of the Z-pointer is ignored. The SPMEN bit will auto-clear upon completion of an SPM instruction, or if no SPM instruction is executed within four clock cycles. During Page Erase and Page Write, the SPMEN bit remains high until the operation is completed.

20. Memory Programming

This section describes the different methods for Programming the ATtiny25/45/85 memories.

20.1 Program And Data Memory Lock Bits

ATtiny25/45/85 provides two Lock bits which can be left unprogrammed (“1”) or can be programmed (“0”) to obtain the additional security listed in Table 20-2. Lock bits can be erased to “1” with the Chip Erase command, only.

Program memory can be read out via the debugWIRE interface when the DWEN fuse is programmed, even if the Lock Bits are set. Thus, when Lock Bit security is required debugWIRE should always be disabled (by clearing the DWEN fuse).

Table 20-1. Lock Bit Byte[1]

Lock Bit	Bit No	Description	Default Value
	7	–	1 (unprogrammed)
	6	–	1 (unprogrammed)
	5	–	1 (unprogrammed)
	4	–	1 (unprogrammed)
	3	–	1 (unprogrammed)
	2	–	1 (unprogrammed)
LB2	1	Lock bit	1 (unprogrammed)
LB1	0	Lock bit	1 (unprogrammed)

Note: 1. “1” means unprogrammed, “0” means programmed

Table 20-2. Lock Bit Protection Modes[1][2]

Memory Lock Bits			Protection Type
LB Mode	LB2	LB1	
1	1	1	No memory lock features enabled.
2	1	0	Further programming of the Flash and EEPROM is disabled in High-voltage and Serial Programming mode. The Fuse bits are locked in both Serial and High-voltage Programming mode.[1] debugWire is disabled.
3	0	0	Further programming and verification of the Flash and EEPROM is disabled in High-voltage and Serial Programming mode. The Fuse bits are locked in both Serial and High-voltage Programming mode.[1] debugWire is disabled.

Notes: 1. Program the Fuse bits before programming the LB1 and LB2.
2. “1” means unprogrammed, “0” means programmed

Lock bits can also be read by device firmware. See section “Reading Lock, Fuse and Signature Data from Software” on page 143.

20.2 Fuse Bytes

ATtiny25/45/85 has three fuse bytes, as described in Table 20-3, Table 20-4, and Table 20-5. Note that fuses are read as logical zero, “0”, when programmed.

Table 20-3. Fuse Extended Byte

Fuse High Byte	Bit No	Description	Default Value
	7	-	1 (unprogrammed)
	6	-	1 (unprogrammed)
	5	-	1 (unprogrammed)
	4	-	1 (unprogrammed)
	3	-	1 (unprogrammed)
	2	-	1 (unprogrammed)
	1	-	1 (unprogrammed)
SELFPRGEN (1)	0	Self-programming enabled	1 (unprogrammed)

Notes: 1. Enables SPM instruction. See “Self-Programming the Flash” on page 141.

Table 20-4. Fuse High Byte

Fuse High Byte	Bit No	Description	Default Value
RSTDISBL (1) (2)	7	External reset disabled	1 (unprogrammed)
DWEN (1) (2) (3)	6	DebugWIRE enabled	1 (unprogrammed)
SPIEN (4)	5	Serial program and data download enabled	0 (programmed) (SPI prog. enabled)
WDTON (5)	4	Watchdog timer always on	1 (unprogrammed)
EESAVE	3	EEPROM preserves chip erase	1 (unprogrammed) (EEPROM not preserved)
BODLEVEL2 (6)	2	Brown-out Detector trigger level	1 (unprogrammed)
BODLEVEL1 (6)	1	Brown-out Detector trigger level	1 (unprogrammed)
BODLEVEL0 (6)	0	Brown-out Detector trigger level	1 (unprogrammed)

Notes:
1. Controls use of $\overline{\text{RESET}}$ pin. See “Alternate Functions of Port B” on page 60.
2. After this fuse has been programmed device can be programmed via high-voltage serial mode, only.
3. Must be unprogrammed when lock bit security is required. See “Program And Data Memory Lock Bits” on page 147.
4. This fuse is not accessible in SPI programming mode.
5. See “WDTCR – Watchdog Timer Control Register” on page 45 for details.
6. See table “BODLEVEL Fuse Coding. TA = -40°C to +85°C” on page 166.

Table 20-5. Fuse Low Byte

Fuse Low Byte	Bit No	Description	Default Value
CKDIV8 (1)	7	Clock divided by 8	0 (programmed)
CKOUT (2)	6	Clock output enabled	1 (unprogrammed)
SUT1 (3)	5	Start-up time setting	1 (unprogrammed)(3)
SUT0 (3)	4	Start-up time setting	0 (programmed)(3)
CKSEL3 (4)	3	Clock source setting	0 (programmed)(4)
CKSEL2 (4)	2	Clock source setting	0 (programmed)(4)
CKSEL1 (4)	1	Clock source setting	1 (unprogrammed)(4)
CKSEL0 (4)	0	Clock source setting	0 (programmed)(4)

Notes:
1. See “System Clock Prescaler” on page 31 for details.
2. Allows system clock to be output on pin. See “Clock Output Buffer” on page 31 for details.
3. The default value gives maximum start-up time for the default clock source. See Table 6-7 on page 28 for details.
4. The default setting selects internal, 8 MHz RC oscillator. See Table 6-6 on page 27 for details.

Note that fuse bits are locked if Lock Bit 1 (LB1) is programmed. Fuse bits should be programmed before lock bits. The status of fuse bits is not affected by chip erase.

Lock bits can also be read by device firmware. See section “Reading Lock, Fuse and Signature Data from Software” on page 143.

20.2.1 Latching of Fuses

Fuse values are latched when the device enters programming mode and changes to fuse values will have no effect until the part leaves programming mode. This does not apply to the EESAVE Fuse which takes effect once it is programmed. Fuses are also latched on power-up.

20.3 Device Signature Imprint Table

The device signature imprint table is a dedicated memory area used for storing miscellaneous device information, such as the device signature and oscillator calibration data. Most of this memory segment is reserved for internal use, as outlined in Table 20-6.

Table 20-6. Contents of Device Signature Imprint Table.

Address	High Byte
0x00	Signature byte 0 (1)
0x01	Calibration data for internal oscillator at 8.0 MHz (2)
0x02	Signature byte 1 (1)
0x03	Calibration data for internal oscillator at 6.4 MHz (2)
0x04	Signature byte 2 (1)
0x05 ... 0x2A	Reserved for internal use

Notes:
1. See section “Signature Bytes” for more information.
2. See section “Calibration Bytes” for more information.

20.3.1 Signature Bytes

All Atmel microcontrollers have a three-byte signature code which identifies the device. This code can be read in both serial and high-voltage programming mode, even when the device is locked.

Signature bytes can also be read by the device firmware. See section “Reading Lock, Fuse and Signature Data from Software” on page 143.

The three signature bytes reside in a separate address space called the device signature imprint table. The signature data for ATtiny25/45/85 is given in Table 20-7.

Table 20-7. Device Signature Bytes

Part	Signature Byte 0	Signature Byte 1	Signature Byte 0
ATtiny25	0x1E	0x91	0x08
ATtiny45	0x1E	0x92	0x06
ATtiny85	0x1E	0x93	0x0B

20.3.2 Calibration Bytes

The device signature imprint table of ATtiny25/45/85 contains two bytes of calibration data for the internal RC Oscillator, as shown in Table 20-6 on page 149. In normal mode of operation the calibration data for 8 MHz operation is automatically fetched and written to the OSCCAL register during reset. In ATtiny15 compatibility mode the calibration data for 6.4 MHz operation is used instead. This procedure guarantees the internal oscillator is always calibrated to the correct frequency.

Calibration bytes can also be read by the device firmware. See section “Reading Lock, Fuse and Signature Data from Software” on page 143.

20.4 Page Size

Table 20-8. No. of Words in a Page and No. of Pages in the Flash

Device	Flash Size	Page Size	PCWORD	No. of Pages	PCPAGE	PCMSB
ATtiny25	1K words (2K bytes)	16 words	PC[3:0]	64	PC[9:4]	9
ATtiny45	2K words (4K bytes)	32 words	PC[4:0]	64	PC[10:5]	10
ATtiny85	4K words (8K bytes)	32 words	PC[4:0]	128	PC[11:5]	11

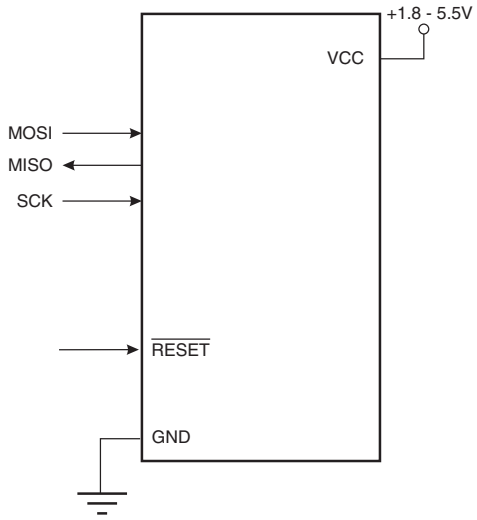
Table 20-9. No. of Words in a Page and No. of Pages in the EEPROM

Device	EEPROM Size	Page Size	PCWORD	No. of Pages	PCPAGE	EEAMSB
ATtiny25	128 bytes	4 bytes	EEA[1:0]	32	EEA[6:2]	6
ATtiny45	256 bytes	4 bytes	EEA[1:0]	64	EEA[7:2]	7
ATtiny85	512 bytes	4 bytes	EEA[1:0]	128	EEA[8:2]	8

20.5 Serial Downloading

Both the Flash and EEPROM memory arrays can be programmed using the serial SPI bus while $\overline{\text{RESET}}$ is pulled to GND. The serial interface consists of pins SCK, MOSI (input) and MISO (output). See below.

Figure 20-1. Serial Programming and Verify[1]



Notes: 1. If the device is clocked by the internal Oscillator, it is no need to connect a clock source to the CLKI pin.

After $\overline{\text{RESET}}$ is set low, the Programming Enable instruction needs to be executed first before program/erase operations can be executed.

Table 20-10. Pin Mapping Serial Programming

Symbol	Pins	I/O	Description
MOSI	PB0	I	Serial Data in
MISO	PB1	O	Serial Data out
SCK	PB2	I	Serial Clock

Note: In Table 20-10 above, the pin mapping for SPI programming is listed. Not all parts use the SPI pins dedicated for the internal SPI interface.

When programming the EEPROM, an auto-erase cycle is built into the self-timed programming operation (in the Serial mode ONLY) and there is no need to first execute the Chip Erase instruction. The Chip Erase operation turns the content of every memory location in both the Program and EEPROM arrays into 0xFF.

Depending on CKSEL Fuses, a valid clock must be present. The minimum low and high periods for the serial clock (SCK) input are defined as follows:

Low: > 2 CPU clock cycles for f_{ck} < 12 MHz, 3 CPU clock cycles for f_{ck} >= 12 MHz

High: > 2 CPU clock cycles for f_{ck} < 12 MHz, 3 CPU clock cycles for f_{ck} >= 12 MHz

20.5.1 Serial Programming Algorithm

When writing serial data to the ATtiny25/45/85, data is clocked on the rising edge of SCK.

When reading data from the ATtiny25/45/85, data is clocked on the falling edge of SCK. See Figure 21-4 and Figure 21-5 for timing details.

To program and verify the ATtiny25/45/85 in the Serial Programming mode, the following sequence is recommended (see four byte instruction formats in Table 20-12):

1. Power-up sequence: apply power between V_{CC} and GND while $\overline{\text{RESET}}$ and SCK are set to “0”
   – In some systems, the programmer can not guarantee that SCK is held low during power-up. In this case, $\overline{\text{RESET}}$ must be given a positive pulse after SCK has been set to '0'. The duration of the pulse must be at least t_{RST} plus two CPU clock cycles. See Table 21-4 on page 165 for minimum pulse width on $\overline{\text{RESET}}$ pin, t_{RST}
2. Wait for at least 20 ms and enable serial programming by sending the Programming Enable serial instruction to pin MOSI.
3. The serial programming instructions will not work if the communication is out of synchronization. When in sync. the second byte (0x53), will echo back when issuing the third byte of the Programming Enable instruction. Whether the echo is correct or not, all four bytes of the instruction must be transmitted. If the 0x53 did not echo back, give $\overline{\text{RESET}}$ a positive pulse and issue a new Programming Enable command.
4. The Flash is programmed one page at a time. The memory page is loaded one byte at a time by supplying the 5 LSB of the address and data together with the Load Program memory Page instruction. To ensure correct loading of the page, the data low byte must be loaded before data high byte is applied for a given address. The Program memory Page is stored by loading the Write Program memory Page instruction with the 6 MSB of the address. If polling (RDY/BSY) is not used, the user must wait at least t_{WD_FLASH} before issuing the next page. (See Table 20-11.) Accessing the serial programming interface before the Flash write operation completes can result in incorrect programming.
5. **A:** The EEPROM array is programmed one byte at a time by supplying the address and data together with the appropriate Write instruction. An EEPROM memory location is first automatically erased before new data is written. If polling (RDY/BSY) is not used, the user must wait at least t_{WD_EEPROM} before issuing the next byte. (See Table 20-11.) In a chip erased device, no 0xFFs in the data file(s) need to be programmed.
   B: The EEPROM array is programmed one page at a time. The Memory page is loaded one byte at a time by supplying the 2 LSB of the address and data together with the Load EEPROM Memory Page instruction. The EEPROM Memory Page is stored by loading the Write EEPROM Memory Page Instruction with the 6 MSB of the address. When using EEPROM page access only byte locations loaded with the Load EEPROM Memory Page instruction is altered. The remaining locations remain unchanged. If polling (RDY/BSY) is not used, the used must wait at least t_{WD_EEPROM} before issuing the next page (See Table 20-9). In a chip erased device, no 0xFF in the data file(s) need to be programmed.
6. Any memory location can be verified by using the Read instruction which returns the content at the selected address at serial output MISO.
7. At the end of the programming session, $\overline{\text{RESET}}$ can be set high to commence normal operation.
8. Power-off sequence (if needed):
   Set $\overline{\text{RESET}}$ to “1”.
   Turn V_{CC} power off.

20.5.2 Serial Programming Instruction set

Table 20-11. Minimum Wait Delay Before Writing the Next Flash or EEPROM Location

Symbol	Minimum Wait Delay
t_{WD_FLASH}	4.5 ms
t_{WD_EEPROM}	4.0 ms
t_{WD_ERASE}	9.0 ms
t_{WD_FUSE}	4.5 ms

Table 20-12 on page 153 and Figure 20-2 on page 154 describes the Instruction set.

Table 20-12. Serial Programming Instruction Set

	Instruction Format			
Instruction/Operation	**Byte 1**	**Byte 2**	**Byte 3**	**Byte4**
Programming Enable	$AC	$53	$00	$00
Chip Erase (Program Memory/EEPROM)	$AC	$80	$00	$00
Poll RDY/$\overline{BSY}$	$F0	$00	$00	data byte out
Load Instructions				
Load Extended Address byte[1]	$4D	$00	Extended adr	$00
Load Program Memory Page, High byte	$48	adr MSB	adr LSB	high data byte in
Load Program Memory Page, Low byte	$40	adr MSB	adr LSB	low data byte in
Load EEPROM Memory Page (page access)	$C1	$00	0000 000aa	data byte in
Read Instructions				
Read Program Memory, High byte	$28	adr MSB	adr LSB	high data byte out
Read Program Memory, Low byte	$20	adr MSB	adr LSB	low data byte out
Read EEPROM Memory	$A0	$00	00aa aaaa	data byte out
Read Lock bits	$58	$00	$00	data byte out
Read Signature Byte	$30	$00	0000 000aa	data byte out
Read Fuse bits	$50	$00	$00	data byte out
Read Fuse High bits	$58	$08	$00	data byte out
Read Extended Fuse Bits	$50	$08	$00	data byte out
Read Calibration Byte	$38	$00	$00	data byte out
Write Instructions[6]				
Write Program Memory Page	$4C	adr MSB	adr LSB	$00
Write EEPROM Memory	$C0	$00	00aa aaaa	data byte in
Write EEPROM Memory Page (page access)	$C2	$00	00aa aa00	$00
Write Lock bits	$AC	$E0	$00	data byte in
Write Fuse bits	$AC	$A0	$00	data byte in
Write Fuse High bits	$AC	$A8	$00	data byte in
Write Extended Fuse Bits	$AC	$A4	$00	data byte in

Notes: 1. Not all instructions are applicable for all parts.

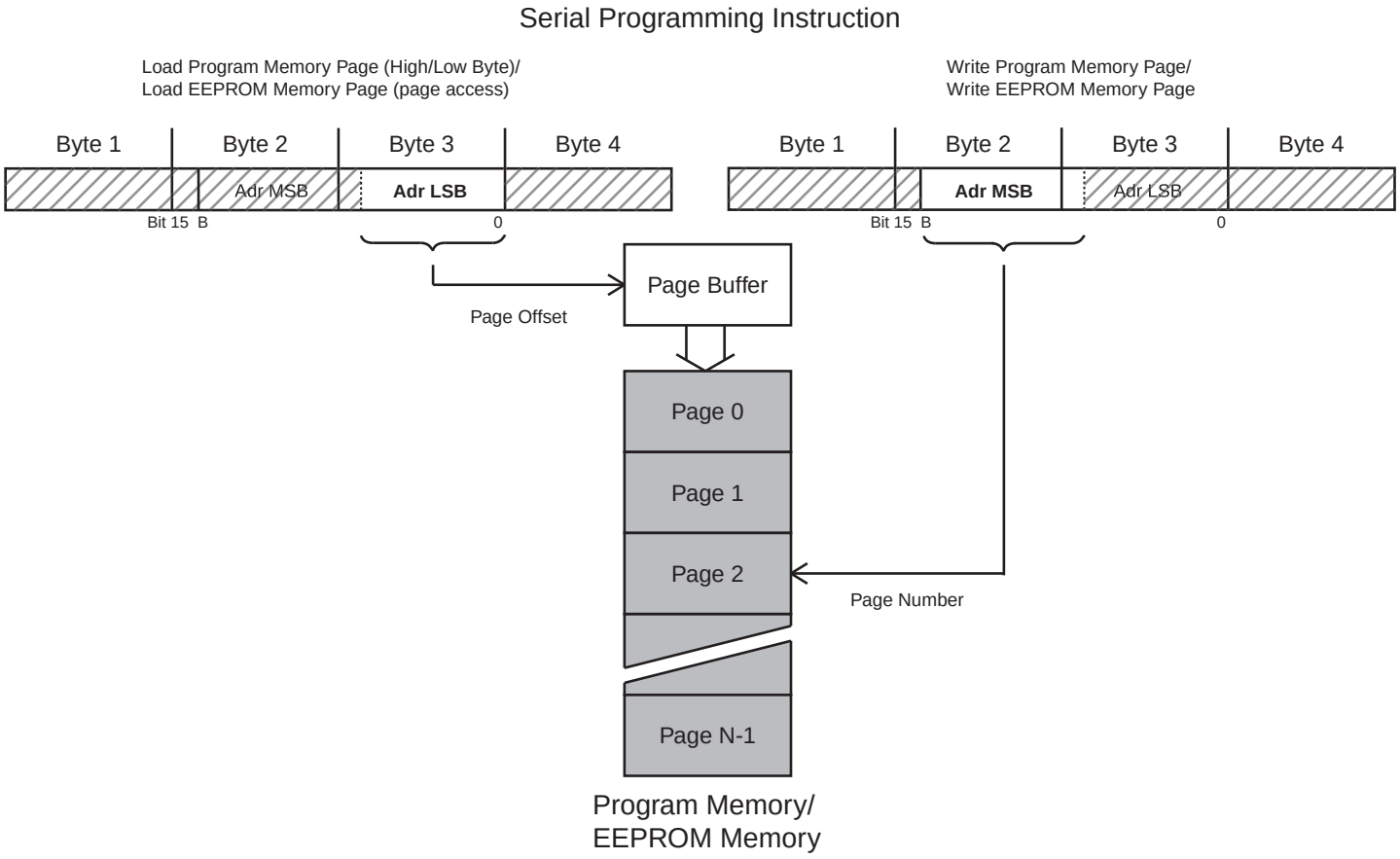
2. a = address
3. Bits are programmed ‘0’, unprogrammed ‘1’.
4. To ensure future compatibility, unused Fuses and Lock bits should be unprogrammed (‘1’) .
5. Refer to the correspondig section for Fuse and Lock bits, Calibration and Signature bytes and Page size.
6. Instructions accessing program memory use a word address. This address may be random within the page range.
7. See htt://www.atmel.com/avr for Application Notes regarding programming and programmers.

If the LSB in RDY/BSY data byte out is ‘1’, a programming operation is still pending. Wait until this bit returns ‘0’ before the next instruction is carried out.

Within the same page, the low data byte must be loaded prior to the high data byte.

After data is loaded to the page buffer, program the EEPROM page, see Figure 20-2 on page 154.

Figure 20-2. Serial Programming Instruction example



20.6 High-voltage Serial Programming

This section describes how to program and verify Flash Program memory, EEPROM Data memory, Lock bits and Fuse bits in the ATtiny25/45/85.

Figure 20-3. High-voltage Serial Programming

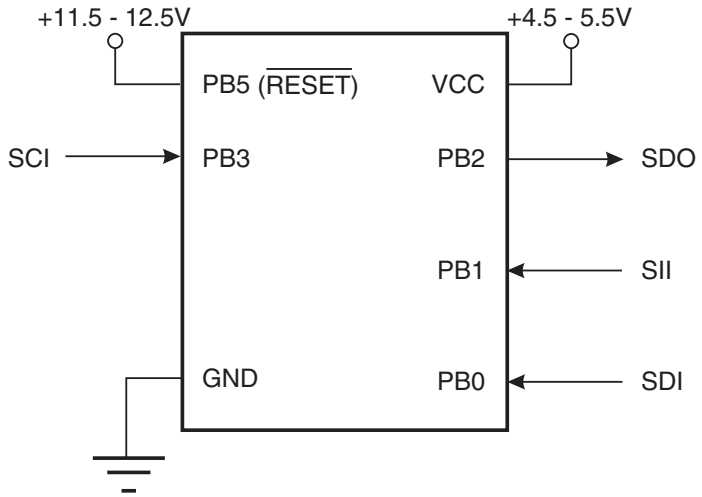


Table 20-13. Pin Name Mapping

Signal Name in High-voltage Serial Programming Mode	Pin Name	I/O	Function
SDI	PB0	I	Serial Data Input
SII	PB1	I	Serial Instruction Input
SDO	PB2	O	Serial Data Output
SCI	PB3	I	Serial Clock Input (min. 220ns period)

The minimum period for the Serial Clock Input (SCI) during High-voltage Serial Programming is 220 ns.

Table 20-14. Pin Values Used to Enter Programming Mode

Pin	Symbol	Value
SDI	Prog_enable[0]	0
SII	Prog_enable[1]	0
SDO	Prog_enable[2]	0

20.7 High-voltage Serial Programming Algorithm

To program and verify the ATtiny25/45/85 in the High-voltage Serial Programming mode, the following sequence is recommended (See instruction formats in Table 20-16):

20.7.1 Enter High-voltage Serial Programming Mode

The following algorithm puts the device in High-voltage Serial Programming mode:

1. Set Prog_enable pins listed in Table 20-14 to “000”, RESET pin and V_{CC} to 0V.
2. Apply 4.5 - 5.5V between V_{CC} and GND. Ensure that V_{CC} reaches at least 1.8V within the next 20 µs.
3. Wait 20 - 60 µs, and apply 11.5 - 12.5V to RESET.
4. Keep the Prog_enable pins unchanged for at least 10 µs after the High-voltage has been applied to ensure the Prog_enable Signature has been latched.
5. Release the Prog_enable[2] pin to avoid drive contention on the Prog_enable[2]/SDO pin.
6. Wait at least 300 µs before giving any serial instructions on SDI/SII.
7. Exit Programming mode by power the device down or by bringing RESET pin to 0V.

If the rise time of the V_{CC} is unable to fulfill the requirements listed above, the following alternative algorithm can be used:

1. Set Prog_enable pins listed in Table 20-14 to “000”, RESET pin and V_{CC} to 0V.
2. Apply 4.5 - 5.5V between V_{CC} and GND.
3. Monitor V_{CC}, and as soon as V_{CC} reaches 0.9 - 1.1V, apply 11.5 - 12.5V to RESET.
4. Keep the Prog_enable pins unchanged for at least 10 µs after the High-voltage has been applied to ensure the Prog_enable Signature has been latched.
5. Release the Prog_enable[2] pin to avoid drive contention on the Prog_enable[2]/SDO pin.
6. Wait until V_{CC} actually reaches 4.5 - 5.5V before giving any serial instructions on SDI/SII.
7. Exit Programming mode by power the device down or by bringing RESET pin to 0V.

Table 20-15. High-voltage Reset Characteristics

Supply Voltage	RESET Pin High-voltage Threshold	Minimum High-voltage Period for Latching Prog_enable
V_{CC}	V_{HVRST}	t_{HVRST}
4.5V	11.5V	100 ns
5.5V	11.5V	100 ns

20.7.2 Considerations for Efficient Programming

The loaded command and address are retained in the device during programming. For efficient programming, the following should be considered.

- The command needs only be loaded once when writing or reading multiple memory locations.
- Skip writing the data value 0xFF that is the contents of the entire EEPROM (unless the EESAVE Fuse is programmed) and Flash after a Chip Erase.
- Address High byte needs only be loaded before programming or reading a new 256 word window in Flash or 256 byte EEPROM. This consideration also applies to Signature bytes reading.

20.7.3 Chip Erase

The Chip Erase will erase the Flash and EEPROM[1] memories plus Lock bits. The Lock bits are not reset until the Program memory has been completely erased. The Fuse bits are not changed. A Chip Erase must be performed before the Flash and/or EEPROM are re-programmed.

Note: 1. The EEPROM memory is preserved during Chip Erase if the EESAVE Fuse is programmed.

1. Load command “Chip Erase” (see Table 20-16).
2. Wait after Instr. 3 until SDO goes high for the “Chip Erase” cycle to finish.
3. Load Command “No Operation”.

20.7.4 Programming the Flash

The Flash is organized in pages, see Table 20-12 on page 153. When programming the Flash, the program data is latched into a page buffer. This allows one page of program data to be programmed simultaneously. The following procedure describes how to program the entire Flash memory:

1. Load Command “Write Flash” (see Table 20-16).
2. Load Flash Page Buffer.
3. Load Flash High Address and Program Page. Wait after Instr. 3 until SDO goes high for the “Page Programming” cycle to finish.
4. Repeat 2 through 3 until the entire Flash is programmed or until all data has been programmed.
5. End Page Programming by Loading Command “No Operation”.

When writing or reading serial data to the ATtiny25/45/85, data is clocked on the rising edge of the serial clock, see Figure 20-5, Figure 21-6 and Table 21-12 for details.

Figure 20-4. Addressing the Flash which is Organized in Pages

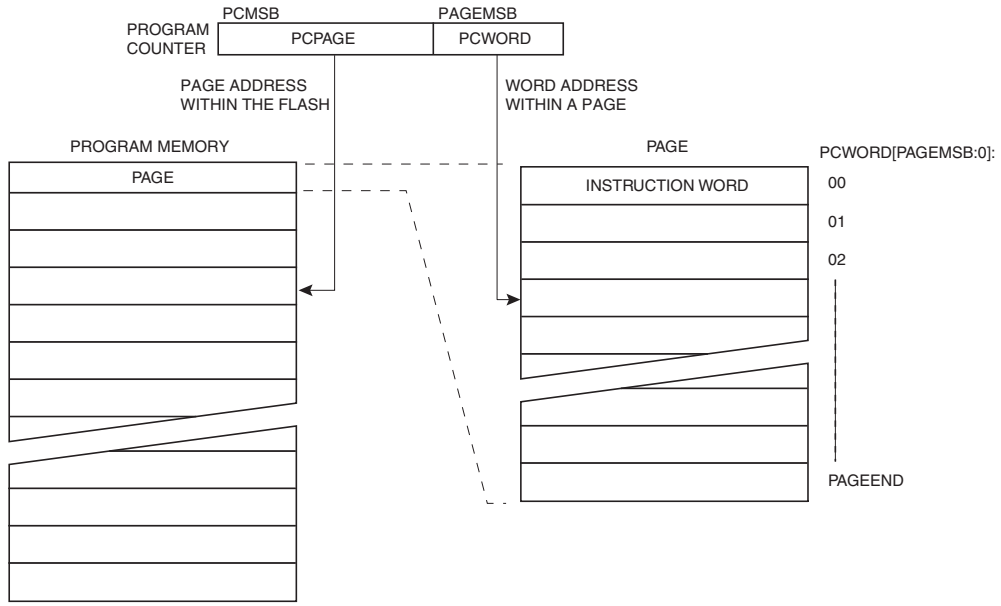
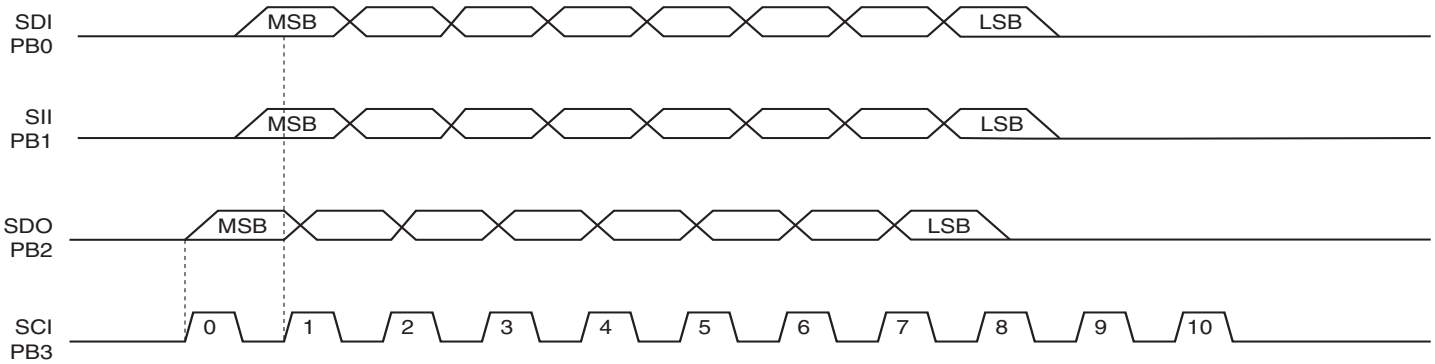


Figure 20-5. High-voltage Serial Programming Waveforms



20.7.5 Programming the EEPROM

The EEPROM is organized in pages, see Table 21-11 on page 170. When programming the EEPROM, the data is latched into a page buffer. This allows one page of data to be programmed simultaneously. The programming algorithm for the EEPROM Data memory is as follows (refer to Table 20-16):

1. Load Command “Write EEPROM”.
2. Load EEPROM Page Buffer.
3. Program EEPROM Page. Wait after Instr. 2 until SDO goes high for the “Page Programming” cycle to finish.
4. Repeat 2 through 3 until the entire EEPROM is programmed or until all data has been programmed.
5. End Page Programming by Loading Command “No Operation”.

20.7.6 Reading the Flash

The algorithm for reading the Flash memory is as follows (refer to Table 20-16):

1. Load Command "Read Flash".
2. Read Flash Low and High Bytes. The contents at the selected address are available at serial output SDO.

20.7.7 Reading the EEPROM

The algorithm for reading the EEPROM memory is as follows (refer to Table 20-16):

1. Load Command “Read EEPROM”.
2. Read EEPROM Byte. The contents at the selected address are available at serial output SDO.

20.7.8 Programming and Reading the Fuse and Lock Bits

The algorithms for programming and reading the Fuse Low/High bits and Lock bits are shown in Table 20-16.

20.7.9 Reading the Signature Bytes and Calibration Byte

The algorithms for reading the Signature bytes and Calibration byte are shown in Table 20-16.

20.7.10 Power-off sequence

Set SCI to “0”. Set RESET to “1”. Turn V_{CC} power off.

Table 20-16. High-voltage Serial Programming Instruction Set for ATtiny25/45/85

Instruction		Instruction Format				Operation Remarks
		Instr.1/5	Instr.2/6	Instr.3	Instr.4	
Chip Erase	SDI SII SDO	0_1000_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx		Wait after Instr.3 until SDO goes high for the Chip Erase cycle to finish.
Load “Write Flash” Command	SDI SII SDO	0_0001_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx				Enter Flash Programming code.
Load Flash Page Buffer	SDI SII SDO	0_**bbbb_bbbb** _00 0_0000_1100_00 x_xxxx_xxxx_xx	0_**eeee_eeee**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_**dddd_dddd**_00 0_0011_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1101_00 x_xxxx_xxxx_xx	Repeat after Instr. 1 - 5 until the entire page buffer is filled or until all data within the page is filled.[(2)]
	SDI SII SDO	0_0000_0000_00 0_0111_1100_00 x_xxxx_xxxx_xx				Instr 5.

Table 20-16. High-voltage Serial Programming Instruction Set for ATtiny25/45/85 (Continued)

Instruction		Instruction Format				Operation Remarks
		Instr.1/5	Instr.2/6	Instr.3	Instr.4	
Load Flash High Address and Program Page	SDI SII SDO	0_0000_000**a**_00 0_0001_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx		Wait after Instr 3 until SDO goes high. Repeat Instr. 2 - 3 for each loaded Flash Page until the entire Flash or all data is programmed. Repeat Instr. 1 for a new 256 byte page.(2)
Load “Read Flash” Command	SDI SII SDO	0_0000_0010_00 0_0100_1100_00 x_xxxx_xxxx_xx				Enter Flash Read mode.
Read Flash Low and High Bytes	SDI SII SDO	0_**bbbb_bbbb**_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_0000_000**a**_00 0_0001_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 **q_qqqq_qqq**x_xx	Repeat Instr. 1, 3 - 6 for each new address. Repeat Instr. 2 for a new 256 byte page.
	SDI SII SDO	0_0000_0000_00 0_0111_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1100_00 **p_pppp_ppp**x_xx			Instr 5 - 6.
Load “Write EEPROM” Command	SDI SII SDO	0_0001_0001_00 0_0100_1100_00 x_xxxx_xxxx_xx				Enter EEPROM Programming mode.
Load EEPROM Page Buffer	SDI SII SDO	0_00**bb_bbbb**_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_**aaaa_aaaa**_00 0_0001_1100_00 x_xxxx_xxxx_xx	0_**eeee_eeee**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1101_00 x_xxxx_xxxx_xx	Repeat Instr. 1 - 5 until the entire page buffer is filled or until all data within the page is filled.(3)
	SDI SII SDO	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx				Instr. 5
Program EEPROM Page	SDI SII SDO	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx			Wait after Instr. 2 until SDO goes high. Repeat Instr. 1 - 2 for each loaded EEPROM page until the entire EEPROM or all data is programmed.
Write EEPROM Byte	SDI SII SDO	0_**bbbb_bbbb**_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_**aaaa_aaaa**_00 0_0001_1100_00 x_xxxx_xxxx_xx	0_**eeee_eeee**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1101_00 x_xxxx_xxxx_xx	Repeat Instr. 1 - 6 for each new address. Wait after Instr. 6 until SDO goes high.(4)
	SDI SII SDO	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx			Instr. 6
Load “Read EEPROM” Command	SDI SII SDO	0_0000_0011_00 0_0100_1100_00 x_xxxx_xxxx_xx				Enter EEPROM Read mode.
Read EEPROM Byte	SDI SII SDO	0_**bbbb_bbbb**_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_**aaaa_aaaa**_00 0_0001_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 **q_qqqq_qqq**0_00	Repeat Instr. 1, 3 - 4 for each new address. Repeat Instr. 2 for a new 256 byte page.
Write Fuse Low Bits	SDI SII SDO	0_0100_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_**A987_6543**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx	Wait after Instr. 4 until SDO goes high. Write **A - 3** = “0” to program the Fuse bit.

Table 20-16. High-voltage Serial Programming Instruction Set for ATtiny25/45/85 (Continued)

Instruction		Instruction Format				Operation Remarks
		Instr.1/5	Instr.2/6	Instr.3	Instr.4	
Write Fuse High Bits	SDI SII SDO	0_0100_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_**IHGF_EDCB**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1100_00 x_xxxx_xxxx_xx	Wait after Instr. 4 until SDO goes high. Write **I - B** = “0” to program the Fuse bit.
Write Fuse Extended Bits	SDI SII SDO	0_0100_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_000**J**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_0110_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1110_00 x_xxxx_xxxx_xx	Wait after Instr. 4 until SDO goes high. Write **J** = “0” to program the Fuse bit.
Write Lock Bits	SDI SII SDO	0_0010_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_00**21**_00 0_0010_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_0100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 x_xxxx_xxxx_xx	Wait after Instr. 4 until SDO goes high. Write **2 - 1** = “0” to program the Lock bit.
Read Fuse Low Bits	SDI SII SDO	0_0000_0100_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 **A_9876_543**x_xx		Reading **A - 3** = “0” means the Fuse bit is programmed.
Read Fuse High Bits	SDI SII SDO	0_0000_0100_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1010_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1110_00 **I_HGFE_DCB**x_xx		Reading **I - B** = “0” means the Fuse bit is programmed.
Read Fuse Extended Bits	SDI SII SDO	0_0000_0100_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1010_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1110_00 x_xxxx_xxJx_xx		Reading **J** = “0” means the Fuse bit is programmed.
Read Lock Bits	SDI SII SDO	0_0000_0100_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1100_00 x_xxxx_x**21**x_xx		Reading **2, 1** = “0” means the Lock bit is programmed.
Read Signature Bytes	SDI SII SDO	0_0000_1000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_00**bb**_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0110_1100_00 **q_qqqq_qqq**x_xx	Repeats Instr 2 4 for each signature byte address.
Read Calibration Byte	SDI SII SDO	0_0000_1000_00 0_0100_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0000_1100_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1000_00 x_xxxx_xxxx_xx	0_0000_0000_00 0_0111_1100_00 **p_pppp_ppp**x_xx	
Load “No Operation” Command	SDI SII SDO	0_0000_0000_00 0_0100_1100_00 x_xxxx_xxxx_xx				

Notes:
1. **a** = address high bits, **b** = address low bits, **d** = data in high bits, **e** = data in low bits, **p** = data out high bits, **q** = data out low bits, x = don’t care, ***1*** = Lock Bit1, ***2*** = Lock Bit2, **3** = CKSEL0 Fuse, **4** = CKSEL1 Fuse, **5** = CKSEL2 Fuse, **6** = CKSEL3 Fuse, **7** = SUT0 Fuse, **8** = SUT1 Fuse, **9** = CKOUT Fuse, **A** = CKDIV8 Fuse, **B** = BODLEVEL0 Fuse, **C** = BODLEVEL1 Fuse, **D** = BODLEVEL2 Fuse, **E** = EESAVE Fuse, **F** = WDTON Fuse, **G** = SPIEN Fuse, **H** = DWEN Fuse, **I** = RSTDISBL Fuse, **J** = SELFPRGEN Fuse
2. For page sizes less than 256 words, parts of the address (bbbb_bbbb) will be parts of the page address.
3. For page sizes less than 256 bytes, parts of the address (bbbb_bbbb) will be parts of the page address.
4. The EEPROM is written page-wise. But only the bytes that are loaded into the page are actually written to the EEPROM. Page-wise EEPROM access is more efficient when multiple bytes are to be written to the same page. Note that auto-erase of EEPROM is not available in High-voltage Serial Programming, only in SPI Programming.

21. Electrical Characteristics

21.1 Absolute Maximum Ratings*

Operating Temperature	-55°C to +125°C
Storage Temperature	-65°C to +150°C
Voltage on any Pin except $\overline{\text{RESET}}$ with respect to Ground	-0.5V to V_{CC}+0.5V
Voltage on $\overline{\text{RESET}}$ with respect to Ground	-0.5V to +13.0V
Maximum Operating Voltage	6.0V
DC Current per I/O Pin	40.0 mA
DC Current V_{CC} and GND Pins	200.0 mA

*NOTICE: Stresses beyond those listed under “Absolute Maximum Ratings” may cause permanent damage to the device. This is a stress rating only and functional operation of the device at these or other conditions beyond those indicated in the operational sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

21.2 DC Characteristics

Table 21-1. DC Characteristics. T_A = -40°C to +85°C

Symbol	Parameter	Condition	Min.	Typ.(1)	Max.	Units
V_{IL}	Input Low-voltage, except XTAL1 and $\overline{\text{RESET}}$ pin	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	-0.5 -0.5		$0.2V_{CC}$(3) $0.3V_{CC}$(3)	V V
V_{IH}	Input High-voltage, except XTAL1 and $\overline{\text{RESET}}$ pin	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	$0.7V_{CC}$(2) $0.6V_{CC}$(2)		V_{CC} +0.5 V_{CC} +0.5	V V
V_{IL1}	Input Low-voltage, XTAL1 pin, External Clock Selected	V_{CC} = 1.8V - 5.5V	-0.5		$0.1V_{CC}$(3)	V
V_{IH1}	Input High-voltage, XTAL1 pin, External Clock Selected	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	$0.8V_{CC}$(2) $0.7V_{CC}$(2)		V_{CC} +0.5 V_{CC} +0.5	V V
V_{IL2}	Input Low-voltage, $\overline{\text{RESET}}$ pin	V_{CC} = 1.8V - 5.5V	-0.5		$0.2V_{CC}$(3)	V V
V_{IH2}	Input High-voltage, $\overline{\text{RESET}}$ pin	V_{CC} = 1.8V - 5.5V	$0.9V_{CC}$(2)		V_{CC} +0.5	V
V_{IL3}	Input Low-voltage, $\overline{\text{RESET}}$ pin as I/O	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	-0.5 -0.5		$0.2V_{CC}$(3) $0.3V_{CC}$(3)	V V
V_{IH3}	Input High-voltage, $\overline{\text{RESET}}$ pin as I/O	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	$0.7V_{CC}$(2) $0.6V_{CC}$(2)		V_{CC} +0.5 V_{CC} +0.5	V V
V_{OL}	Output Low-voltage,(4) Port B (except $\overline{\text{RESET}}$) (6)	I_{OL} = 10 mA, V_{CC} = 5V I_{OL} = 5 mA, V_{CC} = 3V			0.6 0.5	V V
V_{OH}	Output High-voltage, (5) Port B (except $\overline{\text{RESET}}$) (6)	I_{OH} = -10 mA, V_{CC} = 5V I_{OH} = -5 mA, V_{CC} = 3V	4.3 2.5			V V
I_{IL}	Input Leakage Current I/O Pin	V_{CC} = 5.5V, pin low (absolute value)		< 0.05	1	µA
I_{IH}	Input Leakage Current I/O Pin	V_{CC} = 5.5V, pin high (absolute value)		< 0.05	1	µA
R_{RST}	Reset Pull-up Resistor	V_{CC} = 5.5V, input low	30		60	kΩ

Table 21-1. DC Characteristics. T_A = -40°C to +85°C (Continued)

Symbol	Parameter	Condition	Min.	Typ.[1]	Max.	Units
R_{pu}	I/O Pin Pull-up Resistor	V_{CC} = 5.5V, input low	20		50	kΩ
I_{CC}	Power Supply Current [7]	Active 1 MHz, V_{CC} = 2V		0.3	0.55	mA
		Active 4 MHz, V_{CC} = 3V		1.5	2.5	mA
		Active 8 MHz, V_{CC} = 5V		5	8	mA
		Idle 1 MHz, V_{CC} = 2V		0.1	0.2	mA
		Idle 4 MHz, V_{CC} = 3V		0.35	0.6	mA
		Idle 8 MHz, V_{CC} = 5V		1.2	2	mA
	Power-down mode [8]	WDT enabled, V_{CC} = 3V			10	µA
		WDT disabled, V_{CC} = 3V			2	µA

Notes:
1. Typical values at 25°C.
2. “Min” means the lowest value where the pin is guaranteed to be read as high.
3. “Max” means the highest value where the pin is guaranteed to be read as low.
4. Although each I/O port can sink more than the test conditions (10 mA at V_{CC} = 5V, 5 mA at V_{CC} = 3V) under steady state conditions (non-transient), the following must be observed:
   1] The sum of all IOL, for all ports, should not exceed 60 mA.
   If IOL exceeds the test condition, VOL may exceed the related specification. Pins are not guaranteed to sink current greater than the listed test condition.
5. Although each I/O port can source more than the test conditions (10 mA at V_{CC} = 5V, 5 mA at V_{CC} = 3V) under steady state conditions (non-transient), the following must be observed:
   1] The sum of all IOH, for all ports, should not exceed 60 mA.
   If IOH exceeds the test condition, VOH may exceed the related specification. Pins are not guaranteed to source current greater than the listed test condition.
6. The $\overline{\text{RESET}}$ pin must tolerate high voltages when entering and operating in programming modes and, as a consequence, has a weak drive strength as compared to regular I/O pins. See Figure 22-23, Figure 22-24, Figure 22-25, and Figure 22-26 (starting on page 184).
7. Values are with external clock using methods described in “Minimizing Power Consumption” on page 36. Power Reduction is enabled (PRR = 0xFF) and there is no I/O drive.
8. Brown-Out Detection (BOD) disabled.

21.3 Speed

Figure 21-1. Maximum Frequency vs. V_{CC}

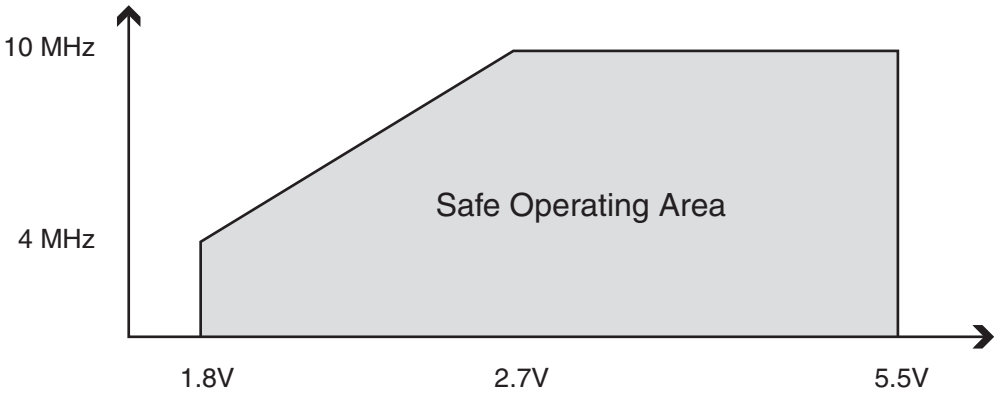
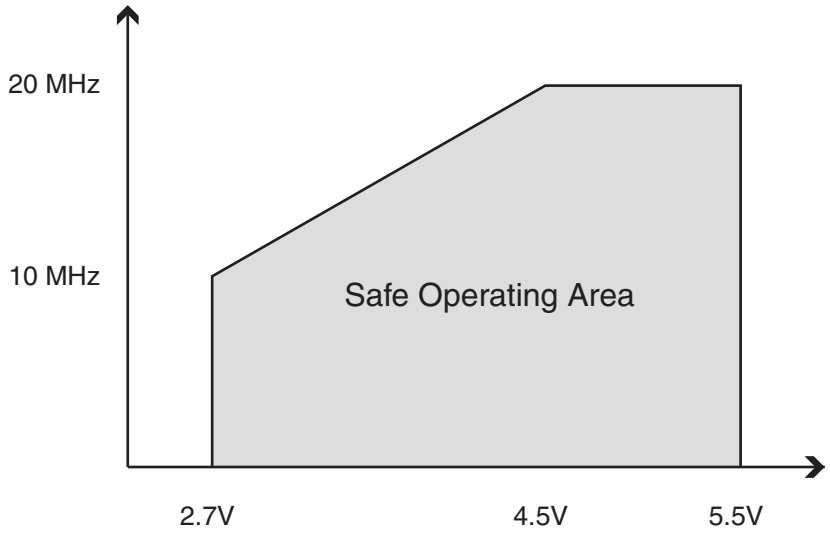


Figure 21-2. Maximum Frequency vs. V_{CC}



21.4 Clock Characteristics

21.4.1 Calibrated Internal RC Oscillator Accuracy

It is possible to manually calibrate the internal oscillator to be more accurate than default factory calibration. Please note that the oscillator frequency depends on temperature and voltage. Voltage and temperature characteristics can be found in Figure 22-40 on page 193 and Figure 22-41 on page 193.

Table 21-2. Calibration Accuracy of Internal RC Oscillator

Calibration Method	Target Frequency	V_{CC}	Temperature	Accuracy at given Voltage & Temperature (1)
Factory Calibration	8.0 MHz (2)	3V	25°C	±10%
User Calibration	Fixed frequency within: 6 – 8 MHz	Fixed voltage within: 1.8V - 5.5V (3) 2.7V - 5.5V (4)	Fixed temperature within: -40°C to +85°C	±1%

Notes:
1. Accuracy of oscillator frequency at calibration point (fixed temperature and fixed voltage).
2. ATtiny25/V, only: 6.4 MHz in ATtiny15 Compatibility Mode.
3. Voltage range for ATtiny25V/45V/85V.
4. Voltage range for ATtiny25/45/85.

21.4.2 External Clock Drive

Figure 21-3. External Clock Drive Waveforms

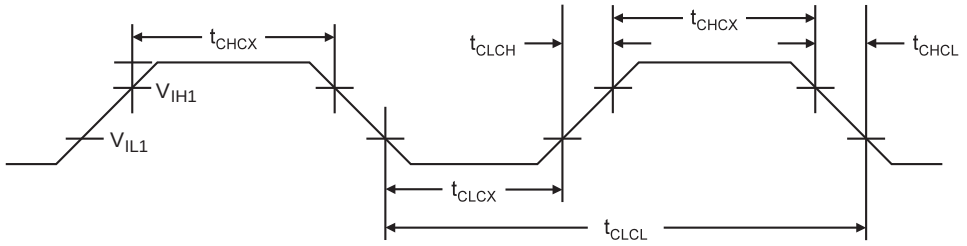


Table 21-3. External Clock Drive Characteristics

Symbol	Parameter	V_{CC} = 1.8 - 5.5V		V_{CC} = 2.7 - 5.5V		V_{CC} = 4.5 - 5.5V		Units
		Min.	Max.	Min.	Max.	Min.	Max.	
$1/t_{CLCL}$	Clock Frequency	0	4	0	10	0	20	MHz
t_{CLCL}	Clock Period	250		100		50		ns
t_{CHCX}	High Time	100		40		20		ns
t_{CLCX}	Low Time	100		40		20		ns
t_{CLCH}	Rise Time		2.0		1.6		0.5	µs
t_{CHCL}	Fall Time		2.0		1.6		0.5	µs
Δt_{CLCL}	Change in period from one clock cycle to the next		2		2		2	%

21.5 System and Reset Characteristics

Table 21-4. Reset, Brown-out and Internal Voltage Characteristics

Symbol	Parameter	Condition	Min[1]	Typ[1]	Max[1]	Units
V_{RST}	$\overline{\text{RESET}}$ Pin Threshold Voltage	V_{CC} = 3V	0.2 V_{CC}		0.9 V_{CC}	V
t_{RST}	Minimum pulse width on $\overline{\text{RESET}}$ Pin	V_{CC} = 3V			2.5	µs
V_{HYST}	Brown-out Detector Hysteresis			50		mV
t_{BOD}	Min Pulse Width on Brown-out Reset			2		µs
V_{BG}	Bandgap reference voltage	V_{CC} = 5.5V T_A = 25°C	1.0	1.1	1.2	V
t_{BG}	Bandgap reference start-up time	V_{CC} = 2.7V T_A = 25°C		40	70	µs
I_{BG}	Bandgap reference current consumption	V_{CC} = 2.7V T_A = 25°C		15		µA

Note: 1. Values are guidelines only.

Two versions of power-on reset have been implemented, as follows.

21.5.1 Standard Power-On Reset

This implementation of power-on reset existed in early versions of ATtiny25/45/85. The table below describes the characteristics of this power-on reset and it is valid for the following devices, only:

- ATtiny25, revision D, and older
- ATtiny45, revision F, and older
- ATtiny85, revision B, and newer

Note: Revisions are marked on the package (packages 8P3 and 8S2: bottom, package 20M1: top)

Table 21-5. Characteristics of Standard Power-On Reset. T_A = -40° to +85°C

Symbol	Parameter	Min[1]	Typ[1]	Max[1]	Units
V_{POR}	Release threshold of power-on reset [2]	0.7	1.0	1.4	V
V_{POA}	Activation threshold of power-on reset [3]	0.05	0.9	1.3	V
SR_{ON}	Power-on slope rate	0.01		4.5	V/ms

Note: 1. Values are guidelines, only
2. Threshold where device is released from reset when voltage is rising
3. The power-on reset will not work unless the supply voltage has been below V_{POA}

21.5.2 Enhanced Power-On Reset

This implementation of power-on reset exists in newer versions of ATtiny25/45/85. The table below describes the characteristics of this power-on reset and it is valid for the following devices, only:

- ATtiny25, revision E, and newer

- ATtiny45, revision G, and newer
- ATtiny85, revision C, and newer

Table 21-6. Characteristics of Enhanced Power-On Reset. T_A = -40°C to +85°C

Symbol	Parameter	Min[1]	Typ[1]	Max[1]	Units
V_{POR}	Release threshold of power-on reset [2]	1.1	1.4	1.6	V
V_{POA}	Activation threshold of power-on reset [3]	0.6	1.3	1.6	V
SR_{ON}	Power-On Slope Rate	0.01			V/ms

Note: 1. Values are guidelines, only
2. Threshold where device is released from reset when voltage is rising
3. The Power-on Reset will not work unless the supply voltage has been below V_{POT} (falling)

21.6 Brown-Out Detection

Table 21-7. BODLEVEL Fuse Coding. T_A = -40°C to +85°C

BODLEVEL[2:0] Fuses	Min[1]	Typ[1]	Max[1]	Units
111	BOD Disabled			
110	1.7	1.8	2.0	V
101	2.5	2.7	2.9	
100	4.1	4.3	4.5	
0XX	Reserved			

Note: 1. V_{BOT} may be below nominal minimum operating voltage for some devices. For devices where this is the case, the device is tested down to $V_{CC} = V_{BOT}$ during the production test. This guarantees that a Brown-out Reset will occur before V_{CC} drops to a voltage where correct operation of the microcontroller is no longer guaranteed.

21.7 ADC Characteristics

Table 21-8. ADC Characteristics, Single Ended Channels. T_A = -40°C to +85°C

Symbol	Parameter	Condition	Min	Typ	Max	Units
	Resolution				10	Bits
	Absolute accuracy (Including INL, DNL, and Quantization, Gain and Offset errors)	V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz		2		LSB
		V_{REF} = 4V, V_{CC} = 4V, ADC clock = 1 MHz		3		LSB
		V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz Noise Reduction Mode		1.5		LSB
		V_{REF} = 4V, V_{CC} = 4V, ADC clock = 1 MHz Noise Reduction Mode		2.5		LSB
	Integral Non-linearity (INL) (Accuracy after offset and gain calibration)	V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz		1		LSB
	Differential Non-linearity (DNL)	V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz		0.5		LSB
	Gain Error	V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz		2.5		LSB
	Offset Error	V_{REF} = 4V, V_{CC} = 4V, ADC clock = 200 kHz		1.5		LSB
	Conversion Time	Free Running Conversion	14		280	µs
	Clock Frequency		50		1000	kHz
V_{IN}	Input Voltage		GND		V_{REF}	V
	Input Bandwidth			38.4		kHz
AREF	External Reference Voltage		2.0		V_{CC}	V
V_{INT}	Internal Voltage Reference		1.0	1.1	1.2	V
	Internal 2.56V Reference (1)	V_{CC} > 3.0V	2.3	2.56	2.8	V
R_{REF}				32		kΩ
R_{AIN}	Analog Input Resistance			100		MΩ
	ADC Output		0		1023	LSB

Note: 1. Values are guidelines only.

Table 21-9. ADC Characteristics, Differential Channels (Unipolar Mode). T_A = -40°C to +85°C

Symbol	Parameter	Condition	Min	Typ	Max	Units
	Resolution	Gain = 1x			10	Bits
		Gain = 20x			10	Bits
	Absolute accuracy (Including INL, DNL, and Quantization, Gain and Offset Errors)	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		10.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		20.0		LSB
	Integral Non-Linearity (INL) (Accuracy after Offset and Gain Calibration)	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		4.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		10.0		LSB
	Gain Error	Gain = 1x		10.0		LSB
		Gain = 20x		15.0		LSB
	Offset Error	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		3.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		4.0		LSB
	Conversion Time	Free Running Conversion	70		280	µs
	Clock Frequency		50		200	kHz
V_{IN}	Input Voltage		GND		V_{CC}	V
V_{DIFF}	Input Differential Voltage				V_{REF}/Gain	V
	Input Bandwidth			4		kHz
AREF	External Reference Voltage		2.0		V_{CC} - 1.0	V
V_{INT}	Internal Voltage Reference		1.0	1.1	1.2	V
	Internal 2.56V Reference (1)	V_{CC} > 3.0V	2.3	2.56	2.8	V
R_{REF}	Reference Input Resistance			32		kΩ
R_{AIN}	Analog Input Resistance			100		MΩ
	ADC Conversion Output		0		1023	LSB

Note: 1. Values are guidelines only.

Table 21-10. ADC Characteristics, Differential Channels (Bipolar Mode). T_A = -40°C to +85°C

Symbol	Parameter	Condition	Min	Typ	Max	Units
	Resolution	Gain = 1x			10	Bits
		Gain = 20x			10	Bits
	Absolute accuracy (Including INL, DNL, and Quantization, Gain and Offset Errors)	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		8.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		8.0		LSB
	Integral Non-Linearity (INL) (Accuracy after Offset and Gain Calibration)	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		4.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		5.0		LSB
	Gain Error	Gain = 1x		4.0		LSB
		Gain = 20x		5.0		LSB
	Offset Error	Gain = 1x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		3.0		LSB
		Gain = 20x V_{REF} = 4V, V_{CC} = 5V ADC clock = 50 - 200 kHz		4.0		LSB
	Conversion Time	Free Running Conversion	70		280	µs
	Clock Frequency		50		200	kHz
V_{IN}	Input Voltage		GND		V_{CC}	V
V_{DIFF}	Input Differential Voltage				V_{REF}/Gain	V
	Input Bandwidth			4		kHz
AREF	External Reference Voltage		2.0		V_{CC} - 1.0	V
V_{INT}	Internal Voltage Reference		1.0	1.1	1.2	V
	Internal 2.56V Reference (1)	V_{CC} > 3.0V	2.3	2.56	2.8	V
R_{REF}	Reference Input Resistance			32		kΩ
R_{AIN}	Analog Input Resistance			100		MΩ
	ADC Conversion Output		-512		511	LSB

Note: 1. Values are guidelines only.

21.8 Serial Programming Characteristics

Figure 21-4. Serial Programming Waveforms

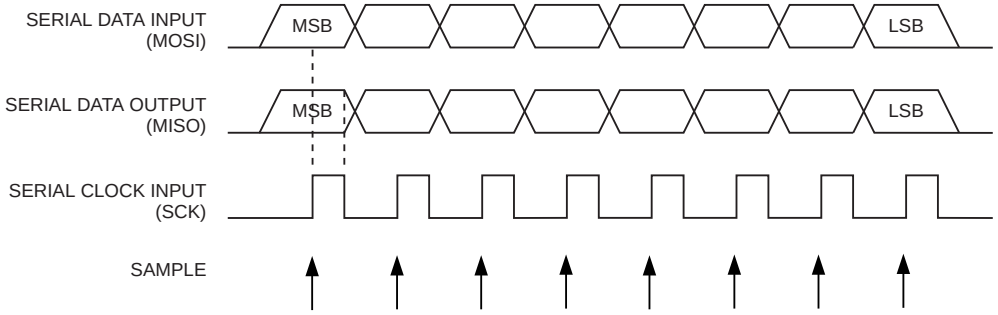


Figure 21-5. Serial Programming Timing

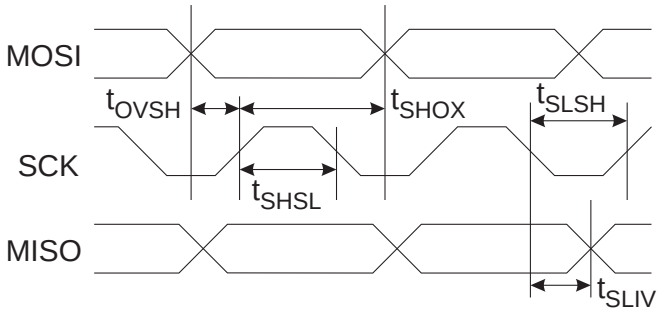


Table 21-11. Serial Programming Characteristics, T_A = -40°C to +85°C, V_{CC} = 1.8 - 5.5V (Unless Otherwise Noted)

Symbol	Parameter	Min	Typ	Max	Units
$1/t_{CLCL}$	Oscillator Frequency (V_{CC} = 1.8 - 5.5V)	0		4	MHz
t_{CLCL}	Oscillator Period (V_{CC} = 1.8 - 5.5V)	250			ns
$1/t_{CLCL}$	Oscillator Frequency (V_{CC} = 2.7 - 5.5V)	0		10	MHz
t_{CLCL}	Oscillator Period (V_{CC} = 2.7 - 5.5V)	100			ns
$1/t_{CLCL}$	Oscillator Frequency (V_{CC} = 4.5V - 5.5V)	0		20	MHz
t_{CLCL}	Oscillator Period (V_{CC} = 4.5V - 5.5V)	50			ns
t_{SHSL}	SCK Pulse Width High	2 t_{CLCL}*			ns
t_{SLSH}	SCK Pulse Width Low	2 t_{CLCL}*			ns
t_{OVSH}	MOSI Setup to SCK High	t_{CLCL}			ns
t_{SHOX}	MOSI Hold after SCK High	2 t_{CLCL}			ns
t_{SLIV}	SCK Low to MISO Valid			100	ns

Note: 1. 2 t_{CLCL} for f_{ck} < 12 MHz, 3 t_{CLCL} for f_{ck} >= 12 MHz

21.9 High-voltage Serial Programming Characteristics

Figure 21-6. High-voltage Serial Programming Timing

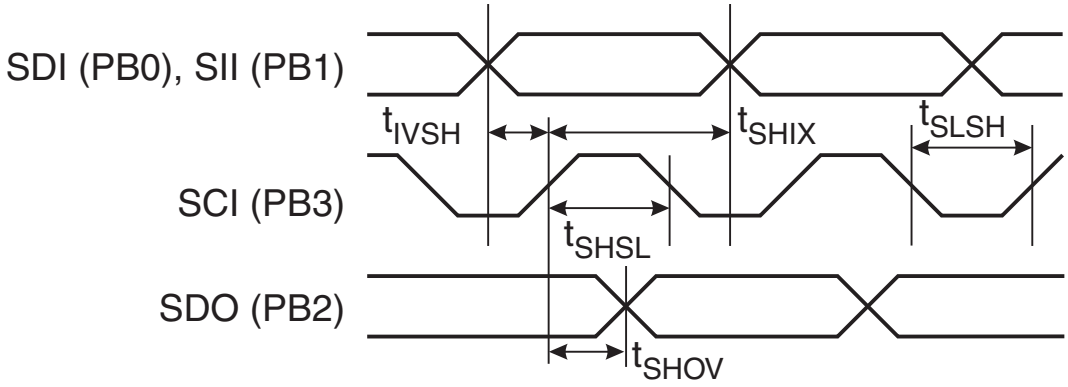


Table 21-12. High-voltage Serial Programming Characteristics T_A = 25°C ± 10%, V_{CC} = 5.0V ± 10% (Unless otherwise noted)

Symbol	Parameter	Min	Typ	Max	Units
t_{SHSL}	SCI (PB3) Pulse Width High	125			ns
t_{SLSH}	SCI (PB3) Pulse Width Low	125			ns
t_{IVSH}	SDI (PB0), SII (PB1) Valid to SCI (PB3) High	50			ns
t_{SHIX}	SDI (PB0), SII (PB1) Hold after SCI (PB3) High	50			ns
t_{SHOV}	SCI (PB3) High to SDO (PB2) Valid		16		ns
t_{WLWH_PFB}	Wait after Instr. 3 for Write Fuse Bits		2.5		ms

22. Typical Characteristics

The data contained in this section is largely based on simulations and characterization of similar devices in the same process and design methods. Thus, the data should be treated as indications of how the part will behave.

The following charts show typical behavior. These figures are not tested during manufacturing. All current consumption measurements are performed with all I/O pins configured as inputs and with internal pull-ups enabled. A sine wave generator with rail-to-rail output is used as clock source.

The power consumption in Power-down mode is independent of clock selection.

The current consumption is a function of several factors such as: operating voltage, operating frequency, loading of I/O pins, switching rate of I/O pins, code executed and ambient temperature. The dominating factors are operating voltage and frequency.

The current drawn from capacitive loaded pins may be estimated (for one pin) as C_L*V_{CC}*f where C_L = load capacitance, V_{CC} = operating voltage and f = average switching frequency of I/O pin.

The parts are characterized at frequencies higher than test limits. Parts are not guaranteed to function properly at frequencies higher than the ordering code indicates.

The difference between current consumption in Power-down mode with Watchdog Timer enabled and Power-down mode with Watchdog Timer disabled represents the differential current drawn by the Watchdog Timer.

22.1 Active Supply Current

Figure 22-1. Active Supply Current vs. Low frequency (0.1 - 1.0 MHz)

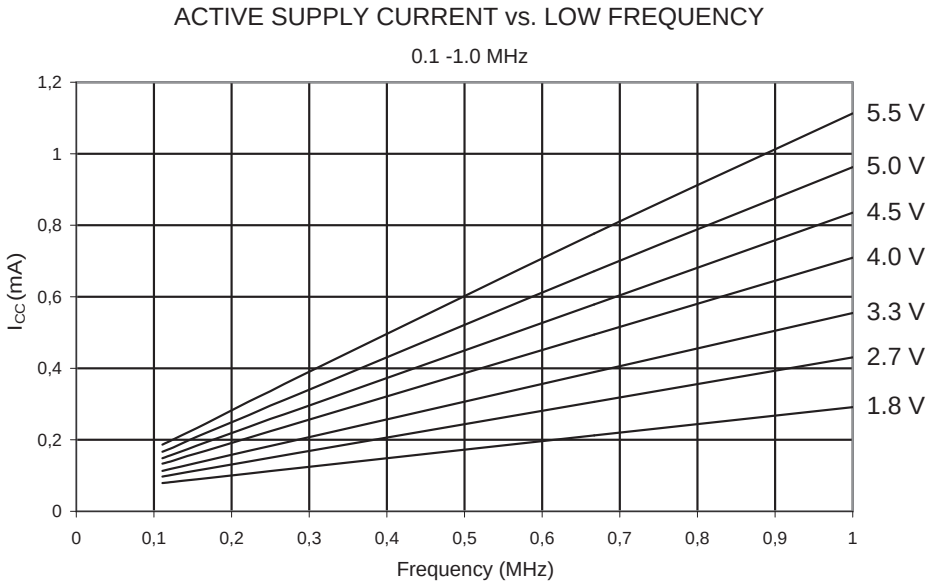


Figure 22-2. Active Supply Current vs. Frequency (1 - 20 MHz)

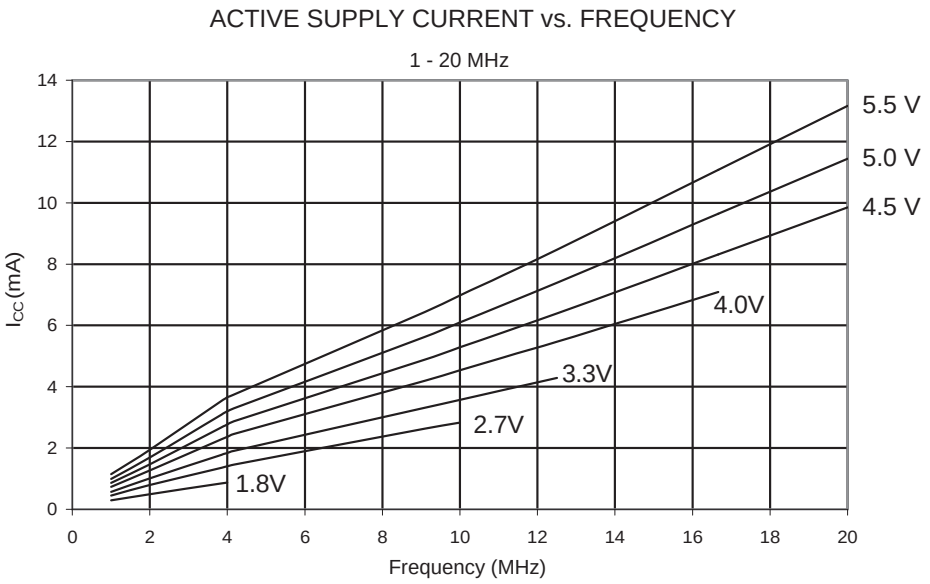


Figure 22-3. Active Supply Current vs. V_{CC} (Internal RC oscillator, 8 MHz)

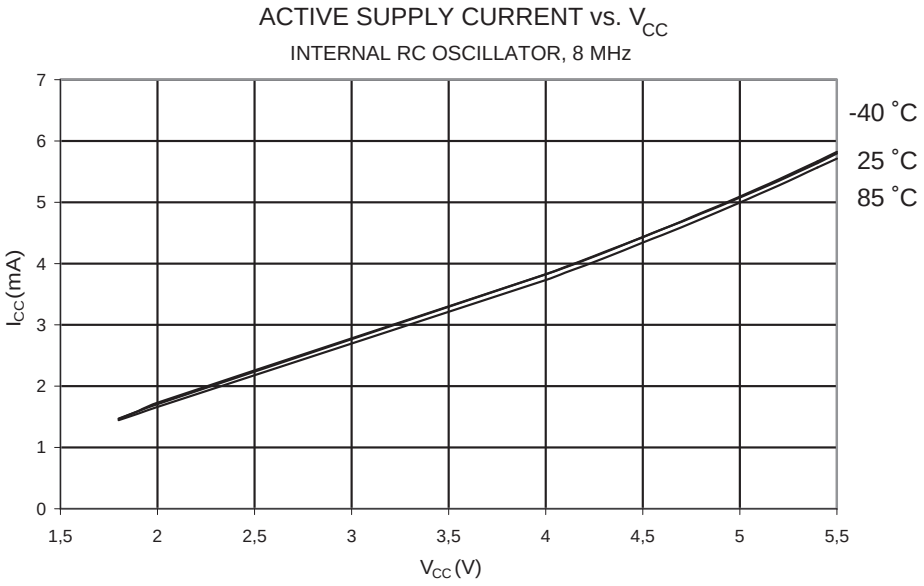


Figure 22-4. Active Supply Current vs. V_{CC} (Internal RC Oscillator, 1 MHz)

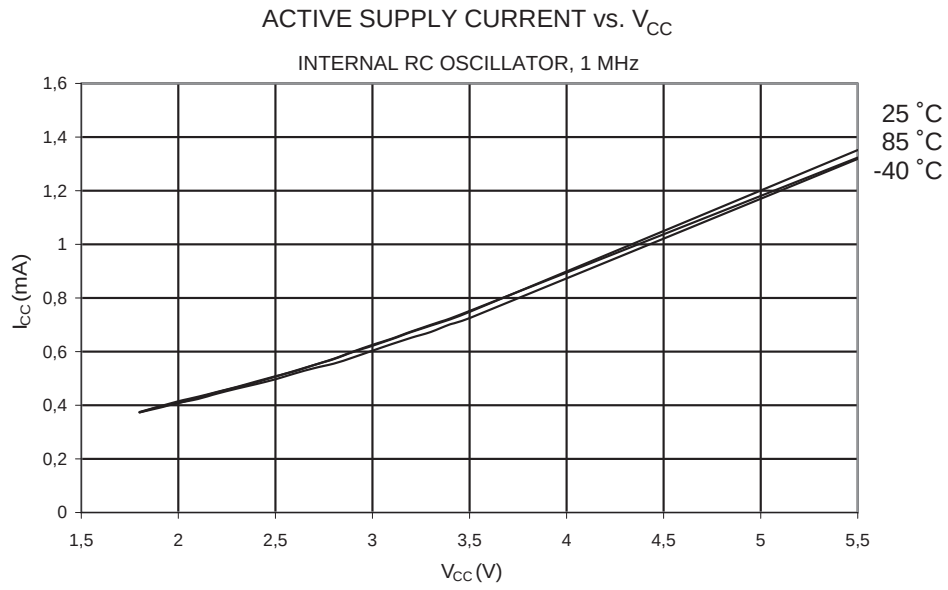
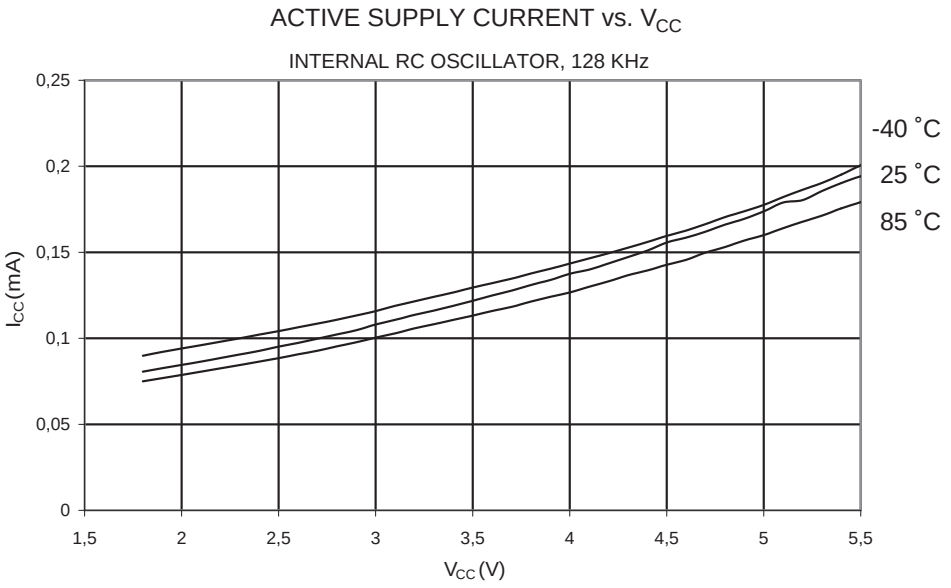


Figure 22-5. Active Supply Current vs. V_{CC} (Internal RC Oscillator, 128 kHz)



22.2 Idle Supply Current

Figure 22-6. Idle Supply Current vs. low Frequency (0.1 - 1.0 MHz)

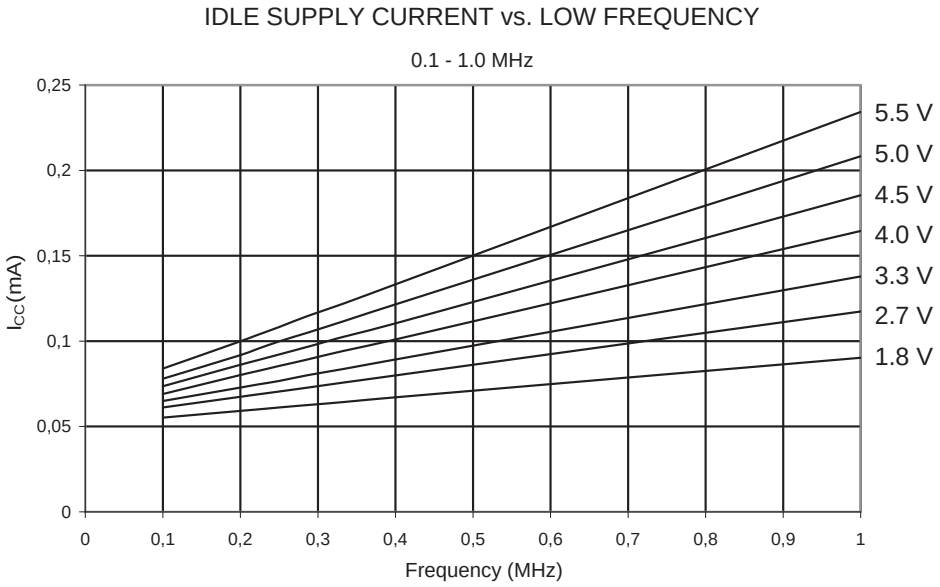


Figure 22-7. Idle Supply Current vs. Frequency (1 - 20 MHz)

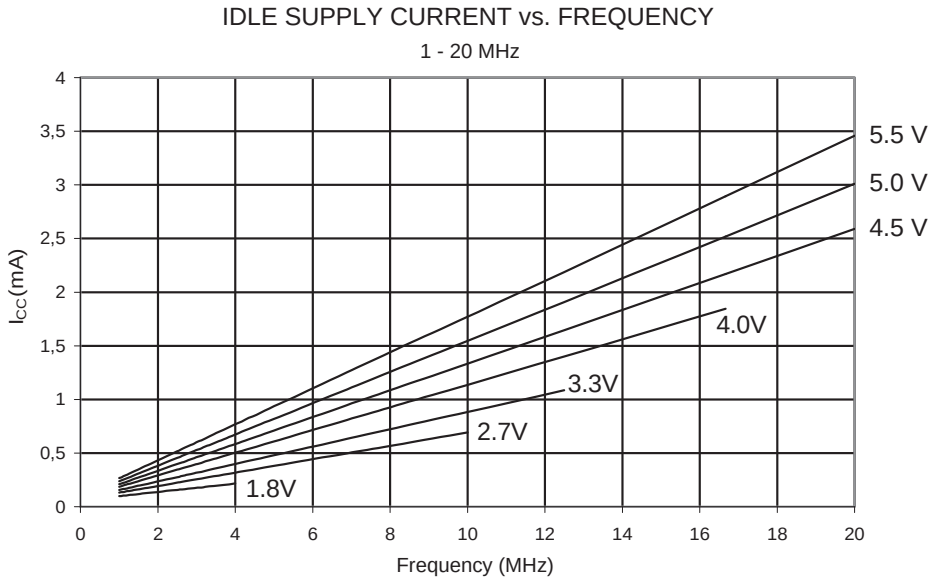


Figure 22-8. Idle Supply Current vs. V_{CC} (Internal RC Oscillator, 8 MHz)I

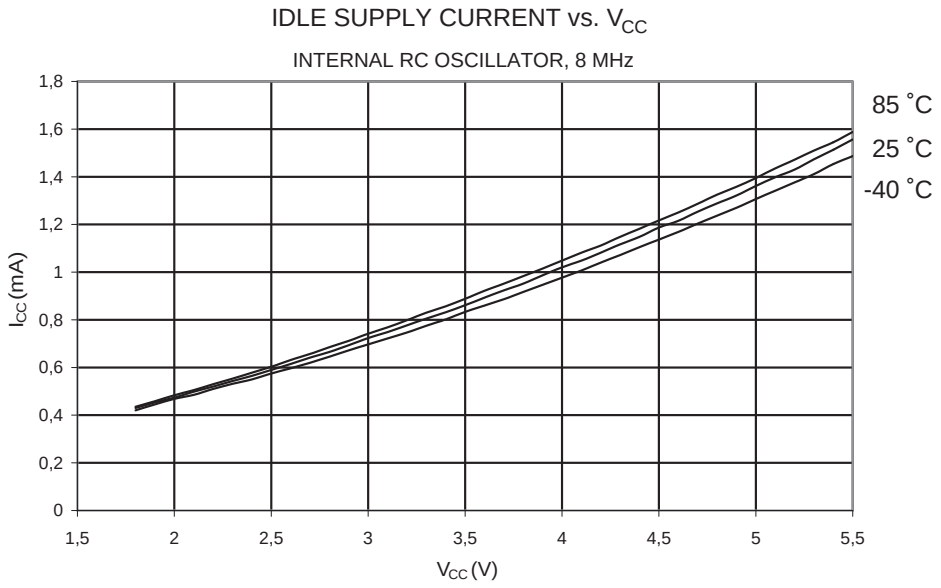


Figure 22-9. Idle Supply Current vs. V_{CC} (Internal RC Oscilllator, 1 MHz)

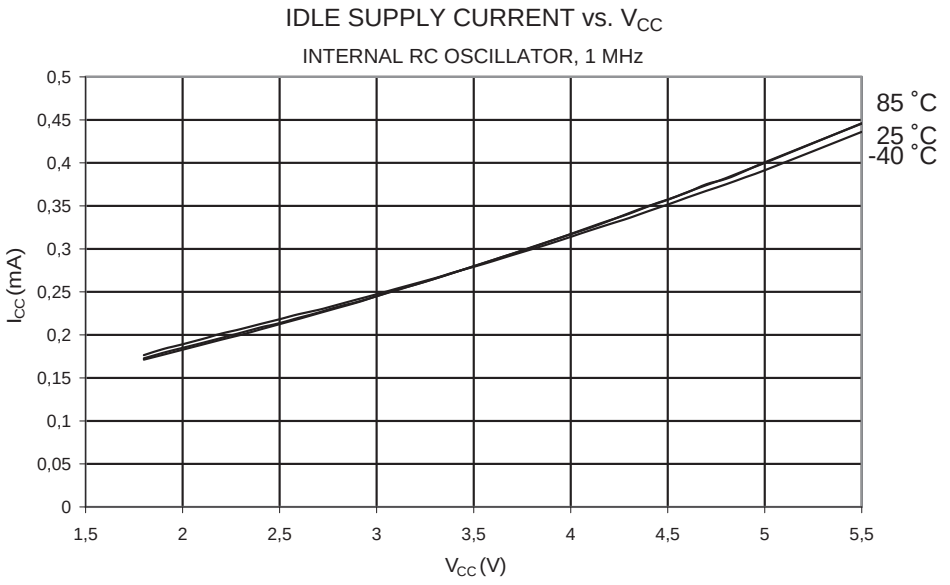
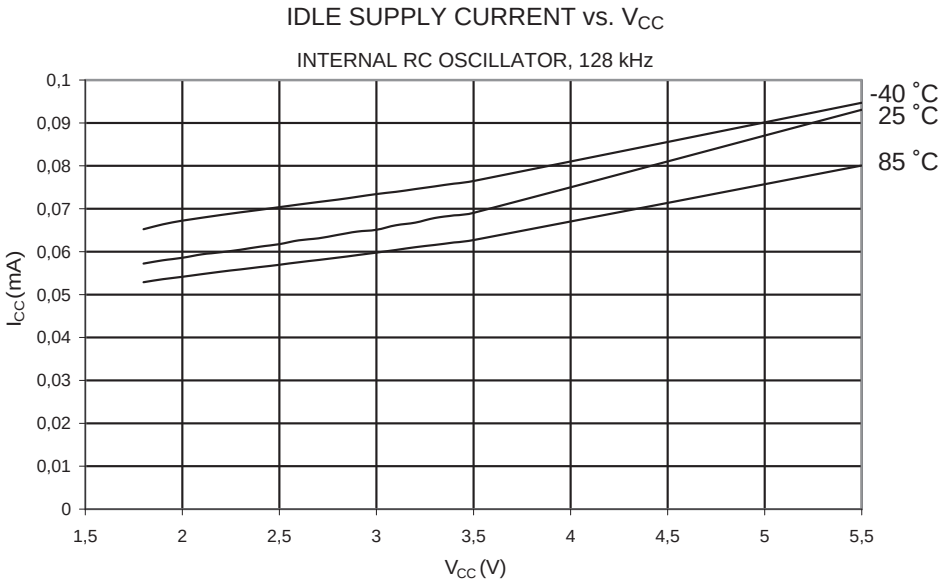


Figure 22-10. Idle Supply Current vs. V_{CC} (Internal RC Oscillator, 128 kHz)



22.3 Supply Current of I/O modules

The tables and formulas below can be used to calculate the additional current consumption for the different I/O modules in Active and Idle mode. The enabling or disabling of the I/O modules are controlled by the Power Reduction Register. See “PRR – Power Reduction Register” on page 38 for details.

Table 22-1. Additional Current Consumption for the different I/O modules (absolute values)

PRR bit	Typical numbers		
	V_{CC} = 2V, f = 1 MHz	V_{CC} = 3V, f = 4 MHz	V_{CC} = 5V, f = 8 MHz
PRTIM1	45 uA	300 uA	1100 uA
PRTIM0	5 uA	30 uA	110 uA
PRUSI	5 uA	25 uA	100 uA
PRADC	15 uA	85 uA	340 uA

Table 22-2. Additional Current Consumption (percentage) in Active and Idle mode

PRR bit	Additional Current consumption compared to Active with external clock (see Figure 22-1 and Figure 22-2)	Additional Current consumption compared to Idle with external clock (see Figure 22-6 and Figure 22-7)
PRTIM1	20 %	80 %
PRTIM0	2 %	10 %
PRUSI	2 %	10 %
PRADC	5 %	25 %

It is possible to calculate the typical current consumption based on the numbers from Table 22-2 for other V_{CC} and frequency settings that listed in Table 22-1.

22.3.1 Example

Calculate the expected current consumption in idle mode with USI, TIMER0, and ADC enabled at V_{CC} = 2.0V and f = 1 MHz. From Table 22-2 on page 177, third column, we see that we need to add 10% for the USI, 25% for the ADC, and 10% for the TIMER0 module. Reading from Figure 22-9, we find that the idle current consumption is ~0,18 mA at V_{CC} = 2.0V and f = 1 MHz. The total current consumption in idle mode with USI, TIMER0, and ADC enabled, gives:

$$I_{CC} = 0{,}18mA \times (1 + 0{,}1 + 0{,}25 + 0{,}1) \approx 0{,}261mA$$

22.4 Power-down Supply Current

Figure 22-11. Power-down Supply Current vs. V_{CC} (Watchdog Timer Disabled)

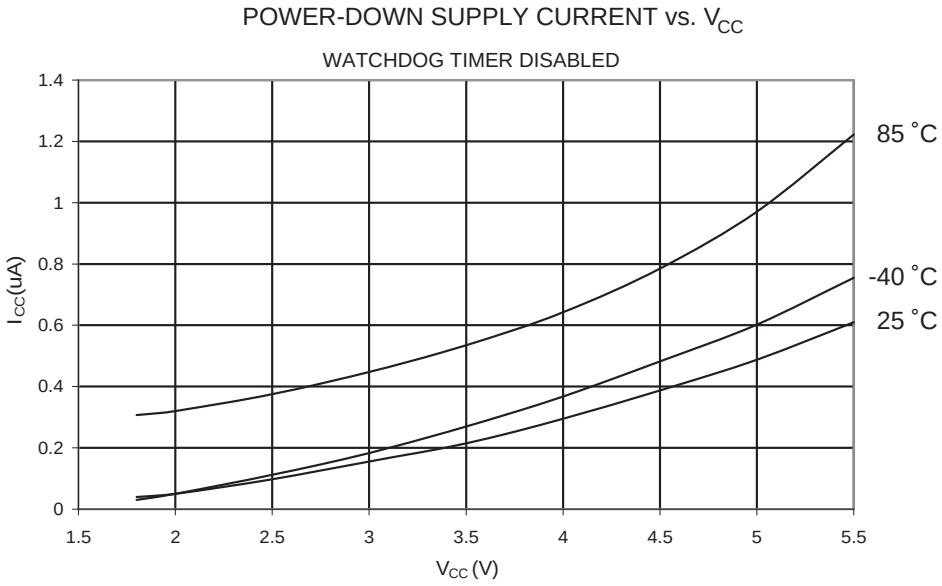
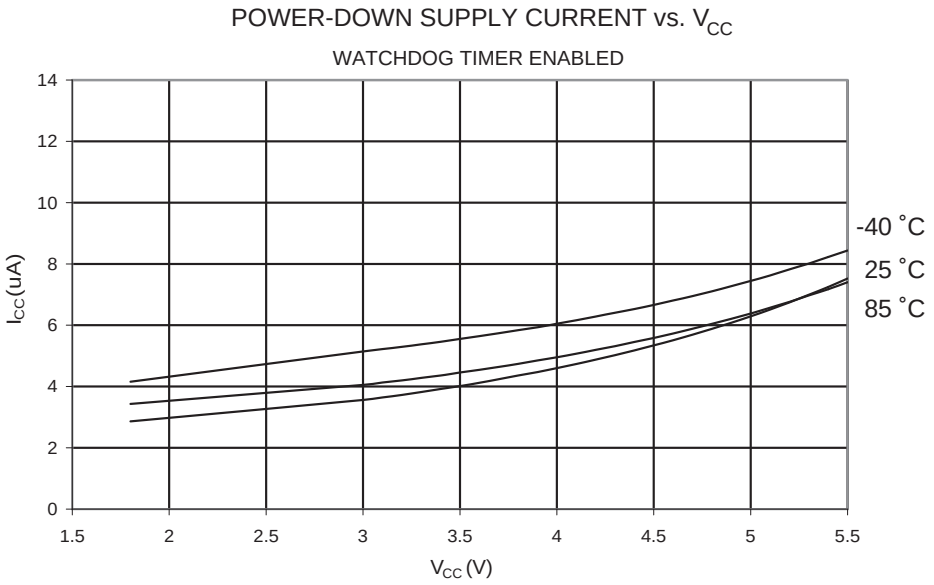


Figure 22-12. Power-down Supply Current vs. V_{CC} (Watchdog Timer Enabled)



22.5 Pin Pull-up

Figure 22-13. I/O Pin Pull-up Resistor Current vs. Input Voltage (V_{CC} = 1.8V)

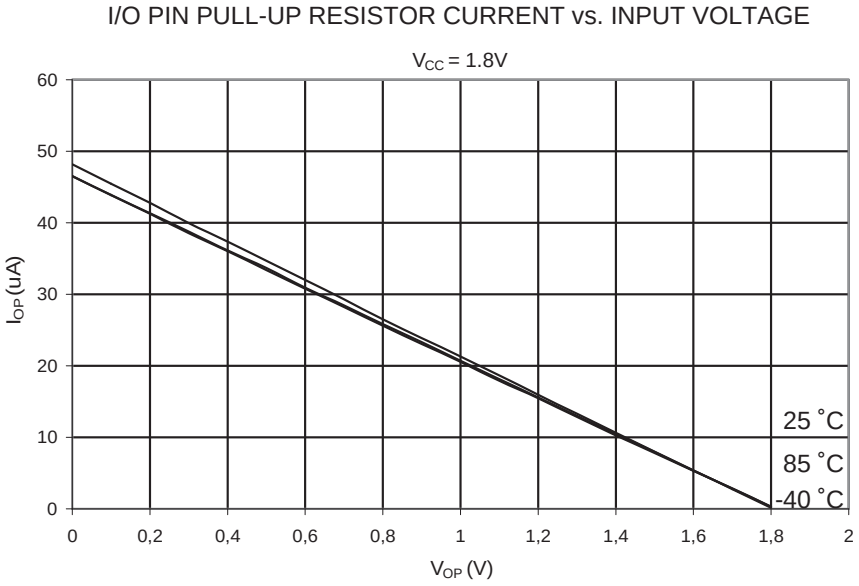


Figure 22-14. I/O Pin Pull-up Resistor Current vs. Input Voltage (V_{CC} = 2.7V)

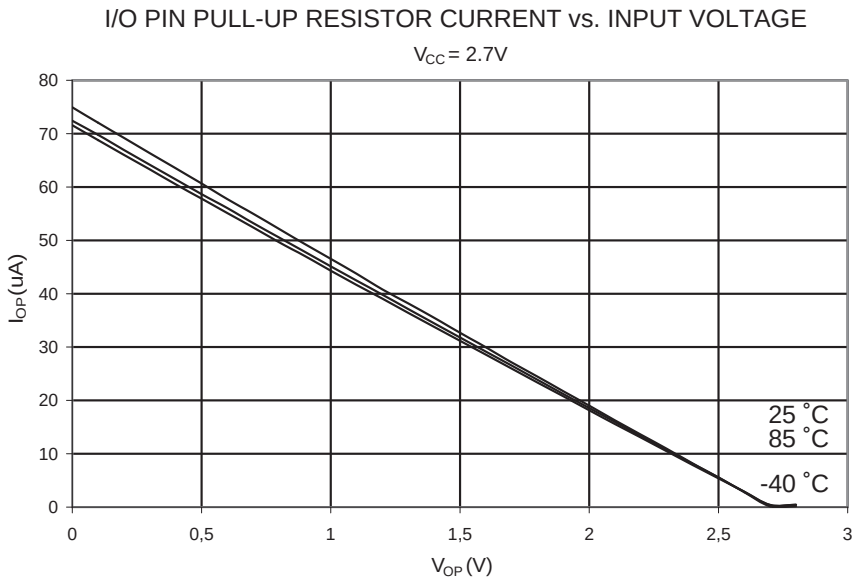


Figure 22-15. I/O Pin Pull-up Resistor Current vs. Input Voltage (V_{CC} = 5V)

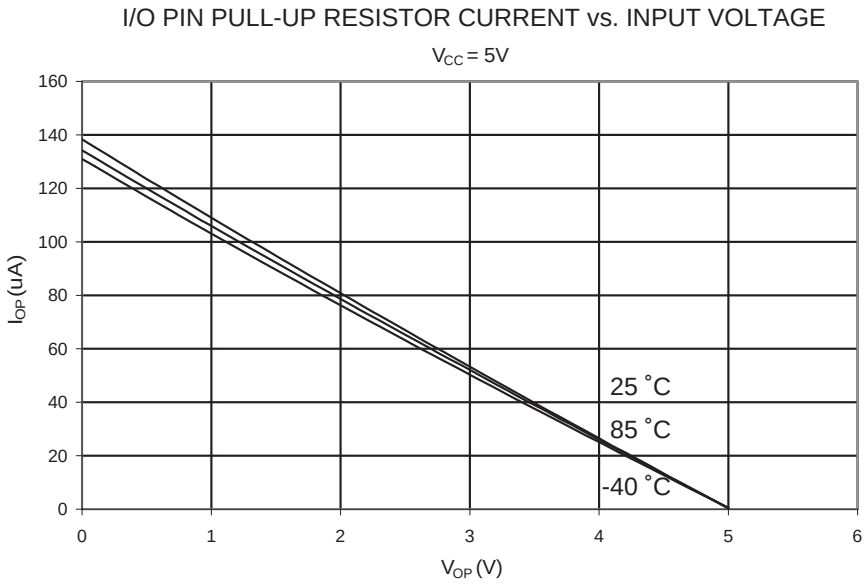


Figure 22-16. Reset Pull-up Resistor Current vs. Reset Pin Voltage (V_{CC} = 1.8V)

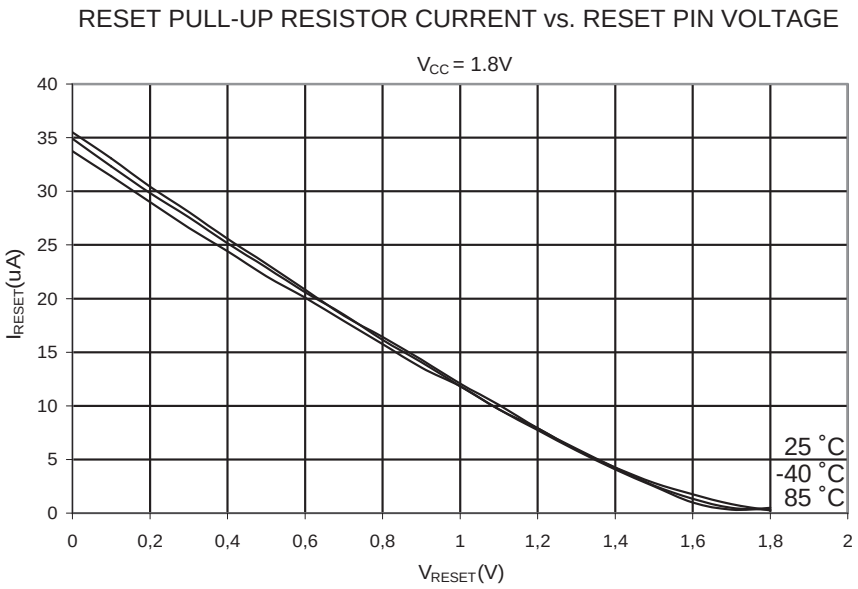


Figure 22-17. Reset Pull-up Resistor Current vs. Reset Pin Voltage (V_{CC} = 2.7V)

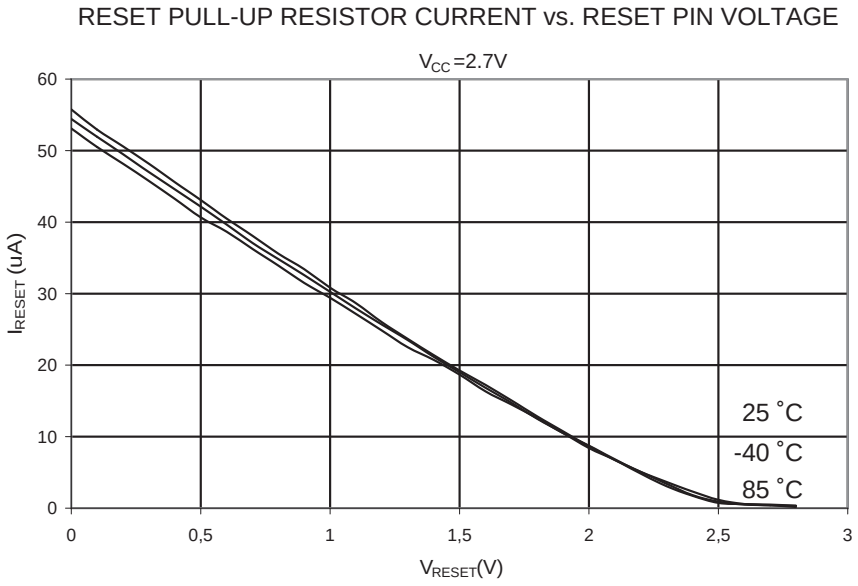
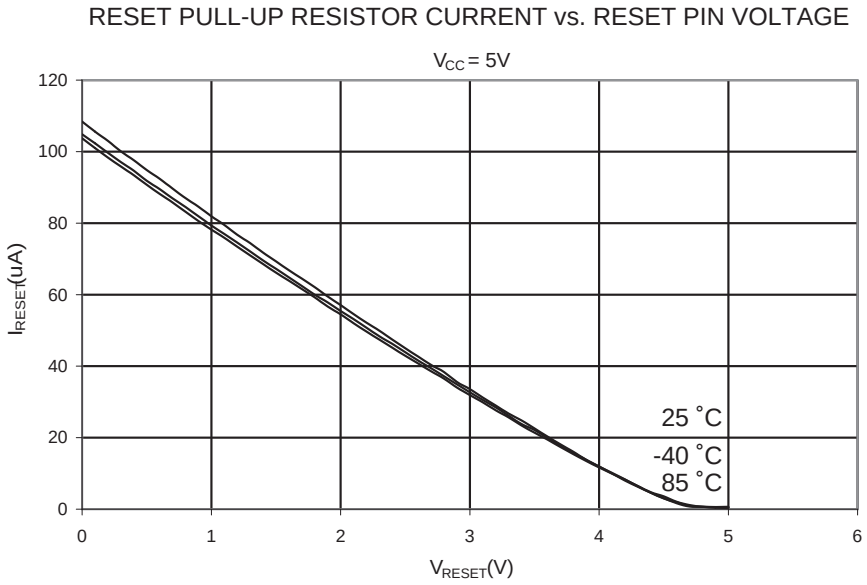


Figure 22-18. Reset Pull-up Resistor Current vs. Reset Pin Voltage (V_{CC} = 5V)



22.6 Pin Driver Strength

Figure 22-19. I/O Pin Output Voltage vs. Sink Current (V_{CC} = 3V)

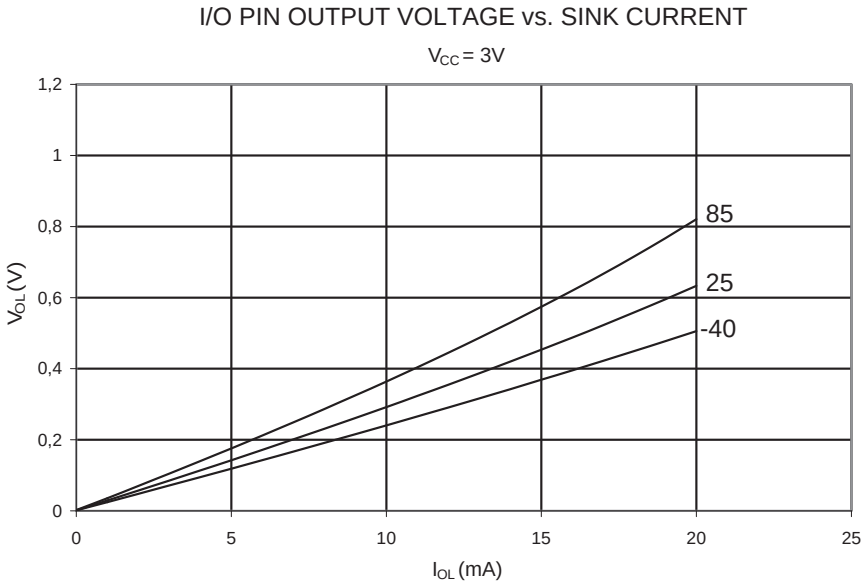


Figure 22-20. I/O Pin Output Voltage vs. Sink Current (V_{CC} = 5V)

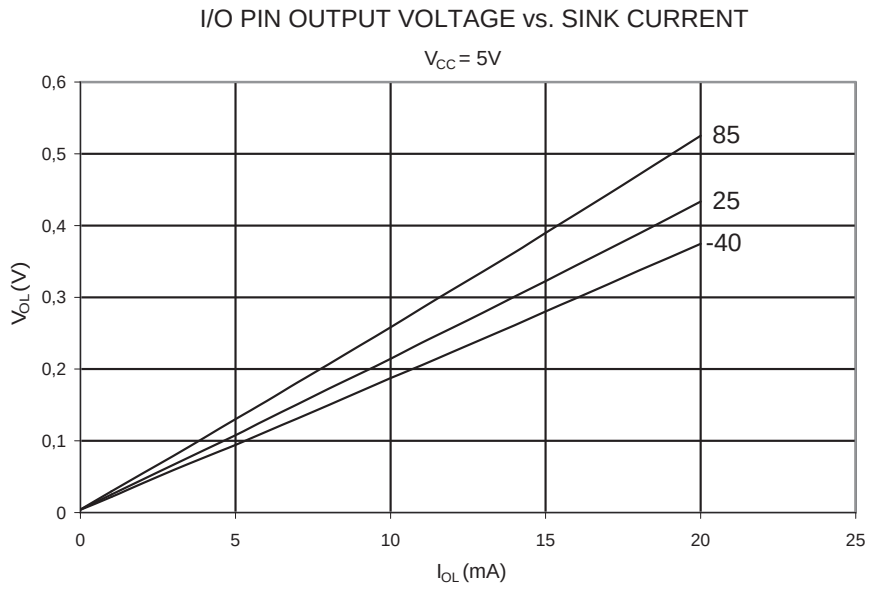


Figure 22-21. I/O Pin Output Voltage vs. Source Current (V_{CC} = 3V)

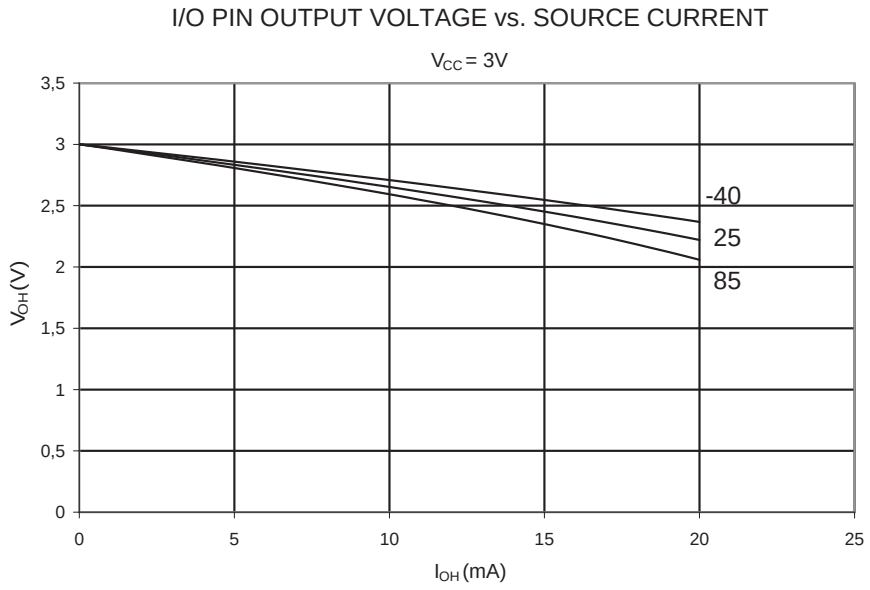


Figure 22-22. I/O Pin Output Voltage vs. Source Current (V_{CC} = 5V)

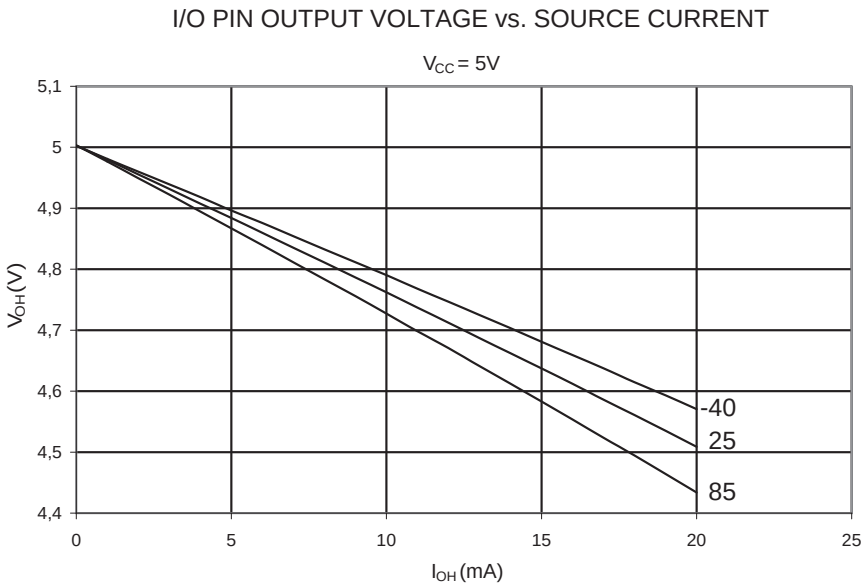


Figure 22-23. Reset Pin Output Voltage vs. Sink Current (V_{CC} = 3V)

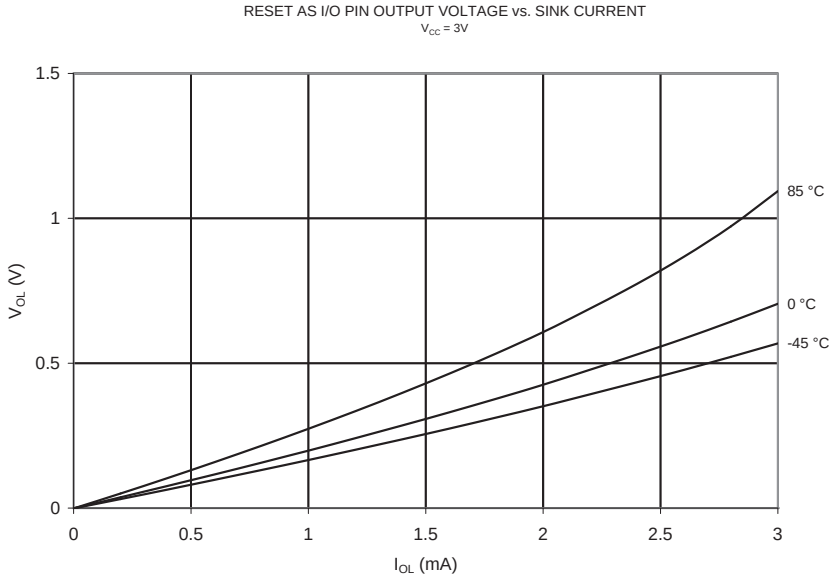


Figure 22-24. Reset Pin Output Voltage vs. Sink Current (V_{CC} = 5V)

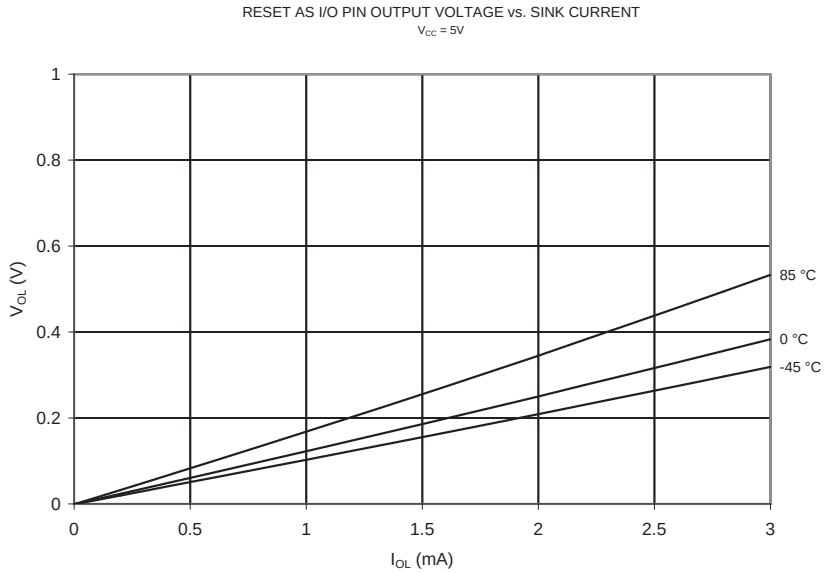


Figure 22-25. Reset Pin Output Voltage vs. Source Current (V_{CC} = 3V)

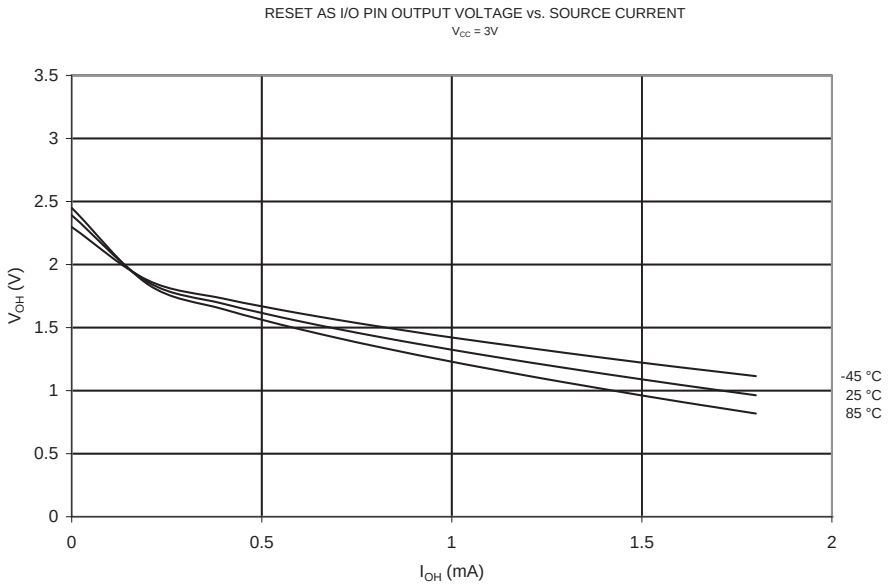
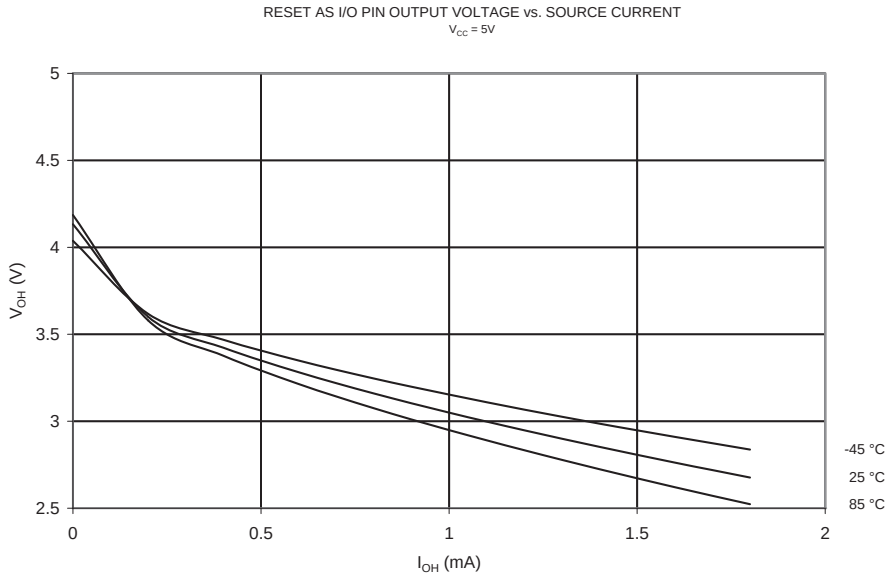


Figure 22-26. Reset Pin Output Voltage vs. Source Current (V_{CC} = 5V)



22.7 Pin Threshold and Hysteresis

Figure 22-27. I/O Pin Input Threshold Voltage vs. V_{CC} (V_{IH}, IO Pin Read as ‘1’)

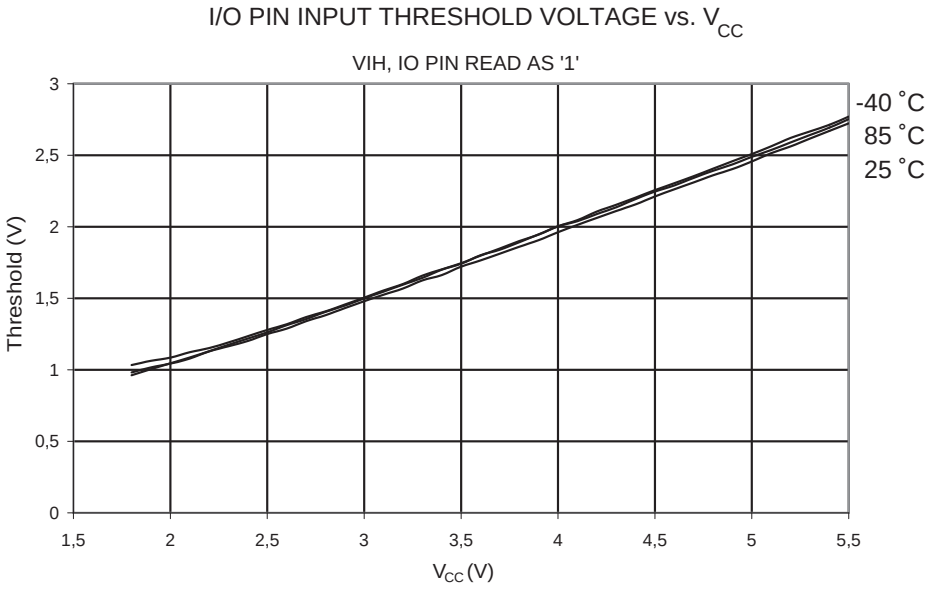


Figure 22-28. I/O Pin Input Threshold Voltage vs. V_{CC} (V_{IL}, IO Pin Read as '0')

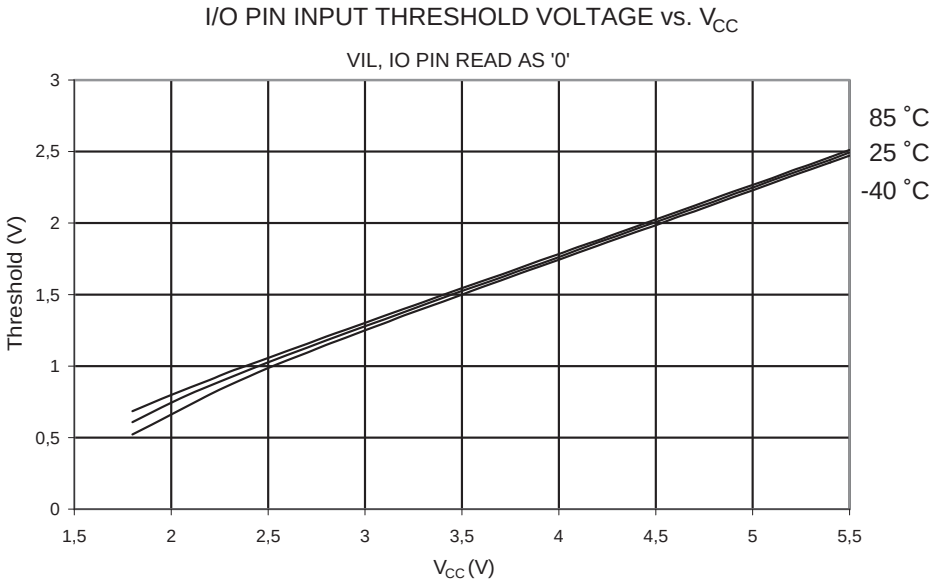


Figure 22-29. I/O Pin Input Hysteresis vs. V_{CC}

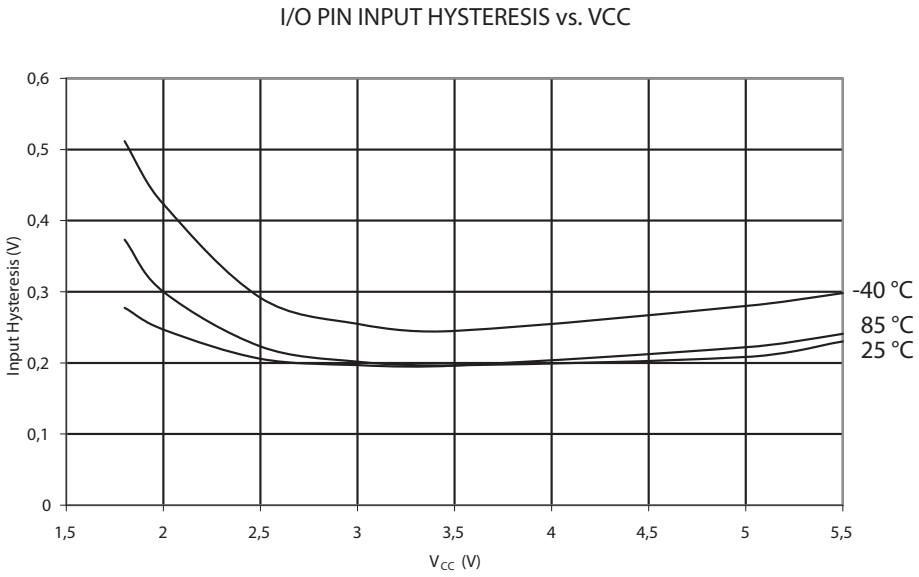


Figure 22-30. Reset Input Threshold Voltage vs. V_{CC} (V_{IH}, IO Pin Read as ‘1’)

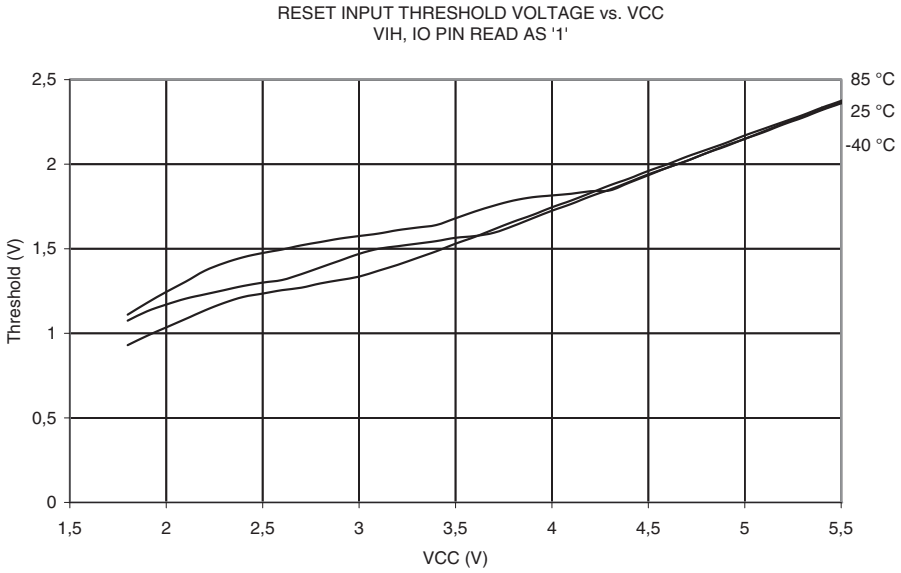


Figure 22-31. Reset Input Threshold Voltage vs. V_{CC} (V_{IL}, IO Pin Read as ‘0’)

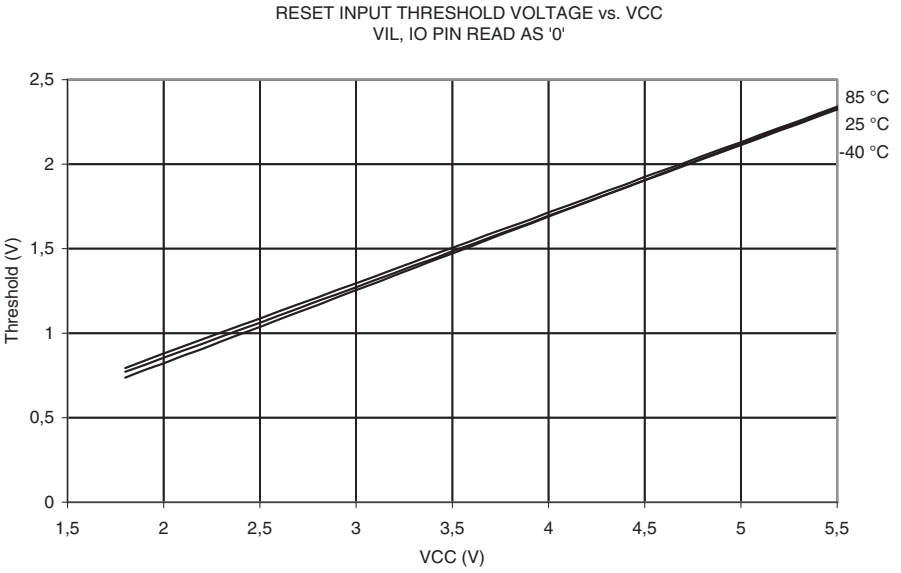
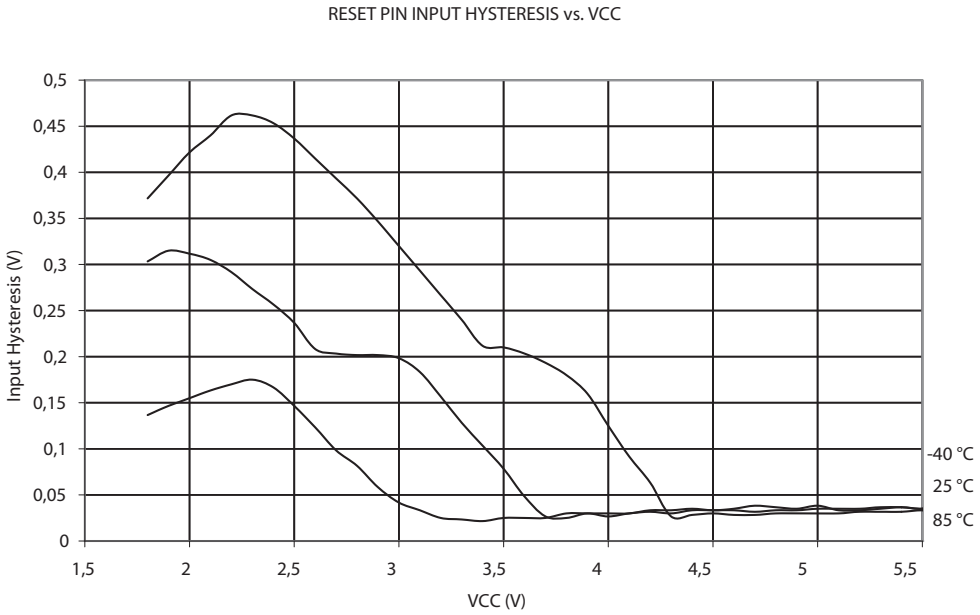


Figure 22-32. Reset Pin Input Hysteresis vs. V_{CC}



22.8 BOD Threshold

Figure 22-33. BOD Threshold vs. Temperature (BOD Level is 4.3V)

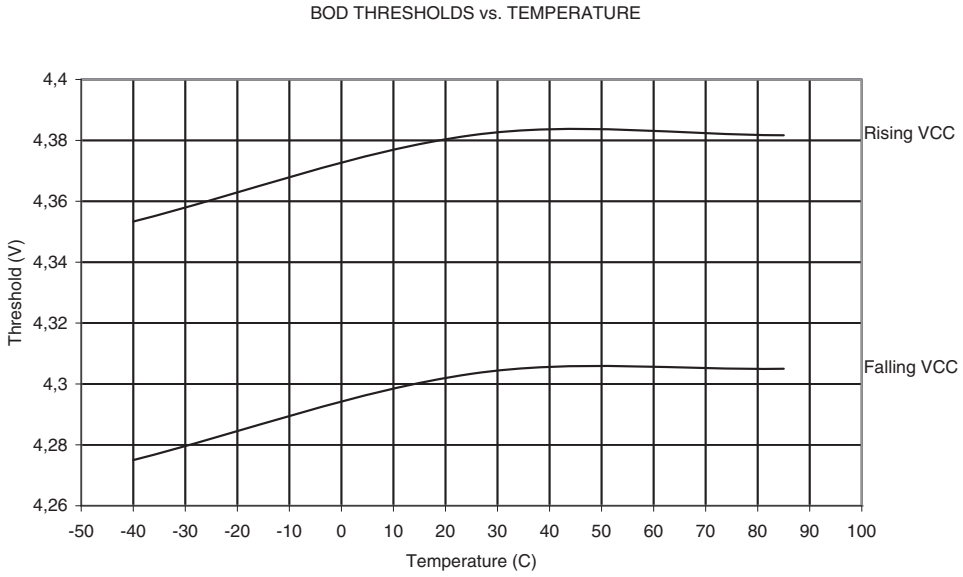


Figure 22-34. BOD Threshold vs. Temperature (BOD Level is 2.7V)

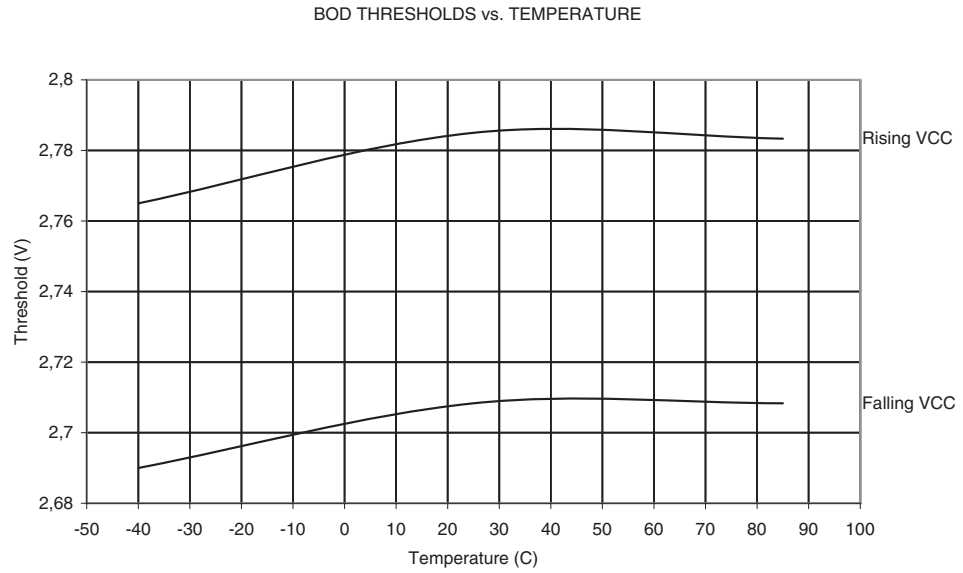


Figure 22-35. BOD Threshold vs. Temperature (BOD Level is 1.8V)

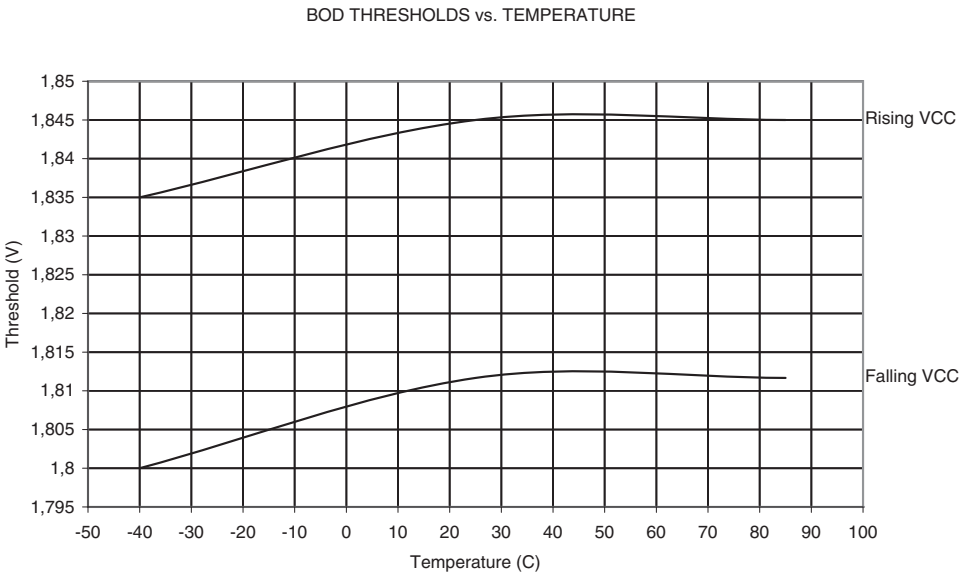


Figure 22-36. Bandgap Voltage vs. Supply Voltage

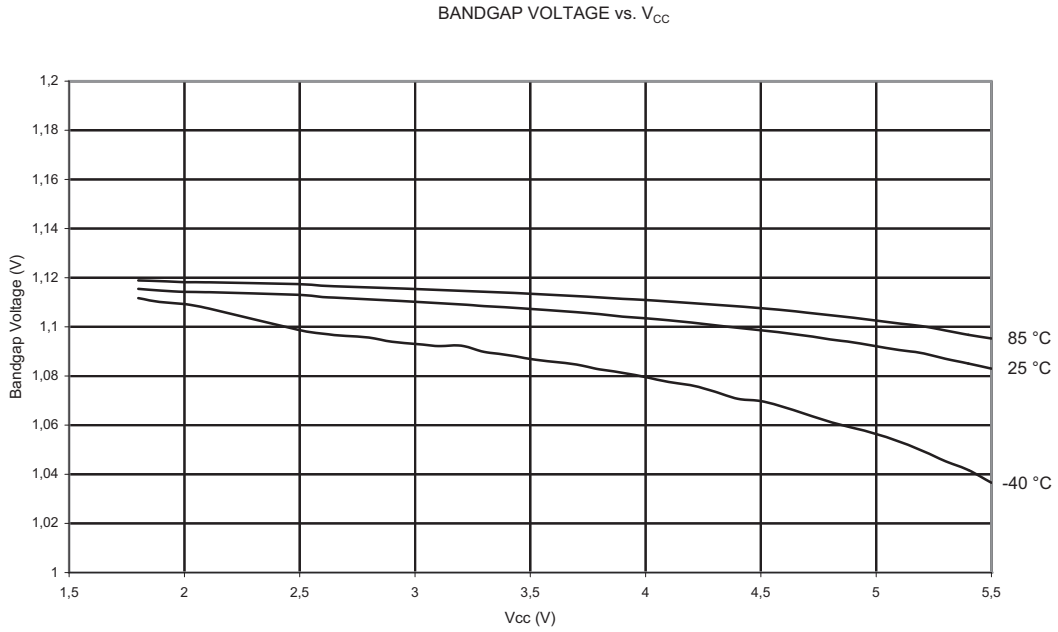
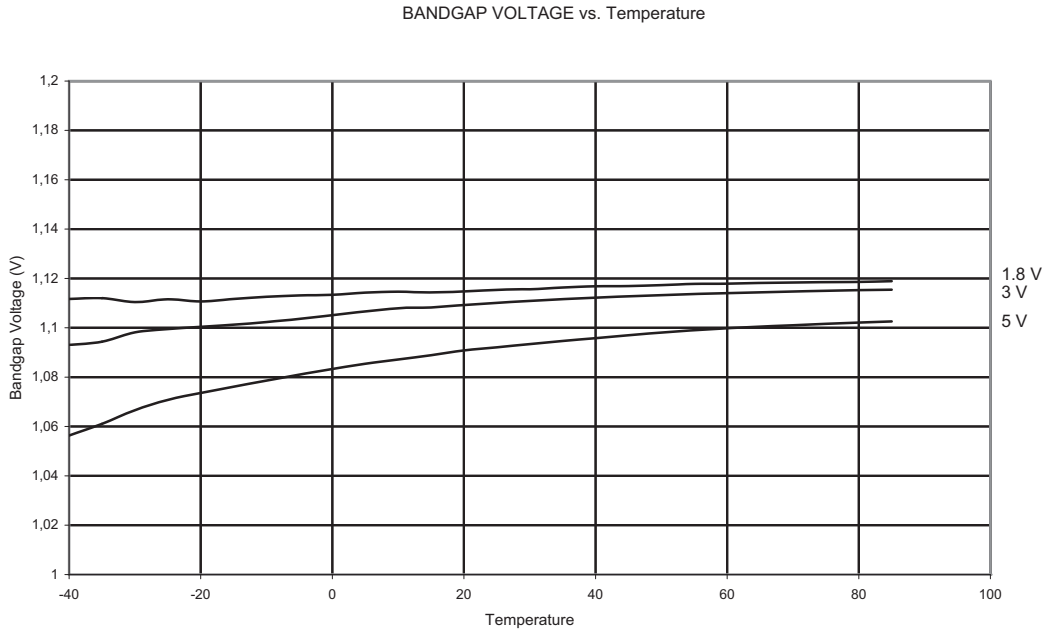


Figure 22-37. Bandgap Voltage vs. Temperature



22.9 Internal Oscillator Speed

Figure 22-38. Watchdog Oscillator Frequency vs. V_{CC}

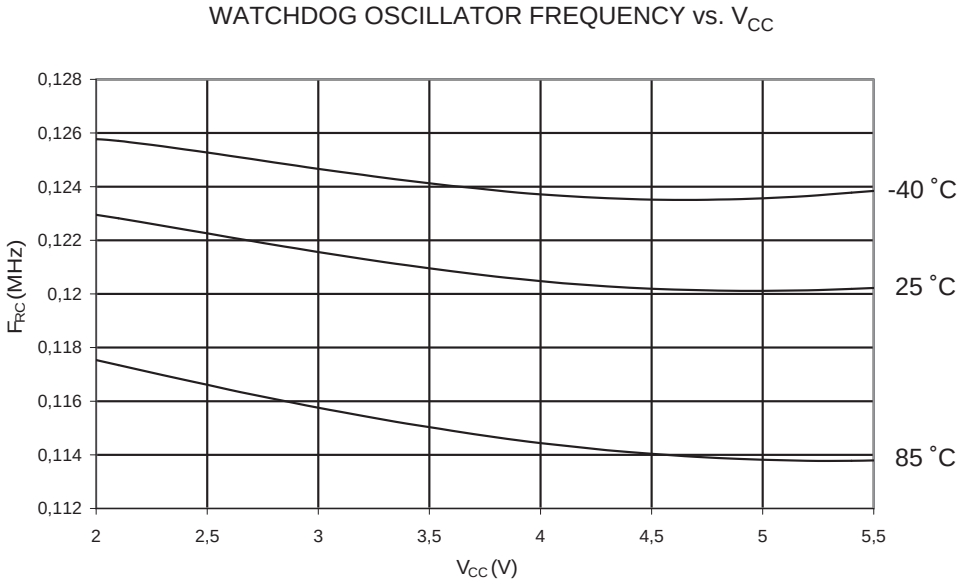


Figure 22-39. Watchdog Oscillator Frequency vs. Temperature

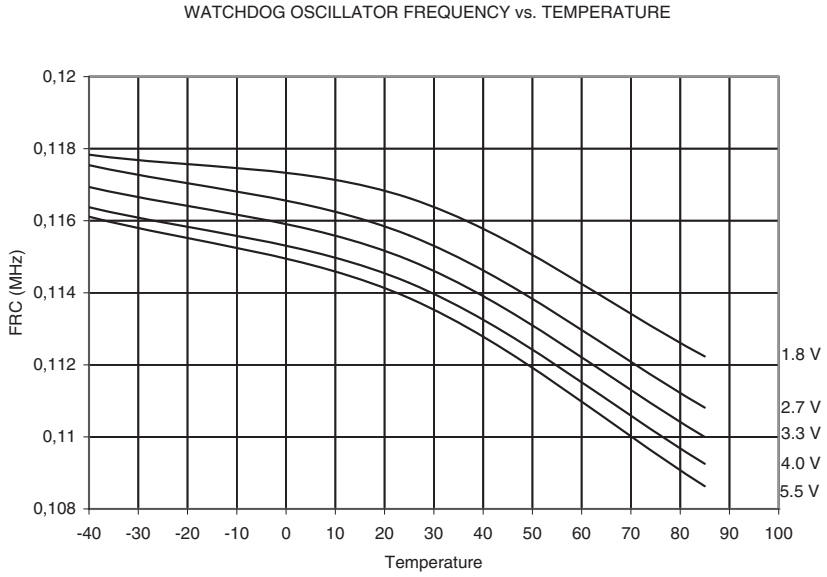


Figure 22-40. Calibrated 8 MHz RC Oscillator Frequency vs. V_{CC}

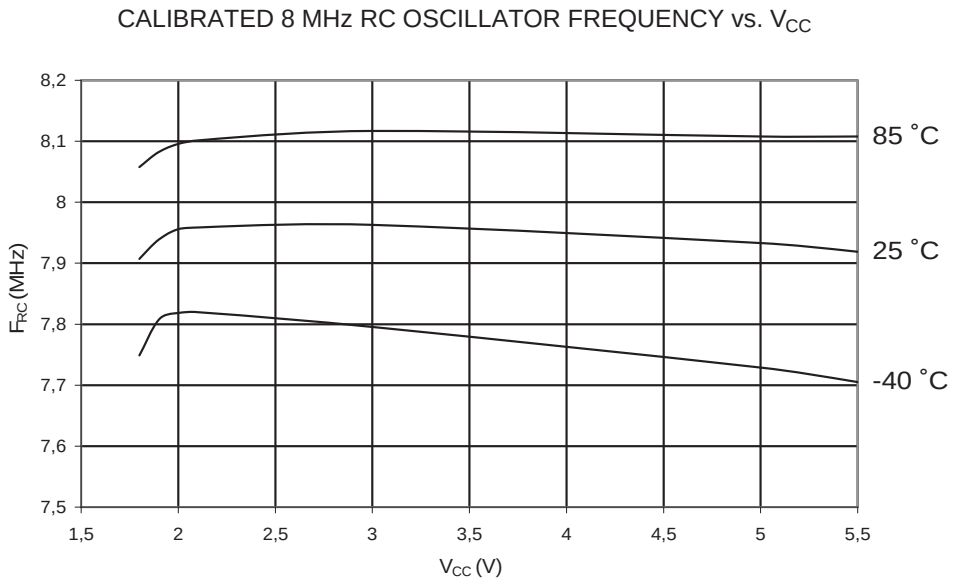


Figure 22-41. Calibrated 8 MHz RC Oscillator Frequency vs. Temperature

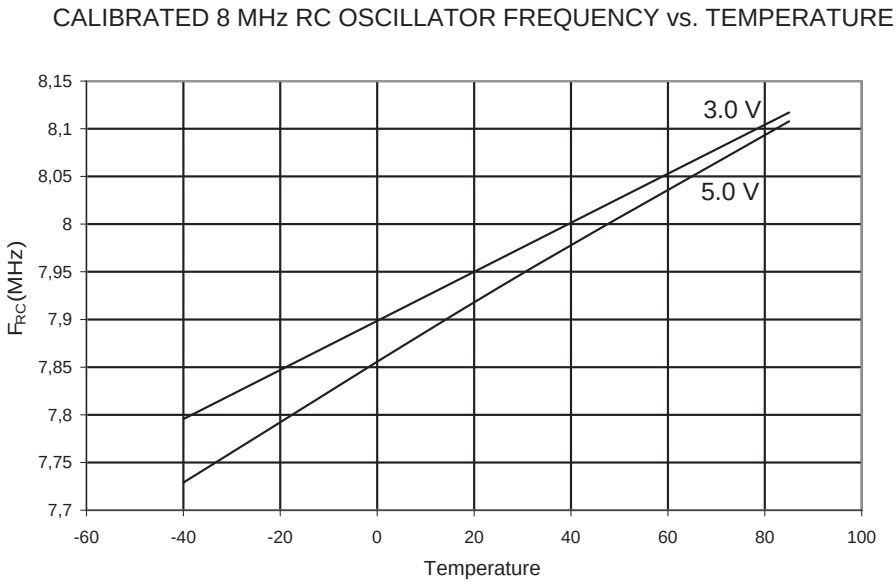


Figure 22-42. Calibrated 8 MHz RC Oscillator Frequency vs. OSCCAL Value

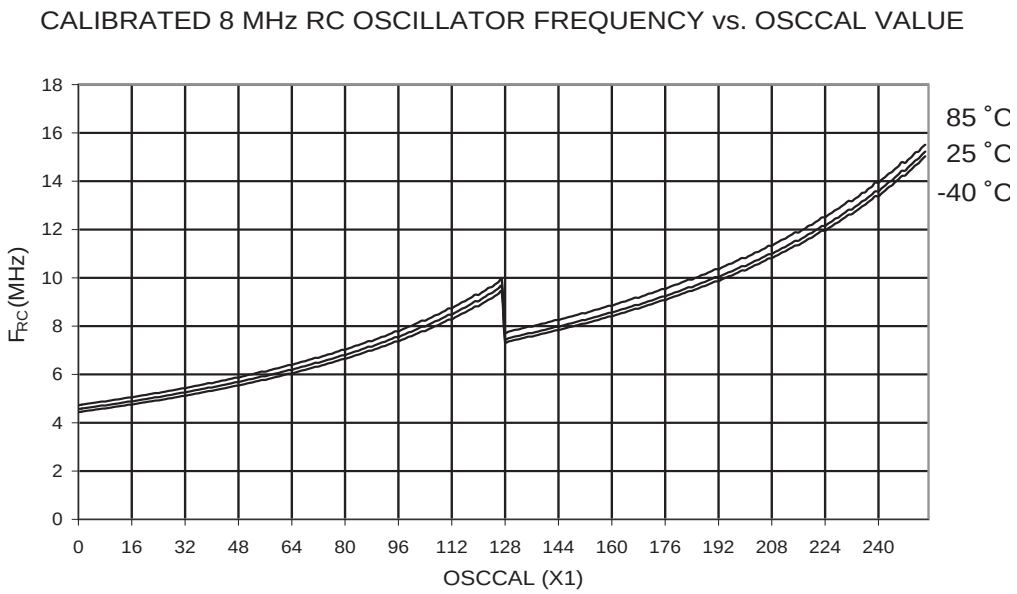


Figure 22-43. Calibrated 1.6 MHz RC Oscillator Frequency vs. V_{CC}

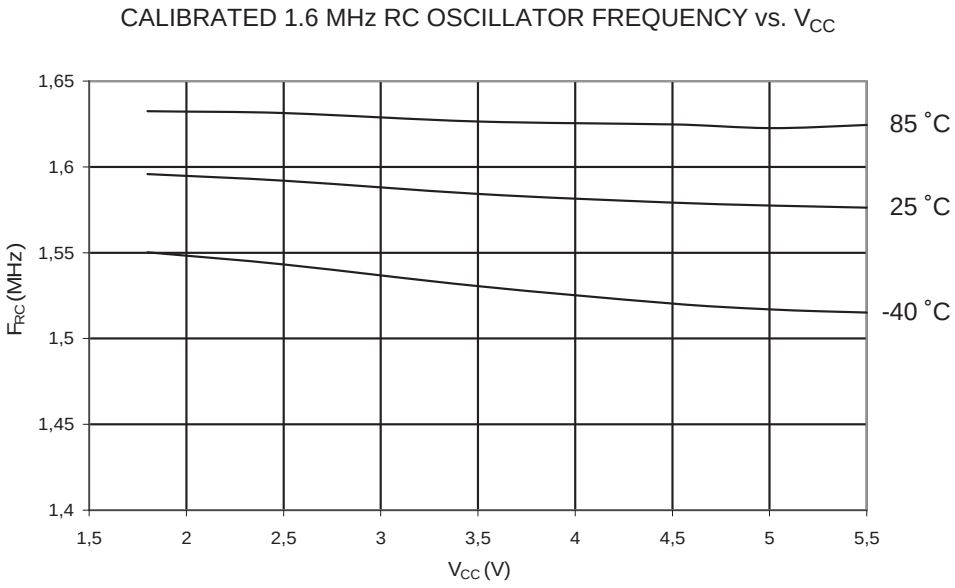


Figure 22-44. Calibrated 1.6 MHz RC Oscillator Frequency vs. Temperature

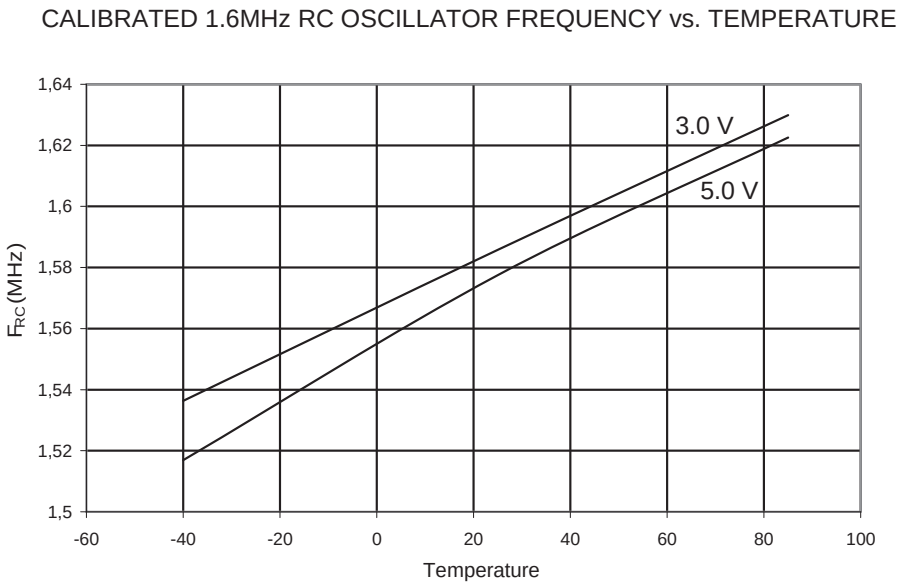
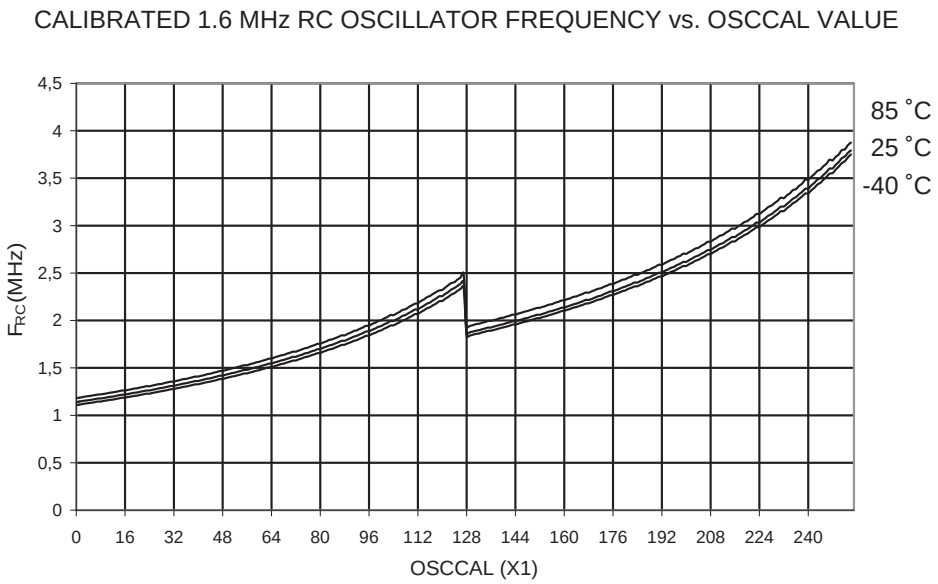


Figure 22-45. Calibrated 1.6 MHz RC Oscillator Frequency vs. OSCCAL Value



22.10 Current Consumption of Peripheral Units

Figure 22-46. Brownout Detector Current vs. V_{CC}

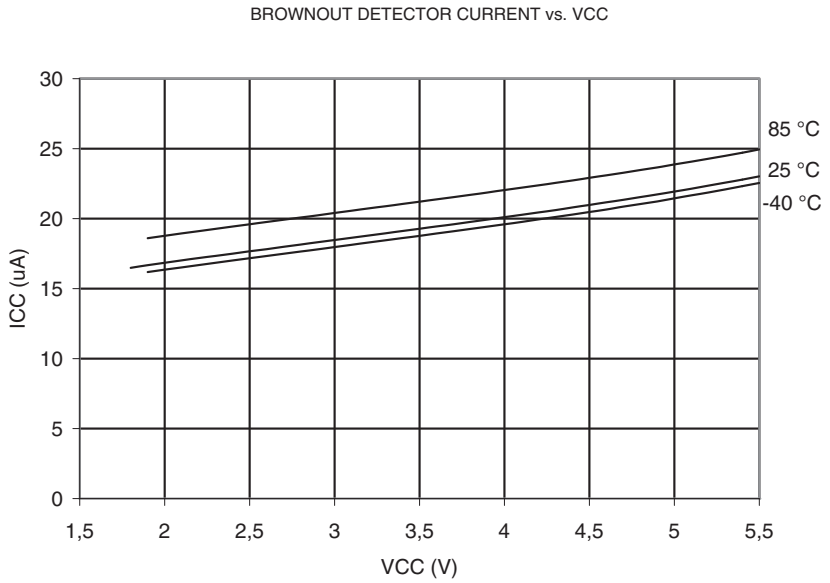


Figure 22-47. ADC Current vs. V_{CC} ($AREF = AV_{CC}$)

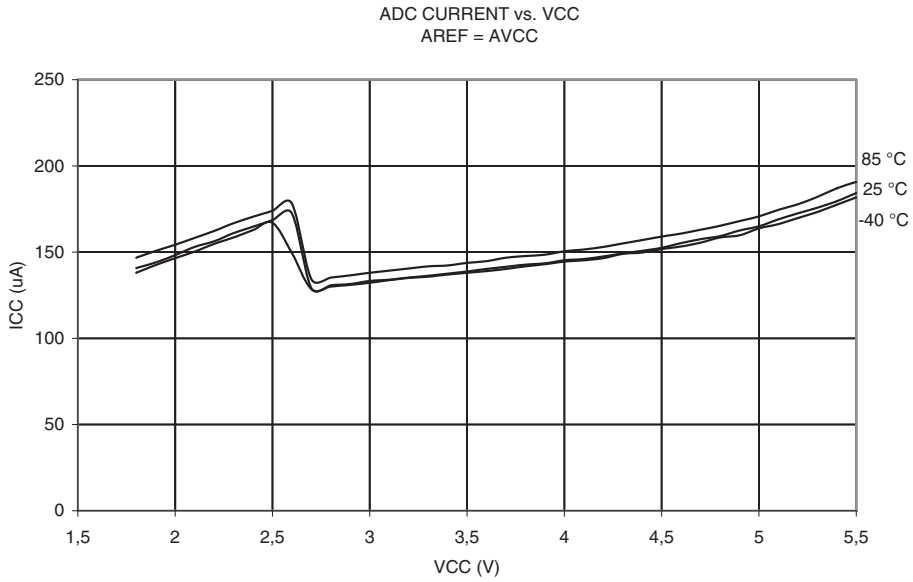


Figure 22-48. Analog Comparator Current vs. V_{CC}

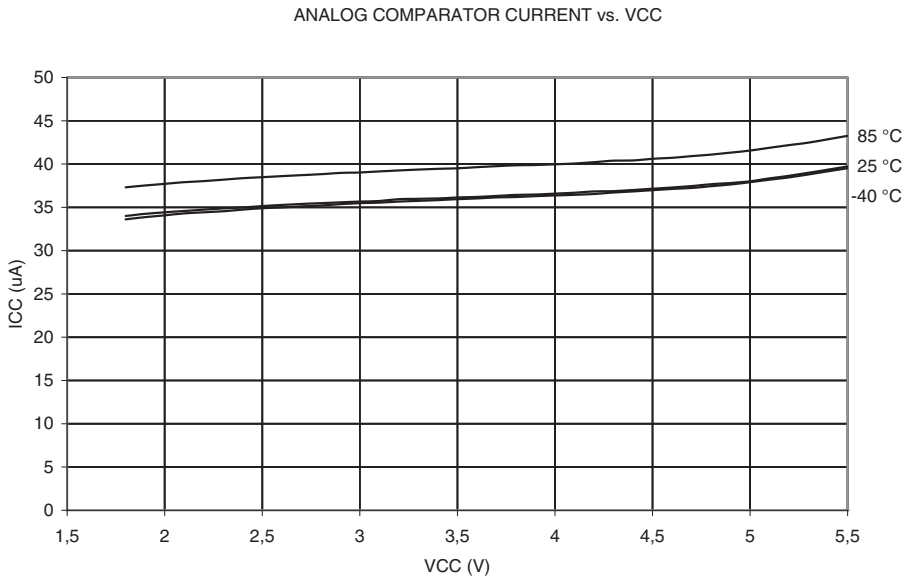
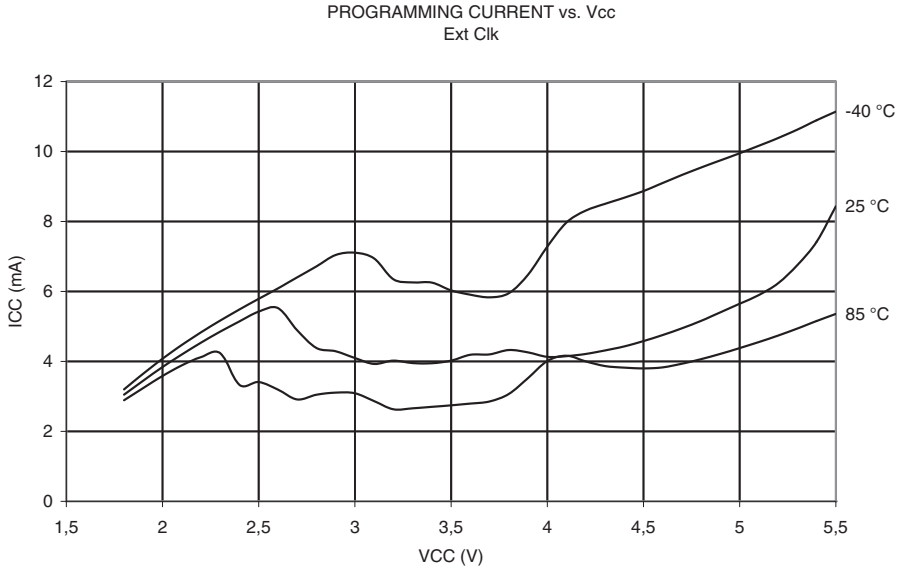


Figure 22-49. Programming Current vs. V_{CC}



22.11 Current Consumption in Reset and Reset Pulsewidth

Figure 22-50. Reset Supply Current vs. V_{CC} (0.1 - 1.0 MHz, Excluding Current Through The Reset Pull-up)

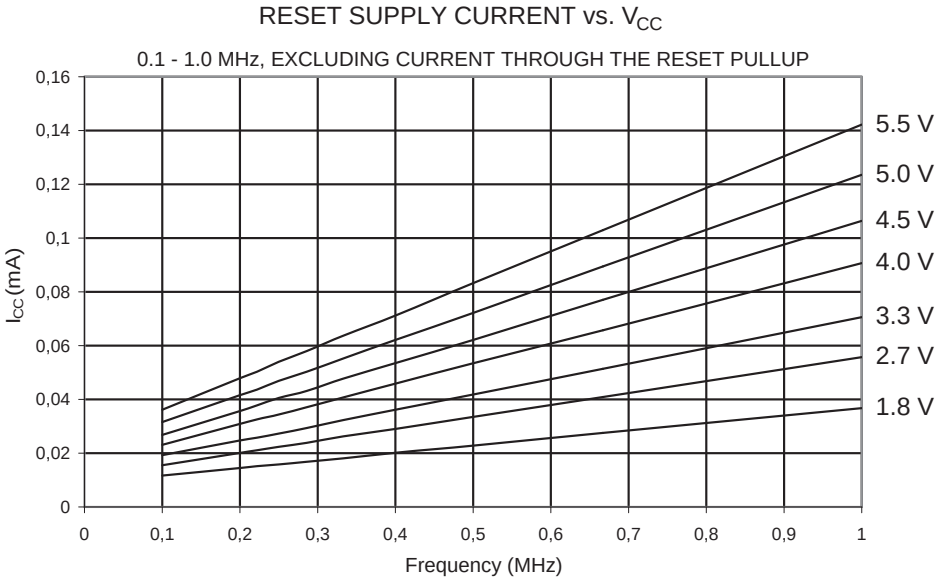


Figure 22-51. Reset Supply Current vs. V_{CC} (1 - 20 MHz, Excluding Current Through The Reset Pull-up)

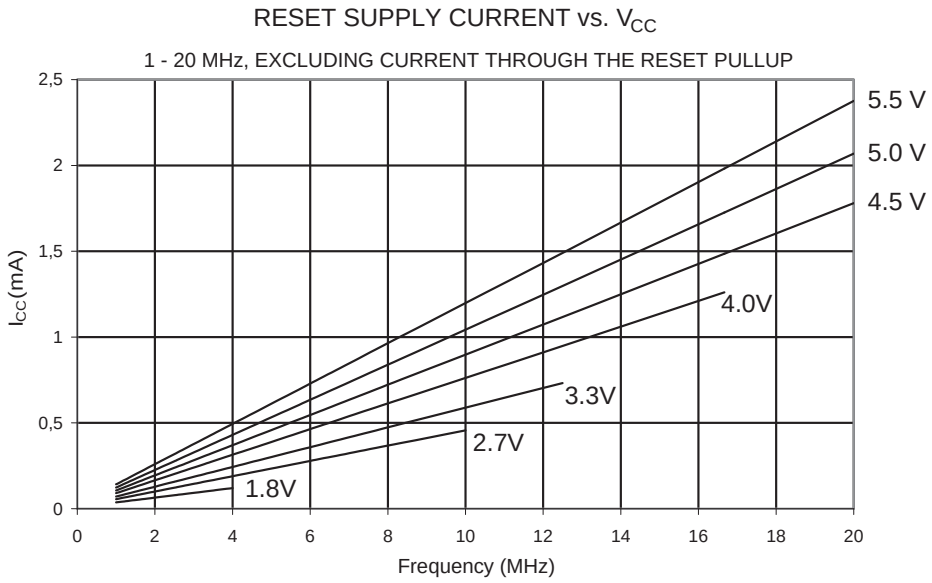
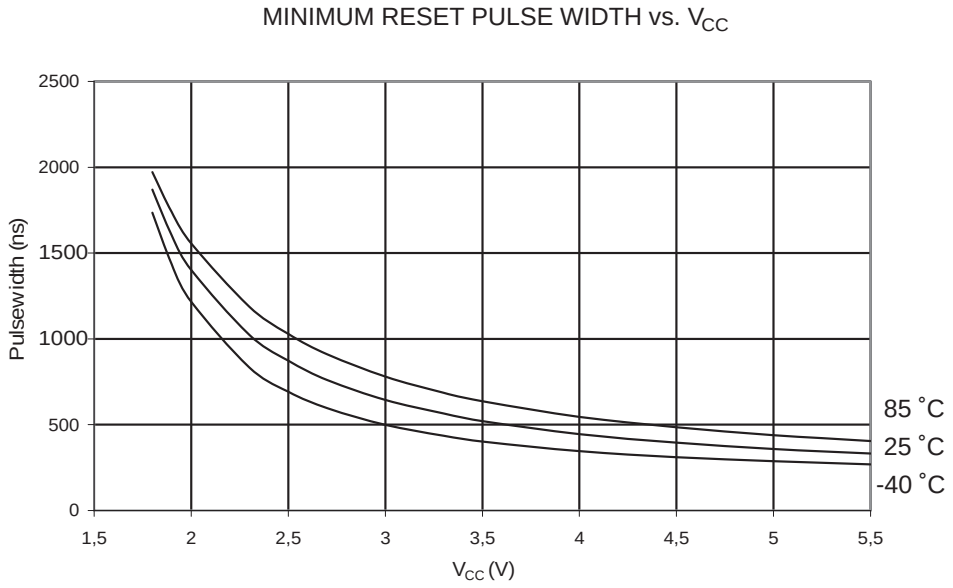


Figure 22-52. Minimum Reset Pulse Width vs. V_{CC}



23. Register Summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
0x3F	SREG	I	T	H	S	V	N	Z	C	page 8
0x3E	SPH	–	–	–	–	–	–	SP9	SP8	page 11
0x3D	SPL	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	page 11
0x3C	Reserved	–								
0x3B	GIMSK	–	INT0	PCIE	–	–	–	–	–	page 51
0x3A	GIFR	–	INTF0	PCIF	–	–	–	–	–	page 52
0x39	TIMSK	–	OCIE1A	OCIE1B	OCIE0A	OCIE0B	TOIE1	TOIE0	–	pages 81, 102
0x38	TIFR	–	OCF1A	OCF1B	OCF0A	OCF0B	TOV1	TOV0	–	page 81
0x37	SPMCSR	–	–	RSIG	CTPB	RFLB	PGWRT	PGERS	SPMEN	page 145
0x36	Reserved	–								
0x35	MCUCR	BODS	PUD	SE	SM1	SM0	BODSE	ISC01	ISC00	pages 37, 51, 64
0x34	MCUSR	–	–	–	–	WDRF	BORF	EXTRF	PORF	page 44,
0x33	TCCR0B	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	page 79
0x32	TCNT0	Timer/Counter0								page 80
0x31	OSCCAL	Oscillator Calibration Register								page 31
0x30	TCCR1	CTC1	PWM1A	COM1A1	COM1A0	CS13	CS12	CS11	CS10	pages 89, 100
0x2F	TCNT1	Timer/Counter1								pages 91, 102
0x2E	OCR1A	Timer/Counter1 Output Compare Register A								pages 91, 102
0x2D	OCR1C	Timer/Counter1 Output Compare Register C								pages 91, 102
0x2C	GTCCR	TSM	PWM1B	COM1B1	COM1B0	FOC1B	FOC1A	PSR1	PSR0	pages 77, 90, 101
0x2B	OCR1B	Timer/Counter1 Output Compare Register B								page 92
0x2A	TCCR0A	COM0A1	COM0A0	COM0B1	COM0B0	–		WGM01	WGM00	page 77
0x29	OCR0A	Timer/Counter0 – Output Compare Register A								page 80
0x28	OCR0B	Timer/Counter0 – Output Compare Register B								page 81
0x27	PLLCSR	LSM	–	–	–	–	PCKE	PLLE	PLOCK	pages 94, 103
0x26	CLKPR	CLKPCE	–	–	–	CLKPS3	CLKPS2	CLKPS1	CLKPS0	page 32
0x25	DT1A	DT1AH3	DT1AH2	DT1AH1	DT1AH0	DT1AL3	DT1AL2	DT1AL1	DT1AL0	page 107
0x24	DT1B	DT1BH3	DT1BH2	DT1BH1	DT1BH0	DT1BL3	DT1BL2	DT1BL1	DT1BL0	page 107
0x23	DTPS1	-	-	-	-	-	-	DTPS11	DTPS10	page 106
0x22	DWDR	DWDR[7:0]								page 140
0x21	WDTCR	WDIF	WDIE	WDP3	WDCE	WDE	WDP2	WDP1	WDP0	page 45
0x20	PRR	–				PRTIM1	PRTIM0	PRUSI	PRADC	page 36
0x1F	EEARH								EEAR8	page 20
0x1E	EEARL	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0	page 21
0x1D	EEDR	EEPROM Data Register								page 21
0x1C	EECR	–	–	EEPM1	EEPM0	EERIE	EEMPE	EEPE	EERE	page 21
0x1B	Reserved	–								
0x1A	Reserved	–								
0x19	Reserved	–								
0x18	PORTB	–	–	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	page 64
0x17	DDRB	–	–	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	page 64
0x16	PINB	–	–	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	page 64
0x15	PCMSK	–	–	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	page 52
0x14	DIDR0	–	–	ADC0D	ADC2D	ADC3D	ADC1D	AIN1D	AIN0D	pages 121, 138
0x13	GPIOR2	General Purpose I/O Register 2								page 10
0x12	GPIOR1	General Purpose I/O Register 1								page 10
0x11	GPIOR0	General Purpose I/O Register 0								page 10
0x10	USIBR	USI Buffer Register								page 115
0x0F	USIDR	USI Data Register								page 115
0x0E	USISR	USISIF	USIOIF	USIPF	USIDC	USICNT3	USICNT2	USICNT1	USICNT0	page 115
0x0D	USICR	USISIE	USIOIE	USIWM1	USIWM0	USICS1	USICS0	USICLK	USITC	page 116
0x0C	Reserved	–								
0x0B	Reserved	–								
0x0A	Reserved	–								
0x09	Reserved	–								
0x08	ACSR	ACD	ACBG	ACO	ACI	ACIE	–	ACIS1	ACIS0	page 120
0x07	ADMUX	REFS1	REFS0	ADLAR	REFS2	MUX3	MUX2	MUX1	MUX0	page 134
0x06	ADCSRA	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	page 136
0x05	ADCH	ADC Data Register High Byte								page 137
0x04	ADCL	ADC Data Register Low Byte								page 137
0x03	ADCSRB	BIN	ACME	IPR	–	–	ADTS2	ADTS1	ADTS0	pages 120, 137
0x02	Reserved	–								
0x01	Reserved	–								
0x00	Reserved	–								

Note: 1. For compatibility with future devices, reserved bits should be written to zero if accessed. Reserved I/O memory addresses

should never be written.

2. I/O Registers within the address range 0x00 - 0x1F are directly bit-accessible using the SBI and CBI instructions. In these registers, the value of single bits can be checked by using the SBIS and SBIC instructions.
3. Some of the Status Flags are cleared by writing a logical one to them. Note that, unlike most other AVRs, the CBI and SBI instructions will only operation the specified bit, and can therefore be used on registers containing such Status Flags. The CBI and SBI instructions work with registers 0x00 to 0x1F only.

24. Instruction Set Summary

Mnemonics	Operands	Description	Operation	Flags	#Clocks
ARITHMETIC AND LOGIC INSTRUCTIONS					
ADD	Rd, Rr	Add two Registers	Rd ← Rd + Rr	Z,C,N,V,H	1
ADC	Rd, Rr	Add with Carry two Registers	Rd ← Rd + Rr + C	Z,C,N,V,H	1
ADIW	Rdl,K	Add Immediate to Word	Rdh:Rdl ← Rdh:Rdl + K	Z,C,N,V,S	2
SUB	Rd, Rr	Subtract two Registers	Rd ← Rd - Rr	Z,C,N,V,H	1
SUBI	Rd, K	Subtract Constant from Register	Rd ← Rd - K	Z,C,N,V,H	1
SBC	Rd, Rr	Subtract with Carry two Registers	Rd ← Rd - Rr - C	Z,C,N,V,H	1
SBCI	Rd, K	Subtract with Carry Constant from Reg.	Rd ← Rd - K - C	Z,C,N,V,H	1
SBIW	Rdl,K	Subtract Immediate from Word	Rdh:Rdl ← Rdh:Rdl - K	Z,C,N,V,S	2
AND	Rd, Rr	Logical AND Registers	Rd ← Rd • Rr	Z,N,V	1
ANDI	Rd, K	Logical AND Register and Constant	Rd ← Rd • K	Z,N,V	1
OR	Rd, Rr	Logical OR Registers	Rd ← Rd v Rr	Z,N,V	1
ORI	Rd, K	Logical OR Register and Constant	Rd ← Rd v K	Z,N,V	1
EOR	Rd, Rr	Exclusive OR Registers	Rd ← Rd ⊕ Rr	Z,N,V	1
COM	Rd	One's Complement	Rd ← 0xFF – Rd	Z,C,N,V	1
NEG	Rd	Two's Complement	Rd ← 0x00 – Rd	Z,C,N,V,H	1
SBR	Rd,K	Set Bit(s) in Register	Rd ← Rd v K	Z,N,V	1
CBR	Rd,K	Clear Bit(s) in Register	Rd ← Rd • (0xFF - K)	Z,N,V	1
INC	Rd	Increment	Rd ← Rd + 1	Z,N,V	1
DEC	Rd	Decrement	Rd ← Rd – 1	Z,N,V	1
TST	Rd	Test for Zero or Minus	Rd ← Rd • Rd	Z,N,V	1
CLR	Rd	Clear Register	Rd ← Rd ⊕ Rd	Z,N,V	1
SER	Rd	Set Register	Rd ← 0xFF	None	1
BRANCH INSTRUCTIONS					
RJMP	k	Relative Jump	PC ← PC + k + 1	None	2
IJMP		Indirect Jump to (Z)	PC ← Z	None	2
RCALL	k	Relative Subroutine Call	PC ← PC + k + 1	None	3
ICALL		Indirect Call to (Z)	PC ← Z	None	3
RET		Subroutine Return	PC ← STACK	None	4
RETI		Interrupt Return	PC ← STACK	I	4
CPSE	Rd,Rr	Compare, Skip if Equal	if (Rd = Rr) PC ← PC + 2 or 3	None	1/2/3
CP	Rd,Rr	Compare	Rd – Rr	Z, N,V,C,H	1
CPC	Rd,Rr	Compare with Carry	Rd – Rr – C	Z, N,V,C,H	1
CPI	Rd,K	Compare Register with Immediate	Rd – K	Z, N,V,C,H	1
SBRC	Rr, b	Skip if Bit in Register Cleared	if (Rr(b)=0) PC ← PC + 2 or 3	None	1/2/3
SBRS	Rr, b	Skip if Bit in Register is Set	if (Rr(b)=1) PC ← PC + 2 or 3	None	1/2/3
SBIC	P, b	Skip if Bit in I/O Register Cleared	if (P(b)=0) PC ← PC + 2 or 3	None	1/2/3
SBIS	P, b	Skip if Bit in I/O Register is Set	if (P(b)=1) PC ← PC + 2 or 3	None	1/2/3
BRBS	s, k	Branch if Status Flag Set	if (SREG(s) = 1) then PC←PC+k + 1	None	1/2
BRBC	s, k	Branch if Status Flag Cleared	if (SREG(s) = 0) then PC←PC+k + 1	None	1/2
BREQ	k	Branch if Equal	if (Z = 1) then PC ← PC + k + 1	None	1/2
BRNE	k	Branch if Not Equal	if (Z = 0) then PC ← PC + k + 1	None	1/2
BRCS	k	Branch if Carry Set	if (C = 1) then PC ← PC + k + 1	None	1/2
BRCC	k	Branch if Carry Cleared	if (C = 0) then PC ← PC + k + 1	None	1/2
BRSH	k	Branch if Same or Higher	if (C = 0) then PC ← PC + k + 1	None	1/2
BRLO	k	Branch if Lower	if (C = 1) then PC ← PC + k + 1	None	1/2
BRMI	k	Branch if Minus	if (N = 1) then PC ← PC + k + 1	None	1/2
BRPL	k	Branch if Plus	if (N = 0) then PC ← PC + k + 1	None	1/2
BRGE	k	Branch if Greater or Equal, Signed	if (N ⊕ V= 0) then PC ← PC + k + 1	None	1/2
BRLT	k	Branch if Less Than Zero, Signed	if (N ⊕ V= 1) then PC ← PC + k + 1	None	1/2
BRHS	k	Branch if Half Carry Flag Set	if (H = 1) then PC ← PC + k + 1	None	1/2
BRHC	k	Branch if Half Carry Flag Cleared	if (H = 0) then PC ← PC + k + 1	None	1/2
BRTS	k	Branch if T Flag Set	if (T = 1) then PC ← PC + k + 1	None	1/2
BRTC	k	Branch if T Flag Cleared	if (T = 0) then PC ← PC + k + 1	None	1/2
BRVS	k	Branch if Overflow Flag is Set	if (V = 1) then PC ← PC + k + 1	None	1/2
BRVC	k	Branch if Overflow Flag is Cleared	if (V = 0) then PC ← PC + k + 1	None	1/2
BRIE	k	Branch if Interrupt Enabled	if (I = 1) then PC ← PC + k + 1	None	1/2
BRID	k	Branch if Interrupt Disabled	if (I = 0) then PC ← PC + k + 1	None	1/2
BIT AND BIT-TEST INSTRUCTIONS					
SBI	P,b	Set Bit in I/O Register	I/O(P,b) ← 1	None	2
CBI	P,b	Clear Bit in I/O Register	I/O(P,b) ← 0	None	2
LSL	Rd	Logical Shift Left	Rd(n+1) ← Rd(n), Rd(0) ← 0	Z,C,N,V	1
LSR	Rd	Logical Shift Right	Rd(n) ← Rd(n+1), Rd(7) ← 0	Z,C,N,V	1
ROL	Rd	Rotate Left Through Carry	Rd(0)←C,Rd(n+1)← Rd(n),C←Rd(7)	Z,C,N,V	1
ROR	Rd	Rotate Right Through Carry	Rd(7)←C,Rd(n)← Rd(n+1),C←Rd(0)	Z,C,N,V	1
ASR	Rd	Arithmetic Shift Right	Rd(n) ← Rd(n+1), n=0..6	Z,C,N,V	1

Mnemonics	Operands	Description	Operation	Flags	#Clocks
SWAP	Rd	Swap Nibbles	Rd(3..0)←Rd(7..4),Rd(7..4)←Rd(3..0)	None	1
BSET	s	Flag Set	SREG(s) ← 1	SREG(s)	1
BCLR	s	Flag Clear	SREG(s) ← 0	SREG(s)	1
BST	Rr, b	Bit Store from Register to T	T ← Rr(b)	T	1
BLD	Rd, b	Bit load from T to Register	Rd(b) ← T	None	1
SEC		Set Carry	C ← 1	C	1
CLC		Clear Carry	C ← 0	C	1
SEN		Set Negative Flag	N ← 1	N	1
CLN		Clear Negative Flag	N ← 0	N	1
SEZ		Set Zero Flag	Z ← 1	Z	1
CLZ		Clear Zero Flag	Z ← 0	Z	1
SEI		Global Interrupt Enable	I ← 1	I	1
CLI		Global Interrupt Disable	I ← 0	I	1
SES		Set Signed Test Flag	S ← 1	S	1
CLS		Clear Signed Test Flag	S ← 0	S	1
SEV		Set Twos Complement Overflow.	V ← 1	V	1
CLV		Clear Twos Complement Overflow	V ← 0	V	1
SET		Set T in SREG	T ← 1	T	1
CLT		Clear T in SREG	T ← 0	T	1
SEH		Set Half Carry Flag in SREG	H ← 1	H	1
CLH		Clear Half Carry Flag in SREG	H ← 0	H	1
DATA TRANSFER INSTRUCTIONS					
MOV	Rd, Rr	Move Between Registers	Rd ← Rr	None	1
MOVW	Rd, Rr	Copy Register Word	Rd+1:Rd ← Rr+1:Rr	None	1
LDI	Rd, K	Load Immediate	Rd ← K	None	1
LD	Rd, X	Load Indirect	Rd ← (X)	None	2
LD	Rd, X+	Load Indirect and Post-Inc.	Rd ← (X), X ← X + 1	None	2
LD	Rd, - X	Load Indirect and Pre-Dec.	X ← X - 1, Rd ← (X)	None	2
LD	Rd, Y	Load Indirect	Rd ← (Y)	None	2
LD	Rd, Y+	Load Indirect and Post-Inc.	Rd ← (Y), Y ← Y + 1	None	2
LD	Rd, - Y	Load Indirect and Pre-Dec.	Y ← Y - 1, Rd ← (Y)	None	2
LDD	Rd,Y+q	Load Indirect with Displacement	Rd ← (Y + q)	None	2
LD	Rd, Z	Load Indirect	Rd ← (Z)	None	2
LD	Rd, Z+	Load Indirect and Post-Inc.	Rd ← (Z), Z ← Z+1	None	2
LD	Rd, -Z	Load Indirect and Pre-Dec.	Z ← Z - 1, Rd ← (Z)	None	2
LDD	Rd, Z+q	Load Indirect with Displacement	Rd ← (Z + q)	None	2
LDS	Rd, k	Load Direct from SRAM	Rd ← (k)	None	2
ST	X, Rr	Store Indirect	(X) ← Rr	None	2
ST	X+, Rr	Store Indirect and Post-Inc.	(X) ← Rr, X ← X + 1	None	2
ST	- X, Rr	Store Indirect and Pre-Dec.	X ← X - 1, (X) ← Rr	None	2
ST	Y, Rr	Store Indirect	(Y) ← Rr	None	2
ST	Y+, Rr	Store Indirect and Post-Inc.	(Y) ← Rr, Y ← Y + 1	None	2
ST	- Y, Rr	Store Indirect and Pre-Dec.	Y ← Y - 1, (Y) ← Rr	None	2
STD	Y+q,Rr	Store Indirect with Displacement	(Y + q) ← Rr	None	2
ST	Z, Rr	Store Indirect	(Z) ← Rr	None	2
ST	Z+, Rr	Store Indirect and Post-Inc.	(Z) ← Rr, Z ← Z + 1	None	2
ST	-Z, Rr	Store Indirect and Pre-Dec.	Z ← Z - 1, (Z) ← Rr	None	2
STD	Z+q,Rr	Store Indirect with Displacement	(Z + q) ← Rr	None	2
STS	k, Rr	Store Direct to SRAM	(k) ← Rr	None	2
LPM		Load Program Memory	R0 ← (Z)	None	3
LPM	Rd, Z	Load Program Memory	Rd ← (Z)	None	3
LPM	Rd, Z+	Load Program Memory and Post-Inc	Rd ← (Z), Z ← Z+1	None	3
SPM		Store Program Memory	(z) ← R1:R0	None	
IN	Rd, P	In Port	Rd ← P	None	1
OUT	P, Rr	Out Port	P ← Rr	None	1
PUSH	Rr	Push Register on Stack	STACK ← Rr	None	2
POP	Rd	Pop Register from Stack	Rd ← STACK	None	2
MCU CONTROL INSTRUCTIONS					
NOP		No Operation		None	1
SLEEP		Sleep	(see specific descr. for Sleep function)	None	1
WDR		Watchdog Reset	(see specific descr. for WDR/Timer)	None	1
BREAK		Break	For On-chip Debug Only	None	N/A

25. Ordering Information

25.1 ATtiny25

Speed (MHz) [1]	Supply Voltage (V)	Temperature Range	Package [2]	Ordering Code [3]
10	1.8 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny25V-10PU
			8S2	ATtiny25V-10SU ATtiny25V-10SUR ATtiny25V-10SH ATtiny25V-10SHR
			S8S1	ATtiny25V-10SSU ATtiny25V-10SSUR ATtiny25V-10SSH ATtiny25V-10SSHR
			20M1	ATtiny25V-10MU ATtiny25V-10MUR
		Industrial (-40°C to +105°C) [5]	8S2	ATtiny25V-10SN ATtiny25V-10SNR
			S8S1	ATtiny25V-10SSN ATtiny25V-10SSNR
		Industrial (-40°C to +125°C) [6]	20M1	ATtiny25V-10MF ATtiny25V-10MFR
20	2.7 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny25-20PU
			8S2	ATtiny25-20SU ATtiny25-20SUR ATtiny25-20SH ATtiny25-20SHR
			S8S1	ATtiny25-20SSU ATtiny25-20SSUR ATtiny25-20SSH ATtiny25-20SSHR
			20M1	ATtiny25-20MU ATtiny25-20MUR
		Industrial (-40°C to +105°C) [5]	8S2	ATtiny25-20SN ATtiny25-20SNR
			S8S1	ATtiny25-20SSN ATtiny25-20SSNR
		Industrial (-40°C to +125°C) [6]	20M1	ATtiny25-20MF ATtiny25-20MFR

Notes:
1. For speed vs. supply voltage, see section 21.3 “Speed” on page 163.
2. All Pb-free, halide-free, fully green, and comply with European directive for Restriction of Hazardous Substances (RoHS).
3. Code indicators: H = NiPdAu lead finish, U/N = matte tin, R = tape & reel.
4. Can also be supplied in wafer form. Contact your local Atmel sales office for ordering information and minimum quantities.
5. For characteristics, see “Appendix A – Specification at 105°C”.
6. For characteristics, see “Appendix B – Specification at 125°C”.

Package Types	
8P3	8-lead, 0.300" Wide, Plastic Dual Inline Package (PDIP)
8S2	8-lead, 0.208" Wide, Plastic Gull-Wing Small Outline (EIAJ SOIC)
S8S1	8-lead, 0.150" Wide, Plastic Gull-Wing Small Outline (JEDEC SOIC)
20M1	20-pad, 4 x 4 x 0.8 mm Body, Quad Flat No-Lead/Micro Lead Frame Package (QFN/MLF)

25.2 ATtiny45

Speed (MHz) [1]	Supply Voltage (V)	Temperature Range	Package [2]	Ordering Code [3]
10	1.8 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny45V-10PU
			8S2	ATtiny45V-10SU ATtiny45V-10SUR ATtiny45V-10SH ATtiny45V-10SHR
			8X	ATtiny45V-10XU ATtiny45V-10XUR
			20M1	ATtiny45V-10MU ATtiny45V-10MUR
20	2.7 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny45-20PU
			8S2	ATtiny45-20SU ATtiny45-20SUR ATtiny45-20SH ATtiny45-20SHR
			8X	ATtiny45-20XU ATtiny45-20XUR
			20M1	ATtiny45-20MU ATtiny45-20MUR

Notes:
1. For speed vs. supply voltage, see section 21.3 “Speed” on page 163.
2. All packages are Pb-free, halide-free and fully green and they comply with the European directive for Restriction of Hazardous Substances (RoHS).
3. Code indicators:
   - H: NiPdAu lead finish
   - U: matte tin
   - R: tape & reel
4. These devices can also be supplied in wafer form. Please contact your local Atmel sales office for detailed ordering information and minimum quantities.

Package Types	
8P3	8-lead, 0.300" Wide, Plastic Dual Inline Package (PDIP)
8S2	8-lead, 0.208" Wide, Plastic Gull-Wing Small Outline (EIAJ SOIC)
8X	8-lead, 4.4 mm Wide, Plastic Thin Shrink Small Outline Package (TSSOP)
20M1	20-pad, 4 x 4 x 0.8 mm Body, Quad Flat No-Lead/Micro Lead Frame Package (QFN/MLF)

25.3 ATtiny85

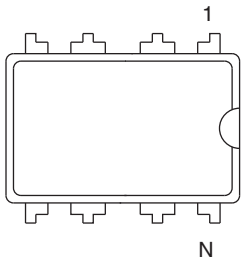
Speed (MHz) [1]	Supply Voltage (V)	Temperature Range	Package [2]	Ordering Code [3]
10	1.8 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny85V-10PU
			8S2	ATtiny85V-10SU ATtiny85V-10SUR ATtiny85V-10SH ATtiny85V-10SHR
			20M1	ATtiny85V-10MU ATtiny85V-10MUR
20	2.7 – 5.5	Industrial (-40°C to +85°C) [4]	8P3	ATtiny85-20PU
			8S2	ATtiny85-20SU ATtiny85-20SUR ATtiny85-20SH ATtiny85-20SHR
			20M1	ATtiny85-20MU ATtiny85-20MUR

Notes:
1. For speed vs. supply voltage, see section 21.3 “Speed” on page 163.
2. All packages are Pb-free, halide-free and fully green and they comply with the European directive for Restriction of Hazardous Substances (RoHS).
3. Code indicators:
   - H: NiPdAu lead finish
   - U: matte tin
   - R: tape & reel
4. These devices can also be supplied in wafer form. Please contact your local Atmel sales office for detailed ordering information and minimum quantities.

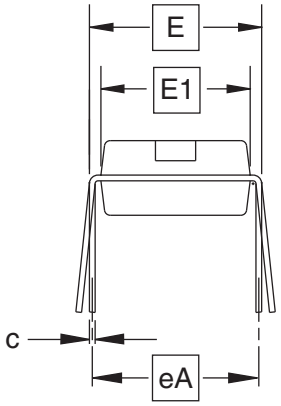
Package Types	
8P3	8-lead, 0.300" Wide, Plastic Dual Inline Package (PDIP)
8S2	8-lead, 0.208" Wide, Plastic Gull-Wing Small Outline (EIAJ SOIC)
20M1	20-pad, 4 x 4 x 0.8 mm Body, Quad Flat No-Lead/Micro Lead Frame Package (QFN/MLF)

26. Packaging Information

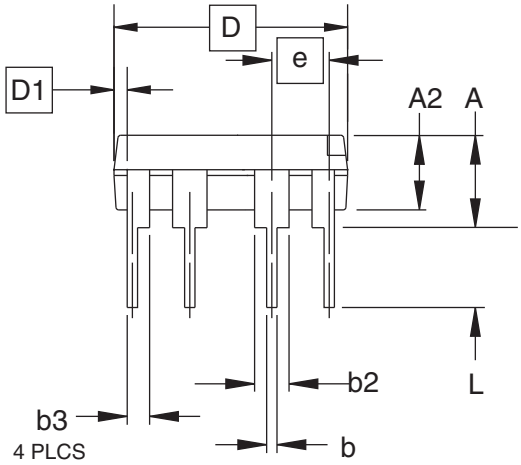
26.1 8P3



Top View



End View



Side View

COMMON DIMENSIONS
(Unit of Measure = inches)

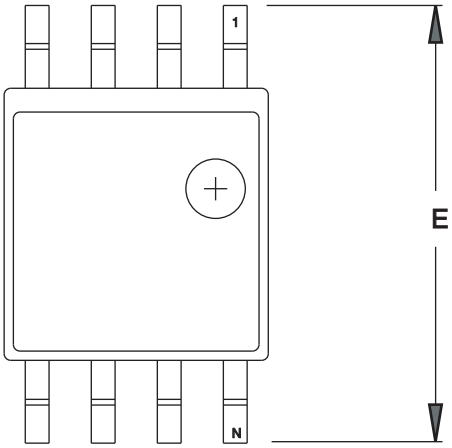
SYMBOL	MIN	NOM	MAX	NOTE
A			0.210	2
A2	0.115	0.130	0.195	
b	0.014	0.018	0.022	5
b2	0.045	0.060	0.070	6
b3	0.030	0.039	0.045	6
c	0.008	0.010	0.014	
D	0.355	0.365	0.400	3
D1	0.005			3
E	0.300	0.310	0.325	4
E1	0.240	0.250	0.280	3
e	0.100 BSC			
eA	0.300 BSC			4
L	0.115	0.130	0.150	2

Notes:
1. This drawing is for general information only; refer to JEDEC Drawing MS-001, Variation BA for additional information.
2. Dimensions A and L are measured with the package seated in JEDEC seating plane Gauge GS-3.
3. D, D1 and E1 dimensions do not include mold Flash or protrusions. Mold Flash or protrusions shall not exceed 0.010 inch.
4. E and eA measured with the leads constrained to be perpendicular to datum.
5. Pointed or rounded lead tips are preferred to ease insertion.
6. b2 and b3 maximum dimensions do not include Dambar protrusions. Dambar protrusions shall not exceed 0.010 (0.25 mm).

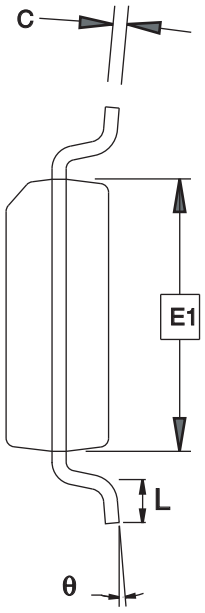
01/09/02

	TITLE	DRAWING NO.	REV.
2325 Orchard Parkway San Jose, CA 95131	**8P3**, 8-lead, 0.300" Wide Body, Plastic Dual In-line Package (PDIP)	8P3	B

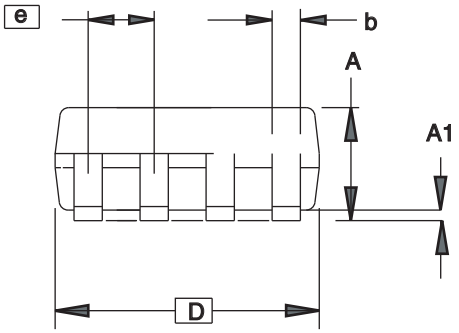
26.2 8S2



TOP VIEW



END VIEW



SIDE VIEW

COMMON DIMENSIONS
(Unit of Measure = mm)

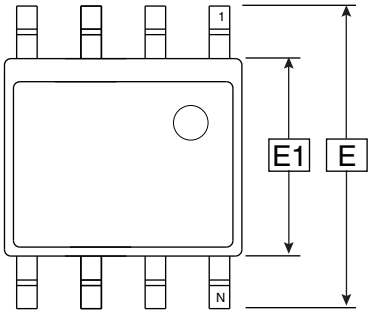
SYMBOL	MIN	NOM	MAX	NOTE
A	1.70		2.16	
A1	0.05		0.25	
b	0.35		0.48	4
C	0.15		0.35	4
D	5.13		5.35	
E1	5.18		5.40	2
E	7.70		8.26	
L	0.51		0.85	
θ	0°		8°	
e	1.27 BSC			3

Notes: 1. This drawing is for general information only; refer to EIAJ Drawing EDR-7320 for additional information.
2. Mismatch of the upper and lower dies and resin burrs aren't included.
3. Determines the true geometric position.
4. Values b,C apply to plated terminal. The standard thickness of the plating layer shall measure between 0.007 to .021 mm.

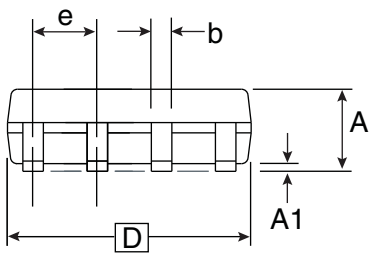
4/15/08

	TITLE	GPC	DRAWING NO.	REV.
Package Drawing Contact: packagedrawings@atmel.com	**8S2,** 8-lead, 0.208” Body, Plastic Small Outline Package (EIAJ)	STN	8S2	F

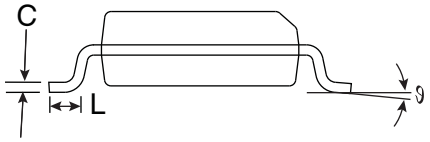
26.3 S8S1



Top View



Side View



End View

COMMON DIMENSIONS
(Unit of Measure = mm)

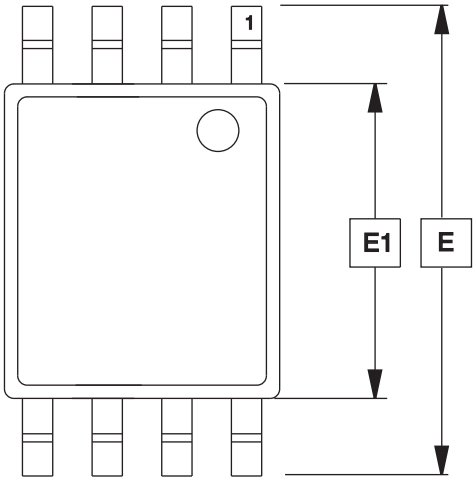
SYMBOL	MIN	NOM	MAX	NOTE
E	5.79		6.20	
E1	3.81		3.99	
A	1.35		1.75	
A1	0.1		0.25	
D	4.80		4.98	
C	0.17		0.25	
b	0.31		0.51	
L	0.4		1.27	
e	1.27 BSC			
θ	0°		8°	

Notes: 1. This drawing is for general information only; refer to JEDEC Drawing MS-012 for proper dimensions, tolerances, datums,etc.

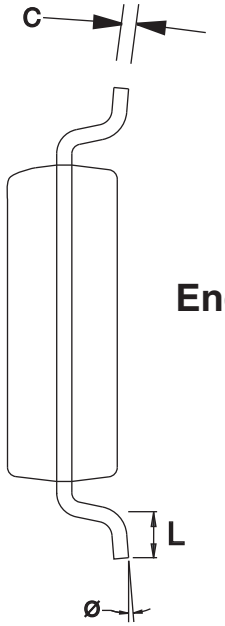
7/28/03

2325 Orchard Parkway San Jose, CA 95131	**TITLE** **S8S1**, 8-lead, 0.150" Wide Body, Plastic Gull Wing Small Outline (JEDEC SOIC)	**DRAWING NO.** S8S1	**REV.** A

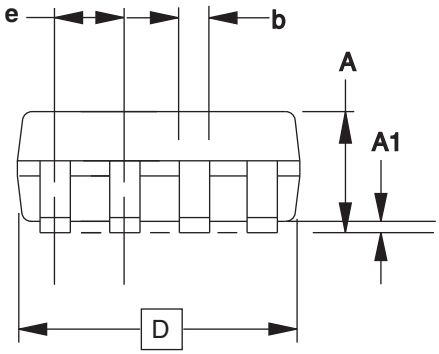
26.4 8X



Top View



End View



Side View

COMMON DIMENSIONS
(Unit of Measure = mm)

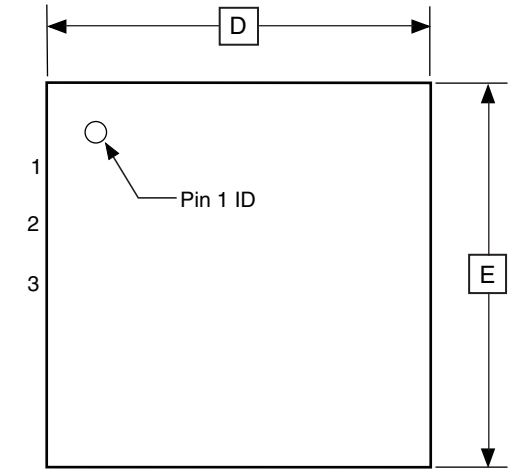
SYMBOL	MIN	NOM	MAX	NOTE
A	1.05	1.10	1.20	
A1	0.05	0.10	0.15	
b	0.25	–	0.30	
C	–	0.127	–	
D	2.90	3.05	3.10	
E1	4.30	4.40	4.50	
E	6.20	6.40	6.60	
e	0.65 TYP			
L	0.50	0.60	0.70	
ø	0°	–	8°	

Note: These drawings are for general information only. Refer to JEDEC Drawing MO-153AC.

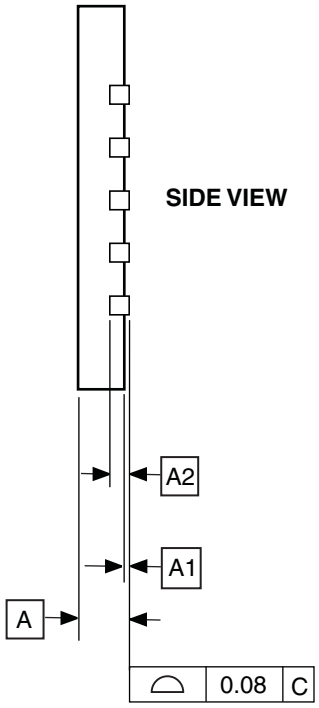
4/14/05

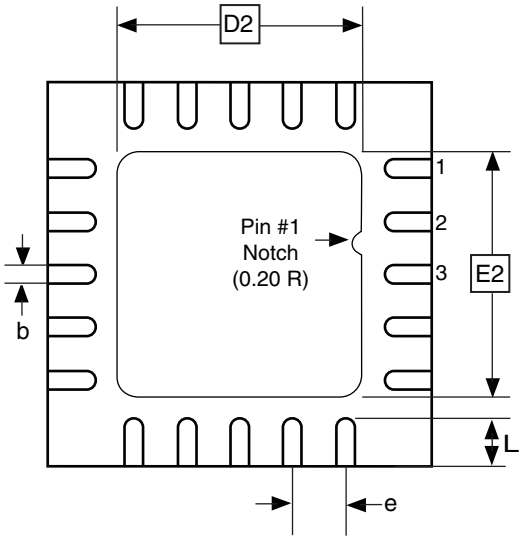
	TITLE	DRAWING NO.	REV.
2325 Orchard Parkway San Jose, CA 95131	**8X**, 8-lead, 4.4 mm Body Width, Plastic Thin Shrink Small Outline Package (TSSOP)	8X	A

26.5 20M1



TOP VIEW





BOTTOM VIEW

COMMON DIMENSIONS
(Unit of Measure = mm)

SYMBOL	MIN	NOM	MAX	NOTE
A	0.70	0.75	0.80	
A1	–	0.01	0.05	
A2		0.20 REF		
b	0.18	0.23	0.30	
D		4.00 BSC		
D2	2.45	2.60	2.75	
E		4.00 BSC		
E2	2.45	2.60	2.75	
e		0.50 BSC		
L	0.35	0.40	0.55	

Note: Reference JEDEC Standard MO-220, Fig. 1 (SAW Singulation) WGGD-5.

10/27/04

	TITLE	DRAWING NO.	REV.
2325 Orchard Parkway San Jose, CA 95131	**20M1**, 20-pad, 4 x 4 x 0.8 mm Body, Lead Pitch 0.50 mm, 2.6 mm Exposed Pad, Micro Lead Frame Package (MLF)	20M1	B

27. Errata

27.1 Errata ATtiny25

The revision letter in this section refers to the revision of the ATtiny25 device.

27.1.1 Rev D – F

No known errata.

27.1.2 Rev B – C

- **EEPROM read may fail at low supply voltage / low clock frequency**

1. **EEPROM read may fail at low supply voltage / low clock frequency**

   Trying to read EEPROM at low clock frequencies and/or low supply voltage may result in invalid data.

   Problem Fix/Workaround

   Do not use the EEPROM when clock frequency is below 1MHz and supply voltage is below 2V. If operating frequency can not be raised above 1MHz then supply voltage should be more than 2V. Similarly, if supply voltage can not be raised above 2V then operating frequency should be more than 1MHz.

   This feature is known to be temperature dependent but it has not been characterised. Guidelines are given for room temperature, only.

27.1.3 Rev A

Not sampled.

27.2 Errata ATtiny45

The revision letter in this section refers to the revision of the ATtiny45 device.

27.2.1 Rev F – G

No known errata

27.2.2 Rev D – E

- **EEPROM read may fail at low supply voltage / low clock frequency**

1. **EEPROM read may fail at low supply voltage / low clock frequency**

   Trying to read EEPROM at low clock frequencies and/or low supply voltage may result in invalid data.

   Problem Fix/Workaround

   Do not use the EEPROM when clock frequency is below 1MHz and supply voltage is below 2V. If operating frequency can not be raised above 1MHz then supply voltage should be more than 2V. Similarly, if supply voltage can not be raised above 2V then operating frequency should be more than 1MHz.

   This feature is known to be temperature dependent but it has not been characterised. Guidelines are given for room temperature, only.

27.2.3 Rev B – C

- **PLL not locking**
- **EEPROM read from application code does not work in Lock Bit Mode 3**
- **EEPROM read may fail at low supply voltage / low clock frequency**
- **Timer Counter 1 PWM output generation on OC1B- XOC1B does not work correctly**

1. **PLL not locking**

   When at frequencies below 6.0 MHz, the PLL will not lock

   Problem fix / Workaround

   When using the PLL, run at 6.0 MHz or higher.

2. **EEPROM read from application code does not work in Lock Bit Mode 3**

   When the Memory Lock Bits LB2 and LB1 are programmed to mode 3, EEPROM read does not work from the application code.

   Problem Fix/Work around

   Do not set Lock Bit Protection Mode 3 when the application code needs to read from EEPROM.

3. **EEPROM read may fail at low supply voltage / low clock frequency**

   Trying to read EEPROM at low clock frequencies and/or low supply voltage may result in invalid data.

   Problem Fix/Workaround

   Do not use the EEPROM when clock frequency is below 1MHz and supply voltage is below 2V. If operating frequency can not be raised above 1MHz then supply voltage should be more than 2V. Similarly, if supply voltage can not be raised above 2V then operating frequency should be more than 1MHz.

   This feature is known to be temperature dependent but it has not been characterised. Guidelines are given for room temperature, only.

4. **Timer Counter 1 PWM output generation on OC1B – XOC1B does not work correctly**

   Timer Counter1 PWM output OC1B-XOC1B does not work correctly. Only in the case when the control bits, COM1B1 and COM1B0 are in the same mode as COM1A1 and COM1A0, respectively, the OC1B-XOC1B output works correctly.

   Problem Fix/Work around

   The only workaround is to use same control setting on COM1A[1:0] and COM1B[1:0] control bits, see table 14-4 in the data sheet. The problem has been fixed for Tiny45 rev D.

27.2.4 Rev A

- **Too high power down power consumption**
- **DebugWIRE looses communication when single stepping into interrupts**
- **PLL not locking**
- **EEPROM read from application code does not work in Lock Bit Mode 3**
- **EEPROM read may fail at low supply voltage / low clock frequency**

1. **Too high power down power consumption**

   Three situations will lead to a too high power down power consumption. These are:

   – An external clock is selected by fuses, but the I/O PORT is still enabled as an output.

   – The EEPROM is read before entering power down.

   – VCC is 4.5 volts or higher.

   Problem fix / Workaround

– When using external clock, avoid setting the clock pin as Output.
– Do not read the EEPROM if power down power consumption is important.
– Use VCC lower than 4.5 Volts.

2. **DebugWIRE looses communication when single stepping into interrupts**
   When receiving an interrupt during single stepping, debugwire will loose

   communication.

   Problem fix / Workaround
   – When singlestepping, disable interrupts.
   – When debugging interrupts, use breakpoints within the interrupt routine, and run into the interrupt.

3. **PLL not locking**
   When at frequencies below 6.0 MHz, the PLL will not lock

   Problem fix / Workaround
   When using the PLL, run at 6.0 MHz or higher.

4. **EEPROM read from application code does not work in Lock Bit Mode 3**
   When the Memory Lock Bits LB2 and LB1 are programmed to mode 3, EEPROM read does not work from the application code.

   Problem Fix/Work around
   Do not set Lock Bit Protection Mode 3 when the application code needs to read from EEPROM.

5. **EEPROM read may fail at low supply voltage / low clock frequency**
   Trying to read EEPROM at low clock frequencies and/or low supply voltage may result in invalid data.

   Problem Fix/Workaround
   Do not use the EEPROM when clock frequency is below 1MHz and supply voltage is below 2V. If operating frequency can not be raised above 1MHz then supply voltage should be more than 2V. Similarly, if supply voltage can not be raised above 2V then operating frequency should be more than 1MHz.

   This feature is known to be temperature dependent but it has not been characterized. Guidelines are given for room temperature, only.

27.3 Errata ATtiny85

The revision letter in this section refers to the revision of the ATtiny85 device.

27.3.1 Rev B – C

No known errata.

27.3.2 Rev A

- **EEPROM read may fail at low supply voltage / low clock frequency**

1. **EEPROM read may fail at low supply voltage / low clock frequency**

   Trying to read EEPROM at low clock frequencies and/or low supply voltage may result in invalid data.

   Problem Fix/Workaround

   Do not use the EEPROM when clock frequency is below 1MHz and supply voltage is below 2V. If operating frequency can not be raised above 1MHz then supply voltage should be more than 2V. Similarly, if supply voltage can not be raised above 2V then operating frequency should be more than 1MHz.

   This feature is known to be temperature dependent but it has not been characterised. Guidelines are given for room temperature, only.

28. Datasheet Revision History

28.1 Rev. 2586Q-08/13

1. “Bit 3 – FOC1B: Force Output Compare Match 1B” description in “GTCCR – General Timer/Counter1 Control Register” on page 90 updated: PB3 in “compare match output pin PB3 (OC1B)” corrected to PB4.

28.2 Rev. 2586P-06/13

1. Updated description of “EEARH – EEPROM Address Register” and “EEARL – EEPROM Address Register” on page 20.

28.3 Rev. 2586O-02/13

Updated ordering codes on page 204, page 205, and page 206.

28.4 Rev. 2586N-04/11

1. Added:
   – Section “Capacitive Touch Sensing” on page 6.
2. Updated:
   – Document template.
   – Removed “Preliminary” on front page. All devices now final and in production.
   – Section “Limitations” on page 36.
   – Program example on page 49.
   – Section “Overview” on page 122.
   – Table 17-4 on page 135.
   – Section “Limitations of debugWIRE” on page 140.
   – Section “Serial Programming Algorithm” on page 151.
   – Table 21-7 on page 166.
   – EEPROM errata on pages 212, 212, 213, 214, and 215
   – Ordering information on pages 204, 205, and 206.

28.5 Rev. 2586M-07/10

1. Clarified Section 6.4 “Clock Output Buffer” on page 31.
2. Added Ordering Codes -SN and -SNR for ATtiny25 extended temperature.

28.6 Rev. 2586L-06/10

1. Added:
   – TSSOP for ATtiny45 in “Features” on page 1, Pinout Figure 1-1 on page 2, Ordering Information in Section 25.2 “ATtiny45” on page 205, and Packaging Information in Section 26.4 “8X” on page 210
   – Table 6-11, “Capacitance of Low-Frequency Crystal Oscillator,” on page 29
   – Figure 22-36 on page 191 and Figure 22-37 on page 191, Typical Characteristics plots for Bandgap Voltage vs. V_{CC} and Temperature
   – Extended temperature in Section 25.1 “ATtiny25” on page 204, Ordering Information
   – Tape & reel part numbers in Ordering Information, in Section 25.1 “ATtiny25” on page 204 and Section 25.2 “ATtiny45” on page 205

2. Updated:
   – “Features” on page 1, removed Preliminary from ATtiny25
   – Section 8.4.2 “Code Example” on page 44
   – “PCMSK – Pin Change Mask Register” on page 52, Bit Descriptions
   – “TCCR1 – Timer/Counter1 Control Register” on page 89 and “GTCCR – General Timer/Counter1 Control Register” on page 90, COM bit descriptions clarified
   – Section 20.3.2 “Calibration Bytes” on page 150, frequencies (8 MHz, 6.4 MHz)
   – Table 20-11, “Minimum Wait Delay Before Writing the Next Flash or EEPROM Location,” on page 153, value for t_{WD_ERASE}
   – Table 20-16, “High-voltage Serial Programming Instruction Set for ATtiny25/45/85,” on page 158
   – Table 21-1, “DC Characteristics. TA = -40°C to +85°C,” on page 161, notes adjusted
   – Table 21-11, “Serial Programming Characteristics, TA = -40°C to +85°C, VCC = 1.8 - 5.5V (Unless Otherwise Noted),” on page 170, added t_{SLIV}
   – Bit syntax throughout the datasheet, e.g. from CS02:0 to CS0[2:0].

28.7 Rev. 2586K-01/08

1. Updated Document Template.
2. Added Sections:
   – “Data Retention” on page 6
   – “Low Level Interrupt” on page 49
   – “Device Signature Imprint Table” on page 149
3. Updated Sections:
   – “Internal PLL for Fast Peripheral Clock Generation - clkPCK” on page 24
   – “System Clock and Clock Options” on page 23
   – “Internal PLL in ATtiny15 Compatibility Mode” on page 24
   – “Sleep Modes” on page 34
   – “Software BOD Disable” on page 35
   – “External Interrupts” on page 49
   – “Timer/Counter1 in PWM Mode” on page 97
   – “USI – Universal Serial Interface” on page 108
   – “Temperature Measurement” on page 133
   – “Reading Lock, Fuse and Signature Data from Software” on page 143
   – “Program And Data Memory Lock Bits” on page 147
   – “Fuse Bytes” on page 148
   – “Signature Bytes” on page 150
   – “Calibration Bytes” on page 150
   – “System and Reset Characteristics” on page 165
4. Added Figures:
   – “Reset Pin Output Voltage vs. Sink Current (VCC = 3V)” on page 184
   – “Reset Pin Output Voltage vs. Sink Current (VCC = 5V)” on page 185
   – “Reset Pin Output Voltage vs. Source Current (VCC = 3V)” on page 185
   – “Reset Pin Output Voltage vs. Source Current (VCC = 5V)” on page 186

5. Updated Figure:
   – “Reset Logic” on page 39
6. Updated Tables:
   – “Start-up Times for Internal Calibrated RC Oscillator Clock” on page 28
   – “Start-up Times for Internal Calibrated RC Oscillator Clock (in ATtiny15 Mode)” on page 28
   – “Start-up Times for the 128 kHz Internal Oscillator” on page 28
   – “Compare Mode Select in PWM Mode” on page 86
   – “Compare Mode Select in PWM Mode” on page 98
   – “DC Characteristics. TA = -40°C to +85°C” on page 161
   – “Calibration Accuracy of Internal RC Oscillator” on page 164
   – “ADC Characteristics” on page 167
7. Updated Code Example in Section:
   – “Write” on page 17
8. Updated Bit Descriptions in:
   – “MCUCR – MCU Control Register” on page 37
   – “Bits 7:6 – COM0A[1:0]: Compare Match Output A Mode” on page 77
   – “Bits 5:4 – COM0B[1:0]: Compare Match Output B Mode” on page 77
   – “Bits 2:0 – ADTS[2:0]: ADC Auto Trigger Source” on page 138
   – “SPMCSR – Store Program Memory Control and Status Register” on page 145.
9. Updated description of feature “EEPROM read may fail at low supply voltage / low clock frequency” in Sections:
   – “Errata ATtiny25” on page 212
   – “Errata ATtiny45” on page 212
   – “Errata ATtiny85” on page 215
10. Updated Package Description in Sections:
   – “ATtiny25” on page 204
   – “ATtiny45” on page 205
   – “ATtiny85” on page 206
11. Updated Package Drawing:
   – “S8S1” on page 209
12. Updated Order Codes for:
   – “ATtiny25” on page 204

28.8 Rev. 2586J-12/06

1. Updated “Low Power Consumption” on page 1.
2. Updated description of instruction length in “Architectural Overview” .
3. Updated Flash size in “In-System Re-programmable Flash Program Memory” on page 15.
4. Updated cross-references in sections “Atomic Byte Programming” , “Erase” and “Write” , starting on page 17.
5. Updated “Atomic Byte Programming” on page 17.
6. Updated “Internal PLL for Fast Peripheral Clock Generation - clkPCK” on page 24.
7. Replaced single clocking system figure with two: Figure 6-2 and Figure 6-3.

8. Updated Table 6-1 on page 25, Table 6-13 on page 30 and Table 6-6 on page 27.
9. Updated “Calibrated Internal Oscillator” on page 27.
10. Updated Table 6-5 on page 26.
11. Updated “OSCCAL – Oscillator Calibration Register” on page 31.
12. Updated “CLKPR – Clock Prescale Register” on page 32.
13. Updated “Power-down Mode” on page 35.
14. Updated “Bit 0” in “PRR – Power Reduction Register” on page 38.
15. Added footnote to Table 8-3 on page 46.
16. Updated Table 10-5 on page 63.
17. Deleted “Bits 7, 2” in “MCUCR – MCU Control Register” on page 64.
18. Updated and moved section “Timer/Counter0 Prescaler and Clock Sources”, now located on page 66.
19. Updated “Timer/Counter1 Initialization for Asynchronous Mode” on page 86.
20. Updated bit description in “PLLCSR – PLL Control and Status Register” on page 94 and “PLLCSR – PLL Control and Status Register” on page 103.
21. Added recommended maximum frequency in“Prescaling and Conversion Timing” on page 125.
22. Updated Figure 17-8 on page 129 .
23. Updated “Temperature Measurement” on page 133.
24. Updated Table 17-3 on page 134.
25. Updated bit R/W descriptions in:
“TIMSK – Timer/Counter Interrupt Mask Register” on page 81,
“TIFR – Timer/Counter Interrupt Flag Register” on page 81,
“TIMSK – Timer/Counter Interrupt Mask Register” on page 92,
“TIFR – Timer/Counter Interrupt Flag Register” on page 93,
“PLLCSR – PLL Control and Status Register” on page 94,
“TIMSK – Timer/Counter Interrupt Mask Register” on page 102,
“TIFR – Timer/Counter Interrupt Flag Register” on page 103,
“PLLCSR – PLL Control and Status Register” on page 103 and
“DIDR0 – Digital Input Disable Register 0” on page 138.
26. Added limitation to “Limitations of debugWIRE” on page 140.
27. Updated “DC Characteristics” on page 161.
28. Updated Table 21-7 on page 166.
29. Updated Figure 21-6 on page 171.
30. Updated Table 21-12 on page 171.
31. Updated Table 22-1 on page 177.
32. Updated Table 22-2 on page 177.
33. Updated Table 22-30, Table 22-31 and Table 22-32, starting on page 188.
34. Updated Table 22-33, Table 22-34 and Table 22-35, starting on page 189.
35. Updated Table 22-39 on page 192.
36. Updated Table 22-46, Table 22-47, Table 22-48 and Table 22-49.

28.9 Rev. 2586I-09/06

1. All Characterization data moved to “Electrical Characteristics” on page 161.
2. All Register Descriptions are gathered up in seperate sections in the end of each chapter.
3. Updated Table 11-3 on page 78, Table 11-5 on page 79, Table 11-6 on page 80 and Table 20-4 on page 148.
4. Updated “Calibrated Internal Oscillator” on page 27.
5. Updated Note in Table 7-1 on page 34.
6. Updated “System Control and Reset” on page 39.
7. Updated Register Description in “I/O Ports” on page 53.
8. Updated Features in “USI – Universal Serial Interface” on page 108.
9. Updated Code Example in “SPI Master Operation Example” on page 110 and “SPI Slave Operation Example” on page 111.
10. Updated “Analog Comparator Multiplexed Input” on page 119.
11. Updated Figure 17-1 on page 123.
12. Updated “Signature Bytes” on page 150.
13. Updated “Electrical Characteristics” on page 161.

28.10 Rev. 2586H-06/06

1. Updated “Calibrated Internal Oscillator” on page 27.
2. Updated Table 6.5.1 on page 31.
3. Added Table 21-2 on page 164.

28.11 Rev. 2586G-05/06

1. Updated “Internal PLL for Fast Peripheral Clock Generation - clkPCK” on page 24.
2. Updated “Default Clock Source” on page 30.
3. Updated “Low-Frequency Crystal Oscillator” on page 29.
4. Updated “Calibrated Internal Oscillator” on page 27.
5. Updated “Clock Output Buffer” on page 31.
6. Updated “Power Management and Sleep Modes” on page 34.
7. Added “Software BOD Disable” on page 35.
8. Updated Figure 16-1 on page 119.
9. Updated “Bit 6 – ACBG: Analog Comparator Bandgap Select” on page 120.
10. Added note for Table 17-2 on page 125.
11. Updated “Register Summary” on page 200.

28.12 Rev. 2586F-04/06

1. Updated “Digital Input Enable and Sleep Modes” on page 57.
2. Updated Table 20-16 on page 158.
3. Updated “Ordering Information” on page 204.

28.13 Rev. 2586E-03/06

1. Updated Features in “Analog to Digital Converter” on page 122.
2. Updated Operation in “Analog to Digital Converter” on page 122.
3. Updated Table 17-2 on page 133.
4. Updated Table 17-3 on page 134.
5. Updated “Errata” on page 212.

28.14 Rev. 2586D-02/06

1. Updated Table 6-13 on page 30, Table 6-10 on page 29, Table 6-3 on page 26, Table 6-9 on page 28, Table 6-5 on page 26, Table 9-1 on page 48,Table 17-4 on page 135, Table 20-16 on page 158, Table 21-8 on page 167.
2. Updated “Timer/Counter1 in PWM Mode” on page 86.
3. Updated text “Bit 2 – TOV1: Timer/Counter1 Overflow Flag” on page 93.
4. Updated values in “DC Characteristics” on page 161.
5. Updated “Register Summary” on page 200.
6. Updated “Ordering Information” on page 204.
7. Updated Rev B and C in “Errata ATtiny45” on page 212.
8. All references to power-save mode are removed.
9. Updated Register Adresses.

28.15 Rev. 2586C-06/05

1. Updated “Features” on page 1.
2. Updated Figure 1-1 on page 2.
3. Updated Code Examples on page 18 and page 19.
4. Moved “Temperature Measurement” to Section 17.12 page 133.
5. Updated “Register Summary” on page 200.
6. Updated “Ordering Information” on page 204.

28.16 Rev. 2586B-05/05

1. CLKI added, instances of EEMWE/EEWE renamed EEMPE/EEPE, removed some TBD.
   Removed “Preliminary Description” from “Temperature Measurement” on page 133.
2. Updated “Features” on page 1.
3. Updated Figure 1-1 on page 2 and Figure 8-1 on page 39.
4. Updated Table 7-2 on page 38, Table 10-4 on page 63, Table 10-5 on page 63
5. Updated “Serial Programming Instruction set” on page 153.
6. Updated SPH register in “Instruction Set Summary” on page 202.
7. Updated “DC Characteristics” on page 161.
8. Updated “Ordering Information” on page 204.
9. Updated “Errata” on page 212.

28.17 Rev. 2586A-02/05

Initial revision.

Table of Contents

Features ***1***

1 Pin Configurations ***2***

1.1 Pin Descriptions 2

2 Overview ***4***

2.1 Block Diagram 4

3 About ***6***

3.1 Resources 6

3.2 Code Examples 6

3.3 Capacitive Touch Sensing 6

3.4 Data Retention 6

4 AVR CPU Core ***7***

4.1 Introduction 7

4.2 Architectural Overview 7

4.3 ALU – Arithmetic Logic Unit 8

4.4 Status Register 8

4.5 General Purpose Register File 10

4.6 Stack Pointer 11

4.7 Instruction Execution Timing 11

4.8 Reset and Interrupt Handling 12

5 AVR Memories ***15***

5.1 In-System Re-programmable Flash Program Memory 15

5.2 SRAM Data Memory 15

5.3 EEPROM Data Memory 16

5.4 I/O Memory 19

5.5 Register Description 20

6 System Clock and Clock Options ***23***

6.1 Clock Systems and their Distribution 23

6.2 Clock Sources 25

6.3 System Clock Prescaler 31

6.4 Clock Output Buffer 31

6.5 Register Description 31

7 Power Management and Sleep Modes ***34***

7.1 Sleep Modes 34

7.2 Software BOD Disable 35
7.3 Power Reduction Register 36
7.4 Minimizing Power Consumption 36
7.5 Register Description 37

8 System Control and Reset 39

8.1 Resetting the AVR 39
8.2 Reset Sources 39
8.3 Internal Voltage Reference 42
8.4 Watchdog Timer 42
8.5 Register Description 44

9 Interrupts 48

9.1 Interrupt Vectors in ATtiny25/45/85 48
9.2 External Interrupts 49
9.3 Register Description 51

10 I/O Ports 53

10.1 Introduction 53
10.2 Ports as General Digital I/O 53
10.3 Alternate Port Functions 57
10.4 Register Description 64

11 8-bit Timer/Counter0 with PWM 65

11.1 Features 65
11.2 Overview 65
11.3 Timer/Counter0 Prescaler and Clock Sources 66
11.4 Counter Unit 68
11.5 Output Compare Unit 69
11.6 Compare Match Output Unit 70
11.7 Modes of Operation 71
11.8 Timer/Counter Timing Diagrams 76
11.9 Register Description 77

12 8-bit Timer/Counter1 83

12.1 Timer/Counter1 Prescaler 83
12.2 Counter and Compare Units 83
12.3 Register Description 89

13 8-bit Timer/Counter1 in ATtiny15 Mode 95

13.1 Timer/Counter1 Prescaler 95

13.2 Counter and Compare Units 95
13.3 Register Description 100

14 Dead Time Generator 105

14.1 Register Description 106

15 USI – Universal Serial Interface 108

15.1 Features 108
15.2 Overview 108
15.3 Functional Descriptions 109
15.4 Alternative USI Usage 114
15.5 Register Descriptions 115

16 Analog Comparator 119

16.1 Analog Comparator Multiplexed Input 119
16.2 Register Description 120

17 Analog to Digital Converter 122

17.1 Features 122
17.2 Overview 122
17.3 Operation 123
17.4 Starting a Conversion 124
17.5 Prescaling and Conversion Timing 125
17.6 Changing Channel or Reference Selection 128
17.7 ADC Noise Canceler 128
17.8 Analog Input Circuitry 129
17.9 Noise Canceling Techniques 129
17.10 ADC Accuracy Definitions 130
17.11 ADC Conversion Result 132
17.12 Temperature Measurement 133
17.13 Register Description 134

18 debugWIRE On-chip Debug System 139

18.1 Features 139
18.2 Overview 139
18.3 Physical Interface 139
18.4 Software Break Points 140
18.5 Limitations of debugWIRE 140
18.6 Register Description 140

19 Self-Programming the Flash 141

19.1 Performing Page Erase by SPM 141
19.2 Filling the Temporary Buffer (Page Loading) 141
19.3 Performing a Page Write 142
19.4 Addressing the Flash During Self-Programming 142
19.5 EEPROM Write Prevents Writing to SPMCSR 142
19.6 Reading Lock, Fuse and Signature Data from Software 143
19.7 Preventing Flash Corruption 144
19.8 Programming Time for Flash when Using SPM 145
19.9 Register Description 145

20 Memory Programming 147

20.1 Program And Data Memory Lock Bits 147
20.2 Fuse Bytes 148
20.3 Device Signature Imprint Table 149
20.4 Page Size 150
20.5 Serial Downloading 151
20.6 High-voltage Serial Programming 155
20.7 High-voltage Serial Programming Algorithm 155

21 Electrical Characteristics 161

21.1 Absolute Maximum Ratings* 161
21.2 DC Characteristics 161
21.3 Speed 163
21.4 Clock Characteristics 164
21.5 System and Reset Characteristics 165
21.6 Brown-Out Detection 166
21.7 ADC Characteristics 167
21.8 Serial Programming Characteristics 170
21.9 High-voltage Serial Programming Characteristics 171

22 Typical Characteristics 172

22.1 Active Supply Current 172
22.2 Idle Supply Current 175
22.3 Supply Current of I/O modules 177
22.4 Power-down Supply Current 178
22.5 Pin Pull-up 179
22.6 Pin Driver Strength 182
22.7 Pin Threshold and Hysteresis 186
22.8 BOD Threshold 189

22.9 Internal Oscillator Speed 192
22.10 Current Consumption of Peripheral Units 196
22.11 Current Consumption in Reset and Reset Pulsewidth 198

23 Register Summary 200

24 Instruction Set Summary 202

25 Ordering Information 204

25.1 ATtiny25 204
25.2 ATtiny45 205
25.3 ATtiny85 206

26 Packaging Information 207

26.1 8P3 207
26.2 8S2 208
26.3 S8S1 209
26.4 8X 210
26.5 20M1 211

27 Errata 212

27.1 Errata ATtiny25 212
27.2 Errata ATtiny45 212
27.3 Errata ATtiny85 215

28 Datasheet Revision History 216

28.1 Rev. 2586Q-08/13 216
28.2 Rev. 2586P-06/13 216
28.3 Rev. 2586O-02/13 216
28.4 Rev. 2586N-04/11 216
28.5 Rev. 2586M-07/10 216
28.6 Rev. 2586L-06/10 216
28.7 Rev. 2586K-01/08 217
28.8 Rev. 2586J-12/06 218
28.9 Rev. 2586I-09/06 220
28.10 Rev. 2586H-06/06 220
28.11 Rev. 2586G-05/06 220
28.12 Rev. 2586F-04/06 220
28.13 Rev. 2586E-03/06 221
28.14 Rev. 2586D-02/06 221
28.15 Rev. 2586C-06/05 221

28.16 Rev. 2586B-05/05 221
28.17 Rev. 2586A-02/05 221

Table of Contents ***i***

Atmel | Enabling Unlimited Possibilities®

Atmel Corporation
1600 Technology Drive
San Jose, CA 95110
USA
Tel: (+1) (408) 441-0311
Fax: (+1) (408) 487-2600
www.atmel.com

Atmel Asia Limited
Unit 01-5 & 16, 19F
BEA Tower, Millennium City 5
418 Kwun Tong Roa
Kwun Tong, Kowloon
HONG KONG
Tel: (+852) 2245-6100
Fax: (+852) 2722-1369

Atmel Munich GmbH
Business Campus
Parkring 4
D-85748 Garching b. Munich
GERMANY
Tel: (+49) 89-31970-0
Fax: (+49) 89-3194621

Atmel Japan G.K.
16F Shin-Osaki Kangyo Bldg
1-6-4 Osaki, Shinagawa-ku
Tokyo 141-0032
JAPAN
Tel: (+81) (3) 6417-0300
Fax: (+81) (3) 6417-0370

© 2013 Atmel Corporation. All rights reserved. / Rev.: 2586Q–AVR–08/2013

Atmel®, Atmel logo and combinations thereof, Enabling Unlimited Possibilities®, AVR®, tinyAVR® and others are registered trademarks or trademarks of Atmel Corporation or its subsidiaries. Other terms and product names may be trademarks of others.

Disclaimer: The information in this document is provided in connection with Atmel products. No license, express or implied, by estoppel or otherwise, to any intellectual property right is granted by this document or in connection with the sale of Atmel products. EXCEPT AS SET FORTH IN THE ATMEL TERMS AND CONDITIONS OF SALES LOCATED ON THE ATMEL WEBSITE, ATMEL ASSUMES NO LIABILITY WHATSOEVER AND DISCLAIMS ANY EXPRESS, IMPLIED OR STATUTORY WARRANTY RELATING TO ITS PRODUCTS INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL ATMEL BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE, SPECIAL OR INCIDENTAL DAMAGES (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS AND PROFITS, BUSINESS INTERRUPTION, OR LOSS OF INFORMATION) ARISING OUT OF THE USE OR INABILITY TO USE THIS DOCUMENT, EVEN IF ATMEL HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. Atmel makes no representations or warranties with respect to the accuracy or completeness of the contents of this document and reserves the right to make changes to specifications and products descriptions at any time without notice. Atmel does not make any commitment to update the information contained herein. Unless specifically provided otherwise, Atmel products are not suitable for, and shall not be used in, automotive applications. Atmel products are not intended, authorized, or warranted for use as components in applications intended to support or sustain life.